Machine Learning

From Intuition to Implementation



A textbook for BCA students
Course code NBCA-601

Unit I Introduction · Unit II Supervised Learning · Unit III Ensembles & SVM
Unit IV Unsupervised Learning · Unit V Neural Networks

How to Use This Book

Learning Objectives

Every chapter opens with a box like this one, telling you exactly what you will be able to do by the end.

Key Idea

The one idea to remember. If you remember nothing else, remember this box.

Definition: Term

Formal definitions you may be asked to write in exams.

Example

Worked examples with real numbers, solved step by step.

Common Mistake

Mistakes students commonly make, and how to avoid them.

Try It Yourself

Short Python experiments. Run them, change numbers, and watch what happens.

Exam Tip

Frequently asked exam questions and how to answer them.

Contents

1 Introduction to Machine Learning **1**

1.1 Why Do We Need Machine Learning? . . . 1

1.2 Artificial Intelligence, Machine Learning and Deep Learning . . . 3

1.3 Where Is Machine Learning Used? . . . 5

1.4 Basic Terminology . . . 5

1.5 Types of Machine Learning Systems . . . 6

1.5.1 Supervised Learning . . . 7

1.5.2 Unsupervised Learning . . . 8

1.5.3 Semi-supervised Learning . . . 9

1.5.4 Reinforcement Learning . . . 10

1.5.5 Batch vs. Online Learning . . . 11

1.5.6 Instance-based vs. Model-based Learning . . . 12

1.6 The Machine Learning Workflow . . . 14

1.7 Main Challenges of Machine Learning . . . 14

1.7.1 Problems with the Data . . . 14

1.7.2 Problems with the Algorithm . . . 15

1.7.3 Testing and Validating . . . 17

1.7.4 No Free Lunch Theorem . . . 18

Exam Corner . . . 18

2 Statistical Learning **22**

2.1 Introduction to Statistical Learning . . . 22

2.1.1 The Statistical Model . . . 23

2.1.2 Why Estimate f? Prediction and Inference . . . 24

2.1.3 Parametric and Non-parametric Methods . . . 24

2.2 Supervised and Unsupervised Learning: The Statistical View . . . 24
2.2.1 Supervised Learning . . . 25
2.2.2 Unsupervised Learning . . . 25
2.3 Loss Functions, Training Loss and Test Loss . . . 26
2.3.1 Loss Functions . . . 26
2.3.2 Risk, Training Loss and Test Loss . . . 27
2.3.3 Splitting the Data . . . 28
2.4 Trade-offs in Statistical Learning . . . 29
2.4.1 The Bias–Variance Trade-off . . . 29
2.4.2 Other Trade-offs . . . 31
2.5 Estimating Risk Statistics . . . 32
2.5.1 Hold-out (Validation-set) Estimate . . . 32
2.5.2 k-Fold Cross-Validation . . . 33
2.5.3 Risk Statistics for Classification . . . 34
2.6 Sampling Distribution of an Estimator . . . 35
2.6.1 Properties of Good Estimators . . . 36
2.6.2 The Sample Mean . . . 36
2.6.3 Central Limit Theorem . . . 37
2.6.4 A Biased Estimator: the Variance . . . 38
Exam Corner . . . 39

3 Distance-Based Methods, Decision Trees and Naive Bayes 43

3.1 Distance-Based Methods . . . 44
3.1.1 Common Distance Measures . . . 44
3.1.2 Feature Scaling . . . 45
3.2 Nearest Neighbours (k-NN) . . . 46
3.2.1 The Algorithm . . . 46
3.2.2 Choosing k . . . 48
3.2.3 Weighted k-NN and k-NN Regression . . . 48
3.2.4 Strengths and Weaknesses . . . 49
3.3 Decision Trees . . . 49
3.3.1 Idea and Structure . . . 49
3.3.2 Measures of Impurity . . . 50

3.3.3 Worked Example: Building a Tree with ID3 . . . 51
3.3.4 Algorithm Summary . . . 53
3.3.5 Overfitting and Pruning . . . 54
3.3.6 Strengths and Weaknesses . . . 54
3.4 Naive Bayes . . . 54
3.4.1 Bayes' Theorem . . . 55
3.4.2 The Naive Bayes Classifier . . . 56
3.4.3 Worked Example: Play-Tennis Again . . . 56
3.4.4 Laplace Smoothing . . . 57
3.4.5 Variants of Naive Bayes . . . 57
3.5 Putting It Together: The Three Methods in Python . . . 59
3.6 Comparison of the Three Methods . . . 60
Exam Corner . . . 61

4 Linear Models and Support Vector Machines 66

4.1 Linear Regression . . . 67
4.1.1 Simple Linear Regression . . . 67
4.1.2 Normal Equation (Matrix Form) . . . 69
4.1.3 Gradient Descent . . . 70
4.1.4 Multiple Linear Regression . . . 72
4.1.5 Evaluating a Regression Model . . . 73
4.1.6 Regularised Linear Regression . . . 73
4.2 Logistic Regression . . . 74
4.2.1 Why Not Use Linear Regression for Classes? . . . 75
4.2.2 Odds and Log-odds . . . 75
4.2.3 Loss Function: Cross-entropy from Maximum Likelihood . . . 76
4.2.4 Real-life Example: Will the Student Pass? . . . 76
4.2.5 Training by Gradient Descent: One Step by Hand . . . 78
4.2.6 Multiclass Logistic Regression . . . 78
4.2.7 Choosing the Threshold and Regularisation . . . 79
4.3 Generalized Linear Models (GLMs) . . . 80
4.3.1 Real-life Example: Poisson Regression for Emergency Calls . . . 81
4.4 Binary Classification and Support Vector Machines . . . 82

4.4.1 Binary Classification . . . 82
4.4.2 The Support Vector Machine (Linear, Hard Margin) . . . 82
4.4.3 Soft-Margin SVM . . . 84
4.5 Choosing Among the Methods of Unit II . . . 86
Exam Corner . . . 87

5 Ensemble Learning 92
5.1 Introduction to Ensemble Learning . . . 92
5.1.1 The Idea: Wisdom of the Crowd . . . 93
5.1.2 Why Does It Work? A Simple Calculation . . . 93
5.1.3 Types of Ensemble Methods . . . 94
5.2 Voting Classifiers . . . 95
5.2.1 Hard vs. Soft Voting: Worked Example . . . 96
5.3 Bagging and Pasting . . . 97
5.3.1 The Bootstrap . . . 97
5.3.2 Algorithm and Aggregation . . . 98
5.3.3 Out-of-Bag (OOB) Evaluation . . . 99
5.4 Random Forests . . . 99
5.4.1 Feature Importance . . . 101
5.5 Boosting . . . 102
5.5.1 AdaBoost (Adaptive Boosting) . . . 102
5.5.2 Gradient Boosting . . . 105
5.6 Stacking . . . 107
5.7 Comparison and Practical Advice . . . 109
Exam Corner . . . 111

6 Support Vector Machines and Naive Bayes Classifiers 116
6.1 Linear SVM Classification . . . 117
6.1.1 The Idea: Build the Widest Road . . . 117
6.1.2 Definitions . . . 118
6.1.3 Soft-Margin SVM: Strict and Relaxed Teachers . . . 118
6.1.4 The Dual Form and Support Vectors . . . 120
6.1.5 Practical Points . . . 120

6.2 Nonlinear SVM Classification . . . 121
6.2.1 When a Straight Line Is Not Enough . . . 121
6.2.2 The Kernel Trick . . . 123
6.2.3 How an RBF SVM Decides: Hills of Influence . . . 124
6.2.4 Choosing C and γ: Real Results . . . 125
6.3 SVM Regression . . . 127
6.3.1 The Idea: Points Inside the Tunnel Are Free . . . 127
6.3.2 Real Results . . . 129
6.4 Naive Bayes Classifiers . . . 130
6.4.1 The Detective Who Multiplies Clues . . . 131
6.4.2 Gaussian Naive Bayes (continuous features) . . . 131
6.4.3 Multinomial Naive Bayes (word counts) . . . 133
6.4.4 Bernoulli Naive Bayes (word present or absent) . . . 134
6.4.5 Generative vs. Discriminative Classifiers . . . 136
6.5 Comparing the Methods of Unit III . . . 137
Exam Corner . . . 137

7 Clustering: K-Means, DBSCAN and Gaussian Mixtures 143
7.1 Introduction to Clustering . . . 143
7.2 K-Means Clustering . . . 145
7.2.1 The Idea: Where to Open the Pizza Shops? . . . 145
7.2.2 Definition and Algorithm . . . 146
7.2.3 Worked Example: K-Means by Hand . . . 147
7.2.4 Initialisation and K-Means++ . . . 149
7.2.5 Choosing the Number of Clusters k . . . 149
7.3 Limits of K-Means . . . 151
7.4 Using Clustering for Image Segmentation . . . 153
7.5 Using Clustering for Preprocessing . . . 155
7.6 Using Clustering for Semi-Supervised Learning . . . 156
7.7 DBSCAN . . . 158
7.7.1 The Idea: Crowds at a Party . . . 158
7.7.2 Definitions . . . 158
7.7.3 Worked Example: DBSCAN by Hand . . . 159

7.7.4 Choosing ε and MinPts . . . 160
7.8 Gaussian Mixture Models . . . 162
7.8.1 The Idea: Overlapping Hills . . . 162
7.8.2 The EM Algorithm . . . 163
7.8.3 Worked Example: One EM Iteration by Hand . . . 163
7.8.4 Covariance Types and Choosing the Number of Components . . . 164
7.8.5 Other Uses and Cautions . . . 166
7.9 Choosing a Clustering Algorithm . . . 167
Exam Corner . . . 168

8 Dimensionality Reduction and Principal Component Analysis 174
8.1 Why Reduce the Number of Features? . . . 174
8.2 The Curse of Dimensionality . . . 175
8.2.1 Cartoon: A Lonely Point in a Big Space . . . 175
8.2.2 Mathematical Facts . . . 176
8.2.3 Real Effect on a Learning Algorithm . . . 177
8.3 Main Approaches for Dimensionality Reduction . . . 178
8.3.1 Cartoon: Shadows versus Unrolling . . . 179
8.3.2 Main Methods . . . 180
8.4 Principal Component Analysis (PCA) . . . 181
8.4.1 The Idea: Find the Best Camera Angle . . . 181
8.4.2 The Mathematics . . . 181
8.4.3 Worked Example: PCA by Hand, Step by Step . . . 182
8.4.4 Choosing the Number of Components . . . 184
8.4.5 Real Example 1: Visualising the Iris Flowers in 2-D . . . 185
8.4.6 Real Example 2: Compressing Handwritten Digits . . . 186
8.4.7 Real Example 3: Why You Must Scale the Features . . . 187
8.4.8 Properties, Limits and Variants . . . 188
8.5 Choosing a Dimensionality-Reduction Method . . . 190
Exam Corner . . . 192

9 Artificial Neural Networks and MLPs with Keras **197**

9.1 From Biological to Artificial Neurons . . . 197

9.1.1 The Biological Inspiration . . . 198

9.1.2 Logic with Neurons . . . 199

9.2 The Perceptron . . . 200

9.2.1 Worked Example: Training a Perceptron to Learn AND . . . 201

9.3 The Multilayer Perceptron (MLP) . . . 202

9.3.1 Forward Pass: A Worked Example . . . 203

9.3.2 Activation Functions . . . 204

9.4 Training a Neural Network: Loss Functions and Backpropagation . . . 207

9.4.1 Loss Functions and Output Layers . . . 207

9.4.2 Gradient Descent and Its Variants . . . 207

9.4.3 Backpropagation . . . 208

9.4.4 Worked Example: One Step of Backpropagation . . . 208

9.4.5 Why Training Can Fail . . . 209

9.4.6 Optimisers . . . 210

9.5 Implementing MLPs with Keras . . . 210

9.5.1 Keras and TensorFlow . . . 210

9.5.2 The Dataset: Fashion-MNIST . . . 211

9.5.3 Building a Classification MLP with the Sequential API . . . 211

9.5.4 Confusion Matrix: Where Does the Network Go Wrong? . . . 214

9.5.5 Regression MLP . . . 215

9.5.6 Functional API: Non-Sequential Networks . . . 215

9.5.7 Callbacks, Saving and Loading . . . 216

9.6 Fine-Tuning the Hyperparameters . . . 216

9.6.1 Number of Neurons . . . 216

9.6.2 The Learning Rate . . . 217

9.6.3 Optimiser . . . 217

9.6.4 Fighting Overfitting: Dropout, L2 and Early Stopping . . . 218

9.6.5 Practical Guidelines . . . 219

9.7 When to Use Neural Networks . . . 219

Exam Corner . . . 220

10 TensorFlow, and Loading and Preprocessing Data **226**

10.1 TensorFlow Basics . . . 227

10.1.1 Tensors . . . 227

10.1.2 Tensor Operations: Real Outputs . . . 228

10.1.3 Automatic Differentiation: GradientTape . . . 230

10.1.4 A Custom Training Loop . . . 231

10.1.5 Speeding Up with `tf.function` . . . 232

10.2 The `tf.data` API . . . 233

10.2.1 Why Do We Need Input Pipelines? . . . 233

10.2.2 Basic Operations: Real Outputs . . . 234

10.2.3 Shuffling with a Buffer . . . 234

10.2.4 Performance: Prefetching and Parallel Processing . . . 235

10.3 Reading Data from CSV and TFRecord Files . . . 237

10.3.1 Reading CSV Files . . . 237

10.3.2 TFRecord Files . . . 238

10.4 Preprocessing the Input Features . . . 239

10.4.1 Numerical Features: Normalization . . . 240

10.4.2 Categorical Features: One-Hot Encoding . . . 240

10.4.3 Embeddings . . . 241

10.4.4 Text Features: TextVectorization . . . 242

10.5 A Complete Pipeline from Data to Model . . . 243

10.6 Putting the Whole Course Together . . . 244

Exam Corner . . . 246

Chapter 1

Introduction to Machine Learning

Learning Objectives

After studying this chapter you will be able to:

- explain the relationship between **AI, Machine Learning and Deep Learning**;
- state and apply Samuel's and Mitchell's definitions of Machine Learning;
- use the vocabulary of ML correctly: *instance, feature, label, training set, model, parameter, hyperparameter*;
- classify ML systems as **supervised, unsupervised, semi-supervised** or **reinforcement**;
- distinguish **batch vs. online** and **instance-based vs. model-based** learning;
- fit a straight-line model by hand using the least-squares formula;
- describe the main **challenges** of ML (data problems, *overfitting*, *underfitting*) and the remedies for each;
- answer 2-, 5- and 10-mark university questions on this unit.

> *Machine learning is the field of study that gives computers the ability to learn without being explicitly programmed.*
>
> — Arthur Samuel, 1959

1.1 Why Do We Need Machine Learning?

Suppose you must write a program that detects *spam e-mail*. The traditional approach is to write rules by hand:

- If the subject contains "FREE" ⇒ spam.
- If the sender is unknown and the body contains "winner" ⇒ spam.

- . . . and hundreds more rules.

This approach has three serious weaknesses.

1. **Long, hard-to-maintain rule lists.** Every new spam trick needs a new rule, and rules start to contradict each other.
2. **No adaptation.** Spammers change "FREE" to "FR33" or "fr.ee" and the rules stop working.
3. **Problems too complex for rules.** No one can write explicit rules to recognise a spoken word or a face in a photo.

A machine-learning program instead studies thousands of e-mails already labelled *spam* or *not spam* and discovers the pattern by itself. When spammers change tactics, we simply retrain it on fresh data.

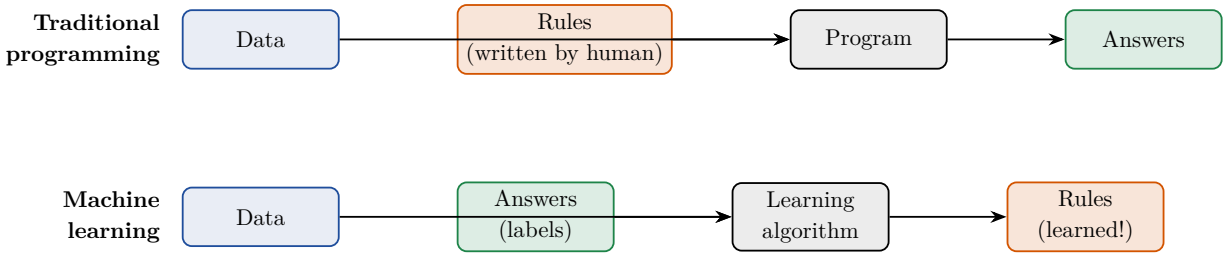


Figure 1.1. Traditional programming versus machine learning.

Reading the Figure

Top row (traditional programming). A programmer writes the *rules*. The program applies those rules to the *data* and produces *answers*.
Bottom row (machine learning). The inputs are swapped. We give the algorithm the *data* together with the correct *answers*, and what comes out is the *rules*. These learned rules can then be applied to new data.
Note the colours: orange marks the *rules*. In the top row a human produces them; in the bottom row the algorithm does. This swap of inputs and outputs is the whole idea of ML.

Key Idea

Traditional programming: *data* + *rules* → *answers*. Machine learning: *data* + *answers* → *rules*.

1.2 Artificial Intelligence, Machine Learning and Deep Learning

Definition: Artificial Intelligence

Artificial Intelligence (AI) is the branch of computer science concerned with building machines that perform tasks that normally require human intelligence, such as reasoning, understanding language, recognising images and making decisions.

Early AI (1950s–1980s) was dominated by **symbolic AI** or *expert systems*, in which experts typed in thousands of "if–then" rules. These systems were useful in narrow areas such as medical diagnosis but broke down on messy real-world data. Since the 1990s the focus has shifted to systems that *learn* from data.

Definition: Machine Learning (Samuel, 1959)

Machine Learning is the field of study that gives computers the ability to learn without being explicitly programmed.

Definition: Machine Learning (Mitchell, 1997)

A computer program is said to **learn** from experience E with respect to some class of tasks T and performance measure P, if its performance at tasks in T, as measured by P, improves with experience E.

Samuel's definition is informal and historical (he wrote a checkers program that improved by playing against itself). Mitchell's definition is more engineering-oriented because it tells us *what to measure*: to claim that a program is learning we must name the task, the metric and the experience.

Definition: Deep Learning

Deep Learning is a subfield of ML that uses *artificial neural networks with many layers* to learn increasingly abstract representations of data (e.g. pixels → edges → shapes → faces). It is the subject of Unit V.

Reading the Figure

The figure is a **set diagram**: each ellipse lies completely inside the one before it.

- *Outer ellipse (AI):* the biggest set. It includes ML but also non-learning AI such as hand-coded chess engines and rule-based expert systems.
- *Middle ellipse (ML):* every ML system is AI, but not every AI system learns.

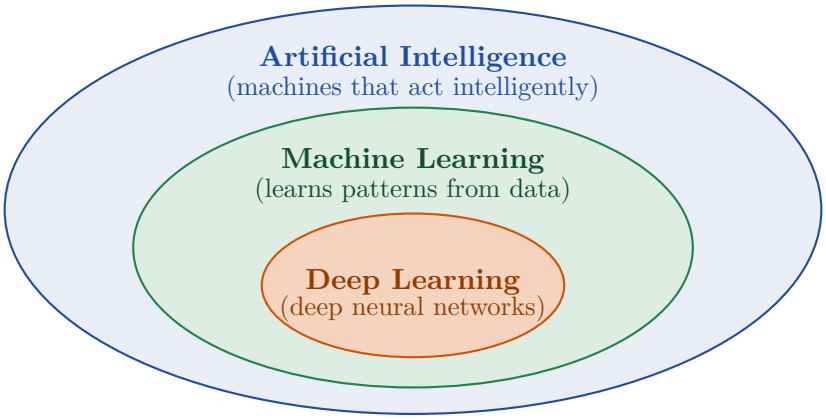


Figure 1.2. AI, ML and Deep Learning as nested sets.

- *Inner ellipse (DL):* deep learning is one family of ML methods. Decision trees, k-NN and SVMs are ML but *not* deep learning.

Remember the order DL $\subset$ ML $\subset$ AI.

Example: Spam filter in (T, P, E) form

Task T	Flag new e-mails as spam or not spam
Performance P	Percentage of e-mails classified correctly (accuracy)
Experience E	A collection of e-mails labelled by users

Check: if accuracy rises from 90% to 96% as we add more labelled mail, the program is learning. If accuracy does not change however much data we add, it is *not* learning, whatever its marketing says.

Example: Which of these is ML?

(a) A program that fixes a date's format using a fixed rule. **Not ML:** no experience is used.

(b) A music app that suggests songs based on what you skipped. **ML:** behaviour (experience) improves recommendations.

(c) A calculator. **Not ML:** its performance never improves with experience.

Exam Tip

"Define Machine Learning" appears almost every year. For full marks write *both* Samuel's and Mitchell's definitions, then give the spam-filter example in (T, P, E) form.

Machine learning

Advantages

- Solves problems too complex for hand-written rules
- Adapts when data changes
- Discovers hidden patterns in large datasets
- Improves as more data becomes available

Disadvantages

- Needs large amounts of good-quality data
- May overfit or inherit bias from the data
- Many models are hard to interpret (black boxes)
- Training can be computationally expensive

Applications

- Spam filtering and recommendation systems
- Medical diagnosis and fraud detection
- Speech, image and language recognition
- Self-driving vehicles

1.3 Where Is Machine Learning Used?

ML is the right tool when (i) rules are too many or too complex, (ii) the environment keeps changing, (iii) the problem has no known algorithmic solution, or (iv) large amounts of data are available to learn from.

Area	Application	ML task
E-mail	Spam filtering	Classification
E-commerce	"Customers also bought"	Recommendation
Health	Detecting disease from X-rays	Image classification
Transport	Self-driving cars	Perception and control
Language	Translation, chatbots	Sequence modelling
Finance	Credit-card fraud detection	Anomaly detection
Real estate	Predicting house prices	Regression
Marketing	Grouping similar customers	Clustering

1.4 Basic Terminology

Before going further, learn this vocabulary. It is used in every chapter.

Term	Meaning
Instance (sample)	One data point, e.g. one e-mail or one house.
Feature (attribute)	An input measurement describing an instance, e.g. area, number of rooms.
Label (target)	The correct output for an instance, e.g. price or "spam".
Training set	Data used to *fit* the model.
Test set	Data held back to *evaluate* the final model on unseen instances.
Model	The function f learned from data, $\hat{y} = f(\boldsymbol{x})$.
Parameters	Numbers learned *by the algorithm* during training (e.g. θ_0, θ_1).
Hyperparameters	Settings chosen *by the human* before training (e.g. k in k-NN, learning rate).

Example: A small dataset

	Area (m^2)	Rooms	Age (yr)	Price (lakh Rs.)
Instance 1	80	2	10	45
Instance 2	120	3	5	78
Instance 3	60	1	20	28

Here *Area, Rooms, Age* are the three **features**; the feature vector of instance 1 is $\boldsymbol{x}_1 = (80, 2, 10)$, and its **label** is $y_1 = 45$. The first three columns together form the input X, and the last column is the target y.

Common Mistake

Do not mix up **parameters** and **hyperparameters**. Parameters are *learned from data*; hyperparameters are *set by you* and are tuned by trying several values and comparing performance on held-out data.

1.5 Types of Machine Learning Systems

ML systems are classified in three independent ways. The same system gets one label from each way, e.g. "a supervised, batch, model-based system".

Q1. How much supervision does it get during training? (supervised, unsupervised, semi-supervised, reinforcement)

Q2. Can it learn incrementally on the fly? (batch vs. online)

Q3. Does it compare new data to stored examples or build a model? (instance-based vs. model-based)

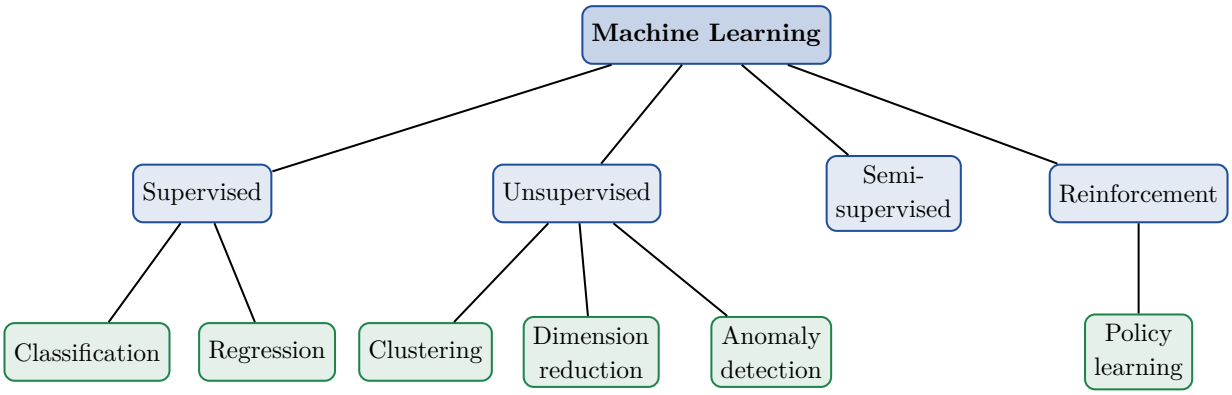


Figure 1.3. Taxonomy of ML systems by the type of supervision. Blue = learning paradigm, green = typical task.

Reading the Figure

Read this **tree from top to bottom**. The root is Machine Learning. The second level (blue) shows the four paradigms; the third level (green) shows the typical *tasks* that each paradigm solves. For example, *classification* and *regression* both need labelled data and therefore sit under *supervised*; *clustering* needs no labels and so sits under *unsupervised*. Semi-supervised learning is a mix of the first two and is usually applied to the same tasks.

1.5.1 Supervised Learning

In supervised learning the training data includes the **correct answer** (the label). The learner is like a student who checks each practice answer against an answer key.

Definition: Supervised Learning

Given a training set $\{(\boldsymbol{x}_i, y_i)\}_{i=1}^{n}$ where $\boldsymbol{x}_i \in \mathbb{R}^d$ is the feature vector and y_i is the label, learn a function f such that $f(\boldsymbol{x}) \approx y$ *for new, unseen inputs.*

Types of supervised problems.

- **Classification:** y is a *category.*
  - *Binary:* two classes (spam / not spam).
  - *Multiclass:* one of many classes (digit 0–9).
  - *Multilabel:* several labels at once (a photo tagged “beach” and “sunset”).
- **Regression:** y is a *real number* (house price, temperature). If there is one feature it is *simple* regression; if more, *multiple* regression.



Figure 1.4. (a) Classification separates two classes with a decision boundary. (b) Regression fits a line that predicts a number.

Reading the Figure

Panel (a): each point is one instance, plotted by two features x_1, x_2. Blue circles belong to class A and orange triangles to class B. The dashed green line is the *decision boundary* the classifier has learned: points on one side are predicted A, points on the other side B. The *output is a category*.
Panel (b): each dot is a house. The horizontal axis is the feature (size) and the vertical axis is the target (price). The green line is the fitted model; to predict the price of a new house, find its size on the horizontal axis and read the height of the line. The *output is a number*.

Supervised learning

Advantages

- Clear goal; accuracy can be measured directly
- Many mature and well-understood algorithms

Disadvantages

- Labelled data is expensive and slow to obtain
- Poor labels (noise) lead to poor models
- Does not generalise beyond the training distribution

Applications

- Spam detection and credit scoring
- House-price and sales prediction
- Disease diagnosis from tests or images

1.5.2 Unsupervised Learning

The training data is **unlabelled**. The system must discover structure on its own, like sorting a heap of unfamiliar coins into groups by size and colour without anyone naming them.

Definition: Unsupervised Learning

Given only inputs $\{\boldsymbol{x}_i\}_{i=1}^{n}$ with no labels, discover hidden structure, such as groups, directions of variation or unusual points, in the data.

- **Clustering:** group similar instances, e.g. segment customers by buying behaviour (K-Means, DBSCAN; Chapter 7).
- **Dimensionality reduction:** compress many features into a few while keeping the information (PCA; Chapter 8). Useful for visualisation and for speeding up learning.
- **Anomaly (outlier) detection:** flag unusual instances, e.g. a suspicious credit-card transaction.
- **Association rule learning:** find relations such as "people who buy bread also buy butter".

Unsupervised learning

Advantages

- No labels needed; works on cheap raw data
- Reveals patterns humans did not expect
- Useful as preprocessing for supervised learning

Disadvantages

- No ground truth, so results are hard to evaluate
- Results can be subjective and depend on parameters
- Sensitive to feature scaling and noise

Applications

- Customer segmentation
- Fraud and fault detection
- Topic discovery in documents
- Data compression and visualisation

1.5.3 Semi-supervised Learning

Definition: Semi-supervised learning

Semi-supervised learning trains a model using a *small amount of labelled data together with a large amount of unlabelled data.*

Labelling is slow and costly (a doctor must label every X-ray), but unlabelled data is cheap. **Semi-supervised** algorithms use a *small labelled* set and a *large unlabelled* set. A typical method is **self-training**: train on the labelled data, predict labels for the unlabelled data, keep the most confident predictions as new labels, and retrain.

Example. Photo apps first cluster faces automatically (unsupervised), then ask you to name each person once (supervised); the name then spreads to every photo in the cluster.

Semi-supervised learning

Advantages

- Greatly reduces the cost of labelling
- Usually more accurate than using the few labels alone

Disadvantages

- Early wrong pseudo-labels can reinforce themselves
- Assumes labelled and unlabelled data come from the same distribution

Applications

- Photo tagging and face grouping
- Speech recognition
- Medical image analysis

1.5.4 Reinforcement Learning

Definition: Reinforcement learning

In **reinforcement learning (RL)** an *agent* interacts with an *environment*: it observes a state, takes an action, and receives a reward or penalty. It learns a *policy* that maximises the total discounted reward $G_t = \sum_k \gamma^k r_{t+k+1}$.

There are no labelled examples. An **agent** observes the **state** of an **environment**, chooses an **action**, and receives a **reward** (positive) or **penalty** (negative). It learns a **policy**, a rule that says which action to take in each state, that maximises the total reward over time.

Term	**Example: robot learning to walk**
Agent	The robot's control program
Environment	The floor, gravity, the robot's body
State s	Joint angles, speed, whether upright
Action a	Move a joint motor
Reward r	$+1$ for moving forward, -10 for falling
Policy π	The learned rule: state $\mapsto$ action

Because rewards may arrive late (a chess move is only rewarded when the game ends), the agent maximises the **discounted return**

$$G_t = r_{t+1} + \gamma\, r_{t+2} + \gamma^2 r_{t+3} + \cdots = \sum_{k=0}^{\infty} \gamma^k\, r_{t+k+1}, \qquad 0 \le \gamma \le 1,$$

where the *discount factor* γ says how much we value future rewards. With $\gamma = 0$ the agent is short-sighted (only the next reward matters); with γ close to 1 it is far-sighted. Famous examples are DeepMind's AlphaGo, which beat the world Go champion, and robots that learn to walk.

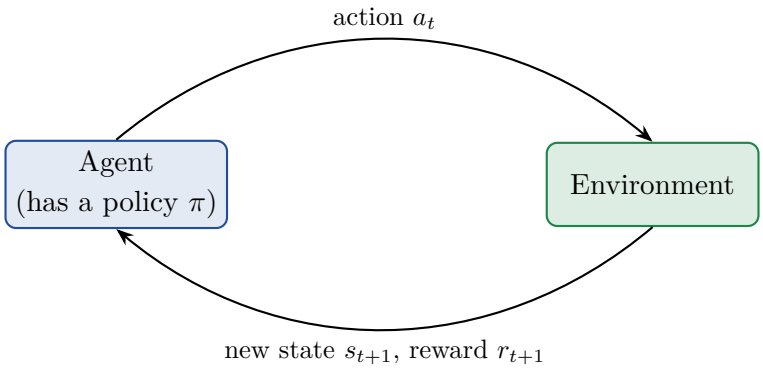


Figure 1.5. The reinforcement-learning loop.

Reading the Figure

The picture shows a **closed loop** that repeats at every time step t. (1) The agent looks at the current state and uses its policy to pick an action a_t (top arrow). (2) The environment reacts and sends back the next state s_{t+1} and a reward r_{t+1} (bottom arrow). (3) The agent uses that feedback to improve its policy, and the loop repeats. Learning happens by *trial and error*: actions that lead to high reward become more likely.

Type	Labels?	Feedback	Typical task	Example
Supervised	Yes	Correct answer	Classification, regression	Spam, prices
Unsupervised	No	None	Clustering, PCA	Customer groups
Semi-supervised	Few	Partial	Mixed	Photo tagging
Reinforcement	No	Reward/penalty	Sequential decisions	Games, robots

Reinforcement learning

Advantages

- Needs no labelled data
- Can learn long-term strategies for sequential decisions
- Adapts to a changing environment

Disadvantages

- Needs very many trials (sample inefficient)
- Designing a good reward function is difficult
- Exploration can be unsafe or costly

Applications

- Game playing (AlphaGo, chess)
- Robotics and self-driving control
- Resource scheduling and recommendation

1.5.5 Batch vs. Online Learning

Batch (offline) learning. The system is trained once on *all* available data, then deployed and no longer learns. To include new data it must be retrained from scratch on old + new data. This is simple but can take hours and a lot of memory.

Online (incremental) learning. The system is trained continuously on single instances or small *mini-batches* as they arrive. Each step is cheap, so it suits streaming data (stock prices) and limited memory; it can even handle datasets too large for memory (*out-of-core learning*).

The key hyperparameter of online learning is the **learning rate**: a *high* rate adapts quickly but forgets old data fast and is sensitive to noise; a *low* rate is stable but slow to adapt. A risk is that *bad data* (a faulty sensor) can quickly degrade a live system, so its performance must be monitored.

Batch (offline) learning

Advantages

- Simple, stable and reproducible
- Any algorithm can be used

Disadvantages

- Cannot learn incrementally; retraining is slow and memory-hungry
- The model becomes outdated as data changes (concept drift)
- Unsuitable for huge or streaming datasets

Applications

- Credit-scoring models retrained monthly
- Offline image classifiers

Online (incremental) learning

Advantages

- Fast, cheap updates; low memory use
- Adapts quickly to changing data
- Handles very large datasets (out-of-core learning) and streams

Disadvantages

- Sensitive to the choice of learning rate
- Bad data can quickly degrade the live model
- Needs continuous monitoring

Applications

- Stock-price prediction
- Daily-updated spam filters
- Sensor and news-feed streams

1.5.6 Instance-based vs. Model-based Learning

Instance-based learning. The system *memorises* the training examples and, for a new point, finds the most *similar* stored ones and copies their answer. *Example:* k-Nearest Neighbours (Chapter 3). No explicit model is built.

Model-based learning. The system builds a *mathematical model* from the examples, then uses only the model (the training data may be thrown away) to predict. *Example:* linear regression.

	Instance-based	**Model-based**
Stores	All training data	Only parameters
Prediction by	Similarity to stored data	Evaluating the learned formula
Training cost	Low (just store)	Higher (fit parameters)
Prediction cost	High (compare to many points)	Low
Example	k-NN	Linear regression

Instance-based learning

Advantages

- No training phase
- Flexible; can fit complex local patterns
- Immediately uses new data

Disadvantages

- Slow prediction and large memory use
- Sensitive to feature scaling and irrelevant features

Applications

- k-Nearest Neighbours
- Recommender systems and pattern recognition

Model-based learning

Advantages

- Fast prediction; compact model
- Often interpretable

Disadvantages

- A wrong model form causes high bias
- Training can be costly

Applications

- Linear and logistic regression
- SVMs and neural networks

Worked example: a model-based learner by hand

Suppose we believe that house price y depends linearly on size x:

$$\hat{y} = \theta_0 + \theta_1 x.$$

Learning means choosing θ_0, θ_1 so that predictions are close to the data. For simple linear regression the least-squares solution is

$$\theta_1 = \frac{\sum_i (x_i - \bar{x})(y_i - \bar{y})}{\sum_i (x_i - \bar{x})^2}, \qquad \theta_0 = \bar{y} - \theta_1 \bar{x}.$$

Example: Fit a line by hand

Data: size x (100 m^2) and price y (lakh Rs.).

x	1	2	3	4	5	$\bar{x} = 3$
y	15	26	29	44	46	$\bar{y} = 32$
$x - \bar{x}$	-2	-1	0	1	2	
$y - \bar{y}$	-17	-6	-3	12	14	
$(x - \bar{x})(y - \bar{y})$	34	6	0	12	28	$\Sigma = 80$
$(x - \bar{x})^2$	4	1	0	1	4	$\Sigma = 10$

Step 1: $\theta_1 = 80/10 = 8$. **Step 2:** $\theta_0 = 32 - 8 \times 3 = 8$.
Model: $\hat{y} = 8 + 8x$. For a new house of size $x = 6$: $\hat{y} = 8 + 8 \times 6 = 56$ lakh Rs..

Try It Yourself

Check the hand calculation with `scikit-learn`. The output should be intercept 8, coefficient 8 and prediction 56.

```
from sklearn.linear_model import LinearRegression

X = [[1], [2], [3], [4], [5]]        # feature: size (100 m^2)
y = [15, 26, 29, 44, 46]             # label: price (lakh Rs)

model = LinearRegression()           # 1. choose the model
model.fit(X, y)                      # 2. learn theta_0 and theta_1
```

```
print(model.intercept_, model.coef_[0])    # -> 8.0  8.0
print(model.predict([[6]]))                # -> [56.]
```

Challenge: add the point (6, 58) to the data. Do the parameters change? Why?

1.6 The Machine Learning Workflow

A real ML project is more than choosing an algorithm. It follows a cycle of steps.

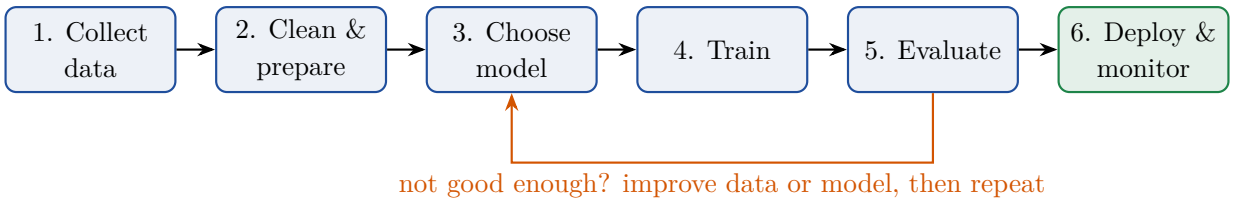


Figure 1.6. The machine-learning workflow.

Reading the Figure

Follow the **blue boxes left to right**; they are the steps in order. The **orange arrow** is the most important part: it shows that evaluation usually sends you *back* to try a different model or better data. ML is therefore iterative.

1. **Collect data:** from databases, sensors, web, surveys.
2. **Clean and prepare:** handle missing values, remove duplicates and outliers, encode text categories as numbers, and *scale* features to comparable ranges. This step often takes most of the project time.
3. **Choose a model** that suits the task (classifier, regressor, clusterer).
4. **Train:** the algorithm finds the parameters that fit the training set.
5. **Evaluate** on data *not* used for training, using the performance measure P.
6. **Deploy and monitor:** use the model on real inputs and watch for performance decay.

1.7 Main Challenges of Machine Learning

Two things can go wrong: *bad data* and *bad algorithm*.

1.7.1 Problems with the Data

1. **Insufficient training data.** A toddler needs to be shown a ball once or twice; ML systems need thousands or millions of examples. Banko and Brill (2001) showed that, given enough

data, even simple algorithms perform almost as well as sophisticated ones: *data matters more than the algorithm.*

2. **Non-representative training data.** The training data must represent the cases the model will meet. If you build a "happiness vs. income" model using only rich countries, it will predict poorly for poor countries. This is **sampling bias**. A famous case is the 1936 *Literary Digest* poll: it surveyed 2.4 million people drawn from telephone and car owners, who were mostly wealthy, and wrongly predicted that Landon would beat Roosevelt. A small *sampling noise* (by chance) arises when the sample is too small; *sampling bias* arises when the method of sampling is flawed, even if the sample is large.

3. **Poor-quality data.** Errors, outliers, noise and missing values hide the patterns. Cleaning (discard outliers, fill or drop missing values) is essential.

4. **Irrelevant features.** "Garbage in, garbage out." **Feature engineering** has three parts: *feature selection* (choose the most useful features), *feature extraction* (combine features, e.g. dimensionality reduction) and *creating new features* by collecting new data.

1.7.2 Problems with the Algorithm

Overfitting. The model performs very well on the training data but badly on new data, because it has *memorised the noise* instead of learning the pattern. It is like a student who memorises last year's paper word for word and then cannot answer a new question. Overfitting happens when the model is *too complex* relative to the amount and noisiness of the data.

Underfitting. The model is too simple to capture the structure of the data, so it does badly even on the training set (like a student who read only the chapter titles).

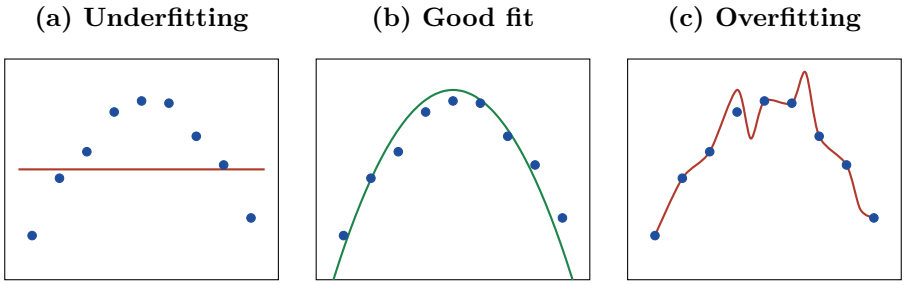


Figure 1.7. The same nine data points fitted by three models of different complexity.

Reading the Figure

All three panels show the *same* blue data points; only the fitted curve (red or green) changes. **(a)** The model is a flat horizontal line. It ignores the rise-and-fall pattern, so errors are large for *every* point: **high training error** ⇒ underfitting. **(b)** A smooth curve passes close to all points without chasing each one: this is the **right complexity** and will generalise well. **(c)** The wiggly curve passes *exactly* through every point, including the random noise. Training error is almost zero, but the zig-zags will mispredict new points: **low training**

error, high test error ⇒ overfitting.

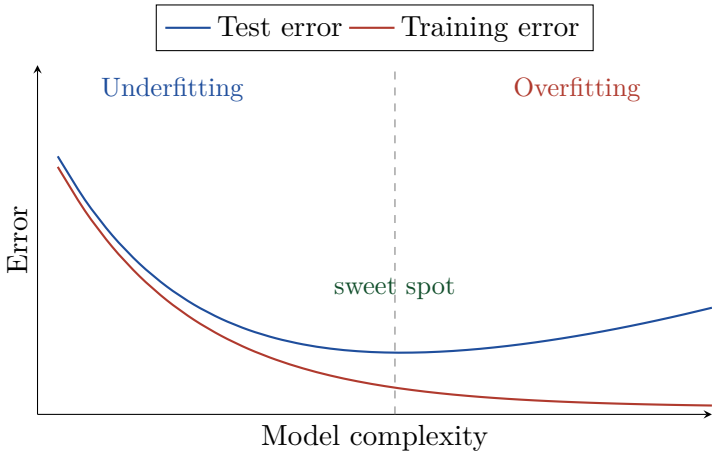


Figure 1.8. Training and test error as model complexity increases.

Reading the Figure

The horizontal axis is how *flexible* the model is (e.g. the degree of a polynomial). The vertical axis is the error.

- **Red curve (training error)** always *falls*: a more flexible model can always fit the training data better.
- **Blue curve (test error)** falls at first and then *rises again*. Beyond the dashed line the model starts fitting noise, so it gets worse on unseen data.
- **Left of the dashed line:** both errors are high ⇒ underfitting. **Right of it:** a growing gap between training and test error ⇒ overfitting.
- **The dashed line** marks the minimum of the test error, the best complexity.

Rule of thumb: judge a model by the *test* curve, never by the training curve.

Common Mistake

Very high accuracy on training data does **not** mean the model is good. Always evaluate on data the model has never seen (Chapter 2).

	Overfitting	**Underfitting**
Model is	too complex	too simple
Training error	very low	high
Test error	high	high
Also called	high variance	high bias
Remedies	more training data; simpler model or fewer features; **regularisation**; remove noise/outliers; early stopping	more complex model; better or more features; reduce regularisation; train longer

Definition: Regularisation

Regularisation means deliberately *constraining* a model to make it simpler and so reduce overfitting, for example by adding a penalty on large parameter values to the loss (Chapter 4). The amount of regularisation is a hyperparameter.

Why do they happen and how can we spot them?

- **Overfitting** is caused by model too complex for the amount of data; too little or noisy training data; too many irrelevant features; training for too long. *How to spot it:* Very low training error but much higher validation or test error.
- **Underfitting** is caused by model too simple (e.g. a straight line for curved data); too few or poorly chosen features; too much regularisation; insufficient training. *How to spot it:* High training error and high test error, close to each other.

1.7.3 Testing and Validating

The only reliable way to know how well a model generalises is to try it on new cases. So we split the data:

- **Training set** (e.g. 60–80%): used to fit the parameters.
- **Validation set** (e.g. 10–20%): used to compare models and tune *hyperparameters*.
- **Test set** (e.g. 10–20%): used *once*, at the very end, to estimate the **generalisation error**.

Common Mistake

If you tune hyperparameters on the *test* set, the test set is no longer "unseen": your reported error will be optimistic. Use the validation set (or cross-validation) for tuning and keep the test set locked away.

1.7.4 No Free Lunch Theorem

Wolpert (1996) proved that, averaged over *all possible problems*, no learning algorithm is better than any other. If we make *no assumption* about the data, there is no reason to prefer a linear model over a neural network. In practice we must therefore (i) make reasonable **assumptions** about the data and (ii) *try several models* and compare them on a validation set.

Key Idea

The goal of ML is **generalisation**: performing well on *new* data, not on the data used for training.

Chapter Summary

- DL $\subset$ ML $\subset$ AI. ML learns *rules from data*; Mitchell's definition uses (T, P, E).
- Vocabulary: instance, feature, label, training/test set, model, parameters (learned) vs. hyperparameters (set by us).
- By supervision: supervised (labels), unsupervised (no labels), semi-supervised (few labels), reinforcement (rewards).
- Supervised tasks: classification (category) and regression (number). Unsupervised tasks: clustering, dimensionality reduction, anomaly detection.
- Batch learns offline from all data; online learns incrementally (learning rate matters). Instance-based memorises; model-based fits parameters.
- Challenges: insufficient data, non-representative data (sampling bias), poor quality, irrelevant features, overfitting, underfitting.
- Fix overfitting by more data, simpler models and regularisation; fix underfitting by richer models and better features.
- Split data into training, validation and test sets; never tune on the test set.

Exam Corner

A. Very Short Answer Questions (2 marks each)

1. Define machine learning. **[2 marks]**
2. What is meant by (T, P, E) in Mitchell's definition? **[2 marks]**
3. Differentiate between a feature and a label. **[2 marks]**
4. Differentiate a parameter from a hyperparameter. **[2 marks]**
5. Name two applications each of classification and regression. **[2 marks]**

6. What is semi-supervised learning? Give one example. **[2 marks]**
7. Define overfitting. **[2 marks]**
8. What is the purpose of the test set? **[2 marks]**

Model Answer: Q2 and Q4

Q2. T is the *task* the program must perform, P is the *performance measure* used to judge it, and E is the *experience* (data) it learns from. A program learns if its P on T improves with E.
Q4. A *parameter* (e.g. θ_1 in linear regression) is learned automatically from the training data. A *hyperparameter* (e.g. k in k-NN or the learning rate) is set by the user before training and tuned using a validation set.

B. Short Answer Questions (5 marks each)

1. Differentiate between AI, machine learning and deep learning with a suitable diagram. **[5 marks]**
2. Compare supervised and unsupervised learning with examples. **[5 marks]**
3. Explain batch and online learning. What is the role of the learning rate? **[5 marks]**
4. Differentiate between instance-based and model-based learning with examples. **[5 marks]**
5. Explain reinforcement learning with a diagram. Define agent, state, action, reward and policy. **[5 marks]**
6. Explain the bias in training data (sampling bias) with an example. **[5 marks]**

Model Answer: B2 (outline for 5 marks)

Supervised learning uses *labelled* data $(\boldsymbol{x}, y)$ and learns a mapping $f(\boldsymbol{x}) \approx y$; tasks are classification and regression (e.g. spam detection, house-price prediction); feedback is the correct answer. **Unsupervised learning** uses *unlabelled* data $\boldsymbol{x}$ and finds hidden structure; tasks are clustering, dimensionality reduction, anomaly detection (e.g. customer segmentation); there is no feedback.

Basis	**Supervised**	**Unsupervised**
Data	Labelled	Unlabelled
Goal	Predict output	Discover structure
Algorithms	k-NN, SVM, linear regression	K-Means, PCA, DBSCAN
Evaluation	Direct (compare with labels)	Indirect

C. Long Answer Questions (10 marks each)

1. Explain the different types of machine-learning systems based on the amount of supervision, with diagrams and examples. **[10 marks]**
2. Describe the main challenges of machine learning. **[10 marks]**
3. What are overfitting and underfitting? Explain with diagrams, state the causes and the remedies for each. **[10 marks]**
4. Using the data $(1, 15), (2, 26), (3, 29), (4, 44), (5, 46)$, fit a straight line $\hat{y} = \theta_0 + \theta_1 x$ by least squares and predict y for $x = 7$. **[10 marks]**

Model Answer: C4 (numerical)

$\bar{x} = 3,\ \bar{y} = 32.\ \sum(x-\bar{x})(y-\bar{y}) = 34+6+0+12+28 = 80;\ \sum(x-\bar{x})^2 = 4+1+0+1+4 = 10.$
$\theta_1 = 80/10 = 8,\ \ \theta_0 = 32 - 8(3) = 8.$ Model: $\hat{y} = 8 + 8x.$
For $x = 7$: $\hat{y} = 8 + 8(7) = \mathbf{64}$.

D. Multiple-Choice Questions

1. Which of the following is *not* a type of ML system by supervision?
   (a) Supervised (b) Unsupervised (c) Reinforcement (d) **Compiled**
2. Grouping customers by buying behaviour without predefined labels is:
   (a) classification (b) regression (c) clustering (d) reinforcement
3. A model with very low training error but high test error is:
   (a) underfitting (b) overfitting (c) well fitted (d) unbiased
4. In Mitchell's definition, accuracy of a spam filter is an example of:
   (a) task (b) experience (c) performance measure (d) model
5. k-Nearest Neighbours is an example of:
   (a) model-based learning (b) instance-based learning (c) online learning (d) RL
6. The discount factor γ is used in:
   (a) clustering (b) regression (c) reinforcement learning (d) PCA
7. Hyperparameters are tuned using the:
   (a) training set (b) validation set (c) test set (d) none

Answers: 1-d, 2-c, 3-b, 4-c, 5-b, 6-c, 7-b.

E. Practice Problems

1. Label each as classification, regression or clustering: (a) predicting tomorrow's temperature, (b) deciding whether a tumour is benign, (c) grouping news articles by topic.

2. Write the (T, P, E) description for a handwriting-recognition program.
3. A robot has $\gamma = 0.9$ and receives rewards 1, 0, 5 at the next three steps (and none afterwards). Compute G_t. *(Answer:* $1 + 0.9 \cdot 0 + 0.81 \cdot 5 = 5.05$*.)*
4. **Coding:** extend the `LinearRegression` program to print predictions for sizes 6, 7 and 8. Plot the data and the fitted line with `matplotlib`.

Quick Revision

Topic	One-line recall
Samuel (1959)	Learning without being explicitly programmed.
Mitchell (1997)	Performance P at task T improves with experience E.
Supervised	Labelled data; classification (category) / regression (number).
Unsupervised	No labels; clustering, PCA, anomaly detection.
Semi-supervised	Few labels + many unlabelled (self-training).
Reinforcement	Agent–environment loop; maximise $G_t = \sum \gamma^k r_{t+k+1}$.
Batch / Online	All data once / incremental; learning rate.
Instance / Model-based	Memorise and compare / fit parameters.
Overfitting	Low train error, high test error $\rightarrow$ more data, regularise, simplify.
Underfitting	High train and test error $\rightarrow$ richer model, better features.
Data splits	Train: fit. Validation: tune. Test: final estimate.
No Free Lunch	No best algorithm for all problems; make assumptions and compare.

Chapter 2

Statistical Learning

Learning Objectives

After studying this chapter you will be able to:

- describe the statistical model $Y = f(X) + \varepsilon$ and explain *prediction* versus *inference*;
- state the statistical view of **supervised** and **unsupervised** learning;
- define and compute common **loss functions** (squared, absolute, 0–1);
- distinguish **training loss**, **test loss** and **risk**, and explain why training loss is optimistic;
- derive and interpret the **bias–variance decomposition** and describe the main **trade-offs** of statistical learning;
- estimate risk using the **hold-out**, **k-fold** and **leave-one-out** methods and compute classification statistics (accuracy, precision, recall, F1);
- explain the **sampling distribution of an estimator**, its bias, variance and standard error, and use the Central Limit Theorem;
- answer 2-, 5- and 10-mark university questions on Unit I.

> *All models are wrong, but some are useful.*
>
> — George Box

2.1 Introduction to Statistical Learning

Chapter 1 told us *what* machine learning is. **Statistical learning** is the mathematical framework that tells us *why it works*, *how well* it works, and *how to measure that*. It treats data as random samples drawn from some unknown real-world process and asks: how can we learn that process from a finite sample?

Definition: Statistical learning

Statistical learning is a set of tools for *estimating the unknown relationship* $Y = f(X) + \varepsilon$ between inputs X and a response Y from data, either to **predict** Y for new X or to **infer** how Y depends on each X_j.

2.1.1 The Statistical Model

Suppose we observe p input variables $X = (X_1, \ldots, X_p)$ (the *features*) and one output variable Y (the *response*). We assume that they are related by

$$Y = f(X) + \varepsilon. \tag{2.1}$$

- f is an **unknown fixed function**: the systematic, predictable part of the relationship.
- ε is a **random error** (noise) with mean zero, $\mathbb{E}[\varepsilon] = 0$, independent of X and with variance σ^2. It stands for everything we did not measure or that is genuinely random.

Statistical learning is therefore *the set of methods for estimating* f from data. Our estimate is written $\hat{f}$.

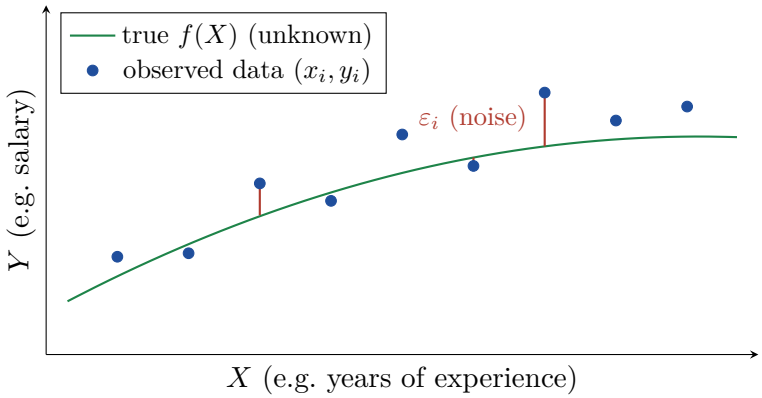


Figure 2.1. Data are generated as $Y = f(X) + \varepsilon$. The curve is the true but unknown f; dots are noisy observations.

Reading the Figure

The **green curve** is the true relationship $f(X)$, which in real life we never see. The **blue dots** are what we actually collect: each dot lies slightly above or below the curve because of random noise. The **short red vertical segments** are examples of the noise term $\varepsilon_i = y_i - f(x_i)$. Our job is to look only at the dots and recover a curve $\hat{f}$ as close to the green one as possible. Because of the noise, we can *never* recover f perfectly.

2.1.2 Why Estimate f? Prediction and Inference

Prediction. We only care about getting accurate predictions $\hat{Y} = \hat{f}(X)$ for new X. $\hat{f}$ can be a "black box". *Example:* forecasting tomorrow's sales.

Inference. We want to *understand* how Y changes with each X_j: which features matter, whether the effect is positive or negative, and how strong it is. $\hat{f}$ must be interpretable. *Example:* how much does one extra year of education raise salary?

Reducible and irreducible error. For a fixed $\hat{f}$ and X,

$$\mathbb{E}[(Y - \hat{Y})^2] = \underbrace{[f(X) - \hat{f}(X)]^2}_{\text{reducible}} + \underbrace{\text{Var}(\varepsilon)}_{\text{irreducible}=\sigma^2} . \tag{2.2}$$

Better methods shrink the reducible part by making $\hat{f}$ closer to f, but *no* method can remove the irreducible part σ^2. This is the lower limit on any model's error.

2.1.3 Parametric and Non-parametric Methods

	Parametric	**Non-parametric**
Idea	Assume a form for f, e.g. $f(X) = \theta_0 + \theta_1 X$, then estimate a few parameters	Make no strong assumption about f; let the data decide its shape
Advantage	Simple, needs little data, easy to interpret	Flexible; can fit any shape
Disadvantage	Wrong assumption ⇒ poor fit	Needs a lot of data; risk of overfitting
Examples	Linear / logistic regression, Naive Bayes	k-NN, decision trees

Statistical learning

Advantages

- Gives a principled way to measure error and uncertainty
- Supports both prediction and inference

Disadvantages

- Conclusions depend on the assumptions made
- The irreducible error σ^2 can never be removed
- Needs enough data to estimate f reliably

Applications

- Forecasting and risk modelling
- Scientific data analysis
- Design and evaluation of ML algorithms

2.2 Supervised and Unsupervised Learning: The Statistical View

2.2.1 Supervised Learning

We observe n pairs $(x_1, y_1), \ldots, (x_n, y_n)$ drawn *independently* from an unknown joint distribution $P(X, Y)$ (this is the **i.i.d. assumption**: independent and identically distributed). We use them to learn a function that predicts Y from X.

Key Idea

What is the best possible predictor?

- For **regression** with squared error, the best predictor is the *conditional mean* $f^*(x) = \mathbb{E}[Y \mid X = x]$ (the **regression function**).
- For **classification**, the best predictor is the **Bayes classifier**: assign x to the class with the highest posterior probability, $\hat{y} = \arg\max_k P(Y = k \mid X = x)$.

Every supervised method is an attempt to approximate one of these using only a finite sample.

The error of the Bayes classifier is the **Bayes error rate**. It is the irreducible error for classification: even the perfect classifier is wrong sometimes when classes overlap.

2.2.2 Unsupervised Learning

We observe only $x_1, \ldots, x_n$ and no y. Statistically, the goal is to learn the structure of the distribution $P(X)$ itself:

- **density estimation:** estimate $p(x)$;
- **clustering:** find groups (modes) in $p(x)$;
- **dimensionality reduction:** find a low-dimensional description of X that retains most of its variation.

Since there is no "correct answer" to compare against, evaluating unsupervised methods is harder and often relies on domain judgement.

	Supervised	**Unsupervised**
Data	(x_i, y_i) pairs	x_i only
Learns	$P(Y \mid X)$ or f	$P(X)$ / structure
Goal	Predict Y for new X	Discover patterns
Loss	Prediction error vs. true y	No direct loss (e.g. within-cluster variance)

2.3 Loss Functions, Training Loss and Test Loss

2.3.1 Loss Functions

To say how good $\hat{f}$ is, we need a number measuring the cost of a wrong prediction.

Definition: Loss function

A **loss function** $L(y, \hat{y}) \geq 0$ measures how bad the prediction $\hat{y}$ is when the true value is y. Lower is better, and $L(y, y) = 0$.

Loss	Formula	Used for / property
Squared error (L2)	$(y - \hat{y})^2$	Regression; punishes large errors heavily; smooth
Absolute error (L1)	$\lvert y - \hat{y} \rvert$	Regression; robust to outliers
0–1 loss	$\mathbb{1}[y \neq \hat{y}]$	Classification; counts mistakes
Cross-entropy (log loss)	$-\sum_k \mathbb{1}[y = k] \ln \hat{p}_k$	Probabilistic classifiers (Chapter 4, 9)

Example: Computing losses

True values $y = (3, 5, 7, 9)$; predictions $\hat{y} = (2, 6, 7, 11)$.

i	1	2	3	4	**Sum**
$y_i - \hat{y}_i$	1	-1	0	-2	
$\lvert y_i - \hat{y}_i \rvert$	1	1	0	2	4
$(y_i - \hat{y}_i)^2$	1	1	0	4	6

$$\text{MAE} = \tfrac{4}{4} = 1, \qquad \text{MSE} = \tfrac{6}{4} = 1.5, \qquad \text{RMSE} = \sqrt{1.5} \approx 1.22.$$

Observation: the single error of 2 contributes $2/4 = 50\%$ of the absolute error but $4/6 \approx 67\%$ of the squared error. Squared loss emphasises large errors.

2.3.2 Risk, Training Loss and Test Loss

Definition: Risk (expected loss, generalisation error)

The **risk** of a function f is its *expected* loss on a new observation drawn from the true distribution:

$$\mathcal{R}(f) = \mathbb{E}_{(X,Y)}\big[L(Y, f(X))\big].$$

The risk is exactly what we want to minimise, but it needs the unknown $P(X, Y)$, so we can never compute it. We estimate it from samples.

Definition: Training loss (empirical risk)

For the training sample $(x_i, y_i)_{i=1}^{n}$,

$$\hat{\mathcal{R}}_{\text{train}}(f) = \frac{1}{n}\sum_{i=1}^{n} L(y_i, f(x_i)).$$

Empirical risk minimisation (ERM): choose $\hat{f} = \arg\min_{f \in \mathcal{F}} \hat{\mathcal{R}}_{\text{train}}(f)$ over a chosen family $\mathcal{F}$ of models.

Definition: Test loss

For a *separate* sample $(x'_j, y'_j)_{j=1}^{m}$ that was **not** used for training,

$$\hat{\mathcal{R}}_{\text{test}}(\hat{f}) = \frac{1}{m}\sum_{j=1}^{m} L(y'_j, \hat{f}(x'_j)).$$

It is an *honest* (unbiased) estimate of the risk $\mathcal{R}(\hat{f})$.

Common Mistake

Training loss is an optimistic estimate of risk. The model was *chosen to minimise it* on exactly those points, so it fits them (and their noise) better than it will fit new data. Test loss $\geq$ training loss on average. Never report training loss as the performance of a model.

Try It Yourself

See the gap between training and test loss for polynomial fits of degree 1, 3 and 9 to noisy data from $y = \sin(2\pi x)$:

```
import numpy as np
rng = np.random.default_rng(0)
x = np.sort(rng.uniform(0, 1, 30))
```

```python
y = np.sin(2*np.pi*x) + rng.normal(0, 0.25, 30)    # f + noise
idx = rng.permutation(30)
tr, te = idx[:20], idx[20:]                          # 20 train / 10 test

def mse(coef, i):
    return np.mean((np.polyval(coef, x[i]) - y[i])**2)

for d in [1, 3, 9]:
    coef = np.polyfit(x[tr], y[tr], d)               # ERM for degree d
    print(d, round(mse(coef, tr), 3), round(mse(coef, te), 3))
```

Output (seed 0):

Degree	Train MSE	Test MSE	Verdict
1	0.222	0.255	underfits (both high)
3	0.041	0.050	good fit
9	0.026	0.102	overfits (test $\approx 4\times$ train)

Notice that the degree-9 model has the *lowest* training loss but a test loss twice that of degree 3. Training loss alone would have picked the wrong model.

Empirical risk minimisation

Advantages

- Simple and general; underlies most ML algorithms

Disadvantages

- Training loss is an *optimistic* estimate of risk
- Overfits if $\mathcal{F}$ is too flexible (needs regularisation or validation)

Applications

- Least-squares regression
- Logistic regression and SVMs
- Neural network training

2.3.3 Splitting the Data

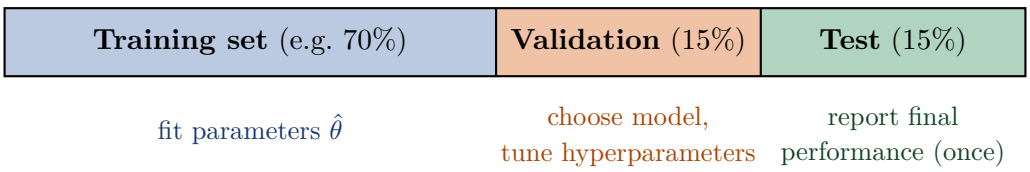


Figure 2.2. Splitting a dataset into training, validation and test sets.

Reading the Figure

The long bar is the *whole* dataset, cut into three **non-overlapping** parts. The label under each part says its single job. The **blue** part is the only one the learning algorithm is allowed to fit. The **orange** part is used to *compare* models and choose hyperparameters. The **green** part is kept locked away until the very end and used *once* to report the final honest performance. Mixing these jobs (for example, tuning on the test part) makes the final number over-optimistic.

2.4 Trade-offs in Statistical Learning

2.4.1 The Bias–Variance Trade-off

Take a *fixed* test point x_0. Imagine collecting many different training sets, each of size n, and fitting $\hat{f}$ on each one. The predictions $\hat{f}(x_0)$ vary from set to set. The expected squared prediction error at x_0 then splits into three parts:

$$\mathbb{E}\Big[\big(y_0 - \hat{f}(x_0)\big)^2\Big] = \underbrace{\Big(\mathbb{E}[\hat{f}(x_0)] - f(x_0)\Big)^2}_{\mathrm{Bias}(\hat{f}(x_0))^2} + \underbrace{\mathbb{E}\Big[\big(\hat{f}(x_0) - \mathbb{E}[\hat{f}(x_0)]\big)^2\Big]}_{\mathrm{Var}(\hat{f}(x_0))} + \underbrace{\sigma^2}_{\text{irreducible}}. \tag{2.3}$$

Definition: Bias and variance of a method

Bias is the error caused by approximating a complicated real problem by a simpler model: how far the *average* prediction is from the truth. **Variance** is how much the prediction would *change* if we trained on a different sample.

Proof sketch of Equation (2.3). Write $y_0 = f + \varepsilon$ and let $\bar{f} = \mathbb{E}[\hat{f}]$. Add and subtract $\bar{f}$:

$$y_0 - \hat{f} = (f - \bar{f}) + (\bar{f} - \hat{f}) + \varepsilon.$$

Square and take expectations. The cross-terms vanish because $\mathbb{E}[\bar{f} - \hat{f}] = 0$ and $\mathbb{E}[\varepsilon] = 0$ with ε independent of $\hat{f}$. What remains is $(f - \bar{f})^2 + \mathbb{E}[(\hat{f} - \bar{f})^2] + \mathbb{E}[\varepsilon^2]$, i.e. $\mathrm{Bias}^2 + \mathrm{Var} + \sigma^2$. ■

Reading the Figure

Think of each **blue dot** as the prediction made by a model trained on a different random sample, and the **red bull's-eye** as the true value.

- *Low bias, low variance:* dots are tightly clustered *on* the centre. This is the ideal.
- *Low bias, high variance:* dots are centred correctly on average but scattered. Predictions depend heavily on the particular sample (**overfitting**).
- *High bias, low variance:* dots are tightly clustered but *away* from the centre. The model is consistently wrong in the same way (**underfitting**).

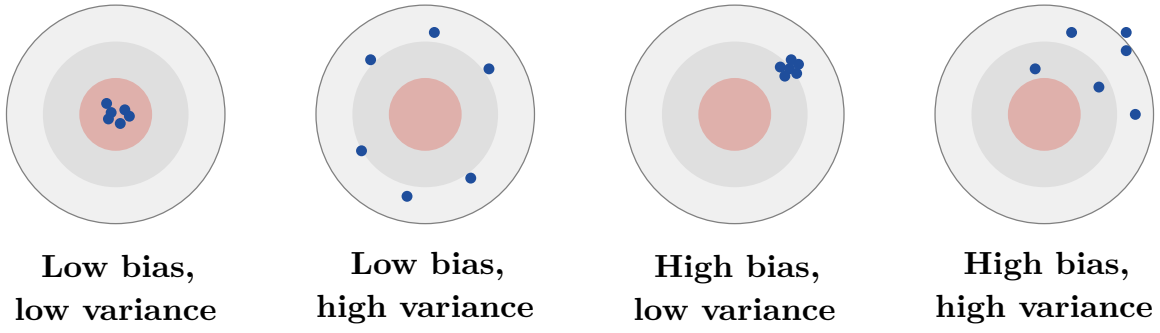


Figure 2.3. Bias and variance explained with a dartboard. Red centre = true value $f(x_0)$; blue dots = predictions from models trained on different samples.

- *High bias, high variance:* dots are scattered and off-centre: the worst case.

Bias is the distance from the *centre of the dots* to the bull's-eye; *variance* is the *spread* of the dots.

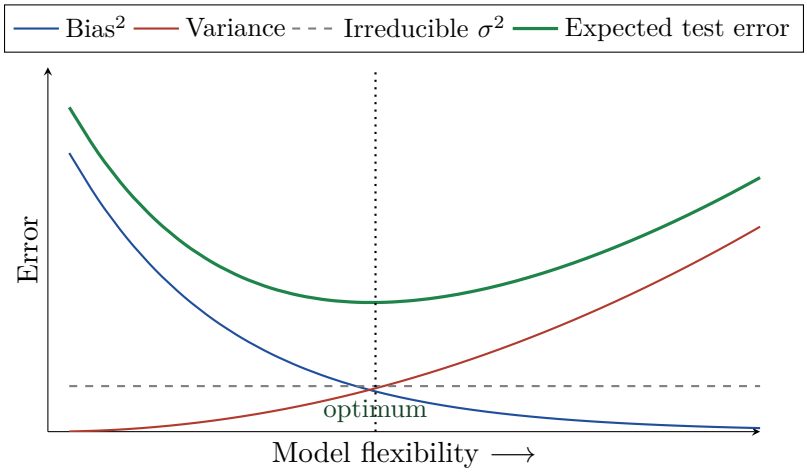


Figure 2.4. Bias^2 falls and variance rises as flexibility increases; expected test error is their sum plus noise.

Reading the Figure

The horizontal axis is how flexible the model is (e.g. polynomial degree, or $1/k$ in k-NN). Read the four curves:

- **Blue (bias^2)** starts high and falls: a flexible model can follow the true f more closely.
- **Red (variance)** starts near zero and rises: a flexible model chases the noise in each training sample.
- **Grey dashed (irreducible error)** is a flat floor that no model can beat.
- **Green (expected test error)** is the *sum* of the three. It is U-shaped: its lowest point (dotted line) is the best flexibility.

Left of the optimum, bias dominates (underfitting); right of it, variance dominates (overfit-

ting). Compare this with Figure 1.8 in Chapter 1.

Example: Putting numbers on the decomposition

At a test point, a model has $\text{Bias}^2 = 4$, $\text{Var} = 3$ and noise $\sigma^2 = 2$.

$$\text{Expected test MSE} = 4 + 3 + 2 = 9.$$

If we switch to a more flexible model with $\text{Bias}^2 = 1$ and $\text{Var} = 5$, the error becomes $1 + 5 + 2 = 8$: lower, even though variance rose, because the bias fell more. If flexibility is pushed further to $\text{Bias}^2 = 0.5$, $\text{Var} = 9$, error is $0.5 + 9 + 2 = 11.5$: worse. This is the trade-off.

How to control the trade-off

- Choose flexibility with cross-validation
- Regularise to reduce variance
- Collect more data to reduce variance
- Ensemble methods (Chapter 5) reduce variance further

2.4.2 Other Trade-offs

1. **Flexibility vs. interpretability.** Simple models (linear regression) are easy to explain but may be too rigid; flexible models (boosting, deep networks) can fit complex patterns but act as black boxes.
2. **Prediction accuracy vs. inference.** If you need to *explain* effects, prefer simple models; if you only need accurate predictions, flexible models are fine.
3. **Model complexity vs. amount of data.** Complex models need much more data. With little data, a simple model usually generalises better.
4. **Training cost vs. accuracy.** More flexible models and more data take longer to train and need more memory.

Reading the Figure

Each box is a family of methods placed by two properties. Moving **right**, models become more flexible (can fit more complicated shapes). Moving **down**, they become harder to interpret. The dashed arrow shows the general trend: *what you gain in flexibility you usually pay for in interpretability.* The positions are rough guides, not exact measurements; these methods are covered in Chapters 3–9.

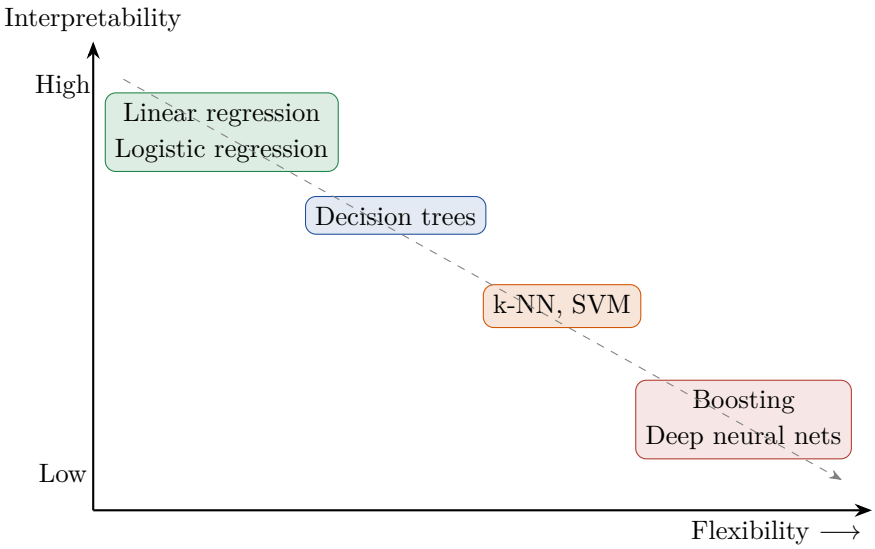


Figure 2.5. Typical position of methods on the flexibility–interpretability trade-off (qualitative).

Exam Tip

The bias–variance trade-off is the single most-asked topic of Unit I. Be ready to (i) state Equation (2.3), (ii) sketch Figure 2.4 with the four curves, and (iii) link high bias to underfitting and high variance to overfitting.

2.5 Estimating Risk Statistics

We cannot compute the true risk $\mathcal{R}(\hat{f})$, so we *estimate* it. A good estimate should be nearly unbiased and have low variance.

2.5.1 Hold-out (Validation-set) Estimate

Split once into training and test parts and compute $\hat{\mathcal{R}}_{\text{test}}$. It is simple and fast, but (i) the number depends on *which* points fell in the test set, and (ii) the model is trained on less data, so it may look slightly worse than it is.

Uncertainty of the estimate. If the m test losses $\ell_j = L(y'_j, \hat{f}(x'_j))$ have sample mean $\bar{\ell} = \hat{\mathcal{R}}_{\text{test}}$ and sample standard deviation s, then

$$\text{SE}(\bar{\ell}) = \frac{s}{\sqrt{m}}, \qquad \text{approximate 95\% CI for the risk:} \quad \bar{\ell} \pm 1.96\ \text{SE}(\bar{\ell}).$$

Example: Confidence interval for test MSE

On $m = 100$ test points the squared errors have mean 4.1 and standard deviation $s = 3$. Then $\text{SE} = 3/\sqrt{100} = 0.3$ and the 95% interval is $4.1 \pm 1.96(0.3) = 4.1 \pm 0.59$, i.e. [3.51, 4.69].

So we may report "test MSE ≈ 4.1, likely between 3.5 and 4.7". Quadrupling the test set to $m = 400$ would halve the standard error to 0.15.

2.5.2 k-Fold Cross-Validation

When data are scarce, we want to use *all* of it for both training and evaluation.

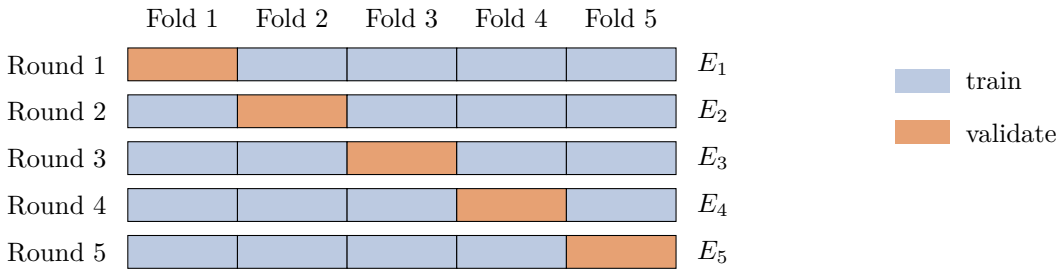


Figure 2.6. 5-fold cross-validation. Each round uses a different fold (orange) for validation and the other four (blue) for training.

Reading the Figure

The dataset is cut into $k = 5$ equal **folds** (the columns). Each **row** is one training round. In Round 1 the model is trained on Folds 2–5 (blue) and its error E_1 is measured on Fold 1 (orange). In Round 2 the orange fold moves to Fold 2, and so on. After five rounds *every point has been used for validation exactly once*, and the final estimate is the average $E_1, \ldots, E_5$ of the five errors.

Definition: k-fold cross-validation estimate

Split the data into k folds. For $i = 1, \ldots, k$ train on all folds except the i-th and compute the error E_i on the i-th. Then

$$\mathrm{CV}_{(k)} = \frac{1}{k} \sum_{i=1}^{k} E_i.$$

Example: 5-fold CV with numbers

Validation errors: $E = (4.2,\ 3.8,\ 4.6,\ 4.0,\ 3.9)$.

$$\mathrm{CV}_{(5)} = \frac{4.2 + 3.8 + 4.6 + 4.0 + 3.9}{5} = \frac{20.5}{5} = 4.1.$$

Deviations from 4.1: $0.1, -0.3, 0.5, -0.1, -0.2$; squares sum to 0.40; $s^2 = 0.40/4 = 0.10$, $s = 0.316$; $\mathrm{SE} = s/\sqrt{5} = 0.141$. So the estimated risk is 4.1 ± 0.14.

Choosing k. **Leave-one-out CV (LOOCV)** is the extreme $k = n$: nearly unbiased but expensive and with high variance. In practice $k = 5$ or $k = 10$ gives a good bias–variance balance.

For classification use *stratified* folds, which keep the class proportions the same in each fold.

	Hold-out	**k-fold CV**	**LOOCV**
Number of fits	1	k	n
Computation	Cheapest	Moderate	Most expensive
Bias of estimate	Higher (less training data)	Low–moderate	Lowest
Variance of estimate	High (one split)	Moderate	High
Typical use	Large datasets	Default choice	Very small datasets

k-fold cross-validation

Advantages

- Uses all data for both training and validation
- More reliable than a single hold-out split
- Standard tool for model selection and tuning

Disadvantages

- Needs k model fits, so it is k times slower
- Not suitable as-is for time-series data (order matters)

Applications

- Hyperparameter tuning
- Comparing different models
- Small datasets

2.5.3 Risk Statistics for Classification

For classification with 0–1 loss, the risk is the **error rate**. We summarise the test predictions in a **confusion matrix**.

		Predicted	
		Positive	Negative
Actual	Positive	TP (true positive)	FN (false negative)
	Negative	FP (false positive)	TN (true negative)

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}, \quad \text{Precision} = \frac{TP}{TP + FP}, \quad \text{Recall} = \frac{TP}{TP + FN},$$

$$\text{Specificity} = \frac{TN}{TN + FP}, \qquad F_1 = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}.$$

Precision answers "of the items I flagged, how many were truly positive?"; *recall* answers "of all true positives, how many did I find?"; F_1 is their harmonic mean.

Example: Spam filter on 100 test e-mails

Actual spam $= 50$, actual non-spam $= 50$. The filter finds $TP = 40$ spam mails, misses $FN = 10$, wrongly flags $FP = 5$ good mails and correctly passes $TN = 45$.

$$\text{Accuracy} = \tfrac{40+45}{100} = 0.85, \qquad \text{Precision} = \tfrac{40}{45} \approx 0.889,$$
$$\text{Recall} = \tfrac{40}{50} = 0.80, \qquad \text{Specificity} = \tfrac{45}{50} = 0.90,$$
$$F_1 = \tfrac{2(0.889)(0.80)}{0.889+0.80} \approx 0.842.$$

Common Mistake

Accuracy can mislead on imbalanced data. If only 1% of transactions are fraud, a program that always says "not fraud" is 99% accurate yet has recall 0 and is useless. Use precision, recall and F_1 instead.

Confusion matrix and classification metrics

Advantages

- Shows *what kind* of errors are made, not only how many
- Works for imbalanced data when precision/recall are used

Disadvantages

- Accuracy misleads on imbalanced classes
- Depends on the chosen decision threshold

Applications

- Medical screening, fraud detection, spam filtering

2.6 Sampling Distribution of an Estimator

Everything we compute from data (a mean, a test error, a regression coefficient) is a *statistic* calculated from a random sample. Draw a different sample and you get a different number. This section studies that randomness.

Definition: Estimator and estimate

An **estimator** $\hat{\theta} = g(X_1, \ldots, X_n)$ is a *rule* (a function of the sample) for guessing an unknown population quantity θ. An **estimate** is the *number* that the rule gives for one particular sample. Since the sample is random, the estimator $\hat{\theta}$ is itself a **random variable**.

Definition: Sampling distribution

The **sampling distribution** of an estimator is its probability distribution over all possible samples of size n from the population. Its *standard deviation* is called the **standard error**, $\mathrm{SE}(\hat{\theta}) = \sqrt{\mathrm{Var}(\hat{\theta})}$.

2.6.1 Properties of Good Estimators

Property	Definition	Meaning
Bias	$\mathrm{Bias}(\hat{\theta}) = \mathbb{E}[\hat{\theta}] - \theta$	Average error; **unbiased** if $= 0$
Variance	$\mathrm{Var}(\hat{\theta}) = \mathbb{E}[(\hat{\theta} - \mathbb{E}\hat{\theta})^2]$	Spread across samples
Standard error	$\mathrm{SE}(\hat{\theta}) = \sqrt{\mathrm{Var}(\hat{\theta})}$	Typical size of the estimation error
MSE	$\mathbb{E}[(\hat{\theta} - \theta)^2] = \mathrm{Var}(\hat{\theta}) + \mathrm{Bias}(\hat{\theta})^2$	Overall accuracy (same decomposition as before!)
Consistency	$\hat{\theta} \to \theta$ as $n \to \infty$	More data gives a better estimate

2.6.2 The Sample Mean

Let $X_1, \dots, X_n$ be i.i.d. with population mean μ and variance σ^2, and let $\bar{X} = \frac{1}{n}\sum X_i$. Then

$$\mathbb{E}[\bar{X}] = \mu \quad \text{(unbiased)}, \qquad \mathrm{Var}(\bar{X}) = \frac{\sigma^2}{n}, \qquad \mathrm{SE}(\bar{X}) = \frac{\sigma}{\sqrt{n}}.$$

Proof. $\mathbb{E}[\bar{X}] = \frac{1}{n}\sum \mathbb{E}[X_i] = \frac{1}{n}(n\mu) = \mu$, and, by independence, $\mathrm{Var}(\bar{X}) = \frac{1}{n^2}\sum \mathrm{Var}(X_i) = \frac{n\sigma^2}{n^2} = \frac{\sigma^2}{n}$. ■

Key Idea

The standard error shrinks like $1/\sqrt{n}$. To make an estimate *twice* as precise you need *four times* as much data.

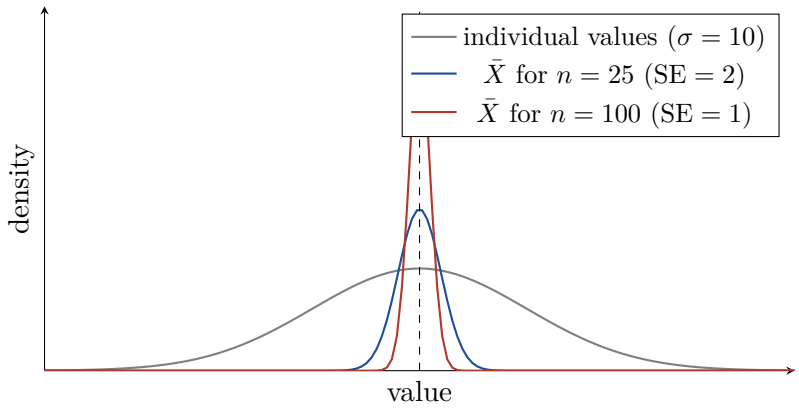


Figure 2.7. Sampling distribution of the sample mean for population mean $\mu = 50$ and $\sigma = 10$.

Reading the Figure

Three bell curves are drawn, all centred on the true mean $\mu = 50$ (dashed line).

- **Grey (widest):** the spread of *individual* observations, $\sigma = 10$.
- **Blue:** the distribution of the *average of 25 observations*. It is still centred at 50 (*unbiased*) but is 5× narrower, SE $= 10/\sqrt{25} = 2$.
- **Red (narrowest and tallest):** the average of 100 observations, SE $= 1$.

The *height* of a curve at a value shows how likely it is that the estimate lands there. The bigger the sample, the more tightly the estimates cluster around the truth. Averages are much more reliable than single observations.

Example: Using the standard error

A population has $\mu = 50$ and $\sigma = 10$. For samples of size $n = 25$, what is $P(\bar{X} > 53)$? SE $= 10/\sqrt{25} = 2$, so $z = \dfrac{53 - 50}{2} = 1.5$ and $P(\bar{X} > 53) = P(Z > 1.5) \approx 0.0668$, about 6.7%. With $n = 100$, SE $= 1$, $z = 3$, and the probability drops to about 0.0013.

2.6.3 Central Limit Theorem

Definition: Central Limit Theorem (CLT)

Let $X_1, \ldots, X_n$ be i.i.d. with mean μ and finite variance σ^2. For large n the sampling distribution of $\bar{X}$ is approximately normal:

$$\bar{X} \;\approx\; \mathcal{N}\!\left(\mu,\, \frac{\sigma^2}{n}\right), \qquad \text{equivalently} \qquad \frac{\bar{X} - \mu}{\sigma/\sqrt{n}} \;\approx\; \mathcal{N}(0, 1),$$

whatever the shape of the original population.

The CLT is the reason the normal curve appears everywhere: test errors, accuracies and averages of many small effects are sums of many small random pieces. It justifies the confidence intervals used in the previous section (a rule of thumb: $n \geq 30$ is usually enough).

95% confidence interval for a mean. $\bar{x} \pm 1.96\,\dfrac{s}{\sqrt{n}}$.

Example: Confidence interval

A sample of $n = 100$ gives $\bar{x} = 52$ and $s = 10$. SE $= 10/10 = 1$, so the 95% interval is 52 ± 1.96, i.e. [50.04, 53.96]. If we repeated the experiment many times, about 95% of intervals built this way would contain the true μ.

2.6.4 A Biased Estimator: the Variance

Not all natural estimators are unbiased. Consider estimating σ^2:

$$S_n^2 = \frac{1}{n}\sum_{i=1}^{n}(X_i - \bar{X})^2, \qquad S_{n-1}^2 = \frac{1}{n-1}\sum_{i=1}^{n}(X_i - \bar{X})^2.$$

It can be shown that $\mathbb{E}[S_n^2] = \dfrac{n-1}{n}\sigma^2 < \sigma^2$, so S_n^2 is **biased** (it underestimates), while S_{n-1}^2 is unbiased. The reason is that deviations are measured from $\bar{X}$, which is itself fitted to the same data and so sits *closer* to the sample than the true μ does. This is the same "fits its own data too well" effect that makes training loss optimistic.

Example: Size of the bias

With $n = 5$ and $\sigma^2 = 10$: $\mathbb{E}[S_n^2] = \frac{4}{5}(10) = 8$ (bias $= -2$), whereas $\mathbb{E}[S_{n-1}^2] = 10$.

Try It Yourself

Verify the standard-error formula and the bias of S_n^2 by simulation:

```
import numpy as np
rng = np.random.default_rng(1)

for n in [5, 25, 100]:
    means = [rng.normal(50, 10, n).mean() for _ in range(10000)]
    print(n, round(np.mean(means), 2), round(np.std(means), 2),
          round(10/np.sqrt(n), 2))       # theory: SE = sigma/sqrt(n)

v0 = [np.var(rng.normal(0, np.sqrt(10), 5), ddof=0) for _ in range(20000)]
v1 = [np.var(rng.normal(0, np.sqrt(10), 5), ddof=1) for _ in range(20000)]
print(round(np.mean(v0), 2), round(np.mean(v1), 2))
```

Typical output:

n	mean of $\bar{X}$	std of $\bar{X}$	theory $10/\sqrt{n}$
5	49.92	4.39	4.47
25	49.98	2.00	2.00
100	50.00	1.01	1.00

The means stay near 50 (unbiased) and the spread follows $\sigma/\sqrt{n}$. The two variance estimates average about 8.0 (ddof = 0, biased) and 9.9 (ddof = 1, unbiased, close to the true 10).

Key Idea

Estimators (including a trained model's predictions and its test error) are random. To trust a number, we need to know **how much it would vary from sample to sample**: that is its **standard error**.

Chapter Summary

- Statistical model: $Y = f(X) + \varepsilon$; we estimate f for *prediction* or *inference*. Error = reducible + irreducible (σ^2).
- Supervised learning estimates $P(Y \mid X)$ (best predictors: conditional mean, Bayes classifier). Unsupervised learning studies $P(X)$.
- Loss measures a single mistake (squared, absolute, 0–1, cross-entropy); **risk** is expected loss; **training loss** (empirical risk) is optimistic; **test loss** is an honest estimate.
- $\mathbb{E}[(y_0 - \hat{f}(x_0))^2] = \text{Bias}^2 + \text{Var} + \sigma^2$. More flexibility lowers bias but raises variance; the best model balances them.
- Other trade-offs: flexibility vs. interpretability, prediction vs. inference, complexity vs. data size.
- Estimate risk by hold-out, k-fold CV ($k = 5$ or 10) or LOOCV; for classification use the confusion matrix, precision, recall and F_1.
- An estimator is a random variable; its spread is the **standard error**. For the sample mean $\text{SE} = \sigma/\sqrt{n}$, and the **CLT** makes $\bar{X}$ approximately normal.
- S_n^2 (divide by n) is biased; S_{n-1}^2 is unbiased.

Exam Corner

A. Very Short Answer Questions (2 marks each)

1. Write the statistical model $Y = f(X) + \varepsilon$ and name each term. **[2 marks]**
2. Differentiate between prediction and inference. **[2 marks]**
3. What is irreducible error? **[2 marks]**
4. Define a loss function. Give two examples. **[2 marks]**
5. Why is training loss an optimistic estimate of the risk? **[2 marks]**
6. Define bias and variance of a learning method. **[2 marks]**
7. What is k-fold cross-validation? **[2 marks]**
8. Define standard error of an estimator. **[2 marks]**

9. State the Central Limit Theorem. **[2 marks]**

10. Define precision and recall. **[2 marks]**

Model Answer: Q5 and Q8

Q5. The model parameters are chosen to *minimise the loss on the training data itself*, so the model partly memorises its noise; on fresh data it performs worse. Hence training loss $\leq$ expected test loss.
Q8. The standard error of an estimator $\hat{\theta}$ is the standard deviation of its sampling distribution, $\mathrm{SE}(\hat{\theta}) = \sqrt{\mathrm{Var}(\hat{\theta})}$. For the sample mean it equals $\sigma/\sqrt{n}$.

B. Short Answer Questions (5 marks each)

1. Distinguish parametric and non-parametric methods with examples. **[5 marks]**

2. Explain training loss, test loss and risk. Why do we need a separate test set? **[5 marks]**

3. Explain the bias–variance trade-off with a diagram. **[5 marks]**

4. Explain k-fold cross-validation with a diagram. Compare it with the hold-out method. **[5 marks]**

5. Explain the confusion matrix and define accuracy, precision, recall and F_1. **[5 marks]**

6. What is the sampling distribution of an estimator? Show that the sample mean is unbiased with $\mathrm{Var}(\bar{X}) = \sigma^2/n$. **[5 marks]**

7. Why is accuracy a poor measure for imbalanced data? Illustrate with an example. **[5 marks]**

Model Answer: B6 (outline for 5 marks)

An *estimator* $\hat{\theta} = g(X_1, \ldots, X_n)$ is a function of a random sample, hence a random variable; its probability distribution over repeated samples of size n is its *sampling distribution*.
For i.i.d. X_i with mean μ and variance σ^2: $\mathbb{E}[\bar{X}] = \frac{1}{n}\sum \mathbb{E}[X_i] = \mu$, so $\bar{X}$ is unbiased. By independence, $\mathrm{Var}(\bar{X}) = \frac{1}{n^2}\sum \mathrm{Var}(X_i) = \frac{n\sigma^2}{n^2} = \frac{\sigma^2}{n}$. Hence $\mathrm{SE}(\bar{X}) = \sigma/\sqrt{n}$; precision improves as n grows. By the CLT, $\bar{X} \approx \mathcal{N}(\mu, \sigma^2/n)$ for large n.

C. Long Answer Questions (10 marks each)

1. Derive the bias–variance decomposition of the expected squared prediction error. Explain its implications for model selection and sketch the curves. **[10 marks]**

2. Describe the different methods of estimating the risk of a model (hold-out, k-fold, LOOCV) with their advantages and disadvantages. **[10 marks]**

3. Explain the sampling distribution of the sample mean. State the CLT and explain, with an example, how it is used to construct a confidence interval. **[10 marks]**

4. Compare supervised and unsupervised learning from a statistical point of view, and discuss the trade-offs in statistical learning. **[10 marks]**

5. A classifier is tested on 200 samples: $TP = 70$, $FN = 30$, $FP = 20$, $TN = 80$. Compute accuracy, precision, recall, specificity and F_1. **[10 marks]**

Model Answer: C5 (numerical)

$n = 200$. Accuracy $= \frac{70+80}{200} = 0.75$. Precision $= \frac{70}{70+20} = 0.778$. Recall $= \frac{70}{70+30} = 0.70$. Specificity $= \frac{80}{80+20} = 0.80$.
$F_1 = \frac{2(0.778)(0.70)}{0.778+0.70} = \frac{1.089}{1.478} \approx \mathbf{0.737}$.

D. Numerical Problems

1. True $y = (2, 4, 6, 8)$, predicted $\hat{y} = (3, 3, 6, 10)$. Compute MAE, MSE and RMSE. *(Answer: $MAE = 1$, $MSE = 1.5$, $RMSE = 1.22$.)*

2. A model has $\text{Bias}^2 = 2.5$, $\text{Var} = 1.5$, $\sigma^2 = 1$. Find the expected test MSE. After switching to a simpler model, $\text{Bias}^2 = 4$, $\text{Var} = 0.5$: which is better? *(Answer: 5; the new one gives 5.5, so the first is better.)*

3. 4-fold CV errors are 6, 5, 7, 6. Find the CV estimate and its standard error. *(Answer: 6; $s = 0.816$, SE $= 0.408$.)*

4. A population has $\mu = 100$ and $\sigma = 20$. For $n = 64$, find $\text{SE}(\bar{X})$ and $P(\bar{X} < 97)$. *(Answer: SE $= 2.5$; $z = -1.2$; $P \approx 0.115$.)*

5. Test squared errors over 400 samples have mean 5 and $s = 4$. Give a 95% CI for the risk. *(Answer: SE $= 0.2$; $5 \pm 0.392 = [4.61, 5.39]$.)*

E. Multiple-Choice Questions

1. The irreducible error in $Y = f(X) + \varepsilon$ is:
   (a) Bias^2 (b) $\text{Var}(\hat{f})$ (c) $\text{Var}(\varepsilon)$ (d) $\hat{\mathcal{R}}_{\text{train}}$

2. As model flexibility increases, typically:
   (a) bias↑, variance↓ (b) bias↓, variance↑ (c) both ↑ (d) both ↓

3. The best predictor under squared loss is:
   (a) the median of Y (b) the mode of Y (c) $\mathbb{E}[Y \mid X]$ (d) $\mathbb{E}[Y]$

4. In 10-fold CV, each observation is used for validation:
   (a) once (b) ten times (c) never (d) a random number of times

5. If $\sigma = 12$ and $n = 36$, then $\text{SE}(\bar{X})$ is:
   (a) 12 (b) 6 (c) 2 (d) 0.33

6. Which estimator of σ^2 is unbiased?
   (a) $\frac{1}{n}\sum(X_i - \bar{X})^2$ (b) $\frac{1}{n-1}\sum(X_i - \bar{X})^2$ (c) $\frac{1}{n+1}\sum(X_i - \bar{X})^2$ (d) none

7. Recall is defined as:
   (a) $\frac{TP}{TP+FP}$ (b) $\frac{TP}{TP+FN}$ (c) $\frac{TN}{TN+FP}$ (d) $\frac{TP+TN}{n}$
8. Which loss is most robust to outliers?
   (a) squared (b) absolute (c) cubic (d) exponential

Answers: 1-c, 2-b, 3-c, 4-a, 5-c, 6-b, 7-b, 8-b.

F. Practice Problems

1. In Figure 2.1, suppose $f(x) = 2x + 1$ and $\sigma = 0.5$. What is the smallest possible expected squared error for any model, and why?
2. Run the polynomial experiment with degrees 2, 5 and 12 and sketch train/test MSE against degree. At which degree does the test MSE start rising?
3. **Coding:** implement 5-fold cross-validation with `cross_val_score` for `LinearRegression` and report the mean and standard error of the scores.
4. Simulate sampling distributions from a *skewed* population (e.g. `rng.exponential(1, n)`) and verify with histograms that $\bar{X}$ becomes bell-shaped as n grows (CLT).

Quick Revision

Topic	One-line recall
Model	$Y = f(X) + \varepsilon$, $\mathbb{E}\varepsilon = 0$, $\operatorname{Var}\varepsilon = \sigma^2$.
Prediction / Inference	Accurate $\hat{Y}$ / understand effect of each X_j.
Error split	Reducible + irreducible σ^2.
Best predictors	$\mathbb{E}[Y \mid X]$ (regression); Bayes classifier $\arg\max_k P(k \mid x)$.
Risk	$\mathbb{E}[L(Y, f(X))]$ (unknown); train loss optimistic; test loss honest.
Bias–variance	$\mathbb{E}[(y_0 - \hat{f})^2] = \text{Bias}^2 + \text{Var} + \sigma^2$; U-shaped test error.
Hold-out / k-fold / LOOCV	1 fit / k fits (use 5 or 10) / n fits.
Classification stats	Acc $= \frac{TP+TN}{n}$, Prec $= \frac{TP}{TP+FP}$, Rec $= \frac{TP}{TP+FN}$, F_1 harmonic mean.
Sampling distribution	Distribution of an estimator over repeated samples; spread = SE.
Sample mean	Unbiased; SE $= \sigma/\sqrt{n}$; 4× data ⇒ 2× precision.
CLT	$\bar{X} \approx \mathcal{N}(\mu, \sigma^2/n)$ for large n; 95% CI $\bar{x} \pm 1.96\,\text{SE}$.
Variance estimator	Divide by $n - 1$ for unbiasedness.

Chapter 3

Distance-Based Methods, Decision Trees and Naive Bayes

Learning Objectives

After studying this chapter you will be able to:

- compute **Euclidean, Manhattan, Minkowski, Chebyshev, Hamming** and **cosine** measures and explain why **feature scaling** matters;
- apply the **k-Nearest Neighbours** algorithm to classification and regression, and explain the effect of k;
- build a **decision tree** by hand using **entropy, information gain** and **Gini index**;
- explain **overfitting in trees** and **pruning**;
- state **Bayes' theorem** and apply the **Naive Bayes** classifier, including **Laplace smoothing** and the **Gaussian** variant;
- compare the three methods and choose between them;
- answer 2-, 5- and 10-mark university questions on these topics.

> *Birds of a feather flock together: the nearest neighbour of a point carries a lot of information about its class.*
>
> — Cover & Hart, 1967 (on nearest neighbours)

This chapter begins Unit II (Supervised Learning). The three methods here are among the simplest and most intuitive classifiers, and each uses a different idea:

Method	Core idea	Type (Chapter 1 / 2)
k-Nearest Neighbours	"You are like your closest neighbours"	instance-based, non-parametric
Decision tree	"Ask a sequence of yes/no questions"	model-based, non-parametric
Naive Bayes	"Use probability and Bayes' rule"	model-based, parametric

3.1 Distance-Based Methods

Many learning algorithms (k-NN, K-Means, SVMs with kernels) depend on one question: *how similar are two instances?* We answer it with a **distance** (small = similar) or a **similarity** (large = similar).

Definition: Distance-based methods

Distance-based methods make predictions or form groups using the *distance (or similarity) between instances* in feature space: close points are assumed to be similar.

Definition: Distance metric

A function $d(\boldsymbol{a}, \boldsymbol{b})$ is a **metric** if for all points: (i) $d(\boldsymbol{a}, \boldsymbol{b}) \geq 0$ and $d(\boldsymbol{a}, \boldsymbol{b}) = 0 \iff \boldsymbol{a} = \boldsymbol{b}$; (ii) symmetry $d(\boldsymbol{a}, \boldsymbol{b}) = d(\boldsymbol{b}, \boldsymbol{a})$; (iii) triangle inequality $d(\boldsymbol{a}, \boldsymbol{c}) \leq d(\boldsymbol{a}, \boldsymbol{b}) + d(\boldsymbol{b}, \boldsymbol{c})$.

3.1.1 Common Distance Measures

For two d-dimensional points $\boldsymbol{a} = (a_1, \ldots, a_d)$ and $\boldsymbol{b} = (b_1, \ldots, b_d)$:

Measure	Formula	Use
Euclidean (L_2)	$\sqrt{\sum_i (a_i - b_i)^2}$	Default for numeric data
Manhattan (L_1)	$\sum_i \lvert a_i - b_i \rvert$	Grid-like paths; less sensitive to outliers
Minkowski (L_p)	$\left(\sum_i \lvert a_i - b_i \rvert^p\right)^{1/p}$	General family ($p = 1, 2$ above)
Chebyshev (L_∞)	$\max_i \lvert a_i - b_i \rvert$	Largest single difference
Hamming	number of positions where $a_i \neq b_i$	Binary / categorical strings
Cosine similarity	$\dfrac{\boldsymbol{a} \cdot \boldsymbol{b}}{\lVert \boldsymbol{a} \rVert \, \lVert \boldsymbol{b} \rVert}$	Text; direction, not size

Example: Computing distances

Let $\boldsymbol{a} = (1, 2, 3)$ and $\boldsymbol{b} = (4, 6, 3)$. The coordinate differences are $3, 4, 0$.

$$d_{L_2} = \sqrt{3^2 + 4^2 + 0^2} = \sqrt{25} = 5, \qquad d_{L_1} = 3 + 4 + 0 = 7,$$
$$d_{L_\infty} = \max(3, 4, 0) = 4, \qquad d_{L_3} = (27 + 64 + 0)^{1/3} = 91^{1/3} \approx 4.50.$$

Cosine similarity: $\boldsymbol{a} \cdot \boldsymbol{b} = 4 + 12 + 9 = 25$, $\lVert \boldsymbol{a} \rVert = \sqrt{14} \approx 3.742$, $\lVert \boldsymbol{b} \rVert = \sqrt{61} \approx 7.810$, so $\cos\theta = 25/29.22 \approx 0.855$.

Hamming: for the bit strings 10110 and 11100 the strings differ in positions 2 and 4, so $d_H = 2$.
Note the ordering $L_\infty \le L_3 \le L_2 \le L_1$ for the same pair of points.

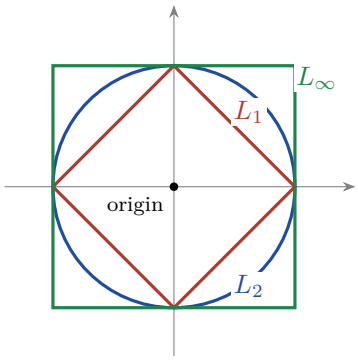


Figure 3.1. The set of all points at distance exactly 1 from the origin, for three metrics.

Reading the Figure

Each coloured outline is the set of points that are *exactly one unit away* from the origin under a different distance. Under **Euclidean** distance (L_2, blue) this set is a *circle*. Under **Manhattan** (L_1, red) it is a *diamond*, because you may only move along the axes. Under **Chebyshev** (L_∞, green) it is a *square*. The point $(1, 1)$ lies on the green square but far outside the red diamond: the *same pair of points* is counted as "close" or "far" depending on the metric. Hence the choice of metric changes which points are neighbours.

3.1.2 Feature Scaling

Distance measures add up differences across all features, so a feature with a *large numeric range* dominates the sum. We must put features on a comparable scale.

Method	Formula	Result
Min–max scaling	$x' = \dfrac{x - x_{\min}}{x_{\max} - x_{\min}}$	Values in $[0, 1]$
Standardisation (z-score)	$x' = \dfrac{x - \mu}{\sigma}$	Mean 0, standard deviation 1

Example: Why scaling matters

Three people: P = (age 25, salary 40000), Q = (45, 42000), R = (26, 80000). Raw Euclidean distances:

$$d(P, Q) = \sqrt{20^2 + 2000^2} \approx 2000.1, \qquad d(P, R) = \sqrt{1^2 + 40000^2} \approx 40000.0.$$

The salary difference swamps age, so Q (20 years older) looks far closer to P than R (1 year older) does, which is silly. Now min–max scale with age range 20–60 and salary range 20 000–100 000:

$$P' = (0.125,\ 0.250), \quad Q' = (0.625,\ 0.275), \quad R' = (0.150,\ 0.750).$$

$d(P', Q') = \sqrt{0.5^2 + 0.025^2} \approx 0.5006$ and $d(P', R') = \sqrt{0.025^2 + 0.5^2} \approx 0.5006$: after scaling, age and salary contribute fairly.

Common Mistake

Always compute the scaling parameters $(x_{\min}, x_{\max}, \mu, \sigma)$ on the **training set only** and then apply the *same* numbers to the test set. Using the test data to compute them leaks information.

Distance-based methods

Advantages

- Intuitive and easy to implement
- Little or no training
- Works for classification, regression and clustering

Disadvantages

- Very sensitive to feature scale and irrelevant features
- Slow for large datasets
- Loses meaning in high dimensions (curse of dimensionality)

Applications

- k-NN and K-Means clustering
- Recommender systems
- Anomaly detection

3.2 Nearest Neighbours (k-NN)

3.2.1 The Algorithm

k-NN is the simplest learning algorithm: it has **no training phase**. It just stores the training data (a *lazy learner*) and does all the work at prediction time.

Definition: k-Nearest Neighbours

To predict for a new point $\boldsymbol{x}_q$:

1. Compute the distance from $\boldsymbol{x}_q$ to every training point.
2. Pick the k training points with the smallest distances (the *neighbourhood* $N_k(\boldsymbol{x}_q)$).
3. **Classification:** output the *majority class* among the k neighbours.
   Regression: output the *mean* of their target values.

Example: k-NN classification by hand

Training data (two features, two classes) and the query $\boldsymbol{x}_q = (5, 4)$:

Point	x_1	x_2	Class	Distance to $(5,4)$	Rank
P_4	6	5	B	$\sqrt{1+1} = 1.414$	1
P_5	7	4	B	$\sqrt{4+0} = 2.000$	2–3
P_6	5	6	B	$\sqrt{0+4} = 2.000$	2–3
P_7	4	2	A	$\sqrt{1+4} = 2.236$	4
P_3	3	2	A	$\sqrt{4+4} = 2.828$	5
P_2	2	3	A	$\sqrt{9+1} = 3.162$	6
P_1	1	1	A	$\sqrt{16+9} = 5.000$	7

k	Neighbours	Prediction
1	P_4 (B)	**B**
3	P_4, P_5, P_6 = B,B,B	**B**
5	B,B,B,A,A (3 vs 2)	**B**
7	all points: 4 A vs 3 B	**A**

The prediction *changes* from B to A when k grows to 7, which shows how sensitive k-NN is to k.

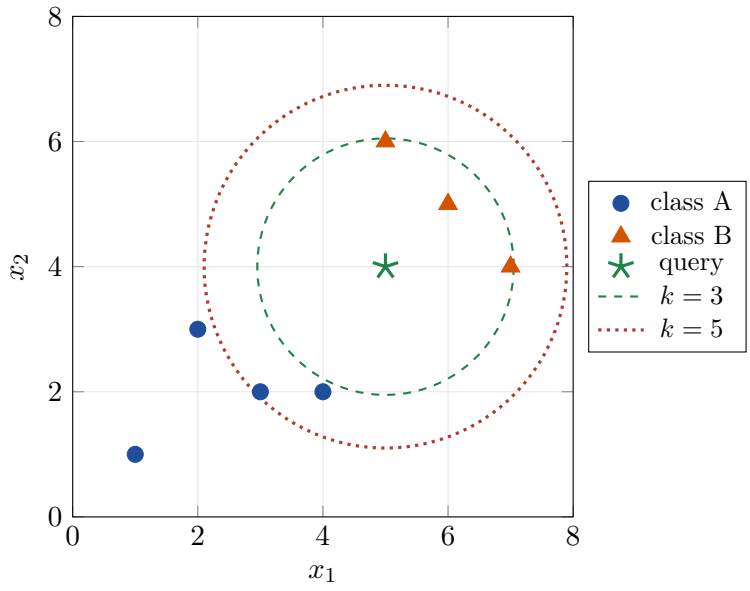


Figure 3.2. Neighbourhoods of the query point for $k = 3$ and $k = 5$.

Reading the Figure

The **green star** is the query point. Blue circles are class A and orange triangles are class B. The **dashed green circle** is just big enough to contain the 3 nearest points: all three are orange, so $k = 3$ votes B. The **dotted red circle** is large enough to contain 5 points: three orange and two blue, so $k = 5$ still votes B, but by a smaller margin. If we kept enlarging the circle to include all 7 points the blue majority would win. The *radius adapts* to the local density of data; this is why k-NN is called *local*.

3.2.2 Choosing k

	Small k (e.g. 1)	**Large k**
Decision boundary	Very jagged, follows every point	Smooth
Bias	Low	High
Variance	High (sensitive to noise)	Low
Risk	Overfitting	Underfitting

k is a **hyperparameter**, chosen by cross-validation (Chapter 2). Use an *odd* k for two-class problems to avoid ties. A common starting point is $k \approx \sqrt{n}$.

Key Idea

$1/k$ is a measure of model flexibility: $k = 1$ is the *most flexible* (and most variable) k-NN; $k = n$ always predicts the overall majority class (the least flexible). This is the bias–variance trade-off of Chapter 2 in action.

3.2.3 Weighted k-NN and k-NN Regression

In **weighted k-NN**, closer neighbours get more influence, with a weight $w_i = 1/d_i$ (or $1/d_i^2$).

Example: k-NN regression

Use the house data of Chapter 1: size $x = 1, 2, 3, 4, 5$ with price $y = 15, 26, 29, 44, 46$. Predict the price at $x_q = 3.4$ with $k = 2$. The two nearest sizes are $x = 3$ (distance 0.4, price 29) and $x = 4$ (distance 0.6, price 44).

- **Plain average:** $\hat{y} = (29 + 44)/2 = 36.5$.
- **Weighted ($w = 1/d$):** $w_1 = 2.5$, $w_2 = 1.667$; $\hat{y} = \dfrac{2.5(29) + 1.667(44)}{2.5 + 1.667} = \dfrac{145.83}{4.167} = 35.0$.

The weighted answer is pulled towards the closer house (price 29). For comparison, the linear model $8 + 8x$ from Chapter 1 predicts 35.2.

3.2.4 Strengths and Weaknesses

Advantages	Disadvantages
Very simple; no training	Slow prediction: must scan all n points ($O(nd)$ per query)
Naturally handles multiple classes	Needs a lot of memory (stores everything)
No assumption about the data shape	Sensitive to feature scale and irrelevant features
Improves as more data arrives	Suffers from the *curse of dimensionality*

Curse of dimensionality. In high dimensions, points become almost equally far apart, so the idea of a "nearest" neighbour loses meaning and k-NN degrades. Remedies: feature selection and dimensionality reduction such as PCA (Chapter 8).

k-Nearest neighbours (k-NN)

Advantages

- Simple; no training; handles many classes
- No assumption about the data distribution
- Improves as more data is added

Disadvantages

- Slow prediction and large memory
- Sensitive to k, scaling and irrelevant features
- Poor in high dimensions

Applications

- Recommendation systems
- Handwriting and pattern recognition
- Credit rating and anomaly detection

3.3 Decision Trees

3.3.1 Idea and Structure

A decision tree predicts by asking a sequence of simple questions about the features, like a game of "twenty questions".

Definition: Decision tree

A **decision tree** is a flowchart-like model in which each **internal node** tests one feature, each **branch** is an outcome of the test, and each **leaf** holds a prediction (a class label for *classification trees*, or a number for *regression trees*). The top node is the **root**.

The learning problem is: *which feature should we test at each node?* We choose the test that makes the resulting groups as **pure** as possible, i.e. each group dominated by one class.

3.3.2 Measures of Impurity

Let a node contain examples of C classes with proportions $p_1, \ldots, p_C$.

Definition: Entropy

$$H = -\sum_{i=1}^{C} p_i \log_2 p_i \qquad (0 \log 0 := 0).$$

Entropy is 0 when the node is pure and is maximal ($\log_2 C$) when all classes are equally likely. It is measured in *bits*.

Definition: Gini index

$$G = 1 - \sum_{i=1}^{C} p_i^2.$$

Gini is 0 for a pure node and equals 0.5 for a 50/50 two-class node. It is the probability that two items drawn at random from the node have different classes.

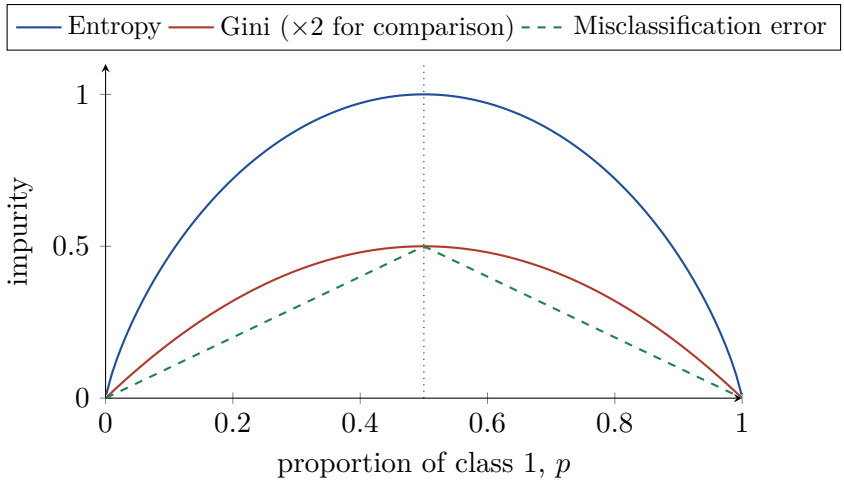


Figure 3.3. Impurity measures for a two-class node as a function of p.

Reading the Figure

The horizontal axis is the **proportion of class 1** in a node: $p = 0$ or $p = 1$ means a *pure* node and $p = 0.5$ is the most mixed. All three curves are **zero at the ends** (pure) and **highest at the middle** (maximum confusion). **Entropy** (blue) is slightly more curved than **Gini** (red; drawn here as $2p(1-p)$ so that it peaks at 1 like entropy). In practice they almost always choose the same splits. The dashed green *misclassification error* has a sharp

corner and is less sensitive to changes, which is why it is rarely used for growing trees.

Definition: Information gain

If a split on feature A divides a set S into subsets S_v (one for each value v of A), then

$$\text{Gain}(S, A) = H(S) - \sum_v \frac{|S_v|}{|S|} H(S_v).$$

It is the *reduction in entropy* achieved by the split. At each node the tree picks the feature with the **highest information gain**. (The ID3 algorithm does exactly this; C4.5 uses the *gain ratio* to avoid favouring features with many values; CART uses the Gini index and binary splits.)

3.3.3 Worked Example: Building a Tree with ID3

We want to decide whether to **play tennis** from the weather. There are 14 past days.

Day	Outlook	Temp.	Humidity	Wind	Play
D1	Sunny	Hot	High	Weak	No
D2	Sunny	Hot	High	Strong	No
D3	Overcast	Hot	High	Weak	Yes
D4	Rain	Mild	High	Weak	Yes
D5	Rain	Cool	Normal	Weak	Yes
D6	Rain	Cool	Normal	Strong	No
D7	Overcast	Cool	Normal	Strong	Yes
D8	Sunny	Mild	High	Weak	No
D9	Sunny	Cool	Normal	Weak	Yes
D10	Rain	Mild	Normal	Weak	Yes
D11	Sunny	Mild	Normal	Strong	Yes
D12	Overcast	Mild	High	Strong	Yes
D13	Overcast	Hot	Normal	Weak	Yes
D14	Rain	Mild	High	Strong	No

Step 1: entropy of the whole set. There are 9 Yes and 5 No:

$$H(S) = -\tfrac{9}{14}\log_2\tfrac{9}{14} - \tfrac{5}{14}\log_2\tfrac{5}{14} = 0.410 + 0.530 = \mathbf{0.940}.$$

Step 2: information gain of every feature.

Split	Values (Yes, No)	Entropies	Gain
Outlook	Sunny (2,3), Overcast (4,0), Rain (3,2)	0.971, 0, 0.971	**0.247**
Humidity	High (3,4), Normal (6,1)	0.985, 0.592	0.152
Wind	Weak (6,2), Strong (3,3)	0.811, 1.000	0.048
Temperature	Hot (2,2), Mild (4,2), Cool (3,1)	1.000, 0.918, 0.811	0.029

Detail for Outlook:

$$\text{Gain} = 0.940 - \tfrac{5}{14}(0.971) - \tfrac{4}{14}(0) - \tfrac{5}{14}(0.971) = 0.940 - 0.694 = 0.247.$$

Outlook has the highest gain, so it becomes the root.

Step 3: recurse on each branch.

- **Overcast:** all 4 days are Yes ⇒ pure ⇒ leaf *Yes*.
- **Sunny** (D1, D2, D8, D9, D11; 2 Yes, 3 No): splitting on Humidity gives High → {D1, D2, D8} all No, Normal → {D9, D11} all Yes. Both pure, so Humidity is chosen.
- **Rain** (D4, D5, D6, D10, D14; 3 Yes, 2 No): splitting on Wind gives Weak → {D4, D5, D10} all Yes, Strong → {D6, D14} all No.

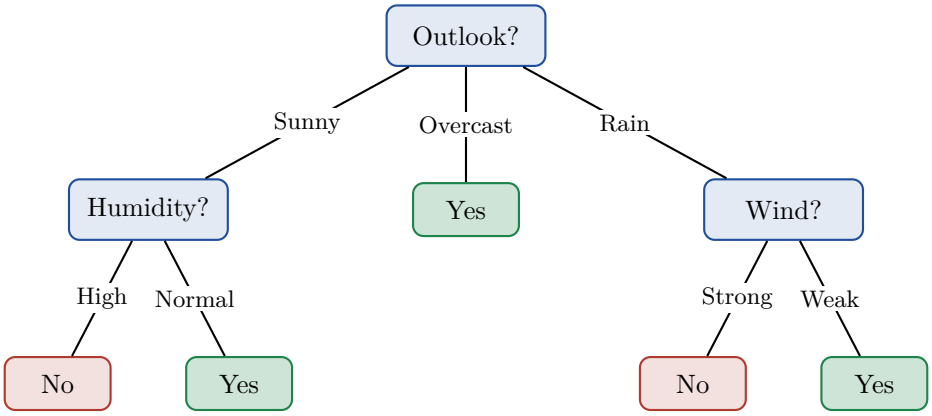


Figure 3.4. The decision tree learned by ID3 from the Play-Tennis data.

Reading the Figure

Blue boxes are decision nodes (questions), **green** and **red** boxes are leaves (final answers: Yes/No). To classify a new day, start at the *root* and follow the branch matching the day's value at each question until you reach a leaf. For example, a day that is *Rain* and *Weak wind*: Outlook = Rain → Wind? → Weak → **Yes**. The tree also reads as rules: *IF Outlook=Overcast THEN Play=Yes; IF Outlook=Sunny AND Humidity=High THEN No;* ... This is why trees are so easy to *interpret*.

The same split using the Gini index. $G(S) = 1 - (\frac{9}{14})^2 - (\frac{5}{14})^2 = 0.459$. For Outlook, $G(\text{Sunny}) = G(\text{Rain}) = 1 - (0.4)^2 - (0.6)^2 = 0.48$ and $G(\text{Overcast}) = 0$, so the weighted Gini is $\frac{5}{14}(0.48) + \frac{4}{14}(0) + \frac{5}{14}(0.48) = 0.343$ and the Gini gain is $0.459 - 0.343 = 0.116$, again the best feature.

3.3.4 Algorithm Summary

1. Start with all training data at the root.
2. If the node is pure (or a stopping rule applies), make it a leaf labelled with the majority class.
3. Otherwise, evaluate every candidate split; choose the one with the highest gain (or largest drop in Gini).
4. Split the data and **recurse** on each child.

Continuous features. Sort the values, try thresholds halfway between consecutive values (e.g. "Age ≤ 30?"), and choose the threshold with the best gain.

Regression trees. Leaves store the *mean* target of their training points; the best split minimises the sum of squared errors (variance) in the two children.

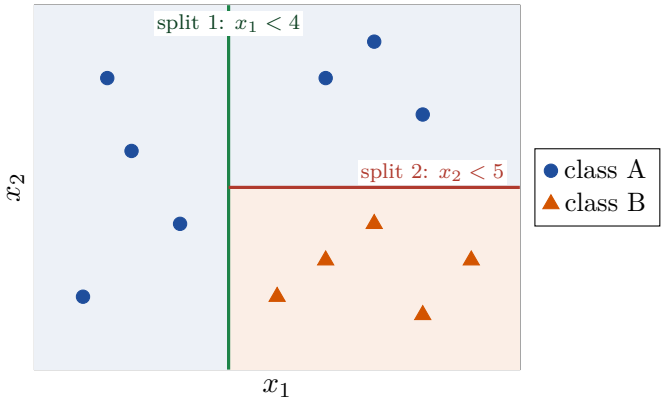


Figure 3.5. A decision tree with two splits carves the feature space into axis-parallel rectangles.

Reading the Figure

Every split in a tree tests *one feature against a threshold*, which draws a **straight line parallel to an axis**. The **green vertical line** is the root test $x_1 < 4$: everything to its left is class A. On the right, the **red horizontal line** tests $x_2 < 5$: below it is class B (orange), above it class A. The final result is three *rectangular regions*, each assigned one class. Trees therefore produce *boxy* decision boundaries, which suit data with axis-aligned structure but approximate diagonal boundaries only with many small steps.

3.3.5 Overfitting and Pruning

A tree allowed to grow until every leaf is pure will memorise the training data, noise included (low bias, **high variance**; see Chapter 2). We control complexity by **pruning**.

Pre-pruning (early stopping)	Stop growing early: limit the depth (`max_depth`), require a minimum number of samples to split (`min_samples_split`) or in a leaf (`min_samples_leaf`), or require a minimum gain.
Post-pruning	Grow the full tree, then *cut back* branches that do not improve accuracy on a validation set (reduced-error pruning) or minimise cost $=$ error $+ \alpha \cdot$ (number of leaves) (cost-complexity pruning).

3.3.6 Strengths and Weaknesses

Advantages	**Disadvantages**
Easy to understand and visualise; human-readable rules	Prone to overfitting
Handles numeric and categorical features	Unstable: small data changes can give a very different tree
No feature scaling needed	Axis-parallel (boxy) boundaries only
Fast prediction; performs feature selection implicitly	Biased towards features with many values

The instability of single trees is fixed by combining many of them into *ensembles* such as Random Forests (Chapter 5).

Decision trees

Advantages

- Easy to understand, visualise and convert to rules
- Handles numeric and categorical features; no scaling needed
- Fast prediction

Disadvantages

- Prone to overfitting; needs pruning
- Unstable: small data changes can change the tree
- Only axis-parallel (boxy) boundaries

Applications

- Credit approval and medical diagnosis
- Customer churn prediction
- Rule extraction; base learner in Random Forests

3.4 Naive Bayes

3.4.1 Bayes' Theorem

Definition: Bayes' theorem

For a class C and observed evidence $\boldsymbol{x}$,

$$P(C \mid \boldsymbol{x}) = \frac{P(\boldsymbol{x} \mid C)\, P(C)}{P(\boldsymbol{x})}, \qquad P(\boldsymbol{x}) = \sum_{c} P(\boldsymbol{x} \mid c) P(c).$$

Here $P(C \mid \boldsymbol{x})$ is the **posterior**, $P(\boldsymbol{x} \mid C)$ the **likelihood**, $P(C)$ the **prior** and $P(\boldsymbol{x})$ the **evidence**.

It follows directly from the definition of conditional probability, $P(C \mid \boldsymbol{x})P(\boldsymbol{x}) = P(\boldsymbol{x}, C) = P(\boldsymbol{x} \mid C)P(C)$. In words: *posterior* (what we believe after seeing the evidence) $\propto$ *likelihood* (how well the class explains the evidence) $\times$ *prior* (what we believed before).

Example: Spam and the word "offer"

Suppose 30% of all e-mail is spam, so $P(\text{spam}) = 0.3$ and $P(\text{ham}) = 0.7$. The word "offer" occurs in 60% of spam, $P(\text{offer} \mid \text{spam}) = 0.6$, but only in 10% of ham, $P(\text{offer} \mid \text{ham}) = 0.1$. An e-mail contains "offer". How likely is it to be spam?

$$P(\text{spam} \mid \text{offer}) = \frac{0.6 \times 0.3}{0.6 \times 0.3 + 0.1 \times 0.7} = \frac{0.18}{0.18 + 0.07} = \frac{0.18}{0.25} = \mathbf{0.72}.$$

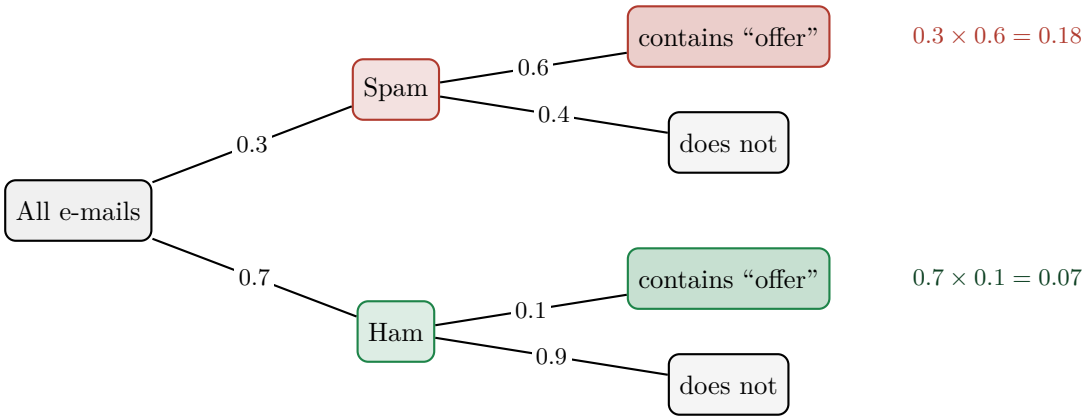


Figure 3.6. Probability tree for the spam example. Only the two "offer" leaves matter once we know the word appeared.

Reading the Figure

Read the tree **left to right**, *multiplying probabilities along a path.* First the e-mail is spam (0.3) or ham (0.7). Then, given its class, it contains "offer" with probability 0.6 (spam) or 0.1 (ham). The **two paths that end in "contains offer"** have joint probabilities 0.18 (spam) and 0.07 (ham). Because we *know* the word appeared, we only look at those two paths, and the posterior is the spam share of their total: $0.18/(0.18 + 0.07) = 0.72$. The

prior of 30% has been **updated** to 72% by the evidence.

3.4.2 The Naive Bayes Classifier

Definition: Naive Bayes classifier

Naive Bayes is a *supervised, probabilistic* classifier based on **Bayes' theorem** with the "naive" assumption that features are *conditionally independent* given the class: $\hat{y} = \arg\max_c P(c) \prod_j P(x_j \mid c)$.

With many features $\boldsymbol{x} = (x_1, \ldots, x_d)$, the likelihood $P(x_1, \ldots, x_d \mid C)$ would need an enormous number of probabilities. Naive Bayes makes one simplifying assumption.

Definition: Naive (conditional independence) assumption

Given the class, the features are **independent**:

$$P(x_1, \ldots, x_d \mid C) = \prod_{j=1}^{d} P(x_j \mid C).$$

The classification rule becomes (the denominator $P(\boldsymbol{x})$ is the same for every class, so it can be dropped):

$$\hat{y} = \arg\max_c \; P(c) \prod_{j=1}^{d} P(x_j \mid c). \tag{3.1}$$

The assumption is usually *false* (in e-mail, "free" and "offer" tend to appear together), hence "naive", yet the classifier works surprisingly well, because we only need the class with the *largest* score, not exact probabilities.

Training = counting. Estimate $P(c) = N_c/N$ and $P(x_j = v \mid c) = \dfrac{\text{count}(x_j = v,\ \text{class } c)}{N_c}$ from the training data.

3.4.3 Worked Example: Play-Tennis Again

Classify the new day $\boldsymbol{x} = (\text{Sunny},\ \text{Cool},\ \text{High},\ \text{Strong})$ using the 14-day table.

Priors: $P(\text{Yes}) = \frac{9}{14}$, $P(\text{No}) = \frac{5}{14}$.

Likelihood	Given Yes (9 days)	Given No (5 days)
$P(\text{Sunny} \mid \cdot)$	2/9	3/5
$P(\text{Cool} \mid \cdot)$	3/9	1/5
$P(\text{High} \mid \cdot)$	3/9	4/5
$P(\text{Strong} \mid \cdot)$	3/9	3/5

$$\text{score}(\text{Yes}) = \tfrac{9}{14} \cdot \tfrac{2}{9} \cdot \tfrac{3}{9} \cdot \tfrac{3}{9} \cdot \tfrac{3}{9} = 0.00529, \qquad \text{score}(\text{No}) = \tfrac{5}{14} \cdot \tfrac{3}{5} \cdot \tfrac{1}{5} \cdot \tfrac{4}{5} \cdot \tfrac{3}{5} = 0.02057.$$

Decision: $0.02057 > 0.00529 \Rightarrow$ predict **No**.

Posterior probabilities (normalise): $P(\text{No} \mid \boldsymbol{x}) = \dfrac{0.02057}{0.02057 + 0.00529} = 0.795$ and $P(\text{Yes} \mid \boldsymbol{x}) = 0.205$.

3.4.4 Laplace Smoothing

Common Mistake

The zero-frequency problem. If a value never occurs with a class in training (e.g. "Overcast" never occurs with "No"), then $P(\text{Overcast} \mid \text{No}) = 0$ and the *whole product becomes zero*, whatever the other features say.

Laplace (add-one) smoothing adds a pseudo-count α (usually 1) to every count:

$$P(x_j = v \mid c) = \frac{\text{count}(x_j = v, c) + \alpha}{N_c + \alpha V_j},$$

where V_j is the number of different values of feature j.

Example: Smoothing

$\text{count}(\text{Overcast}, \text{No}) = 0$, $N_{\text{No}} = 5$, and Outlook has $V = 3$ values. With $\alpha = 1$: $P(\text{Overcast} \mid \text{No}) = \dfrac{0+1}{5+3} = \dfrac{1}{8} = 0.125$ instead of 0.

3.4.5 Variants of Naive Bayes

Variant	Feature type	Model for $P(x_j \mid c)$
Gaussian	Continuous	Normal density with class-wise mean μ_{jc} and variance σ^2_{jc}
Multinomial	Counts (word counts)	Relative frequency of word in class (with smoothing)
Bernoulli	Binary (word present/absent)	Probability of presence/absence

For the **Gaussian** variant,

$$P(x_j \mid c) = \frac{1}{\sqrt{2\pi}\,\sigma_{jc}} \exp\left(-\frac{(x_j - \mu_{jc})^2}{2\sigma_{jc}^2}\right).$$

Example: Gaussian Naive Bayes with one feature

Heights: males have $\mu = 175$, $\sigma = 7$ cm; females have $\mu = 162$, $\sigma = 6$ cm; priors are equal. Classify a person of height 170 cm.

$$P(170 \mid \text{male}) = \tfrac{1}{7\sqrt{2\pi}} e^{-25/98} = 0.0570 \times 0.775 = 0.0442,$$
$$P(170 \mid \text{female}) = \tfrac{1}{6\sqrt{2\pi}} e^{-64/72} = 0.0665 \times 0.411 = 0.0273.$$

Since the priors are equal, the larger likelihood wins: **male**, with posterior $0.0442/(0.0442 + 0.0273) \approx 0.62$. (Only 62% confident because 170 is plausible for both.)

Key Idea

Numerical tip. Multiplying many small probabilities underflows to 0. Real implementations add **logarithms** instead: $\hat{y} = \arg\max_c \left[\ln P(c) + \sum_j \ln P(x_j \mid c)\right]$.

Advantages	Disadvantages
Very fast to train and predict (just counting)	Independence assumption rarely true
Works well with little data and many features	Probability estimates can be poorly calibrated
Excellent baseline for text (spam, sentiment)	Zero-frequency problem (needs smoothing)
Handles multiple classes naturally	Cannot learn interactions between features

Naive Bayes

Advantages

- Very fast to train and predict
- Works well with little data and many features
- Excellent baseline for text classification

Disadvantages

- Independence assumption is rarely true
- Probability estimates can be poorly calibrated
- Zero-frequency problem (needs smoothing)

Applications

- Spam filtering
- Sentiment analysis
- Document and news classification
- Medical diagnosis

3.5 Putting It Together: The Three Methods in Python

Try It Yourself

Compare all three on the classic *Iris* flower dataset (150 flowers, 4 measurements, 3 species).

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split,
    cross_val_score
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline
from sklearn.neighbors import KNeighborsClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.naive_bayes import GaussianNB

X, y = load_iris(return_X_y=True)
Xtr, Xte, ytr, yte = train_test_split(
    X, y, test_size=0.3, random_state=42, stratify=y)

models = {
  "k-NN (k=5)": make_pipeline(StandardScaler(),
                              KNeighborsClassifier(n_neighbors=5)
                                 ),
  "Decision tree": DecisionTreeClassifier(max_depth=3,
     random_state=0),
  "Naive Bayes": GaussianNB(),
}
for name, m in models.items():
    m.fit(Xtr, ytr)
    cv = cross_val_score(m, X, y, cv=5).mean()
    print(f"{name:14s} test = {m.score(Xte, yte):.3f}  CV = {cv
       :.3f}")
```

Output:

Model	Test accuracy	5-fold CV accuracy
k-NN ($k = 5$)	0.911	0.960
Decision tree (depth 3)	0.933	0.960
Naive Bayes	0.911	0.953

All three do well here. The single test split (45 flowers) is noisy, so the *cross-validated* numbers are more reliable and show the methods are practically tied.

Effect of k (same data, scaled):

k	Train acc.	Test acc.	Comment
1	1.000	0.933	memorises training data
3	0.971	0.911	
5	0.981	0.911	
15	0.981	0.933	smooth boundary
51	0.876	0.822	**underfits**: both fall

A huge k averages over too much of the data and accuracy drops on *both* sets: underfitting, exactly as predicted.

A tiny tree you can read (`max_depth=2`, printed with `export_text`):

```
|--- petal_wid <= 0.70            -> class 0 (setosa)
|--- petal_wid >  0.70
|   |--- petal_wid <= 1.55        -> class 1 (versicolor)
|   |--- petal_wid >  1.55        -> class 2 (virginica)
```

Just *two questions about petal width* separate the three species.

3.6 Comparison of the Three Methods

	k-NN	**Decision tree**	**Naive Bayes**
Learner type	Lazy, instance-based	Eager, model-based	Eager, model-based
Training cost	None	Moderate	Very low
Prediction cost	High	Very low	Very low
Assumption	Similar points share classes	Axis-aligned splits	Features independent given class
Needs scaling?	**Yes**	No	No (Gaussian: no)
Interpretable?	Low	**High**	Moderate
Main hyperparameter	k, metric	depth, min samples	smoothing α
Typical weakness	Slow; high dimensions	Overfits; unstable	Independence assumption

Chapter Summary

- Distance measures (L_1, L_2, L_p, L_∞, Hamming, cosine) define similarity; **scale features** first (min–max or z-score, fitted on training data only).
- **k-NN** stores data and predicts by majority vote (classification) or mean (regression) of

the k closest points. Small $k \Rightarrow$ high variance; large $k \Rightarrow$ high bias.

- **Decision trees** split on the feature with highest **information gain** (entropy) or largest Gini reduction. Full trees overfit; control with **pruning**.
- Entropy $H = -\sum p_i \log_2 p_i$; Gini $= 1 - \sum p_i^2$; Gain $= H(S) - \sum \frac{|S_v|}{|S|} H(S_v)$.
- **Naive Bayes**: $\hat{y} = \arg\max_c P(c) \prod_j P(x_j \mid c)$, assuming conditional independence; use **Laplace smoothing** and log-probabilities; variants: Gaussian, multinomial, Bernoulli.
- In the Play-Tennis example the tree root is Outlook (gain 0.247) and Naive Bayes predicts "No" with posterior 0.795 for (Sunny, Cool, High, Strong).

Exam Corner

A. Very Short Answer Questions (2 marks each)

1. Define Euclidean and Manhattan distance. **[2 marks]**
2. Why is feature scaling necessary for k-NN? **[2 marks]**
3. Why is k-NN called a lazy learner? **[2 marks]**
4. What happens when $k = 1$ and when $k = n$ in k-NN? **[2 marks]**
5. Define entropy and Gini index. **[2 marks]**
6. What is information gain? **[2 marks]**
7. What is pruning? Name its two types. **[2 marks]**
8. State Bayes' theorem. **[2 marks]**
9. Why is Naive Bayes called "naive"? **[2 marks]**
10. What is Laplace smoothing and why is it needed? **[2 marks]**

Model Answer: Q4 and Q10

Q4. For $k = 1$ the prediction is the label of the single nearest point: the boundary is very jagged, bias is low, variance is high (overfitting). For $k = n$ every query gets the overall majority class: the model is a constant, bias is high and variance is zero (underfitting).
Q10. Laplace smoothing adds a pseudo-count (usually 1) to every feature-value count so that $P(x_j \mid c) = \frac{\text{count}+1}{N_c + V_j}$. It is needed to avoid zero probabilities for unseen feature–class combinations, which would force the whole product $\prod P(x_j \mid c)$ to zero.

B. Short Answer Questions (5 marks each)

1. Explain the k-NN algorithm for classification with an example. **[5 marks]**
2. Discuss the effect of the value of k on k-NN (bias, variance, over/underfitting). **[5 marks]**
3. List the advantages and disadvantages of k-NN. What is the curse of dimensionality? **[5 marks]**
4. Explain the structure of a decision tree. Differentiate entropy and Gini index. **[5 marks]**
5. Explain overfitting in decision trees. How can it be reduced? **[5 marks]**
6. Explain the Naive Bayes classifier and its independence assumption. **[5 marks]**
7. Compare k-NN, decision trees and Naive Bayes in a table. **[5 marks]**

Model Answer: B4 (outline for 5 marks)

Structure: root node (first test), internal nodes (feature tests), branches (outcomes), leaves (class label or value); classification of a new point follows the path from root to a leaf. *Entropy* $H = -\sum p_i \log_2 p_i$: measures disorder of class labels; 0 for a pure node, 1 for a 50/50 binary node. *Gini* $= 1 - \sum p_i^2$: probability of mis-classifying a random item; 0 for pure, 0.5 for 50/50.

	Entropy	**Gini**
Formula	$-\sum p_i \log_2 p_i$	$1 - \sum p_i^2$
Range (2 classes)	0 to 1	0 to 0.5
Cost	Needs logarithm	Cheaper (no log)
Used in	ID3, C4.5	CART

C. Long Answer Questions (10 marks each)

1. Explain the ID3 algorithm. Using the 14-day Play-Tennis table, compute the information gain of *Outlook* and show that it is chosen as the root. **[10 marks]**
2. Explain the Naive Bayes classifier with Laplace smoothing. Use the Play-Tennis data to classify (Sunny, Cool, High, Strong). **[10 marks]**
3. Describe the k-NN algorithm for classification and regression. Illustrate with a numerical example and discuss the choice of k and distance metric. **[10 marks]**
4. Explain distance measures used in machine learning with examples. Why must features be scaled? **[10 marks]**
5. Discuss pre-pruning and post-pruning in decision trees. How do trees compare to k-NN and Naive Bayes in terms of interpretability and cost? **[10 marks]**

Model Answer: C1 (key steps)

1. $H(S) = 0.940$ (9 Yes, 5 No). 2. Outlook: Sunny (2Y,3N) $H = 0.971$; Overcast (4Y,0N) $H = 0$; Rain (3Y,2N) $H = 0.971$. 3. Gain $= 0.940 - \frac{5}{14}(0.971) - \frac{4}{14}(0) - \frac{5}{14}(0.971) = 0.247$. 4. Gains of the others: Humidity 0.152, Wind 0.048, Temperature 0.029. Outlook is largest, so it is the root. 5. Overcast is pure (Yes); Sunny splits on Humidity; Rain splits on Wind. Draw the tree of Figure 3.4.

D. Numerical Problems

1. Find the Euclidean and Manhattan distance between $(2, 3)$ and $(6, 6)$. *(Answer: 5 and 7.)*
2. Training points: $(1, 2)$ A, $(2, 1)$ A, $(3, 3)$ A, $(4, 5)$ B, $(5, 4)$ B. Classify the query $(4, 4)$ using $k = 3$ and $k = 5$ (Euclidean). *(Distances: $1, 1, 1.414, 3.606, 3.606$. Answer: $k = 3$: B,B,A $\rightarrow$ **B**; $k = 5$: 2B vs 3A $\rightarrow$ **A**.)*
3. Distances to a query are 1 and 2 with target values 10 and 20. Give the weighted $(1/d)$ k-NN regression value for $k = 2$. *(Answer: $\frac{1 \cdot 10 + 0.5 \cdot 20}{1.5} = 13.33$.)*
4. Compute the entropy of a node with 6 positive and 2 negative examples. *(Answer: $-0.75 \log_2 0.75 - 0.25 \log_2 0.25 = 0.811$.)*
5. A node with 4 positives and 4 negatives is split into $(3+, 1-)$ and $(1+, 3-)$. Compute the information gain. *(Answer: $1 - \frac{4}{8}(0.811) - \frac{4}{8}(0.811) = 0.189$.)*
6. Compute the Gini index of a node with 3 "yes" and 1 "no". *(Answer: $1 - 0.5625 - 0.0625 = 0.375$.)*
7. $P(\text{spam}) = 0.4$, $P(\text{"free"} \mid \text{spam}) = 0.5$, $P(\text{"free"} \mid \text{ham}) = 0.05$. Find $P(\text{spam} \mid \text{"free"})$. *(Answer: $\frac{0.4 \cdot 0.5}{0.4 \cdot 0.5 + 0.6 \cdot 0.05} = \frac{0.20}{0.23} = 0.870$.)*
8. A word never appears in the 100 words of class "ham"; vocabulary size is 50. Find its smoothed probability. *(Answer: $\frac{0+1}{100+50} = 0.0067$.)*

E. Multiple-Choice Questions

1. k-NN does its main computation during:
   (a) training (b) prediction (c) pruning (d) scaling
2. Which distance is the sum of absolute coordinate differences?
   (a) Euclidean (b) Chebyshev (c) Manhattan (d) Cosine
3. The entropy of a pure node is:
   (a) 1 (b) 0.5 (c) 0 (d) ∞
4. A decision tree chooses the split with the:
   (a) lowest gain (b) highest information gain (c) most branches (d) fewest samples
5. Which method needs feature scaling the most?
   (a) Decision tree (b) Naive Bayes (c) k-NN (d) none

6. Overfitting of a decision tree is reduced by:
   (a) growing it deeper (b) pruning (c) removing the root (d) adding duplicate data

7. In Naive Bayes the features are assumed to be:
   (a) identical (b) correlated (c) conditionally independent given the class (d) Gaussian only

8. Laplace smoothing prevents:
   (a) overfitting of trees (b) zero probabilities (c) scaling errors (d) ties in k-NN

9. The Gini index of a 50/50 two-class node is:
   (a) 1 (b) 0.5 (c) 0.25 (d) 0

10. A very large k in k-NN typically causes:
    (a) overfitting (b) underfitting (c) zero error (d) no change

Answers: 1-b, 2-c, 3-c, 4-b, 5-c, 6-b, 7-c, 8-b, 9-b, 10-b.

F. Practice Problems

1. Using the Play-Tennis table, find the best feature to split on inside the *Sunny* branch using information gain, and verify that Humidity gives gain 0.971.

2. Classify (Overcast, Mild, High, Weak) with Naive Bayes using add-one smoothing.

3. **Coding:** using `sklearn`, plot training and test accuracy of k-NN on Iris for $k = 1, 3, \dots, 31$ and identify the best k by 5-fold cross-validation.

4. **Coding:** fit `DecisionTreeClassifier` with `max_depth` from 1 to 10 and plot train vs. test accuracy. At which depth does overfitting begin?

Quick Revision

Topic	One-line recall
Euclidean / Manhattan	$\sqrt{\sum (a_i - b_i)^2}$ / $\sum \lvert a_i - b_i \rvert$.
Scaling	Min–max $\frac{x - x_{\min}}{x_{\max} - x_{\min}}$; z-score $\frac{x - \mu}{\sigma}$; fit on train only.
k-NN	Lazy; majority vote / mean of k closest; small k overfits, large k underfits.
Weighted k-NN	$w_i = 1/d_i$; closer neighbours count more.
Entropy / Gini	$-\sum p \log_2 p$ / $1 - \sum p^2$; both 0 if pure.
Information gain	$H(S) - \sum \frac{\lvert S_v \rvert}{\lvert S \rvert} H(S_v)$; choose the maximum.
Pruning	Pre (max depth, min samples) / post (cut back, cost-complexity).
Tree boundary	Axis-parallel rectangles.
Bayes' theorem	Posterior = likelihood × prior / evidence.
Naive Bayes	$\arg\max_c P(c) \prod_j P(x_j \mid c)$; conditional independence; Laplace $\frac{n+1}{N_c + V}$.
Play-Tennis results	Root Outlook (gain 0.247); NB for (S,C,H,Str): No, $P = 0.795$.

Chapter 4

Linear Models and Support Vector Machines

Learning Objectives

After studying this chapter you will be able to:

- **define** linear regression, logistic regression, generalized linear models and support vector machines, and write each as a standard exam answer;
- fit a regression line by hand, using both the **least-squares formula** and **gradient descent**, and judge it with **MSE** and R^2;
- explain the **sigmoid**, **odds**, **log-odds** and **cross-entropy loss**, and compute logistic-regression probabilities, boundaries and accuracy end to end;
- describe the three components of a **GLM** and work with **Poisson regression**;
- explain **binary classification**, the **maximum-margin hyperplane**, **support vectors** and the **soft margin**, and solve a small SVM by hand;
- compare linear and logistic regression and say when to use each;
- answer 2-, 5- and 10-mark university questions on Unit II.

> *Simple models are not simple-minded: a good linear model often beats a clever complicated one.*
>
> — Leo Breiman

In Chapter 3 the models were built from *distances*, *questions* and *probabilities*. This chapter studies the most widely used family of models in all of statistics and machine learning: **linear models**, which predict from a *weighted sum* of the features.

Definition: Linear model

A **linear model** computes a score

$$z = \boldsymbol{w} \cdot \boldsymbol{x} + b = w_1 x_1 + w_2 x_2 + \cdots + w_d x_d + b,$$

where $\boldsymbol{w}$ are the **weights** (coefficients) and b is the **bias** (intercept). The final prediction is a function of z: z itself (linear regression), a probability $\sigma(z)$ (logistic regression), or the sign of z (SVM).

Model	Prediction from the score z	Output	Real-life task
Linear regression	$\hat{y} = z$	any real number	Marks from study hours
Logistic regression	$\hat{p} = \dfrac{1}{1+e^{-z}}$	probability in $(0,1)$	Will the student pass?
GLM (Poisson)	$\hat{\mu} = e^{z}$	non-negative count	Number of calls per day
SVM	$\hat{y} = \operatorname{sign}(z)$	class -1 or $+1$	Spam or not spam

4.1 Linear Regression

Definition: Linear regression

Linear regression is a *supervised, parametric* learning method for *regression* problems. It models the target y as a **linear function of the input features** plus random error, $y = \theta_0 + \theta_1 x_1 + \cdots + \theta_d x_d + \varepsilon$, and chooses the parameters that minimise the sum of squared prediction errors (**method of least squares**).

Assumptions of linear regression

- Linearity of the relationship
- Independence of errors
- Constant error variance (homoscedasticity)
- Normally distributed errors (for inference)
- No strong multicollinearity among features

4.1.1 Simple Linear Regression

With one feature x the model is a straight line

$$\hat{y} = \theta_0 + \theta_1 x,$$

where θ_1 is the **slope** (change in y per unit change in x) and θ_0 the **intercept** (value of y at $x = 0$). The error of a prediction is the **residual** $e_i = y_i - \hat{y}_i$. The method of **least squares** chooses θ_0, θ_1 to minimise the **sum of squared errors**

$$\text{SSE} = \sum_{i=1}^{n} (y_i - \theta_0 - \theta_1 x_i)^2.$$

Setting the partial derivatives $\partial \mathrm{SSE}/\partial\theta_0$ and $\partial \mathrm{SSE}/\partial\theta_1$ to zero gives the **closed-form solution**

$$\theta_1 = \frac{\sum_i (x_i - \bar{x})(y_i - \bar{y})}{\sum_i (x_i - \bar{x})^2} = \frac{S_{xy}}{S_{xx}}, \qquad \theta_0 = \bar{y} - \theta_1 \bar{x}. \tag{4.1}$$

Real-life example: study hours and exam marks

A teacher records the hours studied x and the marks y (out of 100) of five students.

Student	A	B	C	D	E	Mean
Hours x	1	2	3	4	5	$\bar{x} = 3$
Marks y	35	45	50	65	70	$\bar{y} = 53$

Example: End-to-end solution

Step 1: deviations from the mean.

$x - \bar{x}$	-2	-1	0	1	2	
$y - \bar{y}$	-18	-8	-3	12	17	
$(x-\bar{x})(y-\bar{y})$	36	8	0	12	34	$S_{xy} = 90$
$(x-\bar{x})^2$	4	1	0	1	4	$S_{xx} = 10$

Step 2: parameters. $\theta_1 = \dfrac{90}{10} = 9$ and $\theta_0 = 53 - 9(3) = 26$. **Model:** $\widehat{\text{marks}} = 26 + 9 \times \text{hours}$.

Interpretation: each extra hour of study adds about **9 marks**; a student who studies 0 hours is expected to score 26.

Step 3: predict. For $x = 6$ hours: $\hat{y} = 26 + 9(6) = \mathbf{80}$ marks.

Step 4: check the fit.

x	1	2	3	4	5	
y	35	45	50	65	70	
$\hat{y} = 26 + 9x$	35	44	53	62	71	
Residual $y - \hat{y}$	0	1	-3	3	-1	$\sum e = 0$
e^2	0	1	9	9	1	$\mathrm{SSE} = 20$
$(y-\bar{y})^2$	324	64	9	144	289	$\mathrm{SST} = 830$

$$\mathrm{MSE} = \frac{\mathrm{SSE}}{n} = \frac{20}{5} = 4, \quad \mathrm{RMSE} = 2, \quad R^2 = 1 - \frac{\mathrm{SSE}}{\mathrm{SST}} = 1 - \frac{20}{830} = \mathbf{0.976}.$$

The line explains about **97.6%** of the variation in marks. The residuals add up to 0, a property of least squares with an intercept.

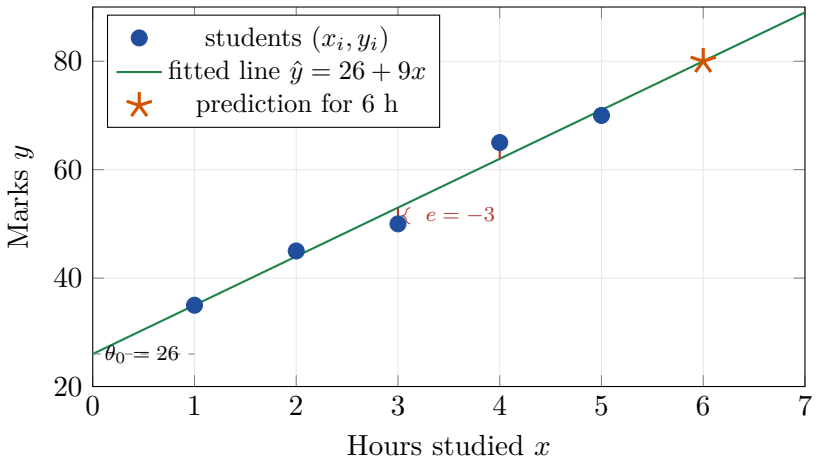


Figure 4.1. Least-squares line for study hours vs. marks. Red segments are residuals.

Reading the Figure

Each **blue dot** is a student. The **green line** is the best straight line, chosen so that the *sum of the squared lengths of the red vertical segments* is as small as possible. A red segment is a **residual**: the gap between the actual marks and the line's prediction. For student C the line predicts 53 but the actual mark is 50, so the residual is -3. The **slope** of the line is 9 (marks per hour), and where it meets the vertical axis it gives the intercept 26. The **orange star** is the prediction for a student who studies 6 hours: we read the height of the line at $x = 6$, giving 80.

4.1.2 Normal Equation (Matrix Form)

For one or many features, stack the inputs into a matrix $\mathbf{X}$ with a leading column of ones (for the intercept) and the targets into a vector $\boldsymbol{y}$. Minimising J gives the **normal equation**

$$\boldsymbol{\theta} = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \boldsymbol{y}. \tag{4.2}$$

Example: Same problem in matrix form

With $\mathbf{X} = \begin{bmatrix} 1 & 1 \\ 1 & 2 \\ 1 & 3 \\ 1 & 4 \\ 1 & 5 \end{bmatrix}$ and $\boldsymbol{y} = (35, 45, 50, 65, 70)^\top$:

$$\mathbf{X}^\top \mathbf{X} = \begin{bmatrix} 5 & 15 \\ 15 & 55 \end{bmatrix}, \quad \mathbf{X}^\top \boldsymbol{y} = \begin{bmatrix} 265 \\ 885 \end{bmatrix}, \quad (\mathbf{X}^\top \mathbf{X})^{-1} = \frac{1}{50} \begin{bmatrix} 55 & -15 \\ -15 & 5 \end{bmatrix}.$$

$$\boldsymbol{\theta} = \frac{1}{50}\begin{bmatrix} 55(265) - 15(885) \\ -15(265) + 5(885) \end{bmatrix} = \frac{1}{50}\begin{bmatrix} 1300 \\ 450 \end{bmatrix} = \begin{bmatrix} 26 \\ 9 \end{bmatrix}.$$

This is the same answer, $\theta_0 = 26$, $\theta_1 = 9$.

The normal equation is exact but requires inverting a $d \times d$ matrix (cost $\approx O(d^3)$), which is slow for thousands of features. For large problems we use an iterative method.

4.1.3 Gradient Descent

Definition: Gradient descent

Gradient descent is an iterative optimisation algorithm that minimises a differentiable loss $J(\boldsymbol{\theta})$ by repeatedly moving the parameters a small step in the direction of *steepest decrease*, i.e. opposite to the gradient:

$$\theta_j \;\leftarrow\; \theta_j - \alpha \frac{\partial J}{\partial \theta_j}, \qquad j = 0, 1, \ldots, d,$$

where $\alpha > 0$ is the **learning rate**. All parameters are updated *simultaneously*.

For linear regression with $J(\boldsymbol{\theta}) = \frac{1}{2n}\sum_i (\hat{y}_i - y_i)^2$:

$$\frac{\partial J}{\partial \theta_0} = \frac{1}{n}\sum_i (\hat{y}_i - y_i), \qquad \frac{\partial J}{\partial \theta_1} = \frac{1}{n}\sum_i (\hat{y}_i - y_i)\, x_i.$$

Example: Gradient descent by hand on the marks data

Start at $\theta_0 = \theta_1 = 0$ and use $\alpha = 0.01$.

Iteration 1. Predictions are all 0, so the errors $\hat{y} - y$ are $-35, -45, -50, -65, -70$.

$$\frac{\partial J}{\partial \theta_0} = \frac{-265}{5} = -53, \qquad \frac{\partial J}{\partial \theta_1} = \frac{-35-90-150-260-350}{5} = \frac{-885}{5} = -177.$$

$$\theta_0 = 0 - 0.01(-53) = 0.53, \qquad \theta_1 = 0 - 0.01(-177) = 1.77.$$

Iteration 2. Predictions $0.53 + 1.77x$ are $2.30, 4.07, 5.84, 7.61, 9.38$; errors $-32.70, -40.93, -44.16, -57.39, -60.62$.

$$\frac{\partial J}{\partial \theta_0} = -47.16, \quad \frac{\partial J}{\partial \theta_1} = -155.94 \;\Rightarrow\; \theta_0 = 1.00,\ \theta_1 = 3.33.$$

Iteration	θ_0	θ_1	Loss J
0	0	0	1487.5
1	0.53	1.77	1166.3
2	1.00	3.33	916.6
3	1.42	4.70	722.4
⋮			
many	$\to 26$	$\to 9$	$\to 2.0$

The loss falls every step and the parameters move towards the exact answer $(26, 9)$ found earlier; with $\alpha = 0.01$ about 20 000 steps are needed to reach it to many decimals (a larger α such as 0.05 is faster, but see the next figure).

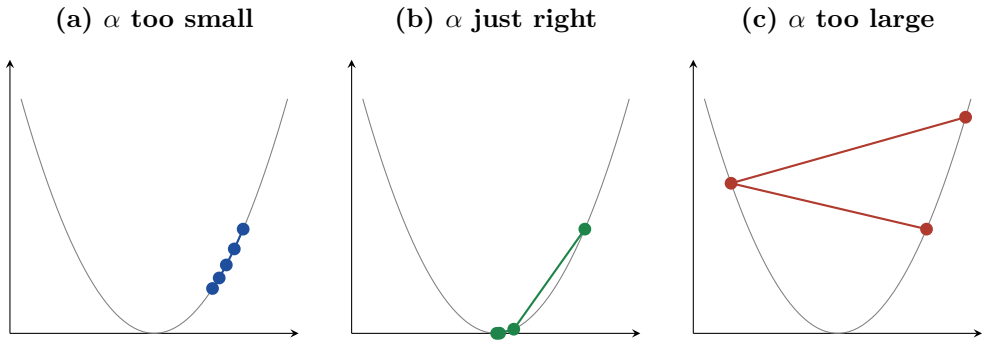


Figure 4.2. Gradient descent on the loss $J(\theta) = \theta^2$ starting from $\theta = 4$ with three learning rates.

Reading the Figure

The grey bowl is a simple loss curve $J(\theta) = \theta^2$; the lowest point (the minimum) is at $\theta = 0$. Each dot is one iteration, joined in order from the start at $\theta = 4$. The update is $\theta \leftarrow \theta(1 - 2\alpha)$.

(a) $\alpha = 0.05$ **(too small):** every step multiplies θ by 0.9: safe, but after five steps we are still far from the bottom. Training is *slow*.

(b) $\alpha = 0.4$ **(good):** each step multiplies by 0.2, and within three steps we are essentially at the minimum.

(c) $\alpha = 1.1$ **(too large):** each step multiplies by -1.2, so we *overshoot* the minimum and land higher on the opposite side each time: the loss *increases* and the algorithm **diverges**.

Lesson: if the loss goes up, reduce the learning rate.

	Normal equation	**Gradient descent**
Learning rate α	Not needed	Must be chosen
Iterations	None (one formula)	Many
Many features (d large)	Slow, $O(d^3)$	Works well
Feature scaling	Not needed	**Needed** for fast convergence
Works for other models	No (linear regression only)	Yes (logistic regression, neural networks)

4.1.4 Multiple Linear Regression

With d features, $\hat{y} = \theta_0 + \theta_1 x_1 + \cdots + \theta_d x_d = \boldsymbol{\theta}^\top \boldsymbol{x}$. Each coefficient θ_j is the change in $\hat{y}$ when x_j increases by one unit *while all other features are held fixed.*

Example: House price

A fitted model for house price (lakh Rs) is

$$\widehat{\text{price}} = 5 + 0.4\,(\text{area in m}^2) + 6\,(\text{bedrooms}) - 0.5\,(\text{age in years}).$$

For a house with 100 m^2, 3 bedrooms and 10 years old: $5 + 0.4(100) + 6(3) - 0.5(10) = 5 + 40 + 18 - 5 = \mathbf{58}$ lakh. Reading the coefficients: one extra bedroom adds 6 lakh and each year of age subtracts 0.5 lakh, other things equal.

Practical points.

- **Categorical features** (e.g. city) are converted to 0/1 *dummy variables* (one-hot encoding).
- **Scale features** before gradient descent so that all weights learn at a similar speed.
- **Multicollinearity:** if two features are strongly correlated (area in m^2 and in ft^2) the coefficients become unstable. Drop one feature or use regularisation.
- **Polynomial regression:** adding $x^2, x^3, \ldots$ as new features gives a *curved* fit, yet the model is still *linear in the parameters*, so all the theory applies. A high degree overfits (Chapter 2).

4.1.5 Evaluating a Regression Model

Metric	Formula	Meaning
MSE	$\frac{1}{n}\sum(y_i - \hat{y}_i)^2$	Average squared error; punishes big errors
RMSE	$\sqrt{\text{MSE}}$	Error in the *units of* y (e.g. marks)
MAE	$\frac{1}{n}\sum\lvert y_i - \hat{y}_i\rvert$	Average absolute error; robust to outliers
R^2	$1 - \dfrac{\text{SSE}}{\text{SST}}$	Fraction of variance explained; 1 is perfect, 0 is no better than predicting $\bar{y}$
Adjusted R^2	$1 - (1 - R^2)\dfrac{n-1}{n-p-1}$	R^2 penalised for the number of features p

For the marks example, $R^2 = 0.976$ and adjusted $R^2 = 1 - 0.0241 \cdot \frac{4}{3} = 0.968$.

Common Mistake

A high R^2 on the *training* data does not guarantee good predictions on new data. Always compute the metrics on a held-out test set. Also remember that **correlation is not causation**: studying more is associated with higher marks, but the regression alone cannot prove it causes them.

4.1.6 Regularised Linear Regression

When there are many features, the least-squares weights can become large and overfit. **Regularisation** adds a penalty on the size of the weights:

$$\text{Ridge (L2):}\;\; J(\boldsymbol{\theta}) + \lambda \sum_{j=1}^{d} \theta_j^2, \qquad \text{Lasso (L1):}\;\; J(\boldsymbol{\theta}) + \lambda \sum_{j=1}^{d} |\theta_j|.$$

- **Ridge** shrinks all weights smoothly towards zero and handles correlated features well.
- **Lasso** can shrink some weights *exactly to zero*, so it performs **automatic feature selection**.
- $\lambda \geq 0$ is a hyperparameter (chosen by cross-validation): $\lambda = 0$ gives ordinary least squares; large λ gives an almost flat model (underfitting).

Try It Yourself

Reproduce the hand results with `scikit-learn`:

```
import numpy as np
```

```python
from sklearn.linear_model import LinearRegression, Ridge

hours = np.array([[1], [2], [3], [4], [5]])
marks = np.array([35, 45, 50, 65, 70])

model = LinearRegression().fit(hours, marks)
print(model.intercept_, model.coef_[0])       # 26.0  9.0
print(model.predict([[6]]))                   # [80.]
print(model.score(hours, marks))              # R^2 = 0.9759

# regularised version: weights shrink a little
print(Ridge(alpha=1.0).fit(hours, marks).coef_)
```

Challenge: add a student with 2 hours and 80 marks (an outlier). How do the slope and R^2 change?

Linear regression

Advantages

- Simple, fast, easy to interpret (each coefficient is the effect of one feature)
- Needs little data
- A good baseline

Disadvantages

- Cannot capture non-linear relationships unless features are transformed
- Sensitive to outliers
- Affected by correlated features

Applications

- House-price prediction
- Sales forecasting
- Estimating marks or salary
- Demand and risk modelling

4.2 Logistic Regression

Definition: Logistic regression

Logistic regression is a *supervised, parametric* method for *classification*. It models the **probability** that an instance belongs to the positive class by passing a linear score through the **sigmoid** function, $P(y = 1 \mid \boldsymbol{x}) = \sigma(\boldsymbol{w} \cdot \boldsymbol{x} + b) = \dfrac{1}{1 + e^{-(\boldsymbol{w} \cdot \boldsymbol{x} + b)}}$, and predicts class 1 if this probability exceeds a threshold (usually 0.5). Despite its name it is a classification algorithm.

Assumptions of logistic regression

- Binary (or categorical) target
- Independent observations
- Log-odds is a linear function of the features
- Little multicollinearity
- Enough data per class

4.2.1 Why Not Use Linear Regression for Classes?

Suppose we code "fail = 0, pass = 1" and fit a straight line. The line gives predictions like -0.3 or 1.4, which are *not valid probabilities*, and one extreme point can tilt the whole line. We need a function that squeezes any real number into $(0, 1)$: the **sigmoid**.

Definition: Sigmoid (logistic) function

$$\sigma(z) = \frac{1}{1 + e^{-z}}, \qquad \sigma(0) = 0.5, \qquad \sigma(-z) = 1 - \sigma(z), \qquad \sigma'(z) = \sigma(z)(1 - \sigma(z)).$$

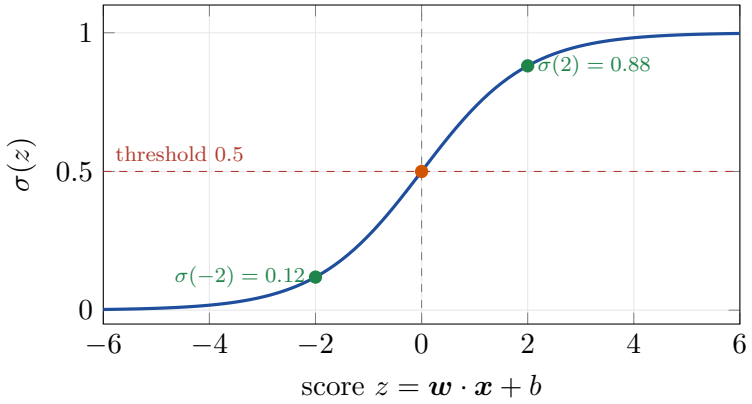


Figure 4.3. The sigmoid function maps any real score to a probability between 0 and 1.

Reading the Figure

The horizontal axis is the **linear score** z (which can be any real number) and the vertical axis is the **probability**. Large positive scores give probabilities near 1 (confident "yes"); large negative scores give probabilities near 0 (confident "no"). At $z = 0$ the curve passes through 0.5, the **decision threshold** (dashed red line): a score above 0 predicts class 1 and a score below 0 predicts class 0. The S-shape is steepest near $z = 0$, where the model is least sure, and flat at the ends, where it is almost certain. The two green dots show the symmetry $\sigma(-2) = 1 - \sigma(2)$.

4.2.2 Odds and Log-odds

Definition: Odds and logit

If an event has probability p, its **odds** are $\dfrac{p}{1-p}$ and its **log-odds (logit)** is $\ln \dfrac{p}{1-p}$.

Logistic regression says that the *log-odds is linear in the features*:

$$\ln \frac{p}{1-p} = \theta_0 + \theta_1 x_1 + \cdots + \theta_d x_d.$$

Hence e^{θ_j} is the **odds ratio**: the factor by which the odds are *multiplied* when x_j increases by one unit. For instance, if $p = 0.8$ the odds are $0.8/0.2 = 4$ ("4 to 1"); if $p = 0.5$ the odds are 1.

4.2.3 Loss Function: Cross-entropy from Maximum Likelihood

For one example with label $y \in \{0, 1\}$ and predicted probability $p = \sigma(z)$, the probability of the observed label is $p^y(1-p)^{1-y}$. Maximising the likelihood of all n independent examples is the same as minimising the **cross-entropy (log) loss**

$$J(\boldsymbol{\theta}) = -\frac{1}{n}\sum_{i=1}^{n}\Big[y_i \ln p_i + (1-y_i)\ln(1-p_i)\Big]. \tag{4.3}$$

Why this loss? If $y = 1$ the loss is $-\ln p$: it is 0 when $p = 1$ and grows to ∞ as $p \to 0$, so *confident wrong answers are punished heavily.* There is no closed-form solution, so we use gradient descent. The gradient has the same neat form as in linear regression:

$$\frac{\partial J}{\partial \theta_j} = \frac{1}{n}\sum_{i=1}^{n}(p_i - y_i)\, x_{ij}.$$

Example: Computing the log-loss

True y	Predicted p	Loss	Comment
1	0.80	$-\ln 0.80 = 0.223$	good
0	0.30	$-\ln 0.70 = 0.357$	fairly good
1	0.10	$-\ln 0.10 = 2.303$	confident and wrong: large loss

Average of the first two: $(0.223 + 0.357)/2 = 0.290$.

4.2.4 Real-life Example: Will the Student Pass?

Twenty students record how many hours they studied and whether they passed. Training logistic regression by gradient descent (equivalently by `scikit-learn`) gives

$$z = -4.08 + 1.50 \times \text{hours}, \qquad P(\text{pass}) = \sigma(z).$$

Example: End-to-end use of the trained model

(a) Probabilities. $P(\text{pass}) = \sigma(-4.08 + 1.50\,h)$:

Hours h	1	2	3	4	5
Score z	-2.58	-1.08	0.42	1.92	3.42
$P(\text{pass}) = \frac{1}{1+e^{-z}}$	0.070	0.254	0.604	0.872	0.968
Prediction (≥ 0.5)	Fail	Fail	Pass	Pass	Pass

For $h = 3$: $e^{-0.42} = 0.657$, so $P = \dfrac{1}{1 + 0.657} = 0.604$.

(b) Decision boundary. $P = 0.5 \iff z = 0 \iff h = \dfrac{4.08}{1.50} = \mathbf{2.72}$ hours: students who study more than about 2.7 hours are predicted to pass.

(c) Meaning of the coefficient. The odds ratio is $e^{1.50} = 4.48$: each extra hour of study multiplies the odds of passing by about **4.5**. (At $h = 2$ the odds are $0.254/0.746 = 0.34$; at $h = 3$ they are $0.604/0.396 = 1.52$; and $1.52/0.34 \approx 4.5$.)

(d) Accuracy. Using the 0.5 rule on the 20 training students: 10 students studied less than 2.72 h (predicted Fail; 8 really failed, 2 passed) and 10 studied more (predicted Pass; 8 passed, 2 failed). So

		Predicted	
		Pass	Fail
Actual	Pass	$TP = 8$	$FN = 2$
	Fail	$FP = 2$	$TN = 8$

Accuracy $= 16/20 = 0.80$, precision $= 8/10 = 0.80$, recall $= 8/10 = 0.80$ (formulas of Chapter 2).

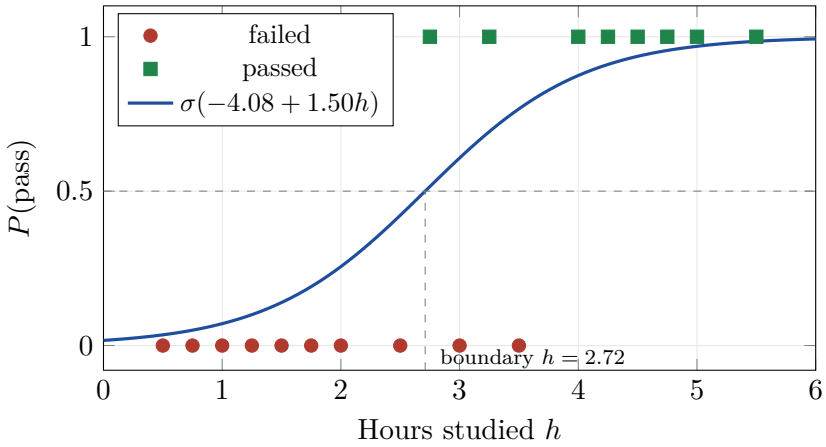


Figure 4.4. Logistic regression fitted to 20 students: the S-curve is the predicted probability of passing.

Reading the Figure

Red dots at height 0 are students who **failed**; green squares at height 1 are students who **passed**. The **blue S-curve** is the model's predicted probability of passing for any number of study hours. It rises slowly at first, climbs quickly around 2–3.5 hours and flattens near 1. Where the curve crosses the horizontal dashed line (probability 0.5), the vertical dashed line marks the **decision boundary** at about 2.7 hours. Note the overlap: some students who studied about 2–3 hours passed and others failed, so the model is naturally *uncertain* there and the curve gives a probability, not a hard yes/no. A straight regression line through these 0/1 points would go below 0 and above 1, which makes no sense for probabilities.

4.2.5 Training by Gradient Descent: One Step by Hand

Example: One gradient step of logistic regression

Five students: $x = (1, 2, 3, 4, 5)$ hours and $y = (0, 0, 1, 0, 1)$. Start with $\theta_0 = \theta_1 = 0$, so $z = 0$ and $p = \sigma(0) = 0.5$ for everyone. Use $\alpha = 0.5$.
Errors $p - y = (0.5,\ 0.5,\ -0.5,\ 0.5,\ -0.5)$.

$$\frac{\partial J}{\partial \theta_0} = \frac{0.5+0.5-0.5+0.5-0.5}{5} = 0.1, \qquad \frac{\partial J}{\partial \theta_1} = \frac{0.5(1)+0.5(2)-0.5(3)+0.5(4)-0.5(5)}{5} = \frac{-0.5}{5} = -0.1.$$

$$\theta_0 = 0 - 0.5(0.1) = -0.05, \qquad \theta_1 = 0 - 0.5(-0.1) = 0.05.$$

The new score is $z = -0.05 + 0.05x$; the 5-hour student now gets $\sigma(0.20) = 0.55$ instead of 0.50. Repeating the step many times converges to the best weights.

4.2.6 Multiclass Logistic Regression

For $K > 2$ classes there are two standard approaches.

- **One-vs-Rest (OvR):** train K binary classifiers ("class k vs. all others") and pick the class with the highest probability.
- **Softmax (multinomial) regression:** compute a score z_k for each class and convert them to probabilities that sum to 1,

$$P(y = k \mid \boldsymbol{x}) = \frac{e^{z_k}}{\sum_{j=1}^{K} e^{z_j}}.$$

Example: Softmax

Scores $z = (2.0,\ 1.0,\ 0.1)$ for three classes: $e^z = (7.389,\ 2.718,\ 1.105)$, sum $= 11.212$, so the probabilities are $(0.659,\ 0.242,\ 0.099)$ and the predicted class is the first.

4.2.7 Choosing the Threshold and Regularisation

- The threshold 0.5 is not sacred. For *cancer screening* a missed patient (false negative) is very costly, so we lower the threshold (say to 0.2) to raise **recall**, accepting more false alarms. For *spam filtering* a good mail sent to spam (false positive) is costly, so we raise it to improve **precision**.
- Logistic regression can be **regularised** just like linear regression (L2 or L1 penalty). In `scikit-learn` the parameter `C` is the *inverse* of regularisation strength: smaller `C` means stronger regularisation.

	Linear regression	**Logistic regression**
Task	Regression (predict a number)	Classification (predict a class / probability)
Output	Any real value	Probability in $(0, 1)$
Function	$\hat{y} = \boldsymbol{\theta}^\top \boldsymbol{x}$	$\hat{p} = \sigma(\boldsymbol{\theta}^\top \boldsymbol{x})$
Loss	Squared error	Cross-entropy (log loss)
Solution	Closed form or gradient descent	Gradient descent only
Metric	MSE, RMSE, R^2	Accuracy, precision, recall, F_1, log-loss
Example	Marks from study hours	Pass/fail from study hours

Try It Yourself

Reproduce the pass/fail model:

```
import numpy as np
from sklearn.linear_model import LogisticRegression

h = np.array([.5,.75,1,1.25,1.5,1.75,1.75,2,2.25,2.5,
              2.75,3,3.25,3.5,4,4.25,4.5,4.75,5,5.5]).reshape(-1,
                  1)
p = np.array([0,0,0,0,0,0,1,0,1,0, 1,0,1,0,1,1,1,1,1,1])

# a very large C = almost no regularisation
clf = LogisticRegression(C=1e6).fit(h, p)
print(clf.intercept_, clf.coef_)            # about -4.08  1.50
print(clf.predict_proba([[3]])[0, 1])       # P(pass | 3 h) =
    0.607
print(clf.score(h, p))                      # accuracy = 0.80
```

Fitted coefficients: intercept -4.079, slope 1.505, boundary at 2.71 hours, training accuracy 0.80, matching the hand calculation. *Challenge:* predict the probabilities for 1, 2, 4 and 5 hours (answer: 0.07, 0.26, 0.88, 0.97).

Logistic regression

Advantages

- Gives *probabilities*, not only labels
- Simple, fast and interpretable (coefficients are log-odds ratios)
- Linear decision boundary
- Works well as a baseline
- Can be regularised

Disadvantages

- Only linear boundaries unless features are transformed
- May underperform on complex data
- Sensitive to outliers and correlated features
- Fails on perfectly separable data without regularisation

Applications

- Spam detection
- Disease diagnosis
- Loan default
- Customer churn
- Pass/fail prediction

4.3 Generalized Linear Models (GLMs)

Definition: Generalized linear model (GLM)

A **generalized linear model (GLM)** is a flexible extension of linear regression that allows the response y to follow any distribution from the **exponential family** (normal, Bernoulli/binomial, Poisson, gamma, ...) and relates its **mean** $\mu = \mathbb{E}[y \mid \boldsymbol{x}]$ to a linear predictor through a **link function**: $g(\mu) = \boldsymbol{\theta}^\top \boldsymbol{x}$.

The three components of a GLM

1. *Random component:* the distribution of y (exponential family).
2. *Systematic component:* the linear predictor $\eta = \theta_0 + \theta_1 x_1 + \cdots + \theta_d x_d$.
3. *Link function:* g, with $g(\mu) = \eta$, i.e. $\mu = g^{-1}(\eta)$

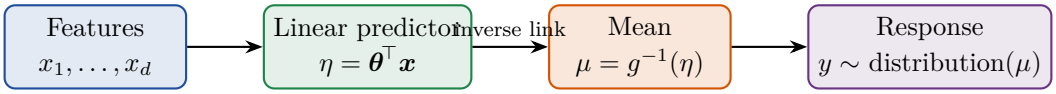


Figure 4.5. Structure of a generalized linear model.

Reading the Figure

Read left to right. The **features** are combined linearly into a score η (green box), exactly as in linear regression. The **inverse link** g^{-1} (arrow label) then converts this unbounded score into the **mean** μ of the response, forcing it into a sensible range (for example $(0, 1)$ for probabilities or $(0, \infty)$ for counts). Finally the **actual outcome** y is a random draw from a distribution (normal, Bernoulli, Poisson, ...) whose mean is μ. Changing the *link* and the *distribution* gives different models: this single picture covers linear, logistic and Poisson regression.

Model	Distribution of y	Link $g(\mu)$	Mean μ	Target type
Linear regression	Normal	identity: μ	η	real number
Logistic regression	Bernoulli	logit: $\ln \frac{\mu}{1-\mu}$	$\frac{1}{1+e^{-\eta}}$	0/1 class
Poisson regression	Poisson	log: $\ln \mu$	e^{η}	count $0, 1, 2, \ldots$
Gamma regression	Gamma	inverse: $1/\mu$	$1/\eta$	positive, skewed

Key Idea

Linear regression and logistic regression are both GLMs. Linear regression uses a Normal distribution with the identity link; logistic regression uses a Bernoulli distribution with the logit link. This is why they share the same gradient formula $\frac{1}{n}\sum(\hat{\mu}_i - y_i)x_{ij}$.

4.3.1 Real-life Example: Poisson Regression for Emergency Calls

A city fire-control room counts the number of emergency calls per day. The count depends on the rainfall x (mm): $y \sim \text{Poisson}(\mu)$ with

$$\ln \mu = 1 + 0.2\,x \qquad \Longleftrightarrow \qquad \mu = e^{\,1+0.2x}.$$

Example: Using the model

Expected calls on a dry day ($x = 0$): $\mu = e^1 = 2.72$.
On a day with 5 mm rain: $\mu = e^{1+1.0} = e^2 = 7.39$ calls.
On a day with 10 mm rain: $\mu = e^3 = 20.09$ calls.
Interpretation of the coefficient: each extra millimetre of rain multiplies the expected number of calls by $e^{0.2} = 1.22$, i.e. an increase of about **22%** per mm. Notice that the prediction is always positive, which an ordinary linear model could not guarantee.
Probability of exactly 7 calls on the 5 mm day: for a Poisson variable $P(y = k) = \frac{\mu^k e^{-\mu}}{k!}$, so $P(y = 7) = \frac{7.39^7 e^{-7.39}}{5040} \approx 0.15$.

Generalized linear models

Advantages

- One unified framework
- Handles counts, proportions and binary data properly
- Predictions always stay in the valid range
- Coefficients remain interpretable

Disadvantages

- Still assumes a linear relationship on the link scale
- Sensitive to outliers
- The distribution must be chosen correctly (e.g. over-dispersed counts break the Poisson model)

Applications

- Insurance claim counts
- Accident/call counts
- Binary outcomes (credit default)
- Proportions
- Waiting times

4.4 Binary Classification and Support Vector Machines

4.4.1 Binary Classification

Definition: Binary classification

Binary classification is the supervised task of assigning each input to one of exactly **two classes**, called the **positive** and **negative** class (coded $\{0, 1\}$ in logistic regression or $\{-1, +1\}$ in SVMs). The classifier learns a **decision function** $f(\boldsymbol{x})$ and predicts the positive class when $f(\boldsymbol{x})$ exceeds a threshold.

A **linear classifier** uses $f(\boldsymbol{x}) = \boldsymbol{w} \cdot \boldsymbol{x} + b$; the set $\{\boldsymbol{x} : \boldsymbol{w} \cdot \boldsymbol{x} + b = 0\}$ is a **hyperplane** (a line in 2-D, a plane in 3-D) that cuts the space into the two decision regions. **Examples:** spam / not spam, fraud / genuine, disease / healthy, loan default / repay. Performance is judged with the confusion matrix, precision, recall, F_1 (Chapter 2) and the **ROC curve** (true-positive rate against false-positive rate for all thresholds; the area under it, **AUC**, is 1 for a perfect classifier and 0.5 for random guessing).

Common Mistake

Many different lines can separate the same two classes perfectly. Which one is *best*? Logistic regression picks the one with the best likelihood; the **SVM** picks the one **farthest from both classes**, which tends to generalise better.

4.4.2 The Support Vector Machine (Linear, Hard Margin)

Definition: Support vector machine (hard margin)

A **Support Vector Machine (SVM)** is a *supervised* learning algorithm for *classification* (and regression) that finds the **maximum-margin hyperplane**: the separating hyperplane $\boldsymbol{w} \cdot \boldsymbol{x} + b = 0$ that is **as far as possible from the nearest training points of both classes**. The nearest points that fix the boundary are called **support vectors**.

Geometry. The distance from a point $\boldsymbol{x}_0$ to the hyperplane $\boldsymbol{w} \cdot \boldsymbol{x} + b = 0$ is $\dfrac{|\boldsymbol{w} \cdot \boldsymbol{x}_0 + b|}{\|\boldsymbol{w}\|}$. We scale $\boldsymbol{w}, b$ so that the closest points satisfy $|\boldsymbol{w} \cdot \boldsymbol{x} + b| = 1$. Then the two *margin lines* are $\boldsymbol{w} \cdot \boldsymbol{x} + b = +1$ and $\boldsymbol{w} \cdot \boldsymbol{x} + b = -1$ and the **margin width** is

$$\frac{1}{\|\boldsymbol{w}\|} + \frac{1}{\|\boldsymbol{w}\|} = \frac{2}{\|\boldsymbol{w}\|}.$$

Making the margin wide means making $\|\boldsymbol{w}\|$ small. The **hard-margin SVM** is therefore the optimisation problem

$$\min_{\boldsymbol{w}, b} \ \tfrac{1}{2}\|\boldsymbol{w}\|^2 \qquad \text{subject to} \qquad y_i \left(\boldsymbol{w} \cdot \boldsymbol{x}_i + b\right) \ \geq \ 1 \quad \text{for all } i. \tag{4.4}$$

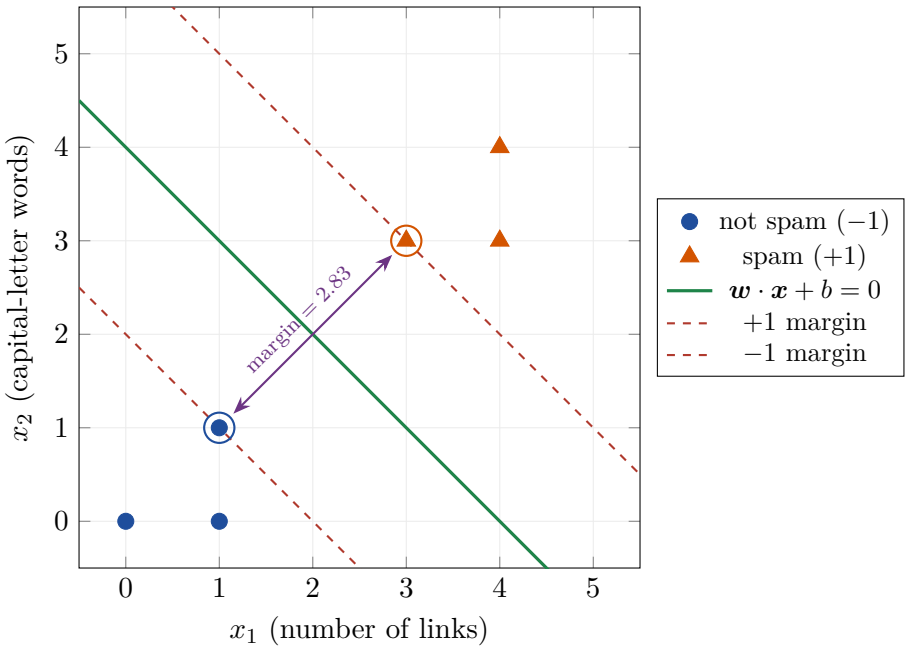


Figure 4.6. Maximum-margin separation of spam and non-spam e-mails. Circled points are the support vectors.

Reading the Figure

Each point is an e-mail described by two features. The **solid green line** is the decision boundary. The two **dashed red lines** are the **margin**: they pass through the nearest point of each class, the **circled support vectors** $(1, 1)$ and $(3, 3)$. The **purple arrow** is the margin width, the gap that the SVM tries to make as large as possible. Only the circled points decide where the line goes. If you moved or deleted the other four points (without crossing the margin), the boundary would *not change*. A different separating line (say, tilted) could also split the classes, but it would pass closer to some points and generalise worse.

Example: SVM by hand on the spam data

Data. Three non-spam mails (class -1): $(1, 1)$, $(0, 0)$, $(1, 0)$. Three spam mails (class $+1$): $(3, 3)$, $(4, 4)$, $(4, 3)$. Features: number of links x_1 and number of words in capitals x_2.
Step 1: find the closest opposite pair. The nearest points are $(1, 1)$ and $(3, 3)$ (distance $\sqrt{8} = 2.83$). They are the support vectors; by symmetry the boundary passes midway, perpendicular to the line joining them, so $\boldsymbol{w} = (a, a)$.
Step 2: impose the margin conditions.

$$\boldsymbol{w} \cdot (3, 3) + b = +1 \;\Rightarrow\; 6a + b = 1, \qquad \boldsymbol{w} \cdot (1, 1) + b = -1 \;\Rightarrow\; 2a + b = -1.$$

Subtracting: $4a = 2 \Rightarrow a = 0.5$; then $b = -1 - 2(0.5) = -2$. So $\boldsymbol{w} = (0.5, 0.5)$, $b = -2$ and the boundary is $0.5x_1 + 0.5x_2 - 2 = 0$, i.e. $x_1 + x_2 = 4$.
Step 3: verify all six constraints $y_i(\boldsymbol{w} \cdot \boldsymbol{x}_i + b) \geq 1$:

Point	$(1,1)$	$(0,0)$	$(1,0)$	$(3,3)$	$(4,4)$	$(4,3)$
$f = 0.5x_1 + 0.5x_2 - 2$	-1	-2	-1.5	1	2	1.5
$y\,f$	1	2	1.5	1	2	1.5
Satisfied?	on margin	yes	yes	on margin	yes	yes

All $y_i f(\boldsymbol{x}_i) \geq 1$, and exactly two points equal 1: they are the support vectors.
Step 4: margin. $\|\boldsymbol{w}\| = \sqrt{0.25 + 0.25} = 0.707$, so the margin is $2/0.707 = \mathbf{2.83}$.
Step 5: predict. A new mail with 4 links and 1 capital word: $f = 0.5(4) + 0.5(1) - 2 = 0.5 > 0 \Rightarrow$ **spam**. A mail with $(1,2)$: $f = 0.5 + 1 - 2 = -0.5 < 0 \Rightarrow$ **not spam**.

4.4.3 Soft-Margin SVM

Real data usually overlap, so no hyperplane separates the classes perfectly (and noisy points would make a hard margin very narrow). The **soft-margin SVM** allows some points to violate the margin by introducing **slack variables** $\xi_i \geq 0$:

$$\min_{\boldsymbol{w},b,\boldsymbol{\xi}} \ \tfrac{1}{2}\|\boldsymbol{w}\|^2 + C\sum_{i=1}^{n}\xi_i \qquad \text{subject to} \qquad y_i(\boldsymbol{w}\cdot\boldsymbol{x}_i + b) \geq 1 - \xi_i, \quad \xi_i \geq 0. \tag{4.5}$$

Here $\xi_i = 0$ for a correctly classified point outside the margin, $0 < \xi_i \leq 1$ for a point inside the margin but on the correct side, and $\xi_i > 1$ for a misclassified point. The hyperparameter C controls the trade-off:

	Large C	**Small C**
Priority	Few training errors	Wide margin
Margin	Narrow	Wide
Bias / variance	Low bias, high variance	Higher bias, lower variance
Risk	Overfitting	Underfitting

Equivalently, the soft-margin SVM minimises the **hinge loss**

$$L_{\text{hinge}} = \max\left(0,\ 1 - y\,f(\boldsymbol{x})\right),$$

which is 0 for points correctly classified *beyond* the margin and grows linearly otherwise. For example, with $y = +1$: $f = 2 \Rightarrow$ loss 0; $f = 0.3 \Rightarrow 0.7$; $f = -0.5 \Rightarrow 1.5$.

Reading the Figure

The horizontal axis is the **margin** $m = y\,f(\boldsymbol{x})$: it is positive when the point is on the *correct* side and negative when on the *wrong* side, and bigger values mean greater confidence. **Grey (0–1 loss):** counts a mistake as 1 and a correct answer as 0, but it is a cliff, so we cannot

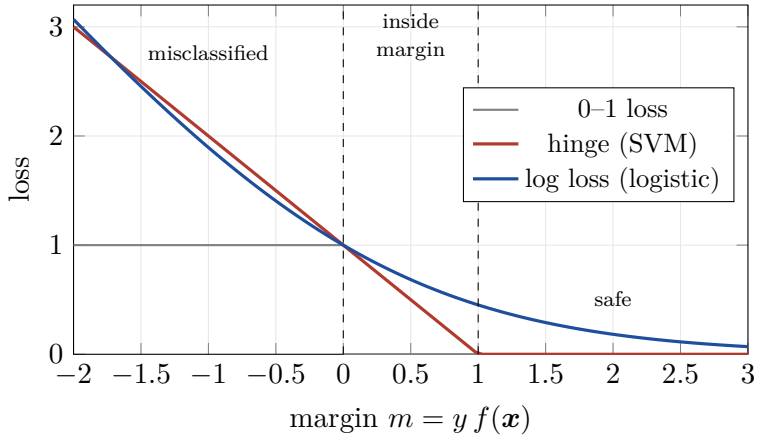


Figure 4.7. Loss as a function of the margin $m = y\,f(\boldsymbol{x})$: 0–1 loss, hinge loss (SVM) and log loss (logistic regression).

use gradients on it. **Red (hinge loss):** zero once $m \geq 1$ ("correct with room to spare"), so points beyond the margin contribute *nothing*, and grows linearly for $m < 1$, including points that are right but inside the margin. **Blue (log loss):** never exactly zero, so every point still matters a little, and it decreases smoothly. Both red and blue curves are smooth-ish *upper bounds* on the 0–1 loss that can be optimised. This single picture shows why an SVM depends only on a few support vectors, while logistic regression uses all points.

Key Idea

SVM vs logistic regression. Both learn a linear boundary. Logistic regression minimises the log loss and outputs *probabilities*; the SVM minimises the hinge loss and maximises the *margin*, depending only on the support vectors. When classes are nearly separable the SVM often generalises slightly better; when you need probabilities, use logistic regression. The *kernel trick* that lets SVMs learn *non-linear* boundaries is covered in Chapter 6.

Try It Yourself

Verify the hand-solved SVM:

```
import numpy as np
from sklearn.svm import SVC

X = np.array([[1,1],[0,0],[1,0],[3,3],[4,4],[4,3]])    # [links, capitals]
y = np.array([-1,-1,-1, 1, 1, 1])                        # -1 ham, +1 spam

svm = SVC(kernel="linear", C=1e6).fit(X, y)    # huge C = hard margin
print(svm.coef_, svm.intercept_)          # [[0.5 0.5]] [-2.]
print(svm.support_vectors_)               # [[1. 1.] [3. 3.]]
```

```
print(svm.predict([[4, 1]]))            # [1]   -> spam
print(svm.decision_function([[4, 1]])) # [0.5]
```

The output reproduces $\boldsymbol{w} = (0.5, 0.5)$, $b = -2$, the two support vectors and $f(4, 1) = 0.5$. *Challenge:* set `C=0.01` and watch what happens to the margin and the support vectors.

Support vector machines

Advantages

- Effective in high dimensions
- Only the support vectors matter, so it is memory-efficient
- Strong generalisation because of the maximum margin
- Flexible through kernels

Disadvantages

- Slow on very large datasets
- Requires feature scaling
- Sensitive to the choice of C and the kernel
- Does not directly output probabilities
- Hard to interpret with nonlinear kernels

Applications

- Text and spam classification
- Image and face recognition
- Handwriting recognition
- Bioinformatics (gene and protein classification)

4.5 Choosing Among the Methods of Unit II

	k-NN	**Decision tree**	**Naive Bayes**	**Linear / Logistic / SVM**
Boundary	Irregular, local	Axis-parallel boxes	Smooth (probabilistic)	Linear hyperplane
Interpretable?	Low	High	Moderate	High (coefficients)
Needs scaling?	Yes	No	No	Yes
Training speed	None	Fast	Very fast	Fast–moderate
Outputs probabilities?	Roughly	Roughly	Yes	Logistic: yes; SVM: no
Good when	Small data, local patterns	Mixed features, rules needed	Text, little data	Many features, linear structure

Chapter Summary

- **Linear regression:** $\hat{y} = \boldsymbol{\theta}^\top \boldsymbol{x}$, fitted by least squares: $\theta_1 = S_{xy}/S_{xx}$, $\theta_0 = \bar{y} - \theta_1 \bar{x}$, or $\boldsymbol{\theta} = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \boldsymbol{y}$, or gradient descent $\theta_j \leftarrow \theta_j - \alpha\, \partial J / \partial \theta_j$. Evaluate with MSE, RMSE, R^2. Ridge (L2) and Lasso (L1) regularise.
- **Logistic regression:** $P(y = 1) = \sigma(\boldsymbol{\theta}^\top \boldsymbol{x})$; log-odds is linear; trained by minimising cross-entropy (maximum likelihood); predicts class 1 if $p \geq 0.5$; e^{θ_j} is the odds ratio. Softmax extends it to many classes.

- **GLM:** distribution from the exponential family + linear predictor + link function $g(\mu) = \eta$. Linear (identity), logistic (logit) and Poisson (log) regression are special cases.
- **Binary classification:** two classes, decision function and hyperplane; evaluate with confusion matrix, precision, recall, F_1, ROC–AUC.
- **SVM:** maximum-margin hyperplane; margin $= 2/\|\boldsymbol{w}\|$; minimise $\frac{1}{2}\|\boldsymbol{w}\|^2$ s.t. $y_i(\boldsymbol{w} \cdot \boldsymbol{x}_i + b) \geq 1$; only **support vectors** matter; soft margin adds slack and C; hinge loss $\max(0, 1 - yf)$.
- Worked examples: marks $= 26 + 9\times$hours ($R^2 = 0.976$); pass probability $\sigma(-4.08 + 1.5h)$ with boundary 2.72 h; Poisson calls $e^{1+0.2x}$; SVM $\boldsymbol{w} = (0.5, 0.5)$, $b = -2$, margin 2.83.

Exam Corner

A. Very Short Answer Questions (2 marks each)

1. Define linear regression. **[2 marks]**
2. What is a residual? What does the method of least squares minimise? **[2 marks]**
3. Write the gradient-descent update rule. What is the role of the learning rate? **[2 marks]**
4. Define R^2. What does $R^2 = 0.9$ mean? **[2 marks]**
5. Differentiate Ridge and Lasso regression. **[2 marks]**
6. Define the sigmoid function. What is its range? **[2 marks]**
7. Define odds and log-odds. **[2 marks]**
8. Why is logistic regression a classification algorithm even though it has "regression" in its name? **[2 marks]**
9. Define a generalized linear model. What is a link function? **[2 marks]**
10. What are support vectors? What is the margin of an SVM? **[2 marks]**
11. What is the role of the parameter C in a soft-margin SVM? **[2 marks]**
12. Define hinge loss. **[2 marks]**

Model Answer: Q2, Q7 and Q10

Q2. A *residual* is the difference between the observed and predicted value, $e_i = y_i - \hat{y}_i$. Least squares chooses the parameters that minimise the *sum of squared residuals* $\sum e_i^2$.
Q7. If an event has probability p, its *odds* are $p/(1-p)$ and its *log-odds* (logit) is $\ln[p/(1-p)]$. Logistic regression models the log-odds as a linear function of the features.

Q10. *Support vectors* are the training points that lie exactly on the margin boundaries (or inside the margin) and that alone determine the position of the separating hyperplane. The *margin* is the distance between the two margin lines, $2/\|\boldsymbol{w}\|$.

B. Short Answer Questions (5 marks each)

1. Explain simple linear regression and derive the least-squares estimates of θ_0 and θ_1. **[5 marks]**
2. Explain gradient descent for linear regression. Discuss the effect of the learning rate. **[5 marks]**
3. Compare the normal equation and gradient descent. **[5 marks]**
4. State the assumptions of linear regression. Explain R^2 and adjusted R^2. **[5 marks]**
5. Explain logistic regression with the sigmoid function. Why is it preferred to linear regression for classification? **[5 marks]**
6. Explain the cross-entropy loss used in logistic regression. **[5 marks]**
7. Explain generalized linear models with the three components and examples. **[5 marks]**
8. Explain the maximum-margin classifier and support vectors with a diagram. **[5 marks]**
9. Differentiate between linear regression and logistic regression. **[5 marks]**
10. Differentiate between hard-margin and soft-margin SVM. **[5 marks]**

Model Answer: B5 (outline for 5 marks)

Definition: logistic regression models $P(y=1 \mid \boldsymbol{x}) = \sigma(z) = 1/(1+e^{-z})$ with $z = \boldsymbol{w}\cdot\boldsymbol{x} + b$; it predicts class 1 if $P \geq 0.5$.
Sigmoid: maps $(-\infty, \infty) \to (0,1)$; $\sigma(0) = 0.5$; S-shaped (draw the curve).
Why not linear regression? (i) a line can give outputs < 0 or > 1, not valid probabilities; (ii) squared error is a poor loss for 0/1 targets; (iii) outliers can shift the line and change predictions; (iv) errors are not normal or of constant variance.
Training: minimise cross-entropy $J = -\frac{1}{n}\sum[y \ln p + (1-y)\ln(1-p)]$ by gradient descent.
Example: pass/fail from study hours.

C. Long Answer Questions (10 marks each)

1. Explain linear regression in detail: model, least-squares method, normal equation, gradient descent, assumptions and evaluation metrics. Fit a line to the data $(1,2), (2,4), (3,5), (4,4), (5,5)$. **[10 marks]**
2. Explain logistic regression: sigmoid, odds and log-odds, decision boundary, cross-entropy loss and gradient-descent training. Explain how it is extended to multiple classes. **[10 marks]**

3. Explain generalized linear models. Show that linear and logistic regression are special cases. Explain Poisson regression with an example. **[10 marks]**

4. Explain the support vector machine: maximum margin, support vectors, hard-margin optimisation problem, soft margin and the role of C. Solve a small numerical example. **[10 marks]**

5. Compare linear regression, logistic regression and SVM with respect to task, loss function, output, advantages and disadvantages. **[10 marks]**

6. Explain overfitting in linear models and how regularisation (Ridge and Lasso) controls it. **[10 marks]**

Model Answer: C1 (numerical part)

$x = (1,2,3,4,5)$, $y = (2,4,5,4,5)$. $\bar{x} = 3$, $\bar{y} = 4$.
$(x-\bar{x})(y-\bar{y}) = (-2)(-2)+(-1)(0)+(0)(1)+(1)(0)+(2)(1) = 4+0+0+0+2 = 6 = S_{xy}$;
$S_{xx} = 4+1+0+1+4 = 10$.
$\theta_1 = 6/10 = 0.6$, $\theta_0 = 4 - 0.6(3) = 2.2$. **Model:** $\hat{y} = 2.2 + 0.6x$. Prediction for $x = 6$: $2.2 + 3.6 = 5.8$.
Residuals: $\hat{y} = (2.8, 3.4, 4.0, 4.6, 5.2)$, $e = (-0.8, 0.6, 1, -0.6, -0.2)$, SSE $= 0.64 + 0.36 + 1 + 0.36 + 0.04 = 2.4$, SST $= 4+0+1+0+1 = 6$, $R^2 = 1 - 2.4/6 = \mathbf{0.60}$.

D. Numerical Problems

1. Fit a line to $x = (2,4,6,8)$, $y = (3,7,5,10)$ and predict y at $x = 10$. *(Answer: $\bar{x} = 5$, $\bar{y} = 6.25$, $S_{xy} = 19$, $S_{xx} = 20$, $\theta_1 = 0.95$, $\theta_0 = 1.5$; $\hat{y}(10) = 11$.)*

2. For problem 1 compute SSE and R^2. *(Answer: $\hat{y} = 3.4, 5.3, 7.2, 9.1$; residuals $-0.4, 1.7, -2.2, 0.9$; $SSE = 8.70$; $SST = 26.75$; $R^2 = 0.675$.)*

3. One gradient-descent step for $\hat{y} = \theta_1 x$ on $(1,2),(2,4)$ with $\theta_1 = 0$, $\alpha = 0.1$, $J = \frac{1}{2n}\sum(\hat{y}-y)^2$. *(Answer: gradient $= \frac{1}{n}\sum(\hat{y}-y)x = \frac{(-2)(1)+(-4)(2)}{2} = -5$; $\theta_1 = 0 - 0.1(-5) = 0.5$.)*

4. Evaluate $\sigma(0)$, $\sigma(2)$ and $\sigma(-2)$. *(Answer: 0.5, 0.881, 0.119.)*

5. A logistic model has $z = -6 + 2x$. Find $P(y=1)$ at $x = 3$ and at $x = 4$, the decision boundary and the odds ratio per unit of x. *(Answer: $\sigma(0) = 0.5$; $\sigma(2) = 0.881$; boundary $x = 3$; $e^2 = 7.39$.)*

6. Compute the log-loss for $(y = 1, p = 0.8)$ and $(y = 0, p = 0.3)$ and the average. *(Answer: 0.223, 0.357; average 0.290.)*

7. One gradient step of logistic regression on $(x,y) = (0,0),(1,1)$ from $\theta_0 = \theta_1 = 0$ with $\alpha = 1$. *(Answer: errors $0.5, -0.5$; $\partial J/\partial\theta_0 = 0$; $\partial J/\partial\theta_1 = \frac{0+(-0.5)(1)}{2} = -0.25$; new $\theta = (0, 0.25)$.)*

8. Softmax of scores $(1,2,3)$. *(Answer: $e^z = 2.718, 7.389, 20.086$, sum $= 30.193$, probabilities $0.090, 0.245, 0.665$.)*

9. A Poisson model has $\ln \mu = 0.2 + 0.5x$. Find the expected count at $x = 2$ and the rate ratio per unit of x. *(Answer: $\mu = e^{1.2} = 3.32$; ratio $e^{0.5} = 1.65$.)*

10. An SVM has $\boldsymbol{w} = (1, 1)$, $b = -3$. Classify $(2, 2)$ and $(1, 1)$, and find the margin. *(Answer: $f(2,2) = 1 \Rightarrow +1$ (on margin); $f(1,1) = -1 \Rightarrow -1$ (on margin); margin $= 2/\sqrt{2} = 1.41$.)*

11. Compute the hinge loss for $(y = +1, f = 0.3)$, $(y = -1, f = -2)$ and $(y = +1, f = -0.5)$. *(Answer: 0.7, 0, 1.5.)*

E. Multiple-Choice Questions

1. Least squares minimises:
   (a) sum of residuals (b) sum of squared residuals (c) largest residual (d) sum of y

2. In gradient descent, a learning rate that is too large may cause:
   (a) slow learning (b) divergence (c) exact solution (d) underfitting

3. The range of the sigmoid function is:
   (a) $(-1, 1)$ (b) $(0, \infty)$ (c) $(0, 1)$ (d) $(-\infty, \infty)$

4. Logistic regression is used for:
   (a) clustering (b) classification (c) dimensionality reduction (d) only regression

5. If $p = 0.8$, the odds are:
   (a) 0.8 (b) 0.25 (c) 4 (d) 1.25

6. The link function of logistic regression is:
   (a) identity (b) log (c) logit (d) inverse

7. Poisson regression is suitable for:
   (a) binary data (b) count data (c) negative values (d) text

8. The margin of an SVM is:
   (a) $\|\boldsymbol{w}\|$ (b) $1/\|\boldsymbol{w}\|$ (c) $2/\|\boldsymbol{w}\|$ (d) $2\|\boldsymbol{w}\|$

9. Lasso regularisation can:
   (a) make weights exactly zero (b) increase variance (c) remove the intercept (d) only increase weights

10. A large value of C in a soft-margin SVM leads to:
    (a) wide margin, many errors (b) narrow margin, few errors (c) no support vectors (d) a non-linear boundary

11. Ridge regression adds a penalty proportional to:
    (a) $\sum |\theta_j|$ (b) $\sum \theta_j^2$ (c) $\max \theta_j$ (d) number of features

12. Which method gives probabilities directly?
    (a) SVM (b) logistic regression (c) hinge loss (d) least squares

Answers: 1-b, 2-b, 3-c, 4-b, 5-c, 6-c, 7-b, 8-c, 9-a, 10-b, 11-b, 12-b.

F. Practice Problems

1. Collect data on 8 friends: hours of sleep and score in a short test. Fit a line by hand and by `scikit-learn` and compare.
2. Using the pass/fail model, find the number of study hours that gives a pass probability of 0.9. *(Hint: solve* $-4.08 + 1.5h = \ln(0.9/0.1) = 2.197$*; answer* $h \approx 4.2$*.)*
3. **Coding:** implement gradient descent for linear regression from scratch with `numpy` and plot the loss against iterations for $\alpha = 0.001, 0.01, 0.05$.
4. **Coding:** train `LogisticRegression` and a linear `SVC` on the *breast cancer* dataset (`load_breast_cancer`), scale the features, and compare accuracy, precision and recall.
5. Draw a labelled diagram of an SVM with the margin, hyperplane and support vectors, and mark where $w \cdot x + b = +1, 0, -1$.

Quick Revision

Topic	One-line recall
Linear regression	$\hat{y} = \theta_0 + \theta_1 x$; minimise SSE; $\theta_1 = \frac{S_{xy}}{S_{xx}}$, $\theta_0 = \bar{y} - \theta_1 \bar{x}$.
Normal equation	$\boldsymbol{\theta} = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \boldsymbol{y}$; exact but $O(d^3)$.
Gradient descent	$\theta \leftarrow \theta - \alpha\, \partial J / \partial \theta$; too small $\rightarrow$ slow, too large $\rightarrow$ diverges.
Metrics	MSE, RMSE, MAE, $R^2 = 1 - \text{SSE/SST}$, adjusted R^2.
Ridge / Lasso	L2 shrinks weights; L1 makes some exactly 0 (feature selection).
Sigmoid	$\sigma(z) = \frac{1}{1+e^{-z}}$; range $(0, 1)$; $\sigma(0) = 0.5$.
Logistic regression	$\ln \frac{p}{1-p} = \boldsymbol{\theta}^\top \boldsymbol{x}$; loss $= -\frac{1}{n} \sum [y \ln p + (1-y) \ln(1-p)]$; odds ratio e^{θ_j}.
Softmax	$\frac{e^{z_k}}{\sum_j e^{z_j}}$ for many classes.
GLM	distribution + linear predictor + link; identity / logit / log.
SVM	max margin $2/\|w\|$; min $\frac{1}{2}\|w\|^2$ s.t. $y_i(w \cdot x_i + b) \geq 1$; support vectors.
Soft margin	slack ξ_i; large C: narrow margin, overfit; small C: wide margin.
Hinge loss	$\max(0, 1 - yf)$; zero beyond the margin.

Chapter 5

Ensemble Learning

Learning Objectives

After studying this chapter you will be able to:

- **define** ensemble learning and explain *why* combining models improves accuracy;
- compute the accuracy of a **majority vote** and explain the role of **diversity**;
- explain **hard** and **soft voting** classifiers and work out an example;
- explain **bagging** and **pasting**, the **bootstrap** and **out-of-bag (OOB)** evaluation;
- describe how a **Random Forest** works and read **feature importances**;
- run **AdaBoost** by hand and explain **gradient boosting**;
- explain **stacking** with a meta-learner;
- compare bagging, boosting and stacking in terms of **bias and variance**;
- answer 2-, 5- and 10-mark university questions on Unit III (ensembles).

> *At a county fair, 800 people guessed the weight of an ox. No one guessed it exactly, but the* average *of all guesses was off by less than one percent.*
>
> — Francis Galton, 1907 (the "wisdom of crowds")

In Chapters 3 and 4 we studied *single* models. This chapter asks a simple question:

If one model is good, are many models better?

The answer is usually **yes**, provided the models are *different from each other*. Ensembles are behind most winning solutions in machine-learning competitions and many real systems.

5.1 Introduction to Ensemble Learning

5.1.1 The Idea: Wisdom of the Crowd

Think of a **hospital**. For a difficult case, one doctor might make a mistake, but if five independent specialists examine the patient and the *majority* agree on a diagnosis, you trust it much more. Or think of a **jury**: twelve people decide together rather than one judge alone.

Definition: Ensemble learning

Ensemble learning combines the predictions of *several models* (called **base learners**, **weak learners** or **predictors**) to produce one final prediction that is usually *more accurate and more stable* than the prediction of any single model.

Definition: Weak learner and strong learner

A **weak learner** is a model only slightly better than random guessing (for two classes, accuracy a little above 50%), for example a very shallow decision tree. A **strong learner** is a model with high accuracy. A key theoretical result says that many weak learners can be combined into a strong learner.

5.1.2 Why Does It Work? A Simple Calculation

Suppose we have n classifiers, each correct with probability $p > 0.5$, and *assume their mistakes are independent*. If we take the **majority vote**, the ensemble is correct whenever more than half of the classifiers are correct. For n classifiers (odd n) this probability is given by the binomial distribution:

$$P(\text{ensemble correct}) = \sum_{k=\lceil (n+1)/2 \rceil}^{n} \binom{n}{k} p^k (1-p)^{n-k}.$$

Example: Three classifiers, each 70% accurate

The majority is correct when at least 2 of the 3 are correct.

Number correct	Probability	Value
All 3 correct	0.7^3	0.343
Exactly 2 correct	$3 \times 0.7^2 \times 0.3$	0.441
At least 2 correct		**0.784**

So three 70% classifiers give a 78.4% ensemble. With **five** classifiers:

$$P = \binom{5}{3} 0.7^3 0.3^2 + \binom{5}{4} 0.7^4 0.3 + 0.7^5 = 0.3087 + 0.3602 + 0.1681 = \mathbf{0.837}.$$

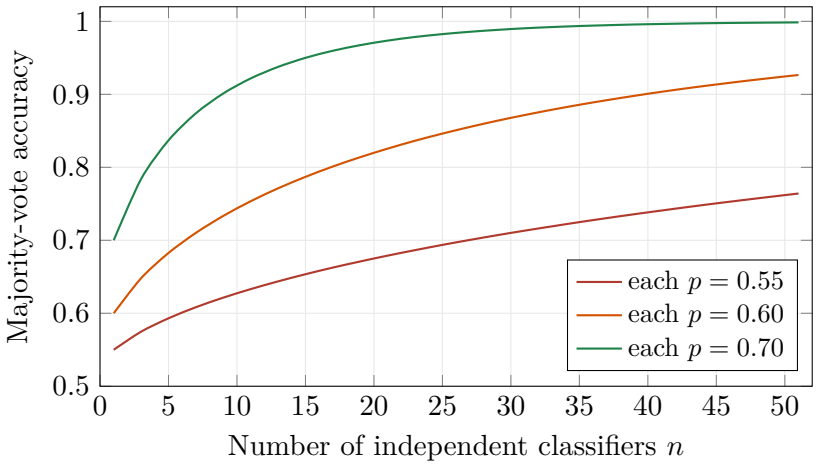


Figure 5.1. Accuracy of a majority vote of n *independent* classifiers, each with accuracy p.

Reading the Figure

The horizontal axis is the **number of classifiers** voting and the vertical axis is the **accuracy of the majority vote**. Each curve is for a different accuracy of the *individual* classifiers (red 55%, orange 60%, green 70%).

Look at the **green curve**: with one classifier the accuracy is 70%; with 5 it is 83.7%; with 21 it is 97.4%; with 51 it is almost 100%. Even the **red curve**, built from classifiers that are barely better than guessing (55%), climbs steadily to about 76% with 51 voters.

Important warning: this picture assumes the classifiers make *independent* errors. In practice they are trained on the same data and their errors are *correlated*, so real gains are smaller. This is why we work hard to make the members **diverse**.

Key Idea

Ensembles work best when the members are (i) each better than random guessing and (ii) as different from each other as possible. Diversity can come from different *algorithms*, different *training samples*, different *features*, or different *hyperparameters*.

5.1.3 Types of Ensemble Methods

Reading the Figure

Read the tree from top to bottom. The three coloured boxes are the three **strategies**.

- **Parallel** (green): the models are trained *independently* (possibly on different samples of the data) and their outputs are combined by voting or averaging. The aim is to reduce **variance**.
- **Sequential** (orange): models are trained *one after another*, and each new model concentrates on the mistakes of the earlier ones. The aim is to reduce **bias**.

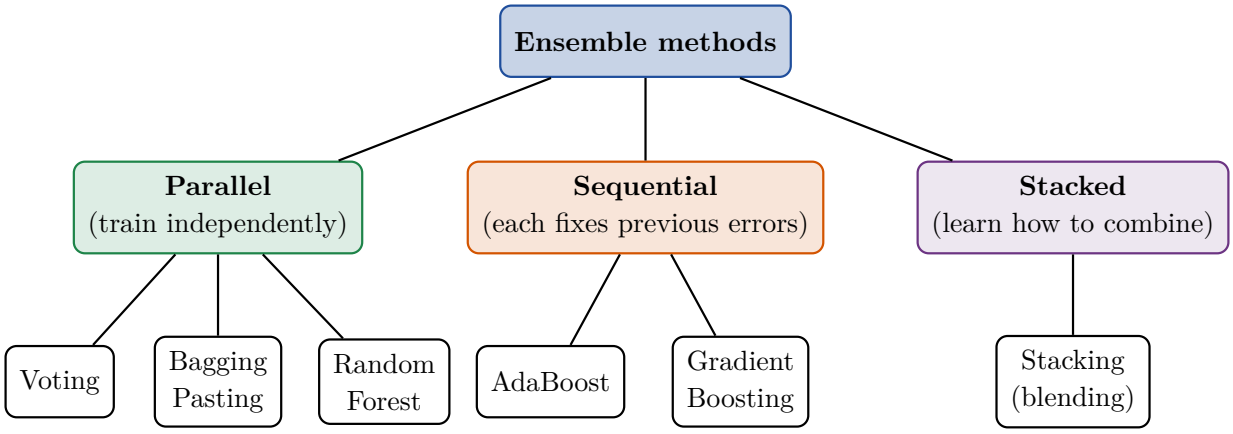


Figure 5.2. Families of ensemble methods covered in this chapter.

- **Stacked** (purple): a second-level model *learns* the best way to combine the first-level models.

Ensemble learning

Advantages

- Higher accuracy than a single model
- Reduces overfitting (lower variance) and is more robust to noise
- Works with almost any base algorithm

Disadvantages

- More computation, memory and prediction time
- Harder to interpret than a single model
- Gains are small if the members are too similar

Applications

- Credit scoring and fraud detection
- Medical diagnosis and recommender systems
- Winning solutions in data-science competitions

5.2 Voting Classifiers

Definition: Voting classifier

A **voting classifier** combines several *different* classifiers trained on the *same data*. In **hard voting** the final class is the one predicted by the *majority* of the classifiers; in **soft voting** the predicted class *probabilities* are averaged and the class with the highest average probability wins.

Reading the Figure

Follow the arrows from left to right. The **same instance $\boldsymbol{x}$** is given to three *different* classifiers. Each returns its own answer (labels A, B, B on the arrows). The orange box combines them: in **hard voting** it counts the votes (two B's against one A), so the **final prediction is B**. Notice that classifier 1 made a "mistake" but the ensemble still reached

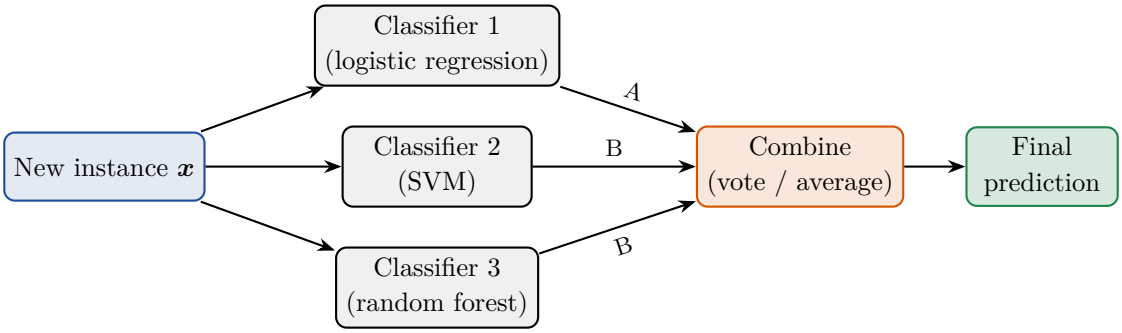


Figure 5.3. A voting ensemble: three classifiers vote on the same instance.

the answer the majority agreed on.

5.2.1 Hard vs. Soft Voting: Worked Example

Example: Which class is the new e-mail?

Three classifiers estimate the probability that an e-mail is **spam** (class A); the other class is not-spam (B).

Classifier	P(A)	P(B)	**Own prediction**
1 (logistic regression)	0.90	0.10	A
2 (SVM)	0.40	0.60	B
3 (random forest)	0.40	0.60	B

Hard voting: votes are A, B, B $\Rightarrow$ **B** wins, 2 to 1.

Soft voting: average probability of A is

$$\bar{P}(A) = \frac{0.90 + 0.40 + 0.40}{3} = \frac{1.70}{3} = 0.567, \qquad \bar{P}(B) = 0.433 \;\Rightarrow\; \textbf{A wins}.$$

Why the difference? Classifier 1 is *very confident* (90%) while the other two are only *mildly* sure (60%). Soft voting lets confident classifiers count for more.

Weighted soft voting: if we trust classifier 1 three times as much (weights $3, 1, 1$):

$$\bar{P}(A) = \frac{3(0.90) + 1(0.40) + 1(0.40)}{3 + 1 + 1} = \frac{3.5}{5} = 0.70 \;\Rightarrow\; \textbf{A}.$$

For **regression**, the ensemble simply **averages** the predictions: $\hat{y} = \frac{1}{m}\sum_{j=1}^{m} \hat{y}_j$.

	Hard voting	**Soft voting**
Uses	Predicted class labels	Predicted probabilities
Rule	Majority vote	Highest average probability
Needs probabilities?	No	Yes
Accuracy	Good	Usually slightly better

Voting classifiers

Advantages

- Very simple to build and often beats the best single member
- Soft voting uses the classifiers' *confidence*

Disadvantages

- Poor members or highly similar members add little
- Needs several trained models, hence more cost

Applications

- Combining different algorithms for text, image or tabular classification

5.3 Bagging and Pasting

Voting uses *different algorithms* on the same data. Another way to create diversity is to use the **same algorithm** but train each copy on a **different random subset of the training data**.

Definition: Bagging and pasting

Both methods train B copies of the same base learner, each on a random sample of the training set, and combine them by voting (classification) or averaging (regression).

- **Bagging** (*bootstrap aggregating*): samples are drawn **with replacement**.
- **Pasting**: samples are drawn **without replacement**.

5.3.1 The Bootstrap

Definition: Bootstrap sample

A **bootstrap sample** of a dataset of size n is obtained by drawing n instances *at random with replacement*. Some instances appear several times and some do not appear at all.

Example: Drawing one bootstrap sample

Original data: instances $\{1, 2, 3, 4, 5\}$ ($n = 5$). One bootstrap sample might be

$$\{2,\ 2,\ 5,\ 1,\ 5\}.$$

Instance 2 and 5 appear twice, instance 1 once, and instances **3 and 4 are missing**. These missing instances are the **out-of-bag (OOB)** instances for this sample; they can be used to test the model trained on the sample.

How many instances are left out? The probability that a particular instance is *not* chosen in one draw is $1 - \frac{1}{n}$; over n draws it is $(1 - \frac{1}{n})^n \to e^{-1} \approx 0.368$ for large n. Hence

- about **63.2%** of the instances appear in each bootstrap sample (at least once);
- about **36.8%** are **out-of-bag**.

(For $n = 100$: $0.99^{100} = 0.366$; a simulation with $n = 1000$ gave 62.2% unique instances.)

5.3.2 Algorithm and Aggregation

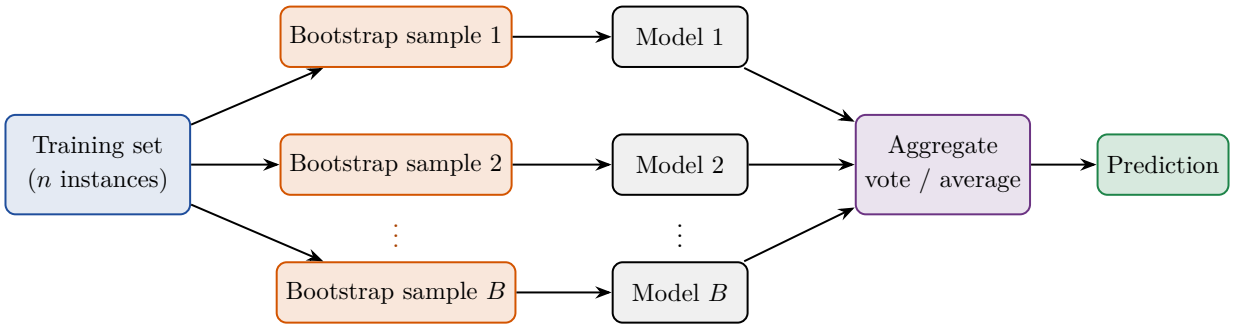


Figure 5.4. The bagging procedure.

Reading the Figure

Start at the blue box on the left. (1) From the training set we draw B different **bootstrap samples** (orange). (2) We train one model on each sample (grey); because the samples differ, the models differ. (3) For a new instance, every model makes a prediction and the purple box **aggregates** them: the **most frequent class** for classification or the **average** for regression. The models can be trained at the same time (in parallel).

Why does bagging reduce variance? Suppose each of B models has prediction variance σ^2 and any two models have correlation ρ. The variance of the average is

$$\mathrm{Var}(\bar{f}) = \rho\,\sigma^2 + \frac{1-\rho}{B}\,\sigma^2. \tag{5.1}$$

Example: Variance of an average of models

Let $\sigma^2 = 4$ and $B = 10$.

- If the models are **independent** ($\rho = 0$): Var $= \frac{4}{10} = 0.4$ (ten times smaller).
- If they are **highly correlated** ($\rho = 0.5$): Var $= 0.5(4) + \frac{0.5}{10}(4) = 2 + 0.2 = 2.2$ (only about half).
- Even with $B \to \infty$ the variance cannot go below $\rho\sigma^2 = 2$.

Lesson: averaging helps most when the models are *uncorrelated*; bagging tries to decorrelate them using different bootstrap samples. Bagging reduces **variance** but *not bias*, so it works best with **low-bias, high-variance** learners such as deep decision trees.

5.3.3 Out-of-Bag (OOB) Evaluation

Each model never saw about 36.8% of the training instances. We can use these OOB instances as a *free validation set*: for each instance, predict using only the models that did **not** have it in their sample, and compute the accuracy. The **OOB score** is a good estimate of test accuracy *without needing a separate validation set.*

	Bagging	**Pasting**
Sampling	With replacement (bootstrap)	Without replacement
Duplicates in a sample?	Yes	No
OOB evaluation?	Yes (about 37% per model)	Not directly
Diversity	Slightly higher	Slightly lower
Typical bias / variance	Slightly higher bias, lower variance	Slightly lower bias

Bagging and pasting

Advantages

- Reduces variance and overfitting
- Parallel training; free OOB validation
- Works with any base learner

Disadvantages

- Does not reduce bias
- Less interpretable and slower than one model
- Little gain for stable learners (e.g. linear regression)

Applications

- Improving decision trees (Random Forests)
- Noisy data problems, regression and classification

5.4 Random Forests

A **Random Forest** is the most popular ensemble method. It is **bagging of decision trees** with one extra trick that makes the trees even more different.

Definition: Random Forest

A **Random Forest** is an ensemble of many decision trees, each trained on a *bootstrap sample* of the data, where at *every split* the tree considers only a **random subset of the features** (typically $\sqrt{p}$ for classification and $p/3$ for regression, with p the total number of features). The forest predicts by *majority vote* (classification) or *average* (regression).

Why the random feature subset? If one feature is very strong, *every* bagged tree would split on it first and all trees would look alike (high correlation ρ in Equation (5.1)). Forcing each split to choose from a random subset **decorrelates** the trees, so averaging them reduces the variance much more.

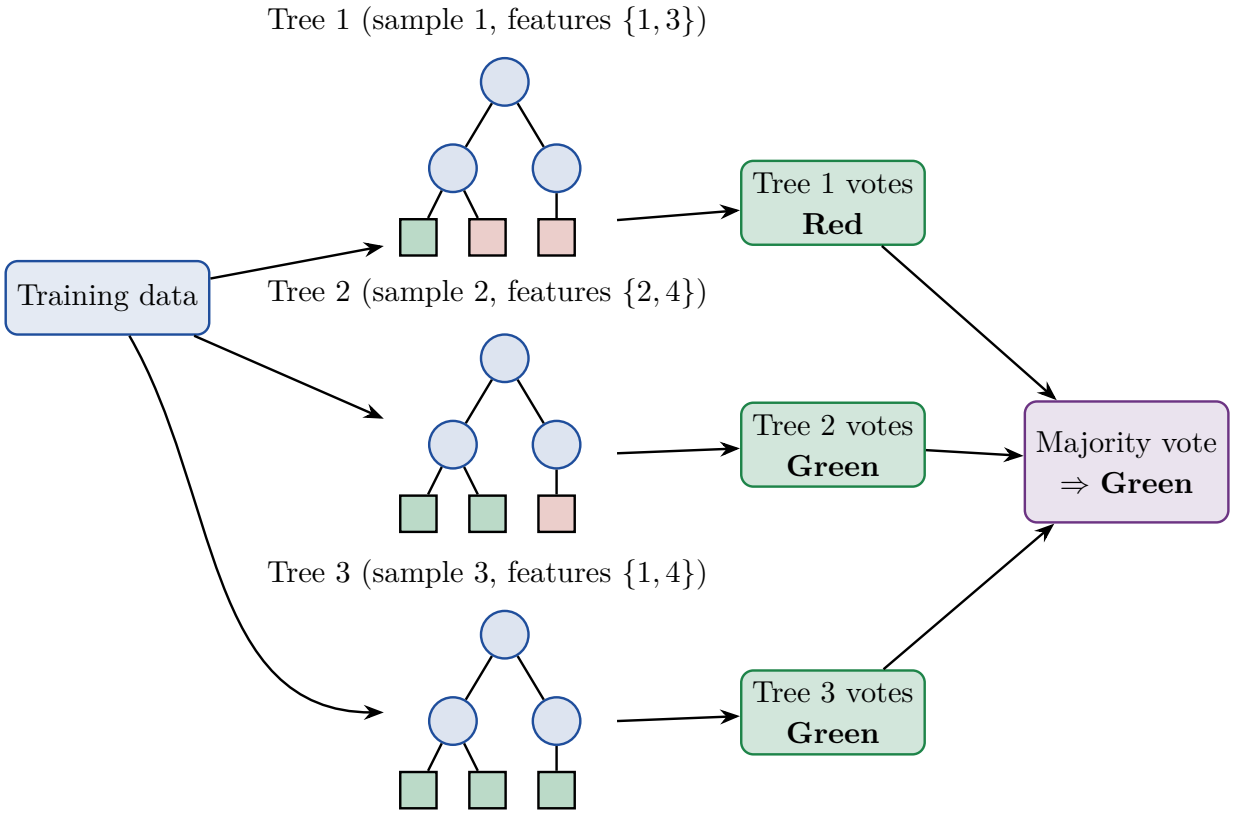


Figure 5.5. A Random Forest of three trees. Each tree sees a different bootstrap sample and different candidate features.

Reading the Figure

The training data (left) is turned into **three different trees**. Each tree is trained on its *own bootstrap sample* and, at each split, can only choose from *its own random features* (shown in brackets), so the trees have different shapes and make different mistakes. The small squares at the bottom of each tree are **leaves**: red or green squares show the class that leaf predicts. For a new instance each tree sends it down to a leaf and **votes** for that leaf's class: Tree 1 says Red; Trees 2 and 3 say Green. The forest takes the **majority**: **Green**.

Example: Voting in a forest of five trees

A forest has 5 trees. For a new patient the trees predict: Disease, Healthy, Disease, Disease, Healthy.

	Disease	**Healthy**
Votes	3	2
Estimated probability	$3/5 = 0.60$	$2/5 = 0.40$

The forest predicts **Disease** (with a "confidence" of 60%). For regression the five tree outputs would be averaged.

5.4.1 Feature Importance

A great by-product: a forest tells us **which features matter**. A feature's **importance** is the average total decrease in impurity (Gini or entropy) from all splits that use it, averaged over all trees and normalised so the importances sum to 1.

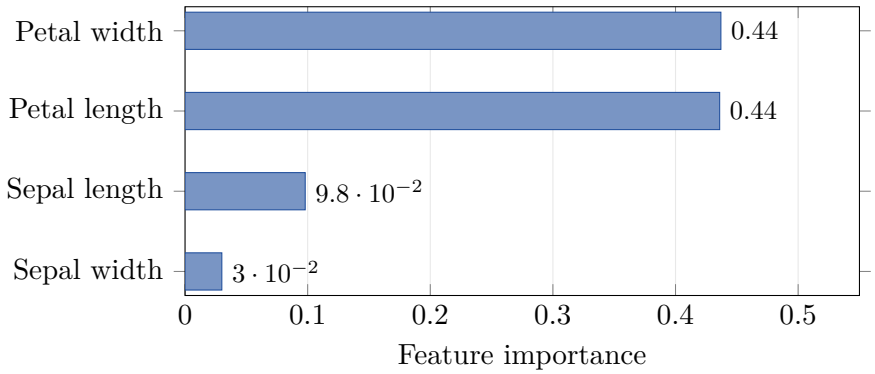


Figure 5.6. Feature importances of a Random Forest (200 trees) on the Iris dataset.

Reading the Figure

Each bar is one of the four measurements of an iris flower; a **longer bar means the forest relied on that feature more**. The two **petal** measurements together account for about 87% of the importance ($0.436 + 0.437$), while the *sepal width* contributes only 3%. This agrees with the single tree of Chapter 3, which separated all three species using only the petal width. Such a ranking helps in **feature selection** and in explaining the model to non-experts.

Main hyperparameters.

- `n_estimators`: number of trees (more is better but slower; 100–500 is typical).

- `max_features`: features tried at each split (default $\sqrt{p}$).
- `max_depth`, `min_samples_leaf`: limit the size of each tree.

Extra-Trees. *Extremely Randomised Trees* go one step further and also choose the *split thresholds* at random: faster to train and even lower variance, at the price of a little more bias.

Random forests

Advantages

- High accuracy; handles many features and mixed data types
- Much less overfitting than a single tree
- Gives feature importance and OOB error; needs little tuning

Disadvantages

- Less interpretable than one tree; larger memory
- Slower prediction with many trees
- May be biased towards features with many categories

Applications

- Credit-risk and churn prediction
- Medical diagnosis, bioinformatics, remote sensing
- Feature selection

5.5 Boosting

Bagging trains models *in parallel* to reduce variance. **Boosting** trains them **one after another**, each model trying to correct the mistakes of the previous ones. It reduces **bias**.

Definition: Boosting

Boosting is a sequential ensemble method that combines many *weak learners* into a strong learner by training them in sequence, where each new learner focuses on the instances that the earlier learners handled badly, and the final prediction is a *weighted* combination of all learners.

Key Idea

Analogy. A student takes a test, then studies *only the questions answered wrongly*, then takes another test, and so on. After several rounds the student has covered the hard questions that a single reading would have missed.

5.5.1 AdaBoost (Adaptive Boosting)

AdaBoost keeps a **weight** on every training instance. After each round it **increases the weights of misclassified instances** so the next learner pays more attention to them. Labels are coded $y_i \in \{-1, +1\}$.

Definition: AdaBoost algorithm

1. Initialise weights $w_i = \dfrac{1}{n}$ for all n instances.
2. For $t = 1, \dots, T$:
   (a) Train a weak learner h_t on the *weighted* data.
   (b) Weighted error: $\varepsilon_t = \sum_{i:\, h_t(\boldsymbol{x}_i) \neq y_i} w_i$.
   (c) Learner weight ("say"): $\alpha_t = \dfrac{1}{2} \ln \dfrac{1 - \varepsilon_t}{\varepsilon_t}$.
   (d) Update weights: $w_i \leftarrow w_i \, e^{-\alpha_t \, y_i \, h_t(\boldsymbol{x}_i)}$, then divide by the sum so that $\sum w_i = 1$. (Correct instances are multiplied by $e^{-\alpha_t} < 1$, wrong ones by $e^{+\alpha_t} > 1$.)
3. Final classifier: $H(\boldsymbol{x}) = \operatorname{sign}\Big(\sum_{t=1}^{T} \alpha_t \, h_t(\boldsymbol{x}) \Big)$.

Meaning of α_t. A learner with small error gets a **large** weight, one with error 0.5 (random guessing) gets $\alpha = 0$, and one with error above 0.5 would get a negative weight.

Error ε_t	0.05	0.10	0.20	0.30	0.50
$\alpha_t = \frac{1}{2} \ln \frac{1-\varepsilon_t}{\varepsilon_t}$	1.47	1.10	0.69	0.42	0

End-to-end example: five instances, three rounds

True labels $y = (+1, +1, -1, -1, +1)$ for instances $1, \dots, 5$. We use weak learners that are each wrong on only one instance.

Example: AdaBoost by hand

Round 1. Weights $w = (0.2, 0.2, 0.2, 0.2, 0.2)$.
Weak learner h_1 predicts $(+, +, -, +, +)$: it is wrong only on **instance 4**.

$$\varepsilon_1 = 0.2, \qquad \alpha_1 = \tfrac{1}{2} \ln \tfrac{0.8}{0.2} = \tfrac{1}{2} \ln 4 = 0.693.$$

Update: correct instances $\times e^{-0.693} = \times 0.5$; instance 4 $\times e^{0.693} = \times 2$.

$$(0.1,\ 0.1,\ 0.1,\ 0.4,\ 0.1) \xrightarrow{\text{sum}=0.8} w = (0.125,\ 0.125,\ 0.125,\ \mathbf{0.5},\ 0.125).$$

Round 2. Instance 4 now carries half of the total weight, so h_2 must get it right. Say h_2 predicts $(-, +, -, -, +)$: wrong only on **instance 1**.

$$\varepsilon_2 = 0.125, \qquad \alpha_2 = \tfrac{1}{2} \ln \tfrac{0.875}{0.125} = \tfrac{1}{2} \ln 7 = 0.973.$$

Correct $\times e^{-0.973} = 0.378$; instance 1 $\times e^{0.973} = 2.646$. After normalising:

$$w = (\mathbf{0.500},\ 0.071,\ 0.071,\ 0.286,\ 0.071).$$

Round 3. Now instance 1 dominates. Say h_3 predicts $(+,+,+,-,+)$: wrong only on **instance 3**.

$$\varepsilon_3 = 0.071, \qquad \alpha_3 = \tfrac{1}{2}\ln\tfrac{0.929}{0.071} = \tfrac{1}{2}\ln 13 = 1.282.$$

Final ensemble: $H(\boldsymbol{x}) = \operatorname{sign}\big(0.693\,h_1 + 0.973\,h_2 + 1.282\,h_3\big)$.

Instance	y	h_1	h_2	h_3	Score	H	Correct?
1	+	+	−	+	$0.693 - 0.973 + 1.282 = +1.00$	+	✓
2	+	+	+	+	$+2.95$	+	✓
3	−	−	−	+	$-0.693 - 0.973 + 1.282 = -0.38$	−	✓
4	−	+	−	−	$0.693 - 0.973 - 1.282 = -1.56$	−	✓
5	+	+	+	+	$+2.95$	+	✓

Conclusion: each weak learner made a mistake, but the weighted ensemble classifies *all five* instances correctly.

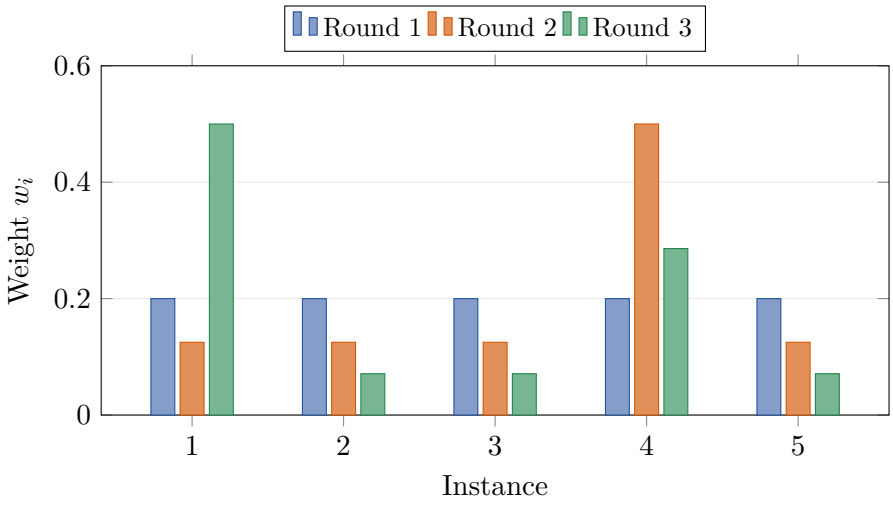


Figure 5.7. How AdaBoost's instance weights change from round to round in the worked example.

Reading the Figure

Each group of bars is one training instance; the three bars show its weight at the start of Round 1 (blue), Round 2 (orange) and Round 3 (green).

- **Round 1 (blue):** all instances are equal (0.2).
- **Round 2 (orange):** instance 4, which learner 1 got wrong, has jumped to 0.5.
- **Round 3 (green):** instance 1, which learner 2 got wrong, now has the biggest weight (0.5) while instance 4 (now handled) has dropped back to 0.286.

The **spotlight moves** to whichever instances are still being misclassified. This is how

boosting makes later learners specialise in the hard cases.

AdaBoost

Advantages

- Turns weak learners into a strong one; reduces bias
- Few hyperparameters; simple to implement

Disadvantages

- Sensitive to noisy data and outliers (they keep gaining weight)
- Sequential, so it cannot be parallelised

Applications

- Face detection (Viola–Jones)
- Text and customer classification

5.5.2 Gradient Boosting

Gradient boosting generalises the idea. Instead of re-weighting instances it fits each new model to the **residuals** (errors) of the current ensemble.

Definition: Gradient boosting (regression with squared loss)

1. Start with a constant prediction $F_0(\boldsymbol{x}) = \bar{y}$ (the mean).
2. For $m = 1, \dots, M$:
   (a) Compute the **residuals** $r_i = y_i - F_{m-1}(\boldsymbol{x}_i)$ (they are the negative gradient of the squared-error loss).
   (b) Fit a small tree h_m to predict the residuals.
   (c) Update: $F_m(\boldsymbol{x}) = F_{m-1}(\boldsymbol{x}) + \eta\, h_m(\boldsymbol{x})$, where $\eta \in (0, 1]$ is the **learning rate** (shrinkage).
3. The final model is $F_M(\boldsymbol{x}) = \bar{y} + \eta \sum_{m=1}^{M} h_m(\boldsymbol{x})$.

Example: Gradient boosting on four points

Predict a salary (in thousand rupees) from years of experience x:

x	1	2	3	4
y	10	12	20	22

Use a decision stump that splits at $x \le 2.5$ and $\eta = 0.5$.

Step 0. $F_0 = \bar{y} = \frac{10+12+20+22}{4} = 16$ for all x. Residuals $r = (-6, -4, 4, 6)$; MSE $= \frac{36+16+16+36}{4} = \mathbf{26}$.

Step 1. The stump predicts the mean residual on each side: left $\frac{-6-4}{2} = -5$, right $\frac{4+6}{2} = 5$.

$$F_1 = F_0 + 0.5\, h_1 = (16 - 2.5,\ 16 - 2.5,\ 16 + 2.5,\ 16 + 2.5) = (13.5,\ 13.5,\ 18.5,\ 18.5).$$

New residuals $(10-13.5,\ 12-13.5,\ 20-18.5,\ 22-18.5) = (-3.5, -1.5, 1.5, 3.5)$; MSE $= \frac{12.25+2.25+2.25+12.25}{4} = \mathbf{7.25}$.

Step 2. Stump on the new residuals: left -2.5, right $+2.5$.

$$F_2 = F_1 + 0.5\, h_2 = (12.25,\ 12.25,\ 19.75,\ 19.75).$$

Residuals $(-2.25, -0.25, 0.25, 2.25)$; MSE $= \frac{5.06+0.06+0.06+5.06}{4} = \mathbf{2.56}$.

Stage	F_0	F_1	F_2
MSE	26.0	7.25	2.56

The error falls at every stage. Prediction for $x = 3$: $F_2(3) = 19.75$ (true value 20).

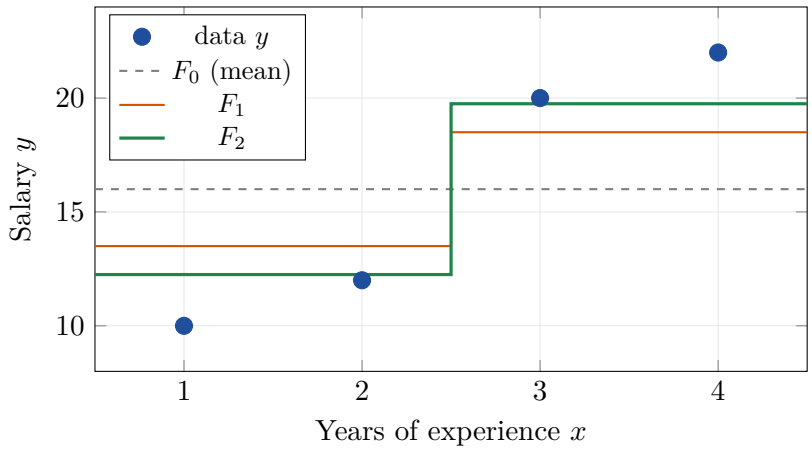


Figure 5.8. Gradient boosting: each stage moves the step-function prediction closer to the data.

Reading the Figure

The blue dots are the data. The **grey dashed line** is the starting guess F_0: the mean, a flat line that is wrong for every point. The **orange step function** F_1 moves the left points down and the right points up, getting closer. The **green step function** F_2 is closer still: its left step (12.25) is near the left points $(10, 12)$ and its right step (19.75) is near $(20, 22)$. Each stage *adds a small correction* to fix the remaining error. With many stages and deeper trees, the curve can follow complex patterns.

Learning rate and number of stages. A small η (e.g. 0.1) makes learning slower but usually generalises better; it must be balanced with the number of trees M (small η needs large M). Too many stages can **overfit**; use early stopping on a validation set. Popular fast implementations are **XGBoost**, **LightGBM** and **CatBoost**.

	AdaBoost	**Gradient boosting**
How errors are handled	Re-weights misclassified instances	Fits new learner to the residuals
Loss function	Exponential loss	Any differentiable loss
Combination	Weighted vote $\sum \alpha_t h_t$	Sum $F_0 + \eta \sum h_m$
Weak learner	Usually a stump	Small tree (depth 3–8)

Gradient boosting

Advantages

- Very high accuracy on structured/tabular data
- Works for regression, classification and ranking with any differentiable loss

Disadvantages

- Many hyperparameters to tune (learning rate, depth, number of trees)
- Can overfit noisy data; sequential training is slower

Applications

- Web search ranking, click-through-rate prediction
- Credit scoring and sales forecasting

5.6 Stacking

Voting uses a fixed rule (majority, average). **Stacking** (stacked generalisation) asks: *why not train a model to learn the best way to combine the predictions?*

Definition: Stacking

Stacking trains several different **base learners** (level-0) and then trains a **meta-learner** (level-1, also called the *blender*) whose *inputs are the predictions of the base learners* and whose output is the final prediction.

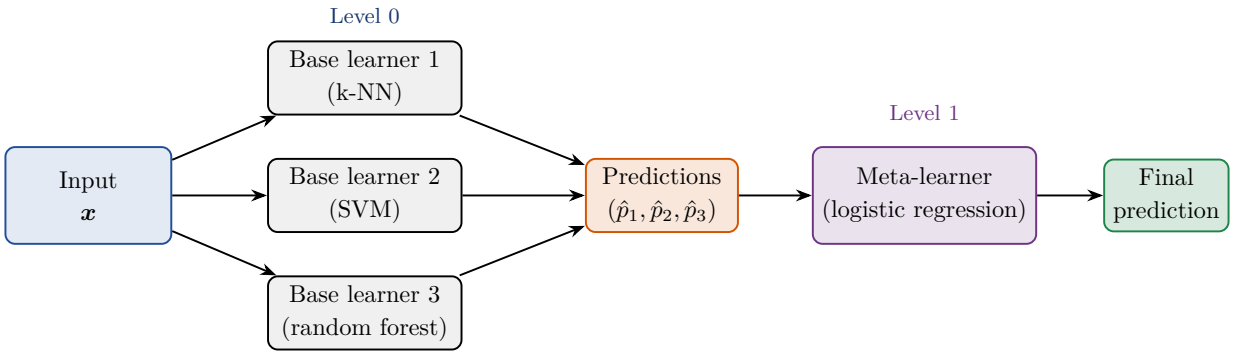


Figure 5.9. Architecture of a stacking ensemble.

Reading the Figure

The input goes to three **level-0** base learners (grey) that are different algorithms. Their three outputs are collected into a small table of predictions (orange), which becomes the *input features* of the **level-1 meta-learner** (purple). The meta-learner has *learned* how much to trust each base learner: for example it may learn that the SVM is most reliable and give it the largest weight. Its output is the final prediction (green).

Avoiding leakage. If the base learners predict on the *same data they were trained on*, their predictions are too good and the meta-learner learns to over-trust them. The standard fix is to use **out-of-fold predictions**: for each training instance use the prediction of a base model that was trained *without* that instance (cross-validation). A simpler variant, **blending**, holds out a separate validation set to train the meta-learner.

Example: A meta-learner in action

Three trained base models predict the probability that a customer will leave (churn) for one customer: $(0.80,\ 0.30,\ 0.60)$. A linear meta-learner has learned the weights $(0.5,\ 0.2,\ 0.3)$.

$$\hat{p} = 0.5(0.80) + 0.2(0.30) + 0.3(0.60) = 0.40 + 0.06 + 0.18 = \mathbf{0.64}.$$

Since $0.64 \geq 0.5$ the customer is predicted to **churn**. A simple average of the three would give 0.567; stacking gave more influence to the first model because the meta-learner found it most reliable.

Stacking

Advantages

- Learns the best combination automatically
- Often more accurate than any single model or simple voting

Disadvantages

- Complex, slow to train and tune
- Risk of information leakage and overfitting
- Hard to interpret

Applications

- Machine-learning competitions (e.g. the Netflix Prize)
- High-stakes prediction where small accuracy gains matter

5.7 Comparison and Practical Advice

	Bagging / Random Forest	**Boosting**	**Stacking**
Training	Parallel	Sequential	Two levels
Data used	Bootstrap samples	Re-weighted data / residuals	Out-of-fold predictions
Main effect	Reduces **variance**	Reduces **bias** (and some variance)	Learns best combination
Base learner	Strong, high variance (deep trees)	Weak, high bias (stumps, shallow trees)	Diverse algorithms
Combination	Vote / average	Weighted sum	Meta-model
Overfitting risk	Low	Higher (noisy data)	Moderate
Parallelisable?	Yes	No	Partly
Examples	Random Forest, Extra-Trees	AdaBoost, XGBoost, LightGBM	StackingClassifier

Try It Yourself

Compare single models and ensembles on the `moons` dataset (two interleaving half circles, with noise).

```
from sklearn.datasets import make_moons
from sklearn.model_selection import train_test_split
from sklearn.ensemble import (VotingClassifier, BaggingClassifier
    ,
        RandomForestClassifier, AdaBoostClassifier,
        GradientBoostingClassifier, StackingClassifier)
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.svm import SVC

X, y = make_moons(n_samples=500, noise=0.30, random_state=42)
Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.3,
                                      random_state=42)

lr = LogisticRegression()
rf = RandomForestClassifier(n_estimators=100, random_state=42)
sv = SVC(probability=True, random_state=42)

# soft voting
vote = VotingClassifier([("lr", lr), ("rf", rf), ("svc", sv)],
                        voting="soft").fit(Xtr, ytr)
print("soft voting:", vote.score(Xte, yte))

# bagging of 200 trees with out-of-bag score
```

```
bag = BaggingClassifier(DecisionTreeClassifier(), n_estimators=200,
      bootstrap=True, oob_score=True, random_state=42).fit(Xtr, ytr)
print("bagging:", bag.score(Xte, yte), "OOB:", bag.oob_score_)
```

Results on the 150 test points:

Model	Test accuracy	Model	Test accuracy
Single decision tree	0.853 (train: 1.000)	AdaBoost (200 stumps)	0.880
Logistic regression	0.860	Gradient boosting	0.907
SVM	0.907	Bagging (200 trees)	0.900 (OOB 0.891)
Hard voting	0.913	Pasting (200 trees)	0.900
Soft voting	**0.933**	Random Forest	0.900 (OOB 0.897)
Stacking	0.900		

What to notice: the single tree *memorises* the training set (train accuracy 1.0) but scores only 0.853 on test data; **bagging the same trees raises test accuracy to** 0.90; and the OOB score (0.891, 0.897) is close to the test accuracy without using the test set. *Honest note:* with only 150 test points one point is 0.67%, so differences of 1–2% between ensembles are within noise; we should not claim that one ensemble is always the best.

Common Mistake

Common mistakes with ensembles.

- Combining models that are all *the same* (no diversity) gives no gain.
- Evaluating a stacking model with base predictions made on the training data causes leakage.
- Using boosting on very noisy labels: the algorithm keeps chasing the noise.
- Assuming more trees always overfit: for Random Forests, more trees never hurt much, but for boosting too many stages can.

Chapter Summary

- **Ensemble learning** combines several diverse models; it works because errors partly cancel (three 70% classifiers give 78.4% by majority vote, five give 83.7%).
- **Voting:** hard (majority of labels) or soft (average of probabilities, usually better).

- **Bagging** (with replacement) and **pasting** (without) train the same learner on random subsets; Var $= \rho\sigma^2 + \frac{1-\rho}{B}\sigma^2$; OOB $\approx 36.8\%$ gives free validation.
- **Random Forest** = bagged trees + random feature subset at each split; gives feature importances.
- **AdaBoost:** re-weight misclassified instances; $\alpha_t = \frac{1}{2}\ln\frac{1-\varepsilon_t}{\varepsilon_t}$; final $H = \text{sign}\sum\alpha_t h_t$.
- **Gradient boosting:** fit each tree to the current residuals; $F_m = F_{m-1} + \eta h_m$.
- **Stacking:** a meta-learner learns to combine base predictions; use out-of-fold predictions.
- Bagging reduces **variance**; boosting reduces **bias**; stacking learns the combination.

Exam Corner

A. Very Short Answer Questions (2 marks each)

1. Define ensemble learning. **[2 marks]**
2. What is a weak learner? **[2 marks]**
3. Differentiate between hard voting and soft voting. **[2 marks]**
4. Differentiate between bagging and pasting. **[2 marks]**
5. What is a bootstrap sample? **[2 marks]**
6. What is an out-of-bag instance? What fraction of the data is OOB? **[2 marks]**
7. How does a Random Forest differ from bagging of decision trees? **[2 marks]**
8. What is feature importance in a Random Forest? **[2 marks]**
9. State the formula for the weight α_t of a learner in AdaBoost. **[2 marks]**
10. What does gradient boosting fit at each stage? **[2 marks]**
11. What is a meta-learner? **[2 marks]**
12. Which ensemble technique reduces variance and which reduces bias? **[2 marks]**

Model Answer: Q6, Q9 and Q12

Q6. When a bootstrap sample of size n is drawn with replacement, some instances are never selected; they are the *out-of-bag* instances of that sample. The chance that an instance is left out is $(1-1/n)^n \to e^{-1} \approx 0.368$, so about **36.8%** of the data is OOB for each model.
Q9. $\alpha_t = \dfrac{1}{2}\ln\dfrac{1-\varepsilon_t}{\varepsilon_t}$, where ε_t is the weighted error of the t-th weak learner.
Q12. Bagging (and Random Forests) mainly reduce *variance*; boosting mainly reduces *bias*.

B. Short Answer Questions (5 marks each)

1. Explain why ensemble methods improve accuracy. What is the role of diversity? **[5 marks]**
2. Explain hard and soft voting classifiers with an example. **[5 marks]**
3. Explain bagging with a diagram. How does out-of-bag evaluation work? **[5 marks]**
4. Differentiate between bagging and pasting. **[5 marks]**
5. Explain the Random Forest algorithm. Why do we choose a random subset of features at each split? **[5 marks]**
6. Explain the AdaBoost algorithm. **[5 marks]**
7. Explain gradient boosting with a suitable example. **[5 marks]**
8. Explain stacking with a neat diagram. **[5 marks]**
9. Differentiate between bagging and boosting. **[5 marks]**

Model Answer: B9 (comparison for 5 marks)

Basis	Bagging	Boosting
Training	Parallel; models independent	Sequential; each model depends on the previous
Data	Bootstrap samples	Re-weighted instances / residuals
Goal	Reduce variance	Reduce bias
Base learner	Strong, unstable (deep trees)	Weak (stumps)
Combination	Equal-weight vote/average	Weighted vote/sum
Noise	Robust	Sensitive
Example	Random Forest	AdaBoost, XGBoost

C. Long Answer Questions (10 marks each)

1. What is ensemble learning? Explain the different types of ensemble methods with diagrams. **[10 marks]**
2. Explain bagging, pasting and Random Forests. Discuss why they reduce variance. **[10 marks]**
3. Explain the AdaBoost algorithm in detail and work through a numerical example. **[10 marks]**
4. Explain gradient boosting and show with a numerical example how the error decreases over stages. **[10 marks]**
5. Explain voting classifiers (hard, soft, weighted) and stacking. Give examples. **[10 marks]**
6. Compare bagging, boosting and stacking in terms of training, bias–variance effect, advantages and disadvantages. **[10 marks]**

Model Answer: C3 (key steps of the numerical part)

Take 5 instances with equal weights 0.2. A weak learner misclassifies one instance: $\varepsilon_1 = 0.2$. $\alpha_1 = \frac{1}{2}\ln\frac{0.8}{0.2} = 0.693$. Correct instances: $0.2 \times e^{-0.693} = 0.1$; misclassified: $0.2 \times e^{0.693} = 0.4$. Sum $= 4(0.1) + 0.4 = 0.8$.
Normalised weights: correct $= 0.1/0.8 = 0.125$, misclassified $= 0.4/0.8 = 0.5$.
Next learner concentrates on the misclassified instance. Final model $H(\boldsymbol{x}) = \text{sign}\left(\sum \alpha_t h_t(\boldsymbol{x})\right)$.

D. Numerical Problems

1. Three independent classifiers each have accuracy 0.8. Find the accuracy of the majority vote. *(Answer:* $3(0.8)^2(0.2) + 0.8^3 = 0.384 + 0.512 = 0.896$*.)*

2. Three classifiers give $P(\text{class } 1) = (0.9,\ 0.45,\ 0.40)$. Find the hard- and soft-voting predictions. *(Answer: hard votes* $1, 0, 0 \Rightarrow$ *class* 0*; soft:* $\bar{P} = 0.583 \Rightarrow$ *class* 1*.)*

3. Using weights $(2, 1, 1)$ for the classifiers of problem 2, find the weighted soft-voting probability. *(Answer:* $\frac{2(0.9)+0.45+0.40}{4} = \frac{2.65}{4} = 0.6625 \Rightarrow$ *class* 1*.)*

4. For $n = 100$ compute the probability that a given instance is *not* in a bootstrap sample. *(Answer:* $(0.99)^{100} = 0.366$*; so* 63.4% *are in the sample.)*

5. $B = 20$ models each have variance 9 and pairwise correlation 0.3. Find the variance of their average and the limit as $B \to \infty$. *(Answer:* $0.3(9) + \frac{0.7(9)}{20} = 2.7 + 0.315 = 3.015$*; limit* $= 2.7$*.)*

6. Find α_t for $\varepsilon_t = 0.1$, 0.3 and 0.5 in AdaBoost. *(Answer:* $\frac{1}{2}\ln 9 = 1.099$*;* $\frac{1}{2}\ln\frac{7}{3} = 0.424$*;* 0*.)*

7. In AdaBoost with 10 instances (weights 0.1) a learner misclassifies 2. Find ε, α and the new normalised weights. *(Answer:* $\varepsilon = 0.2$*,* $\alpha = 0.693$*; correct* $0.05 \to 0.0625$ *each, wrong* $0.2 \to 0.25$ *each.)*

8. Gradient boosting: $y = (2, 4, 9, 11)$ at $x = 1, 2, 3, 4$. $F_0 = \bar{y}$; a stump splits at $x \le 2.5$; $\eta = 0.1$. Find F_1 and the MSE before and after. *(Answer:* $F_0 = 6.5$*; residuals* $(-4.5, -2.5, 2.5, 4.5)$*; stump* $= (-3.5, 3.5)$*;* $F_1 = (6.15, 6.15, 6.85, 6.85)$*; MSE* $13.25 \to 10.92$*.)*

9. A stacking meta-learner has weights $(0.5, 0.2, 0.3)$ and base predictions $(0.8, 0.3, 0.6)$. Find the final probability. *(Answer:* $0.40 + 0.06 + 0.18 = 0.64$*.)*

E. Multiple-Choice Questions

1. Combining the predictions of several models is called:
   (a) regularisation (b) ensemble learning (c) scaling (d) pruning

2. In soft voting the final class is chosen using:
   (a) majority of labels (b) average of predicted probabilities (c) random choice (d) the first classifier

3. Bagging samples the training data:
   (a) without replacement (b) with replacement (c) only once (d) never

4. The fraction of instances that are OOB for large n is about:
   (a) 10% (b) 25% (c) 36.8% (d) 63.2%

5. Random Forests choose, at each split:
   (a) all features (b) one fixed feature (c) a random subset of features (d) no feature

6. Boosting mainly reduces:
   (a) variance (b) bias (c) data size (d) noise

7. Bagging mainly reduces:
   (a) bias (b) variance (c) irreducible error (d) training time

8. In AdaBoost, a learner with error 0.5 gets weight $\alpha =$
   (a) 1 (b) $\ln 2$ (c) 0 (d) ∞

9. Gradient boosting fits each new tree to the:
   (a) original labels (b) residuals (c) random noise (d) features

10. The final model in stacking is called the:
    (a) base learner (b) meta-learner (c) weak learner (d) stump

11. Which method can be trained fully in parallel?
    (a) AdaBoost (b) Gradient boosting (c) Random Forest (d) all of these

12. Ensembles help most when members are:
    (a) identical (b) accurate and diverse (c) inaccurate (d) correlated

Answers: 1-b, 2-b, 3-b, 4-c, 5-c, 6-b, 7-b, 8-c, 9-b, 10-b, 11-c, 12-b.

F. Practice Problems

1. Compute by hand the accuracy of a majority vote of 7 independent classifiers each 60% accurate. *(Answer:* 0.710*, see Figure 5.1.)*

2. Continue the AdaBoost example: suppose a fourth learner has $\varepsilon_4 = 0.1$. Compute α_4 and explain how it changes the combined vote.

3. **Coding:** train a `RandomForestClassifier` on the *wine* dataset (`load_wine`), print the feature importances sorted, and compare accuracy with a single `DecisionTreeClassifier`.

4. **Coding:** plot test accuracy of `GradientBoostingClassifier` against `n_estimators` (1 to 300) for learning rates 0.5 and 0.05; find where it starts to overfit.

5. Draw a diagram of a stacking ensemble and label base learners, predictions and meta-learner.

Quick Revision

Topic	One-line recall
Ensemble	Many diverse models combined; better and more stable than one.
Majority vote	$P = \sum_{k>n/2} \binom{n}{k} p^k (1-p)^{n-k}$; 3 models of 70% $\to$ 78.4%.
Hard / soft voting	Majority of labels / average of probabilities (soft often better).
Bagging / pasting	Bootstrap (with replacement) / without replacement; vote or average.
OOB	$\approx 36.8\%$ left out of each bootstrap sample; free validation.
Variance of average	$\rho\sigma^2 + \frac{1-\rho}{B}\sigma^2$.
Random Forest	Bagged trees + random features per split ($\sqrt{p}$); importances.
AdaBoost	$\alpha = \frac{1}{2}\ln\frac{1-\varepsilon}{\varepsilon}$; $w \leftarrow we^{\mp\alpha}$; $H = \text{sign}\sum \alpha h$.
Gradient boosting	Fit residuals; $F_m = F_{m-1} + \eta h_m$.
Stacking	Meta-learner on out-of-fold base predictions.
Bias–variance	Bagging: variance $\downarrow$; boosting: bias $\downarrow$.

Chapter 6

Support Vector Machines and Naive Bayes Classifiers

Learning Objectives

After studying this chapter you will be able to:

- explain the **maximum-margin idea** of a linear SVM and the role of **support vectors**;
- explain the **soft margin** and the effect of the parameter C;
- use the **dual form** to show that an SVM depends only on dot products between points;
- explain the **kernel trick** and compute **polynomial** and **RBF** kernels by hand;
- describe **nonlinear SVM classification** and tune C and γ;
- explain **SVM regression** with the ε-tube and the ε-insensitive loss;
- apply **Gaussian**, **multinomial** and **Bernoulli Naive Bayes** classifiers, with Laplace smoothing, end to end;
- answer 2-, 5- and 10-mark university questions on Unit III.

> *Nothing is more practical than a good theory.*
>
> — Vladimir Vapnik, inventor of the SVM

This chapter completes Unit III. It has two halves:

- **Support Vector Machines (SVMs):** linear classification, nonlinear classification with kernels, and regression;
- **Naive Bayes classifiers:** the probabilistic family for text, medical and many other problems.

Throughout the chapter you will meet some cartoon characters who help us picture the ideas: a mayor who builds roads, a strict and a relaxed teacher, a detective who loves clues, and a tunnel full of data points.

6.1 Linear SVM Classification

6.1.1 The Idea: Build the Widest Road

Imagine two colonies of houses, the **Blue Colony** and the **Orange Colony**. The mayor wants to build a road between them so that the two colonies are kept apart. Many roads would do the job, but a good mayor builds the **widest road possible**, so that no house is ever too close to the traffic. The houses that end up right at the *edge* of the road are the ones that decide where the road can go.

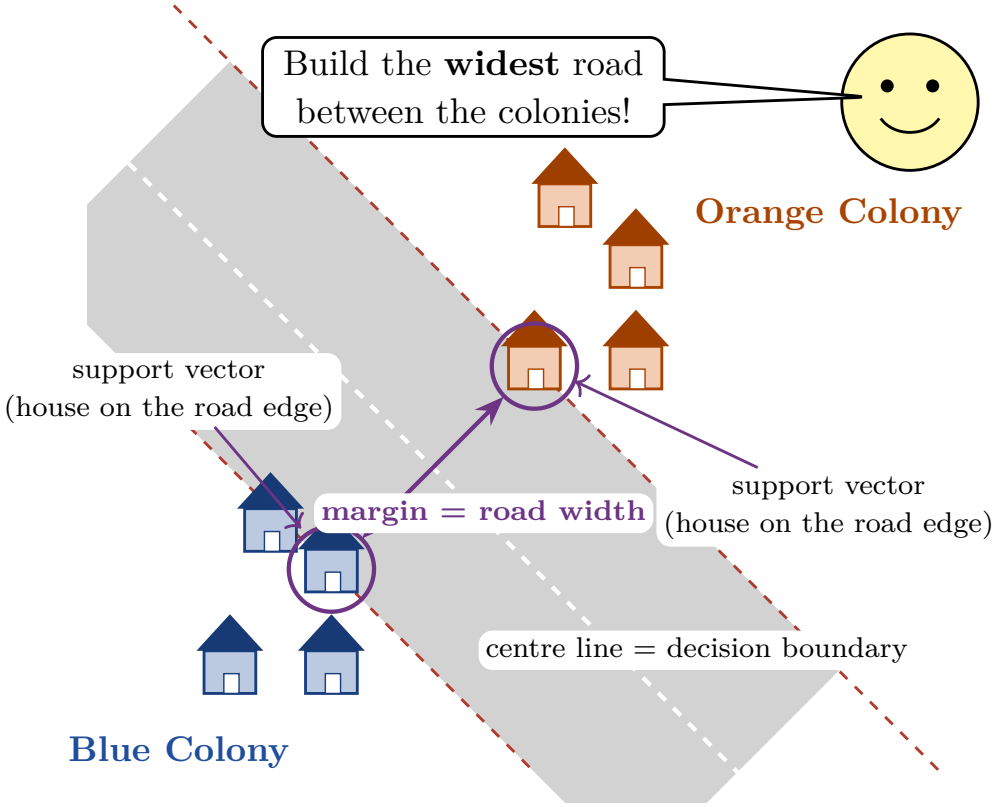


Figure 6.1. A cartoon of the maximum-margin classifier: the widest road between two colonies.

Reading the Figure

Houses are data points: blue houses are one class and orange houses are the other. The **grey road** is the margin and the **white dashed centre line** is the decision boundary. The **red dashed lines** are the edges of the road.

The two **purple circled houses** sit exactly on the edge of the road. They are the **support vectors**. If you moved or removed any *other* house (without letting it cross the edge) the road would stay exactly where it is; but if you moved a circled house, the whole road would shift. The **purple arrow** shows the road width, the **margin**, which the SVM tries to make as large as possible. A wide road means the classifier keeps a safe distance from both classes, so new houses (new data) are less likely to be misclassified.

6.1.2 Definitions

Definition: Support Vector Machine (SVM)

A **Support Vector Machine** is a supervised learning algorithm that classifies data by finding the **maximum-margin hyperplane**: the separating hyperplane $\boldsymbol{w} \cdot \boldsymbol{x} + b = 0$ that is *farthest* from the closest training points of both classes. With labels $y_i \in \{-1, +1\}$ the **hard-margin** problem is

$$\min_{\boldsymbol{w}, b} \ \tfrac{1}{2}\|\boldsymbol{w}\|^2 \qquad \text{subject to} \qquad y_i(\boldsymbol{w} \cdot \boldsymbol{x}_i + b) \geq 1 \quad \text{for all } i,$$

and a new point is classified by $\hat{y} = \operatorname{sign}(\boldsymbol{w} \cdot \boldsymbol{x} + b)$.

Definition: Margin and support vectors

The **margin** is the width $2/\|\boldsymbol{w}\|$ of the empty strip between the two parallel hyperplanes $\boldsymbol{w} \cdot \boldsymbol{x} + b = \pm 1$. The **support vectors** are the training points that lie *on* these hyperplanes (or, for a soft margin, inside them or on the wrong side); they alone determine $\boldsymbol{w}$ and b.

In Chapter 4 we solved a small SVM by hand. Here is a quick reminder of its answer, which we will reuse: for the spam data the support vectors were $(1, 1)$ with $y = -1$ and $(3, 3)$ with $y = +1$, giving $\boldsymbol{w} = (0.5, 0.5)$, $b = -2$ and margin $2/\|\boldsymbol{w}\| = 2.83$.

6.1.3 Soft-Margin SVM: Strict and Relaxed Teachers

Real data usually overlap, so no road can perfectly separate the colonies. A **soft margin** lets a few houses stand on the road or even on the wrong side, but each such house is fined.

Definition: Soft-margin SVM

Introduce a **slack variable** $\xi_i \geq 0$ for each point that measures how far it violates the margin, and solve

$$\min_{\boldsymbol{w}, b, \boldsymbol{\xi}} \ \tfrac{1}{2}\|\boldsymbol{w}\|^2 + C \sum_{i=1}^{n} \xi_i \qquad \text{s.t.} \qquad y_i(\boldsymbol{w} \cdot \boldsymbol{x}_i + b) \geq 1 - \xi_i, \quad \xi_i \geq 0.$$

The hyperparameter $C > 0$ controls the trade-off. Equivalently the SVM minimises the **hinge loss** $\max(0, 1 - y_i f(\boldsymbol{x}_i))$ plus the penalty $\frac{1}{2C}\|\boldsymbol{w}\|^2$.

Think of C as the **strictness of a teacher**.

- **Large C = strict teacher:** "No student may break the rules!" The road becomes narrow and twists to avoid every violation: low bias, *high variance*, risk of overfitting.
- **Small C = relaxed teacher:** "A few mistakes are fine." The road stays wide and smooth: higher bias, *low variance*, risk of underfitting.

(a) $C = 0.01$**: relaxed teacher wide road, margin** 5.0

(b) $C = 100$**: strict teacher narrow road, margin** 0.53

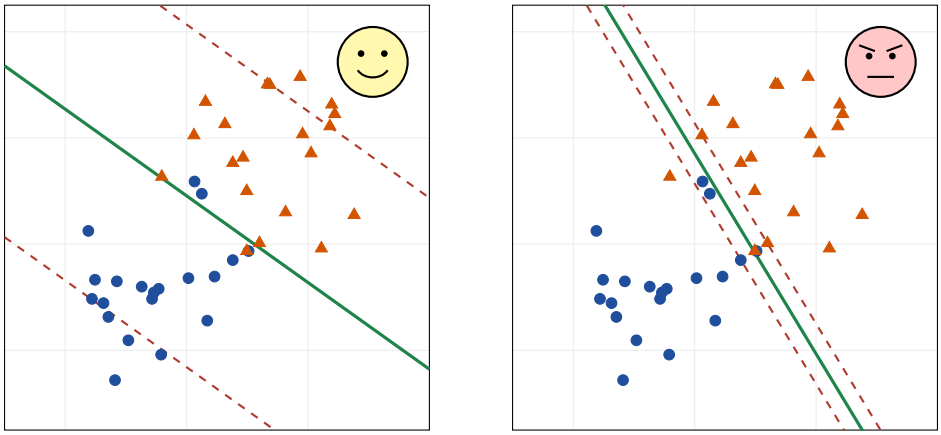


Figure 6.2. The same 40 points fitted with a very small and a very large C (real SVM fits).

Reading the Figure

Both panels show the *same* 40 houses (blue circles and orange triangles; the two colonies overlap a little in the middle). The **green line** is the decision boundary and the **red dashed lines** are the road edges.

(a) Relaxed teacher ($C = 0.01$): the road is very wide (margin 5.0) and covers many houses, so many of them are support vectors. The model pays little attention to the individual rule-breakers and looks only at the overall pattern.
(b) Strict teacher ($C = 100$): the road is narrow (margin 0.53) and the line is tilted to squeeze between the points, because every violation is expensive. The model follows the training data closely.

C	Support vectors	Margin	Train accuracy	Test accuracy (400 pts)
0.01 (relaxed)	32	5.00	0.925	0.953
1	11	1.63	0.900	0.948
100 (strict)	8	0.53	0.950	0.940

Read the table: as C grows, the margin shrinks ($5.00 \to 0.53$), fewer points are support vectors ($32 \to 8$) and the *training* accuracy rises slightly ($0.925 \to 0.950$) while the *test* accuracy drifts down ($0.953 \to 0.940$): the strict teacher fits the training set better but generalises a little worse. (The differences in accuracy are small here, since the two classes are only mildly overlapping and 400 test points is a small sample; the large change in margin and support vectors is the main lesson.)

6.1.4 The Dual Form and Support Vectors

The SVM optimisation problem can be rewritten in an equivalent *dual* form using Lagrange multipliers $\alpha_i \geq 0$, one per training point.

Definition: Dual form of the SVM

$$\max_{\boldsymbol{\alpha}} \sum_i \alpha_i - \frac{1}{2} \sum_i \sum_j \alpha_i \alpha_j \, y_i y_j \, (\boldsymbol{x}_i \cdot \boldsymbol{x}_j) \quad \text{s.t.} \quad \sum_i \alpha_i y_i = 0, \quad 0 \leq \alpha_i \leq C.$$

The weights are $\boldsymbol{w} = \sum_i \alpha_i y_i \boldsymbol{x}_i$ and a new point is classified by

$$f(\boldsymbol{x}) = \sum_i \alpha_i \, y_i \, (\boldsymbol{x}_i \cdot \boldsymbol{x}) + b.$$

Only the **support vectors** have $\alpha_i > 0$; all other points have $\alpha_i = 0$.

Two important facts:

1. The prediction needs only the *support vectors*, which makes it compact.
2. The data enter *only through dot products* $\boldsymbol{x}_i \cdot \boldsymbol{x}_j$. This is the key to the **kernel trick** in the next section.

Example: The dual form on the spam data

Support vectors: $(1, 1)$ with $y = -1$ and $(3, 3)$ with $y = +1$; all other points have $\alpha = 0$.

Solving for α: by symmetry $\alpha_1 = \alpha_2 = \alpha$. Then

$$\boldsymbol{w} = \alpha(-1)(1,1) + \alpha(+1)(3,3) = \alpha(2,2) = (0.5, 0.5) \;\Rightarrow\; \alpha = 0.25.$$

Check $\sum \alpha_i y_i = 0.25(-1) + 0.25(+1) = 0$ ✓. The bias is $b = -2$ as before.

Predicting the new e-mail $(4, 1)$ using only dot products with the support vectors:

$$f = \underbrace{0.25(-1)(1 \cdot 4 + 1 \cdot 1)}_{=-1.25} + \underbrace{0.25(+1)(3 \cdot 4 + 3 \cdot 1)}_{=+3.75} - 2 = \mathbf{0.5} > 0 \;\Rightarrow\; \text{spam}.$$

This matches the value $f(4, 1) = 0.5$ found in Chapter 4 directly from $\boldsymbol{w}$.

Dual objective value: $\sum \alpha_i - \frac{1}{2} \sum_{i,j} \ldots = 0.5 - 0.25 = 0.25 = \frac{1}{2}\|\boldsymbol{w}\|^2$, equal to the primal value: the primal and dual give the same answer.

6.1.5 Practical Points

- **Scale the features.** The margin is measured with distances, so a feature with a huge range (salary in rupees) would dominate one with a tiny range (age in years). Standardise first.
- **Choose C by cross-validation** on a grid such as $10^{-2}, 10^{-1}, \ldots, 10^{3}$.

- **More than two classes:** SVMs are binary. For K classes use *one-vs-rest* (K classifiers, pick the highest score) or *one-vs-one* ($K(K-1)/2$ classifiers that vote).
- **No probabilities by default.** The SVM output $f(\boldsymbol{x})$ is a signed distance, not a probability (scikit-learn can fit one afterwards with `probability=True`).

Linear SVM classification

Advantages

- Wide margin gives good generalisation
- Effective with many features (text)
- Only support vectors matter, so it is compact
- Convex problem: a single global optimum

Disadvantages

- No probabilities by default
- Needs scaled features
- Sensitive to the choice of C
- Slow for very large datasets
- Hard margin fails if classes overlap

Applications

- Spam and news-article classification
- Handwritten digit recognition
- Gene-expression and cancer classification
- Image classification

6.2 Nonlinear SVM Classification

6.2.1 When a Straight Line Is Not Enough

Many real problems cannot be solved by a straight line. For instance, if the "blue" houses are in the *middle* of a village and the "orange" houses are all around them, no straight road can separate them.

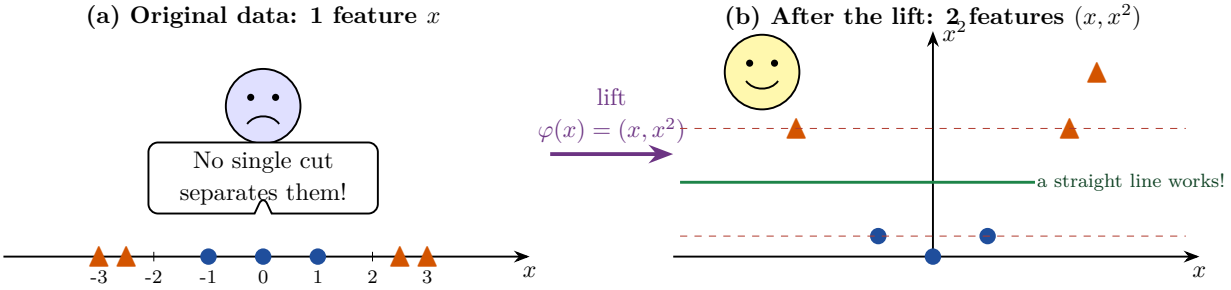


Figure 6.3. The lifting trick: data that cannot be separated in one dimension become separable after adding the feature x^2.

Reading the Figure

Panel (a): the data live on a single number line. The blue circles are in the middle ($x = -1, 0, 1$) and the orange triangles are on both outsides ($x = \pm 2.5, \pm 3$). Wherever we put *one* cut-point on the line, some blue and some orange end up on the same side. The sad face is right: no straight cut works.

Panel (b): we *lift* every point into two dimensions by adding the new feature x^2 (the purple arrow). A point $x = 3$ becomes $(3, 9)$, and $x = 1$ becomes $(1, 1)$. Now the orange

points have large x^2 values and the blue points have small ones, so a *horizontal straight line* (green) separates them perfectly, with red dashed lines marking the margin. Going back to the original line, this horizontal cut corresponds to *two* cut-points, $x \approx -1.9$ and $x \approx +1.9$: a **nonlinear** boundary in x is a **linear** boundary in the lifted space.

Key Idea

A linear model in a higher-dimensional feature space is a nonlinear model in the original space. The SVM can therefore learn curved boundaries by first mapping the data with a function φ and then finding the maximum-margin hyperplane there.

Example: Solving the lifted problem end to end

Data: $x = -1, 0, 1$ are class -1 (blue); $x = \pm 2.5, \pm 3$ are class $+1$ (orange). Map $\varphi(x) = (x, x^2)$.

x	-1	0	1	-2.5	2.5	-3	3
$\varphi(x) = (x, x^2)$	$(-1, 1)$	$(0, 0)$	$(1, 1)$	$(-2.5, 6.25)$	$(2.5, 6.25)$	$(-3, 9)$	$(3, 9)$
Class	$-$	$-$	$-$	$+$	$+$	$+$	$+$

Step 1: closest opposite points. In the lifted space the blue points $(\pm 1, 1)$ have $x^2 = 1$ and the orange points $(\pm 2.5, 6.25)$ have $x^2 = 6.25$. These four are the support vectors. The separating line is horizontal, $\boldsymbol{w} = (0, a)$.

Step 2: margin conditions. $a(6.25) + b = +1$ and $a(1) + b = -1$. Subtracting, $5.25\, a = 2$ so

$$a = 0.381, \qquad b = -1 - 0.381 = -1.381, \qquad f(x) = 0.381\, x^2 - 1.381.$$

Step 3: boundary. $f(x) = 0 \iff x^2 = 3.625 \iff x = \pm 1.90$. The margin is $2/0.381 = 5.25$ units in the x^2 direction.

Step 4: check the others. $f(0) = -1.381 \leq -1$ ✓; $f(\pm 3) = 0.381(9) - 1.381 = 2.05 \geq 1$ ✓.

Step 5: predict. $x = 2$: $f = 0.381(4) - 1.381 = 0.143 > 0 \Rightarrow$ **orange**. $x = 1.5$: $f = 0.381(2.25) - 1.381 = -0.524 < 0 \Rightarrow$ **blue**.

The cost of lifting. Adding more and more features (polynomial features of degree 3, 10, ...) quickly produces an enormous number of columns: d features and degree p give about $\binom{d+p}{p}$ new features. Computing with them directly becomes very slow. The **kernel trick** avoids this cost.

6.2.2 The Kernel Trick

Remember that the dual SVM uses the data only through dot products $\boldsymbol{x}_i \cdot \boldsymbol{x}_j$. After the lift we would need $\varphi(\boldsymbol{x}_i) \cdot \varphi(\boldsymbol{x}_j)$. A kernel computes that number *without ever building* φ.

Definition: Kernel function

A **kernel** is a function $K(\boldsymbol{a}, \boldsymbol{b})$ that equals a dot product in some (possibly very high-dimensional) feature space: $K(\boldsymbol{a}, \boldsymbol{b}) = \varphi(\boldsymbol{a}) \cdot \varphi(\boldsymbol{b})$. The **kernel trick** replaces every dot product $\boldsymbol{x}_i \cdot \boldsymbol{x}_j$ in the SVM by $K(\boldsymbol{x}_i, \boldsymbol{x}_j)$, so the SVM works in the high-dimensional space *at the cost of computing a simple formula*:

$$f(\boldsymbol{x}) = \sum_i \alpha_i \, y_i \, K(\boldsymbol{x}_i, \boldsymbol{x}) + b.$$

Kernel	Formula	Parameters
Linear	$K(\boldsymbol{a}, \boldsymbol{b}) = \boldsymbol{a} \cdot \boldsymbol{b}$	none
Polynomial	$K(\boldsymbol{a}, \boldsymbol{b}) = (\gamma \, \boldsymbol{a} \cdot \boldsymbol{b} + r)^d$	degree d, γ, r
RBF (Gaussian)	$K(\boldsymbol{a}, \boldsymbol{b}) = \exp(-\gamma \lVert \boldsymbol{a} - \boldsymbol{b} \rVert^2)$	$\gamma > 0$
Sigmoid	$K(\boldsymbol{a}, \boldsymbol{b}) = \tanh(\gamma \, \boldsymbol{a} \cdot \boldsymbol{b} + r)$	γ, r

Example: The polynomial kernel really equals a dot product

Let $\boldsymbol{a} = (1, 2)$ and $\boldsymbol{b} = (3, 1)$ and take the kernel $K(\boldsymbol{a}, \boldsymbol{b}) = (\boldsymbol{a} \cdot \boldsymbol{b})^2$.

Using the kernel (one cheap step): $\boldsymbol{a} \cdot \boldsymbol{b} = 3 + 2 = 5$, so $K = 5^2 = \mathbf{25}$.

Using the explicit feature map $\varphi(\boldsymbol{x}) = (x_1^2, \ \sqrt{2}\, x_1 x_2, \ x_2^2)$ (three features):

$$\varphi(\boldsymbol{a}) = (1, \ 2.828, \ 4), \qquad \varphi(\boldsymbol{b}) = (9, \ 4.243, \ 1),$$

$$\varphi(\boldsymbol{a}) \cdot \varphi(\boldsymbol{b}) = 1(9) + 2.828(4.243) + 4(1) = 9 + 12 + 4 = \mathbf{25}.$$

Both ways give 25, but the kernel never needed the three-dimensional vectors.

Example: The RBF kernel as a similarity score

For $\boldsymbol{a} = (1, 2)$ and $\boldsymbol{b} = (3, 1)$: $\lVert \boldsymbol{a} - \boldsymbol{b} \rVert^2 = 2^2 + 1^2 = 5$.

γ	0.1	0.5	2
$K = e^{-5\gamma}$	$e^{-0.5} = 0.607$	$e^{-2.5} = 0.082$	$e^{-10} = 0.00005$

Also $K(\boldsymbol{a}, \boldsymbol{a}) = e^0 = 1$ (identical points are maximally similar). The RBF kernel is a **similarity between** 0 **and** 1 that falls as the points move apart, and γ says *how fast*. The feature space of the RBF kernel has infinitely many dimensions, yet each K is one cheap

calculation.

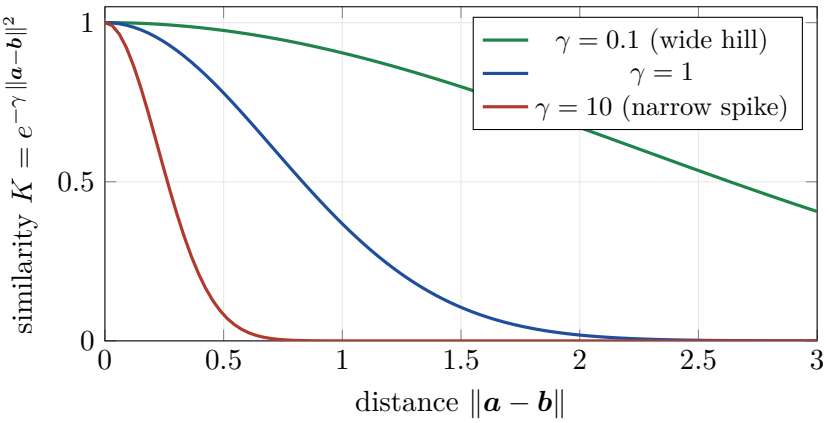


Figure 6.4. The RBF kernel as a function of distance, for three values of γ.

Reading the Figure

The horizontal axis is the **distance** between two points and the vertical axis is their **similarity** (1 = identical, 0 = completely unrelated). Each curve is a different γ.

- **Green ($\gamma = 0.1$):** a wide, gentle hill. Even points that are 2 units apart are still considered fairly similar. Every training point influences a large neighbourhood, so the model is *smooth* (risk of underfitting).
- **Blue ($\gamma = 1$):** a medium hill.
- **Red ($\gamma = 10$):** a narrow spike. Only points very close together are similar. Each training point influences just its immediate surroundings, so the boundary becomes wiggly and tightly follows the data (risk of overfitting).

6.2.3 How an RBF SVM Decides: Hills of Influence

Here is a cartoon way to see the RBF-kernel SVM. Each support vector builds a **hill of influence** around itself: a positive hill for an orange support vector (it "pulls" nearby points to the orange class) and a negative hill (a pit) for a blue one. The decision value $f(\boldsymbol{x})$ at any point is the *sum* of all the hills and pits there.

Reading the Figure

We work in one dimension for clarity, with one blue support vector at $x = 1$ and one orange at $x = 3$.

- The **orange curve** is the positive hill $e^{-(x-3)^2}$ centred on the orange support vector.
- The **blue curve** is the negative pit $-e^{-(x-1)^2}$ centred on the blue support vector.

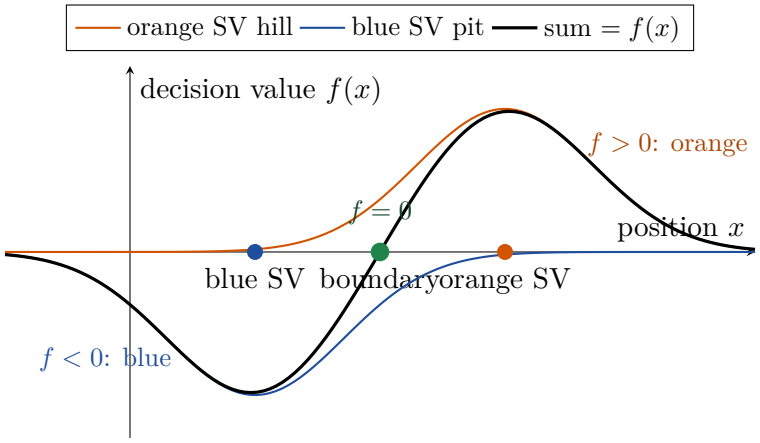


Figure 6.5. Hills of influence: two RBF support vectors ($\gamma = 1$) and the resulting decision value.

- The **black curve** is their sum, the decision value $f(x)$. It is positive on the right (the orange hill wins), negative on the left (the blue pit wins), and crosses **zero at** $x = 2$ (green dot), exactly midway, where both influences are equal.

In more dimensions it is the same picture: the decision boundary is where the hills and pits cancel, which can be a curve of any shape.

6.2.4 Choosing C and γ: Real Results

We trained RBF SVMs on the `moons` data (two interleaving half-circles with noise): 180 points to train and 120 to test.

γ	**Train accuracy**	**Test accuracy**	**Support vectors**	**Diagnosis**
0.1	0.839	0.867	79	too smooth: underfits
1	0.950	**0.992**	54	good
10	0.961	0.975	115	starting to overfit
100	**1.000**	0.850	173	overfits (memorises)

C ($\gamma = 1$)	**Train**	**Test**	**Kernel** ($C = 1$)	**Train**	**Test**
0.1	0.928	0.950	Linear	0.817	0.883
1	0.950	**0.992**	Polynomial (degree 3)	0.944	0.983
10	0.950	0.975	RBF	0.950	**0.992**
1000	0.967	0.933			

Interpretation. (i) With $\gamma = 100$ the training accuracy is a perfect 1.000 but the test accuracy falls to 0.850: the classic overfitting signature. (ii) Too small a γ underfits. (iii) Very large C

(1000) also overfits (0.967 train, 0.933 test). (iv) On this curved data the linear SVM is clearly worse (0.883) than the polynomial and RBF kernels. (Test accuracies are based on 120 points, so one point is 0.8%.)

Try It Yourself

Reproduce the experiment with `scikit-learn`:

```
from sklearn.datasets import make_moons
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline
from sklearn.svm import SVC

X, y = make_moons(n_samples=300, noise=0.25, random_state=0)
Xa, Xb, ya, yb = train_test_split(X, y, test_size=0.4,
                                  random_state=0)

for g in [0.1, 1, 10, 100]:
    model = make_pipeline(StandardScaler(),
                          SVC(kernel="rbf", gamma=g, C=1))
    model.fit(Xa, ya)
    print(g, model.score(Xa, ya), model.score(Xb, yb))
```

Challenge: replace the loop by `GridSearchCV` over $C \in \{0.1, 1, 10, 100\}$ and $\gamma \in \{0.1, 1, 10\}$ and report the best pair.

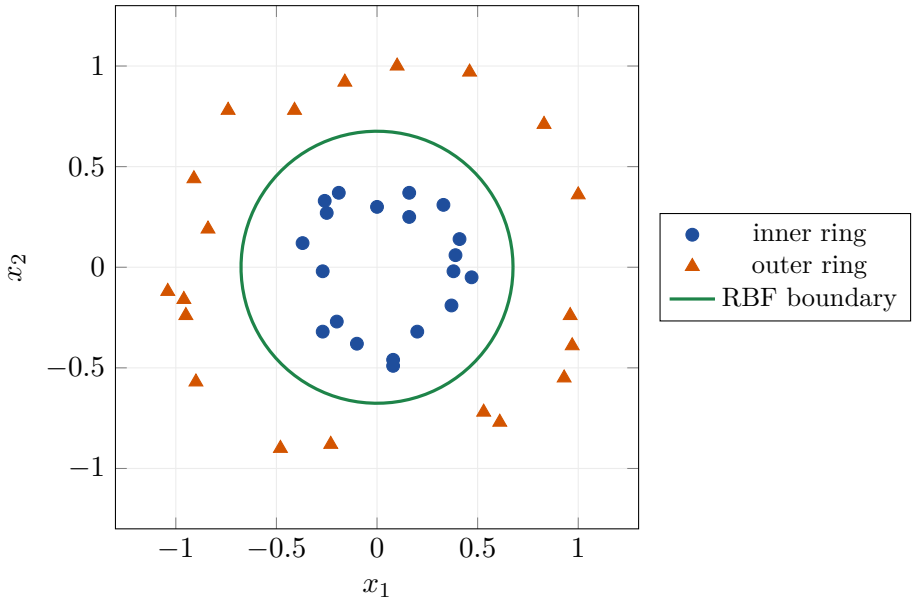


Figure 6.6. An RBF SVM separates two concentric rings that no straight line can (a real fit, 20 points of each class shown).

Reading the Figure

The blue circles form an **inner ring** and the orange triangles an **outer ring**. A straight line cannot separate them: a linear SVM got only 58% of these points right. The **RBF SVM** ($\gamma = 2$, $C = 10$) classified all of them correctly, and its decision boundary (green) is an almost perfect **circle of radius** ≈ 0.68 lying in the gap between the rings. The curved boundary was never specified by us: the kernel allowed the SVM to find it.

Key Idea

Choosing a kernel: start with a *linear* kernel (fast, good for many features and text). If the data look nonlinear, try the *RBF* kernel, which is a good default, and tune C and γ by cross-validation on *scaled* features. Use a polynomial kernel when you believe interactions between features of a known degree matter.

Nonlinear (kernel) SVM

Advantages

- Learns nonlinear boundaries without building features by hand
- Flexible: many kernels
- Works well in high dimensions
- Strong generalisation with the margin

Disadvantages

- Needs careful tuning of C, γ and the kernel
- Training cost grows roughly as n^2 to n^3: slow for $n \gg 10^5$
- Hard to interpret
- Needs feature scaling

Applications

- Face and object recognition
- Handwritten digit recognition
- Protein and DNA classification
- Remote-sensing image classification

6.3 SVM Regression

6.3.1 The Idea: Points Inside the Tunnel Are Free

The SVM idea also works for **regression**. In classification we wanted a road that keeps points *off* the road. In SVM regression we reverse it: we fit a **tunnel** (tube) around the regression line and try to get **as many points as possible inside**. Points inside the tunnel cost nothing; only points *outside* are penalised.

Definition: ε-insensitive loss

$$L_\varepsilon(y, f(\boldsymbol{x})) = \max\left(0,\ |y - f(\boldsymbol{x})| - \varepsilon\right).$$

Errors smaller than ε are ignored (loss 0); larger errors cost the amount *by which they exceed* ε.

Definition: Support vector regression (SVR)

SVR fits a function $f(\boldsymbol{x}) = \boldsymbol{w} \cdot \boldsymbol{x} + b$ (or its kernelised version) that is as *flat* as possible while keeping most training points within a tube of half-width ε:

$$\min_{\boldsymbol{w},b,\boldsymbol{\xi},\boldsymbol{\xi}^*} \ \tfrac{1}{2}\|\boldsymbol{w}\|^2 + C\sum_i(\xi_i + \xi_i^*) \quad \text{s.t.} \quad \begin{cases} y_i - f(\boldsymbol{x}_i) \le \varepsilon + \xi_i \\ f(\boldsymbol{x}_i) - y_i \le \varepsilon + \xi_i^* \\ \xi_i, \xi_i^* \ge 0. \end{cases}$$

The slack variables ξ_i, ξ_i^* measure how far a point lies above or below the tube. The **support vectors** are the points on or outside the tube; points strictly inside have no influence. With a kernel the prediction is $f(\boldsymbol{x}) = \sum_i(\alpha_i - \alpha_i^*)K(\boldsymbol{x}_i, \boldsymbol{x}) + b$.

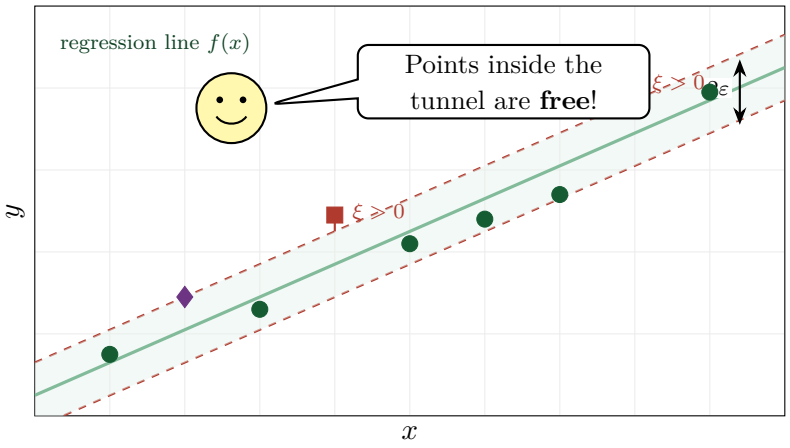


Figure 6.7. Support vector regression: a tube of half-width ε around the fitted line.

Reading the Figure

The **green line** is the fitted regression function and the **red dashed lines** are the tube edges, a distance ε above and below the line (total width 2ε).

- The **green dots** lie *inside* the tunnel. Their errors are smaller than ε, so they cost nothing and *do not affect* the line at all (they are not support vectors).
- The **red squares** lie *outside* the tunnel. They are penalised by the distance ξ to the nearest tube edge (the short red segments). They are support vectors.
- The **purple diamond** sits exactly on the tube edge and is also a support vector.

A wider tunnel (large ε) includes more points, gives a simpler model and fewer support vectors; a narrow tunnel follows the data more closely.

Example: The ε-insensitive loss

Let $\varepsilon = 0.5$. Three predictions:

	Point 1	Point 2	Point 3	Total
True y	3.0	4.1	6.5	
Predicted f	3.3	4.0	5.2	
Error $y - f$	-0.3	0.1	1.3	
$L_\varepsilon = \max(0, \lvert e\rvert - 0.5)$	0	0	0.8	**0.8**
Squared error e^2	0.09	0.01	1.69	1.79

Points 1 and 2 lie inside the tube and are *ignored*; only point 3 is charged ($1.3 - 0.5 = 0.8$). Ordinary squared error would have charged all three points.

6.3.2 Real Results

We fitted noisy data from $y = \sin x$ (60 points: 42 train, 18 test).

Model	Train R^2	Test R^2	Support vectors
Linear SVR	0.539	0.278	39
RBF SVR ($C = 10, \gamma = 1, \varepsilon = 0.1$)	**0.971**	**0.924**	26

ε (RBF SVR)	Support vectors	Test R^2	Comment
0.05	35	0.914	narrow tube, many support vectors
0.2	17	**0.942**	good balance
0.5	4	0.841	tube too wide: underfits

Read the tables. A straight line cannot follow the sine curve ($R^2 = 0.28$ on test data), while the RBF kernel can (0.92). Widening the tube from 0.05 to 0.5 reduces the number of support vectors from 35 to 4, but when the tube is too wide the fit gets worse.

Try It Yourself

```
import numpy as np
from sklearn.svm import SVR
from sklearn.model_selection import train_test_split

rng = np.random.default_rng(0)
```

```
x = np.sort(rng.uniform(0, 6, 60))
y = np.sin(x) + rng.normal(0, 0.15, 60)
xa, xb, ya, yb = train_test_split(x.reshape(-1, 1), y,
                                  test_size=0.3, random_state=0)

svr = SVR(kernel="rbf", C=10, gamma=1, epsilon=0.1).fit(xa, ya)
print(svr.score(xa, ya), svr.score(xb, yb), len(svr.support_))
```

Challenge: try `epsilon=0.3` and `gamma=10` and compare the train and test R^2.

Parameter	Meaning	Effect when increased
ε	Half-width of the tube	Fewer support vectors, simpler (flatter) model
C	Penalty for points outside the tube	Fits training data more closely (overfits)
γ (RBF)	Reach of each support vector	More wiggly curve (overfits)

SVM regression (SVR)

Advantages

- Ignores small errors, so robust to noise
- Sparse: only support vectors matter
- Nonlinear fits through kernels
- Good generalisation

Disadvantages

- Three parameters to tune (ε, C, γ)
- Slow for very large datasets
- Needs feature scaling
- No probabilistic prediction interval

Applications

- Stock and sales forecasting
- Electricity-load and weather prediction
- Estimating material or engineering properties
- Travel-time estimation

6.4 Naive Bayes Classifiers

In Chapter 3 we introduced Bayes' theorem and the Naive Bayes classifier. Here we study the three main *variants*, one for each type of feature, and solve a full text-classification example. First recall the principle.

Definition: Naive Bayes classifier

A **Naive Bayes classifier** predicts the class

$$\hat{y} = \arg\max_{c} \; P(c) \prod_{j=1}^{d} P(x_j \mid c),$$

where the "naive" assumption is that the features are *conditionally independent given the class*. It is a **generative** classifier: it models how each class generates the data, $P(\boldsymbol{x} \mid c)$,

and then uses Bayes' theorem to get $P(c \mid \boldsymbol{x})$.

6.4.1 The Detective Who Multiplies Clues

Think of the classifier as a **detective**. Each feature is a *clue*. The detective does not worry about how the clues are connected (the naive assumption); each clue is allowed to "speak" separately, and the detective **multiplies the votes** of all clues together with the prior suspicion.

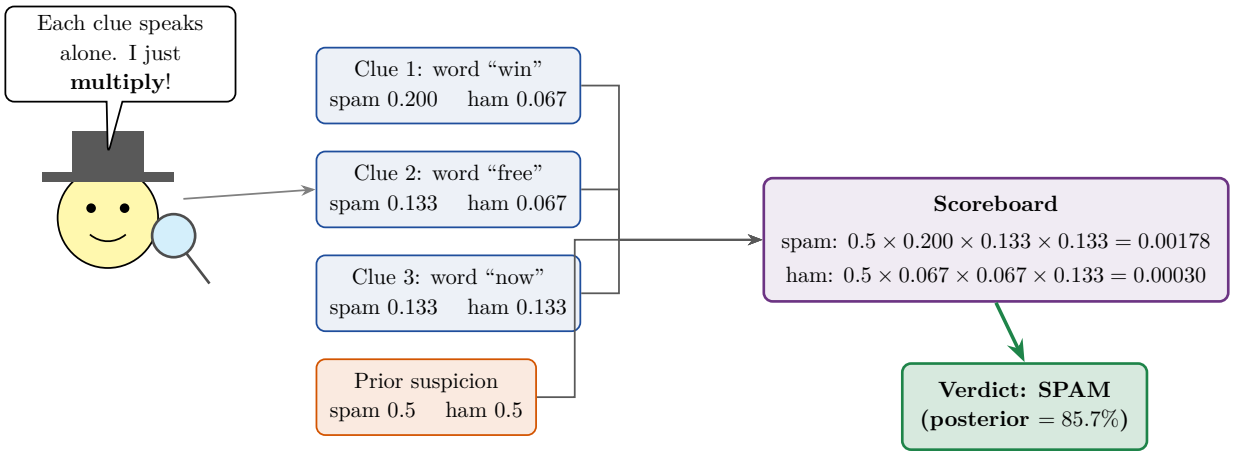


Figure 6.8. Naive Bayes as a detective: the evidence of each word is multiplied with the prior.

Reading the Figure

The detective is examining the message "**win free now**". The three blue cards are the clues, each giving how likely the word is under *spam* and under *ham* (probabilities estimated from a tiny training set, worked out in detail below). The orange card is the prior: before reading the message we think spam and ham are equally likely. The **purple scoreboard** multiplies the prior with each clue. Spam scores 0.00178 and ham only 0.00030, so the **verdict is spam**. The word "win" is the strongest clue because it is three times more likely in spam; the word "now" is no help at all (equal probability), so it simply cancels out.

6.4.2 Gaussian Naive Bayes (continuous features)

Definition: Gaussian Naive Bayes

For **continuous** features, $P(x_j \mid c)$ is modelled by a normal density with a *class-specific* mean μ_{jc} and variance σ_{jc}^2 estimated from the training data of class c:

$$P(x_j \mid c) = \frac{1}{\sqrt{2\pi}\,\sigma_{jc}} \exp\Big(- \frac{(x_j - \mu_{jc})^2}{2\sigma_{jc}^2} \Big).$$

Real-life example: ripe or unripe mango. A fruit seller measures the *sweetness score* and the *colour* of mangoes. From past data: ripe mangoes have sweetness $\sim \mathcal{N}(6, 1)$ and are yellow

with probability 0.9; unripe mangoes have sweetness $\sim \mathcal{N}(3,1)$ and are yellow with probability 0.2. Both classes are equally common.

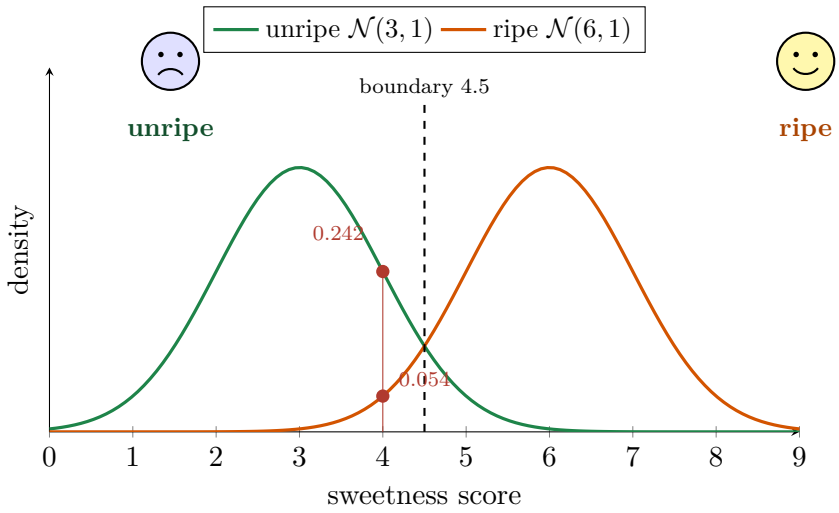


Figure 6.9. Class-conditional densities of sweetness for ripe and unripe mangoes (Gaussian Naive Bayes in one dimension).

Reading the Figure

Each bell curve shows how sweetness is distributed in one class: unripe mangoes cluster around 3 (green) and ripe ones around 6 (orange). The curves cross at 4.5, halfway between the means: with equal priors, a mango sweeter than 4.5 is classified ripe and a less sweet one unripe. For a mango of sweetness 4 (red dots) the unripe curve is higher (0.242) than the ripe curve (0.054), so the model leans towards *unripe*.

Example: Mango classification, one feature

A mango has sweetness $x = 4$. Equal priors 0.5 each.

$$P(4 \mid \text{ripe}) = \tfrac{1}{\sqrt{2\pi}}\, e^{-(4-6)^2/2} = 0.3989\, e^{-2} = 0.0540,$$

$$P(4 \mid \text{unripe}) = \tfrac{1}{\sqrt{2\pi}}\, e^{-(4-3)^2/2} = 0.3989\, e^{-0.5} = 0.2420.$$

$$P(\text{ripe} \mid x=4) = \frac{0.5(0.0540)}{0.5(0.0540)+0.5(0.2420)} = \frac{0.0540}{0.2960} = \mathbf{0.182} \;\Rightarrow\; \text{unripe}.$$

Effect of the prior. If ripe mangoes were four times as common (priors 0.8 ripe, 0.2 unripe): $P(\text{ripe} \mid x=4) = \frac{0.8(0.0540)}{0.8(0.0540)+0.2(0.2420)} = \frac{0.0432}{0.0916} = 0.472$, much closer to 0.5. A strong prior can move the decision.

Example: Mango classification, two features (sweetness and colour)

A mango has sweetness 4.8 and is **yellow**. Equal priors.

Quantity	Ripe	Unripe
Prior $P(c)$	0.5	0.5
$P(\text{sweetness} = 4.8 \mid c)$ (Gaussian)	$\varphi(-1.2) = 0.1942$	$\varphi(1.8) = 0.0790$
$P(\text{yellow} \mid c)$	0.9	0.2
Score $= P(c)\prod P(x_j \mid c)$	$0.5(0.1942)(0.9) = 0.0874$	$0.5(0.0790)(0.2) = 0.0079$

$$P(\text{ripe} \mid \text{data}) = \frac{0.0874}{0.0874 + 0.0079} = \mathbf{0.917} \;\Rightarrow\; \text{ripe}.$$

Teamwork of the clues: sweetness 4.8 alone is a moderately ripe-looking value ($P = 0.1942/(0.1942 + 0.0790) = 0.71$), and the yellow colour makes it much more convincing (0.917).

6.4.3 Multinomial Naive Bayes (word counts)

Definition: Multinomial Naive Bayes

For **count data** such as word counts in a document, a class c is described by a probability $P(w \mid c)$ for each word w in the vocabulary of size V. With Laplace smoothing α,

$$P(w \mid c) = \frac{N_{wc} + \alpha}{N_c + \alpha V},$$

where N_{wc} is the number of times word w appears in training documents of class c and N_c is the total number of word occurrences in class c. A document with words $w_1, \ldots, w_m$ is scored by $P(c)\prod_{k=1}^{m} P(w_k \mid c)$.

Real-life example: spam or ham SMS. A tiny training set of four messages:

Message	Class	Words
“win cash now”	spam	win, cash, now
“win free prize”	spam	win, free, prize
“meet me now”	ham	meet, me, now
“lunch with me”	ham	lunch, with, me

Example: Multinomial Naive Bayes, end to end

Step 1: priors. 2 spam and 2 ham messages: $P(\text{spam}) = P(\text{ham}) = 0.5$.

Step 2: counts. The vocabulary has $V = 9$ words: cash, free, lunch, me, meet, now, prize, win, with. Each class has $N_c = 6$ word occurrences.

Step 3: smoothed probabilities with $\alpha = 1$, so every denominator is $6 + 9 = 15$:

Word	win	cash	now	free	prize	meet	me	lunch	with
Count in spam	2	1	1	1	1	0	0	0	0
$P(w \mid \text{spam})$	$\frac{3}{15}$	$\frac{2}{15}$	$\frac{2}{15}$	$\frac{2}{15}$	$\frac{2}{15}$	$\frac{1}{15}$	$\frac{1}{15}$	$\frac{1}{15}$	$\frac{1}{15}$
Count in ham	0	0	1	0	0	1	2	1	1
$P(w \mid \text{ham})$	$\frac{1}{15}$	$\frac{1}{15}$	$\frac{2}{15}$	$\frac{1}{15}$	$\frac{1}{15}$	$\frac{2}{15}$	$\frac{3}{15}$	$\frac{2}{15}$	$\frac{2}{15}$

Step 4: classify "win free now".

$$\text{score}(\text{spam}) = 0.5 \cdot \tfrac{3}{15} \cdot \tfrac{2}{15} \cdot \tfrac{2}{15} = 0.5 \cdot \tfrac{12}{3375} = 0.001778,$$
$$\text{score}(\text{ham}) = 0.5 \cdot \tfrac{1}{15} \cdot \tfrac{1}{15} \cdot \tfrac{2}{15} = 0.5 \cdot \tfrac{2}{3375} = 0.000296.$$

$$P(\text{spam} \mid \text{message}) = \frac{12}{12+2} = \frac{6}{7} = \mathbf{0.857} \;\Rightarrow\; \textbf{spam}.$$

Step 5: classify "meet me now". spam: $0.5 \cdot \frac{1}{15} \cdot \frac{1}{15} \cdot \frac{2}{15} \to 2$ units; ham: $0.5 \cdot \frac{2}{15} \cdot \frac{3}{15} \cdot \frac{2}{15} \to 12$ units. So $P(\text{ham}) = \frac{12}{14} = 0.857$: **ham**.

Why smoothing matters. Without smoothing, $P(\text{win} \mid \text{ham}) = \frac{0}{6} = 0$, and any message containing "win" would get a ham score of exactly 0, whatever its other words. With $\alpha = 1$ it is $\frac{1}{15}$: small but not zero.

6.4.4 Bernoulli Naive Bayes (word present or absent)

Definition: Bernoulli Naive Bayes

For **binary** features $x_j \in \{0, 1\}$ (word present or absent), each feature has a probability $p_{jc} = P(x_j = 1 \mid c)$ and

$$P(x_j \mid c) = p_{jc}^{x_j}(1 - p_{jc})^{1-x_j}, \qquad p_{jc} = \frac{n_{jc} + \alpha}{N_c^{\text{docs}} + 2\alpha},$$

where n_{jc} is the number of documents of class c containing feature j. Unlike the multinomial model, it *explicitly uses the absence of a word* as evidence.

Example: Bernoulli Naive Bayes on the same messages

Each class has 2 documents, so $p_{jc} = \frac{n_{jc}+1}{4}$.

Word	win	cash	now	free	prize	meet	me	lunch	with
$P(\text{present} \mid \text{spam})$	0.75	0.5	0.5	0.5	0.5	0.25	0.25	0.25	0.25
$P(\text{present} \mid \text{ham})$	0.25	0.25	0.5	0.25	0.25	0.5	0.75	0.5	0.5

Classify "win free now": present words win, free, now; **absent** words cash, prize, meet, me, lunch, with.

$$\text{spam: } 0.5 \times \underbrace{0.75 \cdot 0.5 \cdot 0.5}_{\text{present}} \times \underbrace{0.5 \cdot 0.5 \cdot 0.75 \cdot 0.75 \cdot 0.75 \cdot 0.75}_{\text{absent}} = 0.01483,$$

$$\text{ham: } 0.5 \times \underbrace{0.25 \cdot 0.25 \cdot 0.5}_{\text{present}} \times \underbrace{0.75 \cdot 0.75 \cdot 0.5 \cdot 0.25 \cdot 0.5 \cdot 0.5}_{\text{absent}} = 0.00055.$$

$P(\text{spam}) = \dfrac{0.01483}{0.01483 + 0.00055} = \mathbf{0.964}$: spam, with an even higher confidence than the multinomial model, because the *absence* of ham-typical words (me, meet, lunch, with) is also evidence.

Variant	**Feature type**	**Models $P(x_j \mid c)$ by**	**Typical use**
Gaussian	Continuous measurements	Normal density per class	Medical measurements, sensors
Multinomial	Counts (word counts, frequencies)	Word probabilities with smoothing	Text classification, spam
Bernoulli	Binary (present/absent)	Presence probability per feature	Short texts, binary indicators

Try It Yourself

Confirm the hand calculations with `scikit-learn`:

```
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.naive_bayes import MultinomialNB, BernoulliNB

docs = ["win cash now", "win free prize", "meet me now",
        "lunch with me"]
labels = [1, 1, 0, 0]                          # 1 = spam, 0 = ham

cv = CountVectorizer(binary=False)
mnb = MultinomialNB(alpha=1).fit(cv.fit_transform(docs), labels)
print(mnb.predict_proba(cv.transform(["win free now"])))
# -> [[0.143 0.857]]   P(ham), P(spam)

cvb = CountVectorizer(binary=True)
bnb = BernoulliNB(alpha=1).fit(cvb.fit_transform(docs), labels)
```

```
print(bnb.predict_proba(cvb.transform(["win free now"])))
# -> [[0.036 0.964]]
```

The outputs, 0.857 (multinomial) and 0.964 (Bernoulli) for "spam", agree with the hand calculation.

6.4.5 Generative vs. Discriminative Classifiers

	Naive Bayes (generative)	**Logistic regression / SVM (discriminative)**
What is learned	$P(\boldsymbol{x} \mid c)$ and $P(c)$	$P(c \mid \boldsymbol{x})$ or the boundary directly
Training	Counting; very fast	Optimisation; slower
Data needed	Works with little data	Needs more data, but usually more accurate with plenty
Feature dependence	Assumed independent	Can use correlated features
Probabilities	Often over-confident	Logistic: better calibrated

Common Mistake

Naive Bayes often classifies well even when its probabilities are *wrong*. If the posterior says 0.999, do not trust that number literally: the independence assumption multiplies correlated evidence several times and makes the model over-confident. Use it to *rank* or *decide*, not to quote exact probabilities.

Naive Bayes classifiers

Advantages

- Extremely fast to train and predict
- Works with little training data and many features
- Handles multiple classes naturally
- Excellent baseline for text

Disadvantages

- Independence assumption is rarely true
- Probabilities over-confident
- Zero-frequency problem (needs smoothing)
- Cannot learn feature interactions

Applications

- Spam filtering and sentiment analysis
- News and document categorisation
- Medical diagnosis (Gaussian)
- Real-time prediction

6.5 Comparing the Methods of Unit III

	Kernel SVM	**Naive Bayes**	**Random Forest / Boosting**
Training speed	Slow for large n	**Very fast**	Moderate
Prediction speed	Fast (few support vectors)	**Very fast**	Moderate
Nonlinear?	Yes (kernel)	Mild	**Yes**
Needs scaling?	**Yes**	No	No
Probabilities?	Not directly	Yes (over-confident)	Yes
Works with little data	Yes	**Yes**	Less so
Interpretability	Low	Moderate	Low–moderate

Chapter Summary

- **Linear SVM:** the maximum-margin hyperplane; margin $2/\|\boldsymbol{w}\|$; support vectors lie on the road edge and alone determine the model.
- **Soft margin:** slack ξ_i and parameter C; large C = strict (narrow margin, overfit), small C = relaxed (wide margin, underfit).
- **Dual form:** $f(\boldsymbol{x}) = \sum \alpha_i y_i (\boldsymbol{x}_i \cdot \boldsymbol{x}) + b$; only support vectors have $\alpha_i > 0$; data appear only through dot products.
- **Nonlinear SVM:** lift the data with φ; the **kernel trick** replaces dot products by $K(\boldsymbol{a}, \boldsymbol{b}) = \varphi(\boldsymbol{a}) \cdot \varphi(\boldsymbol{b})$. Polynomial: $(\gamma \boldsymbol{a} \cdot \boldsymbol{b} + r)^d$; RBF: $e^{-\gamma \|\boldsymbol{a}-\boldsymbol{b}\|^2}$. Large γ or C overfits.
- **SVR:** ε-tube; loss $\max(0, |y - f| - \varepsilon)$; points inside the tube are free; support vectors are on or outside.
- **Naive Bayes:** $\hat{y} = \arg\max_c P(c) \prod_j P(x_j \mid c)$; Gaussian (continuous), multinomial (counts), Bernoulli (binary); use Laplace smoothing $\frac{N_{wc}+\alpha}{N_c+\alpha V}$.
- Worked results: SVM lifted boundary at $x = \pm 1.90$; RBF kernel $e^{-0.5 \cdot 5} = 0.082$; SMS "win free now" spam with 0.857 (multinomial) and 0.964 (Bernoulli); mango ripe with 0.917.

Exam Corner

A. Very Short Answer Questions (2 marks each)

1. Define a support vector machine. **[2 marks]**
2. What are support vectors? What is the margin? **[2 marks]**

3. What is the role of the parameter C in a soft-margin SVM? **[2 marks]**
4. Write the decision function of an SVM in dual form. **[2 marks]**
5. What is the kernel trick? **[2 marks]**
6. Write the formulas of the polynomial and RBF kernels. **[2 marks]**
7. What does a large γ do in the RBF kernel? **[2 marks]**
8. Define the ε-insensitive loss. **[2 marks]**
9. What is the effect of increasing ε in SVR? **[2 marks]**
10. State the independence assumption of Naive Bayes. **[2 marks]**
11. What is Laplace smoothing? **[2 marks]**
12. Name the three types of Naive Bayes and their feature types. **[2 marks]**

Model Answer: Q5, Q7 and Q9

Q5. The kernel trick is the replacement of every dot product $\boldsymbol{x}_i \cdot \boldsymbol{x}_j$ in the SVM by a kernel $K(\boldsymbol{x}_i, \boldsymbol{x}_j) = \varphi(\boldsymbol{x}_i) \cdot \varphi(\boldsymbol{x}_j)$. The SVM then works in a high-dimensional feature space without ever computing the mapping φ, which makes nonlinear classification possible at low cost.

Q7. A large γ makes the similarity fall off very quickly with distance: every training point influences only a tiny neighbourhood, producing a very wiggly decision boundary. It reduces bias but increases variance, so it *overfits*.

Q9. A larger ε gives a wider tube; more points fall inside, fewer are support vectors, and the model becomes simpler (flatter). If ε is too large the model underfits.

B. Short Answer Questions (5 marks each)

1. Explain the maximum-margin classifier and support vectors with a diagram. **[5 marks]**
2. Explain the soft-margin SVM. Discuss the effect of C. **[5 marks]**
3. Explain the dual form of the SVM and why it is important for kernels. **[5 marks]**
4. Why do we need nonlinear SVMs? Explain the lifting idea with an example. **[5 marks]**
5. Explain the kernel trick. Describe the linear, polynomial and RBF kernels. **[5 marks]**
6. Explain the effect of C and γ on an RBF SVM. **[5 marks]**
7. Explain SVM regression with the ε-tube. **[5 marks]**
8. Explain the Gaussian, multinomial and Bernoulli Naive Bayes classifiers. **[5 marks]**
9. Explain why Laplace smoothing is needed in Naive Bayes with an example. **[5 marks]**
10. Compare generative and discriminative classifiers. **[5 marks]**

Model Answer: B6 (outline for 5 marks)

Parameter C (penalty for margin violations): large $C \to$ few violations, narrow margin, complex boundary, *low bias, high variance* (overfit); small $C \to$ wide margin, simple boundary, *higher bias, low variance* (underfit).
Parameter γ (reach of each support vector in $e^{-\gamma\|a-b\|^2}$): large $\gamma \to$ narrow influence, wiggly boundary, overfit; small $\gamma \to$ wide influence, smooth boundary, underfit.
Example: on the moons data ($\gamma = 0.1, 1, 100$) test accuracy was 0.867, 0.992, 0.850 while training accuracy rose to 1.0 at $\gamma = 100$. Both parameters are chosen by cross-validation on scaled features.

C. Long Answer Questions (10 marks each)

1. Explain linear SVM classification: the margin, the optimisation problem, support vectors, soft margin and the dual form. Solve a small numerical example. **[10 marks]**
2. Explain nonlinear SVM classification. Describe the kernel trick with the polynomial and RBF kernels, including a numerical verification that $K(\boldsymbol{a}, \boldsymbol{b}) = \varphi(\boldsymbol{a}) \cdot \varphi(\boldsymbol{b})$. **[10 marks]**
3. Explain SVM regression: the ε-insensitive loss, the optimisation problem, support vectors and the roles of ε, C and γ. **[10 marks]**
4. Explain the Naive Bayes classifier. Classify a message using the multinomial model with Laplace smoothing for a given training set. **[10 marks]**
5. Describe the Gaussian, multinomial and Bernoulli Naive Bayes classifiers with formulas, examples, advantages and disadvantages. **[10 marks]**
6. Compare SVM, Naive Bayes and ensemble methods (Random Forest / boosting) for classification. **[10 marks]**

Model Answer: C2 (numerical verification)

Take $\boldsymbol{a} = (1, 2)$, $\boldsymbol{b} = (3, 1)$ and $K(\boldsymbol{a}, \boldsymbol{b}) = (\boldsymbol{a} \cdot \boldsymbol{b})^2$.
Via the kernel: $\boldsymbol{a} \cdot \boldsymbol{b} = 1(3) + 2(1) = 5$, $K = 25$.
Via the feature map $\varphi(\boldsymbol{x}) = (x_1^2, \sqrt{2}x_1x_2, x_2^2)$: $\varphi(\boldsymbol{a}) = (1, 2\sqrt{2}, 4)$, $\varphi(\boldsymbol{b}) = (9, 3\sqrt{2}, 1)$; $\varphi(\boldsymbol{a}) \cdot \varphi(\boldsymbol{b}) = 9 + 2\sqrt{2} \cdot 3\sqrt{2} + 4 = 9 + 12 + 4 = 25$.
Both give 25, so the kernel equals the dot product in the 3-D feature space while needing only one 2-D dot product.

D. Numerical Problems

1. An SVM has $\boldsymbol{w} = (2, 1)$, $b = -4$. Classify $(1, 1)$ and $(3, 2)$ and find the margin. *(Answer:* $f(1, 1) = -1 \Rightarrow -1$ *(on the margin);* $f(3, 2) = 4 \Rightarrow +1$*; margin* $= 2/\sqrt{5} = 0.894$*.)*

2. In 1-D the support vectors are $x = 1$ ($y = -1$) and $x = 3$ ($y = +1$). Find w, b, the margin and the dual values α_i. *(Answer:* $w + b = -1$, $3w + b = 1 \Rightarrow w = 1$, $b = -2$*; margin* $= 2$*;* $w = \sum \alpha_i y_i x_i = 3\alpha - \alpha = 2\alpha = 1 \Rightarrow \alpha = 0.5$ *each; check* $\sum \alpha_i y_i = 0$*.)*

3. Using the dual weights $\alpha = 0.5$ at $x = 1$ ($y = -1$) and $x = 3$ ($y = +1$) with $b = -2$, compute $f(2.5)$. *(Answer:* $0.5(-1)(1 \cdot 2.5) + 0.5(1)(3 \cdot 2.5) - 2 = -1.25 + 3.75 - 2 = 0.5 > 0$*.)*

4. Compute the polynomial kernel $K(\boldsymbol{a}, \boldsymbol{b}) = (\boldsymbol{a} \cdot \boldsymbol{b} + 1)^2$ for $\boldsymbol{a} = (1, 2)$, $\boldsymbol{b} = (2, 1)$. *(Answer:* $\boldsymbol{a} \cdot \boldsymbol{b} = 4$*;* $K = 25$*.)*

5. Compute the RBF kernel for $\boldsymbol{a} = (1, 2)$, $\boldsymbol{b} = (2, 1)$ with $\gamma = 0.5$. *(Answer:* $\|\boldsymbol{a} - \boldsymbol{b}\|^2 = 2$*;* $K = e^{-1} = 0.368$*.)*

6. With $\varphi(x) = (x, x^2)$ compute $\varphi(2) \cdot \varphi(3)$ and the matching kernel $K(2, 3) = 2 \cdot 3 + (2 \cdot 3)^2$. *(Answer:* $\varphi(2) \cdot \varphi(3) = (2, 4) \cdot (3, 9) = 6 + 36 = 42$*;* $K = 6 + 36 = 42$*.)*

7. In SVR with $\varepsilon = 0.3$: $y = (2.0, 3.0, 5.0)$, $f = (2.2, 3.5, 4.1)$. Find the total ε-insensitive loss. *(Answer: errors* $-0.2, -0.5, 0.9$*; losses* $0, 0.2, 0.6$*; total* 0.8*.)*

8. Gaussian NB: ripe $\sim \mathcal{N}(6, 1)$, unripe $\sim \mathcal{N}(3, 1)$, equal priors. Find $P(\text{ripe} \mid x = 4)$. *(Answer: densities* 0.0540 *and* 0.2420*; posterior* 0.182*.)*

9. Using the SMS example, classify "win now" with multinomial NB. *(Answer: spam* $0.5 \cdot \frac{3}{15} \cdot \frac{2}{15} = 0.01333$*; ham* $0.5 \cdot \frac{1}{15} \cdot \frac{2}{15} = 0.00444$*;* $P(\text{spam}) = 0.75$*.)*

10. A word never occurs in the 200 word-tokens of a class; the vocabulary has 1000 words. Find its Laplace-smoothed probability ($\alpha = 1$). *(Answer:* $\frac{0+1}{200+1000} = 0.00083$*.)*

E. Multiple-Choice Questions

1. The support vectors of an SVM are:
   (a) all training points (b) points on or inside the margin (c) the outliers only (d) the test points

2. The margin of a linear SVM equals:
   (a) $\|\boldsymbol{w}\|$ (b) $1/\|\boldsymbol{w}\|$ (c) $2/\|\boldsymbol{w}\|$ (d) $\|\boldsymbol{w}\|^2$

3. A very large C in a soft-margin SVM tends to:
   (a) widen the margin (b) narrow the margin and overfit (c) remove all support vectors (d) make the model linear

4. In the dual form the data appear only through:
   (a) squares (b) dot products (c) means (d) labels

5. The RBF kernel is:
   (a) $\boldsymbol{a} \cdot \boldsymbol{b}$ (b) $(\boldsymbol{a} \cdot \boldsymbol{b} + 1)^2$ (c) $e^{-\gamma\|\boldsymbol{a}-\boldsymbol{b}\|^2}$ (d) $\tanh(\boldsymbol{a} \cdot \boldsymbol{b})$

6. A very large γ in an RBF SVM leads to:
   (a) underfitting (b) overfitting (c) a linear boundary (d) fewer features

7. In SVR, points strictly inside the ε-tube:
   (a) are support vectors (b) have no effect on the model (c) have the largest loss (d) are

removed from the data

8. Increasing ε in SVR typically:
   (a) increases support vectors (b) decreases support vectors (c) changes nothing (d) adds features

9. Naive Bayes assumes features are:
   (a) identical (b) conditionally independent given the class (c) all Gaussian (d) uncorrelated with the class

10. Which variant suits word counts?
    (a) Gaussian (b) Multinomial (c) Linear (d) Sigmoid

11. Laplace smoothing prevents:
    (a) overfitting of trees (b) zero probabilities (c) scaling problems (d) ties

12. Which classifier is *generative*?
    (a) SVM (b) logistic regression (c) Naive Bayes (d) k-NN

Answers: 1-b, 2-c, 3-b, 4-b, 5-c, 6-b, 7-b, 8-b, 9-b, 10-b, 11-b, 12-c.

F. Practice Problems

1. Using the 1-D lifted example, classify $x = -2$ and $x = 0.5$ with $f(x) = 0.381x^2 - 1.381$. *(Answer:* $f(-2) = 0.143 \Rightarrow$ *orange;* $f(0.5) = -1.286 \Rightarrow$ *blue.)*

2. Draw the margin and support vectors for the data $(1,1), (2,2)$ in class -1 and $(4,4), (5,5)$ in class $+1$. Find $\boldsymbol{w}$, b and the margin. *(Hint: closest pair* $(2,2)$ *and* $(4,4)$*.)*

3. **Coding:** train `SVC` with linear, polynomial and RBF kernels on `make_circles` and report the accuracies.

4. **Coding:** use `GridSearchCV` to tune C and γ of an RBF SVM on the *breast cancer* dataset (after scaling).

5. **Coding:** write your own spam examples and train `MultinomialNB` with `alpha=1` and `alpha=0.01`; compare predicted probabilities.

Quick Revision

Topic	One-line recall
Linear SVM	Max-margin hyperplane; margin $2/\|w\|$; min $\frac{1}{2}\|w\|^2$ s.t. $y_i(w \cdot x_i + b) \geq 1$.
Support vectors	Points on / inside the margin; only they determine w, b.
Soft margin	Slack ξ_i, penalty C: large C strict (overfit), small C relaxed (underfit).
Dual form	$f(x) = \sum \alpha_i y_i (x_i \cdot x) + b$; $\alpha_i > 0$ only for support vectors.
Kernel trick	Replace dot product by $K(a, b) = \varphi(a) \cdot \varphi(b)$.
Kernels	Linear $a \cdot b$; poly $(\gamma a \cdot b + r)^d$; RBF $e^{-\gamma \|a-b\|^2}$.
C, γ	Large C / large γ: overfit; small: underfit.
SVR	ε-tube; loss $\max(0, \|y - f\| - \varepsilon)$; points inside are free.
Naive Bayes	$\arg\max_c P(c) \prod_j P(x_j \mid c)$; generative; conditional independence.
Variants	Gaussian (continuous), multinomial (counts), Bernoulli (binary).
Smoothing	$\frac{N_{wc}+\alpha}{N_c+\alpha V}$ avoids zero probabilities.

Chapter 7

Clustering: K-Means, DBSCAN and Gaussian Mixtures

Learning Objectives

After studying this chapter you will be able to:

- define **clustering** and explain how it differs from classification;
- run the **K-Means** algorithm by hand, explain **inertia**, **K-Means++** and how to choose k with the **elbow** and **silhouette** methods;
- explain the **limits of K-Means** and recognise when it fails;
- use clustering for **image segmentation**, **preprocessing** and **semi-supervised learning**;
- explain **DBSCAN** (core, border and noise points) and apply it to a small dataset by hand;
- explain **Gaussian mixture models**, the **EM algorithm** and **soft clustering**, and carry out an EM step by hand;
- choose between K-Means, DBSCAN and GMM;
- answer 2-, 5- and 10-mark university questions on this part of Unit IV.

> *Supervised learning answers "which of the known groups is this?" Clustering answers "are there any groups at all?"*
>
> — a data scientist's saying

7.1 Introduction to Clustering

Imagine you pour a bag of mixed socks onto the floor. Nobody tells you the categories, but you quickly start making piles: black socks here, white socks there, striped socks over there. You have just **clustered** the socks, using only how similar they look.

Definition: Clustering

Clustering is an *unsupervised* learning task: given unlabelled data $\{\boldsymbol{x}_i\}_{i=1}^n$, divide it into groups (**clusters**) so that instances in the *same* cluster are *similar* to each other and instances in *different* clusters are *dissimilar*. There are no correct labels to learn from; the groups are discovered from the structure of the data.

Clustering is not classification.

	Classification	**Clustering**
Labels in training data?	Yes (supervised)	No (unsupervised)
Groups known beforehand?	Yes, fixed classes	No, discovered from data
Goal	Predict the class of a new item	Find natural groups
Evaluation	Accuracy against true labels	Cohesion/separation measures
Example	Spam or not spam	Group customers by buying habits

Why cluster?

- **Customer segmentation:** group shoppers so that each group gets a suitable offer.
- **Image segmentation:** split a picture into regions (Section 7.4).
- **Anomaly detection:** points that belong to no cluster are suspicious.
- **Data exploration and compression:** summarise millions of points by a few cluster centres.
- **Helping other learning tasks:** preprocessing and semi-supervised learning (Sections 7.5 and 7.6).

Families of clustering methods. This chapter studies one method from each of three families:

Family	Method	Idea
Centroid-based	K-Means	Each cluster is represented by its centre; points go to the nearest centre
Density-based	DBSCAN	A cluster is a dense region separated from other dense regions by sparse regions
Model-based	Gaussian mixtures	Data come from a mixture of probability distributions; each point belongs to every cluster with some probability

(A fourth family, *hierarchical clustering*, builds a tree of nested clusters; it is not part of this syllabus.)

7.2 K-Means Clustering

7.2.1 The Idea: Where to Open the Pizza Shops?

A pizza company wants to open $k = 2$ shops in a town. Every house will order from its *nearest* shop, and the company wants the delivery distances to be as short as possible. A simple strategy is:

1. Open the shops anywhere (even at random).
2. Every house chooses its nearest shop.
3. Each shop *moves to the middle* of the houses that chose it.
4. Repeat steps 2–3 until nobody changes their mind.

This is exactly the K-Means algorithm: the **shops are the centroids** and the **houses are the data points**.

Reading the Figure

Left: the two shops have been opened on houses C (blue) and F (orange). The dashed lines show each remaining house choosing the *closest* shop: B joins the blue shop; A, D and E join the orange shop. This is the **assignment step**.

Right: the houses are now coloured by their shop. Each shop moves to the *average position* of its own customers: the blue shop moves from $(5, 2)$ up to $(5, 2.5)$ and the orange shop moves from $(4, 2)$ to $(2.75, 2.5)$. This is the **update step**. After the move some houses may be nearer to the *other* shop, so we repeat. The full run for this example is worked out in Section 7.2.3.

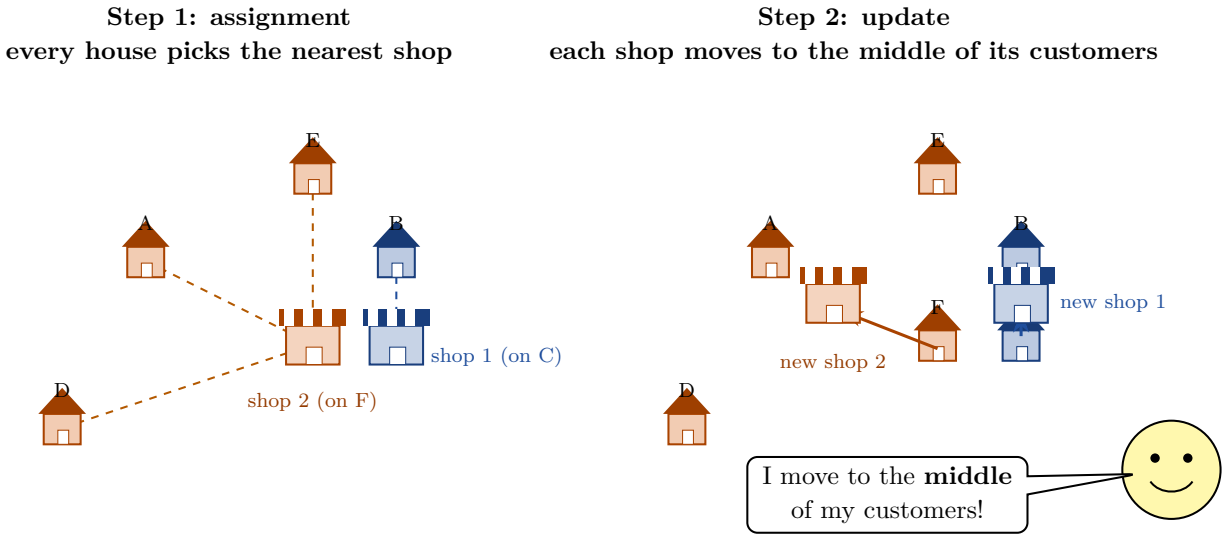


Figure 7.1. The two steps of K-Means as a pizza-shop story: assign houses to the nearest shop, then move each shop to the middle of its customers.

7.2.2 Definition and Algorithm

Definition: K-Means clustering

K-Means partitions n points into k clusters $C_1, \dots, C_k$ with centres (**centroids**) $\boldsymbol{\mu}_1, \dots, \boldsymbol{\mu}_k$ by minimising the **within-cluster sum of squares**, also called the **inertia**:

$$J = \sum_{j=1}^{k} \sum_{\boldsymbol{x}_i \in C_j} \|\boldsymbol{x}_i - \boldsymbol{\mu}_j\|^2.$$

Each centroid is the *mean* of the points in its cluster, $\boldsymbol{\mu}_j = \frac{1}{|C_j|} \sum_{\boldsymbol{x}_i \in C_j} \boldsymbol{x}_i$.

Definition: The K-Means (Lloyd's) algorithm

1. Choose the number of clusters k and initialise k centroids (e.g. k random data points).
2. **Assignment step:** assign every point to its *nearest* centroid (Euclidean distance).
3. **Update step:** replace every centroid by the *mean* of the points assigned to it.
4. Repeat steps 2–3 until the assignments no longer change (or the centroids move less than a tiny tolerance).

Why it works. The assignment step can only decrease J (every point moves to a centre that is at least as close), and the update step can only decrease J (the mean is the point that minimises the sum of squared distances to a set). So J never increases and, since there are finitely many partitions, the algorithm must stop. The result, however, is only a **local minimum** of J: it depends on the starting centroids.

7.2.3 Worked Example: K-Means by Hand

Data: six points $A(2,3)$, $B(5,3)$, $C(5,2)$, $D(1,1)$, $E(4,4)$, $F(4,2)$ and $k=2$. Initial centroids: $\boldsymbol{\mu}_1 = C = (5,2)$ and $\boldsymbol{\mu}_2 = F = (4,2)$.

Example: Iteration 1

Euclidean distances from every point to the two centroids and the resulting cluster:

Point	Coordinates	$d(\cdot, \boldsymbol{\mu}_1 = (5,2))$	$d(\cdot, \boldsymbol{\mu}_2 = (4,2))$	Cluster
A	$(2,3)$	$\sqrt{9+1} = 3.162$	$\sqrt{4+1} = 2.236$	2
B	$(5,3)$	$\sqrt{0+1} = 1.000$	$\sqrt{1+1} = 1.414$	1
C	$(5,2)$	0	1.000	1
D	$(1,1)$	$\sqrt{16+1} = 4.123$	$\sqrt{9+1} = 3.162$	2
E	$(4,4)$	$\sqrt{1+4} = 2.236$	$\sqrt{0+4} = 2.000$	2
F	$(4,2)$	1.000	0	2

Inertia with the starting centroids: cluster 1 $\{B, C\}$: $1^2 + 0^2 = 1$; cluster 2 $\{A, D, E, F\}$: $2.236^2 + 3.162^2 + 2^2 + 0 = 5 + 10 + 4 + 0 = 19$; total $J = \mathbf{20}$.

Update step:

$$\boldsymbol{\mu}_1 = \left(\tfrac{5+5}{2}, \tfrac{3+2}{2}\right) = (5,\ 2.5), \qquad \boldsymbol{\mu}_2 = \left(\tfrac{2+1+4+4}{4}, \tfrac{3+1+4+2}{4}\right) = (2.75,\ 2.5).$$

New inertia $J = \mathbf{12.25}$.

Example: Iteration 2

With $\boldsymbol{\mu}_1 = (5, 2.5)$ and $\boldsymbol{\mu}_2 = (2.75, 2.5)$:

Point	$d(\cdot, \boldsymbol{\mu}_1)$	$d(\cdot, \boldsymbol{\mu}_2)$	Cluster	Changed?
A $(2,3)$	3.041	0.901	2	no
B $(5,3)$	0.500	2.305	1	no
C $(5,2)$	0.500	2.305	1	no
D $(1,1)$	4.272	2.305	2	no
E $(4,4)$	1.803	1.953	**1**	**yes** (was 2)
F $(4,2)$	1.118	1.346	**1**	**yes** (was 2)

Points E and F *change their mind*. Inertia with the old centroids is 11.125. Update:

$$\boldsymbol{\mu}_1 = \left(\tfrac{5+5+4+4}{4}, \tfrac{3+2+4+2}{4}\right) = (4.5,\ 2.75), \qquad \boldsymbol{\mu}_2 = \left(\tfrac{2+1}{2}, \tfrac{3+1}{2}\right) = (1.5,\ 2).$$

New inertia $J = \mathbf{6.25}$.

Example: Iteration 3: convergence

With $\boldsymbol{\mu}_1 = (4.5, 2.75)$ and $\boldsymbol{\mu}_2 = (1.5, 2)$ every point stays in the same cluster as before (B, C, E, F in cluster 1 and A, D in cluster 2), so the centroids do not move: the algorithm has **converged**.

	Cluster 1	**Cluster 2**
Members	B, C, E, F	A, D
Centroid	(4.5, 2.75)	(1.5, 2.0)
Inertia contribution	3.75	2.50

Final inertia $J = 3.75 + 2.50 = \mathbf{6.25}$.

Stage	Start	After update 1	After assignment 2	After update 2
Inertia J	20	12.25	11.125	**6.25**

The inertia **never increases**, confirming the argument above.

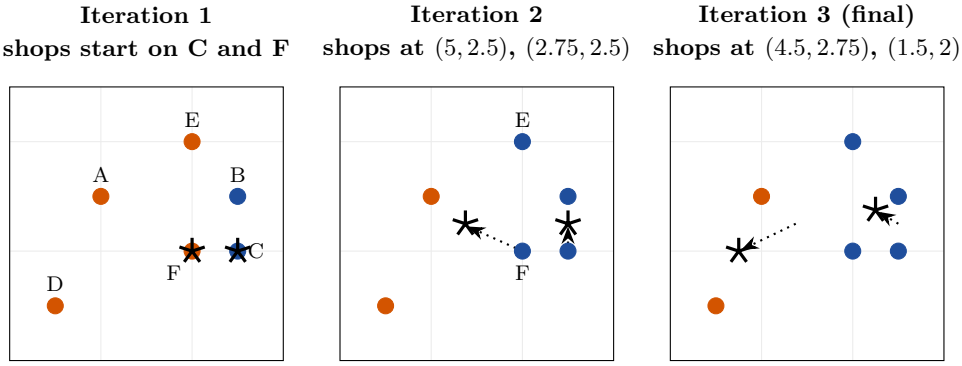


Figure 7.2. The K-Means run of the worked example. Stars are centroids; colours show the current cluster.

Reading the Figure

Each panel is one iteration; the **stars** are the centroids and the **colours** show the cluster each point currently belongs to (blue = cluster 1, orange = cluster 2). In the first panel the two centroids start almost on top of each other, so the clustering is poor. In the second panel the dotted arrows show the centroids moving; points E and F have turned blue because they are now closer to the blue star. In the third panel the centroids have settled in the middle of their groups and nothing changes any more.

7.2.4 Initialisation and K-Means++

Because K-Means only finds a local minimum, a bad start can give a bad result. If both initial shops land in the same neighbourhood, the final clusters can be poor.

Definition: K-Means++ initialisation

K-Means++ chooses the first centroid uniformly at random from the data; each further centroid is chosen from the remaining points with probability *proportional to* $D(\boldsymbol{x})^2$, the squared distance from the point to the nearest centroid already chosen. This spreads the starting centroids far apart.

Example: K-Means++ probabilities

One-dimensional points 0, 1, 10; suppose the first centroid is 0.

$$D^2 = (0,\ 1,\ 100) \ \Rightarrow\ P = \left(0,\ \tfrac{1}{101},\ \tfrac{100}{101}\right) = (0,\ 0.0099,\ 0.990).$$

The next centroid is almost certainly the far-away point 10, giving well-separated starting centres.

Real results. We ran K-Means with $k = 4$ on 300 points from four well-separated blobs, using a *single* initialisation each time, 100 different random seeds:

Initialisation	Mean inertia	Worst inertia	Runs reaching the best solution
Random points	1513.6	6825.4	60 of 100
K-Means++	279.9	279.9	**100 of 100**

With random starts, 40% of the runs got stuck in a bad local minimum (the worst had inertia 24 times larger!), while K-Means++ found the best solution every time. In practice we also run the algorithm several times (`n_init=10`) and keep the run with the lowest inertia.

7.2.5 Choosing the Number of Clusters k

K-Means needs k *in advance*. Inertia always decreases as k grows (with $k = n$ it is 0), so we cannot simply minimise it. Two standard methods help.

1. The elbow method. Run K-Means for $k = 1, 2, 3, \ldots$ and plot the inertia against k. Choose the k at the "elbow", where adding more clusters stops giving a big improvement.

2. The silhouette score.

Definition: Silhouette coefficient

For a point i let $a(i)$ be the **average distance to the other points in its own cluster** (cohesion) and $b(i)$ the **smallest average distance to the points of any other cluster** (separation). Then

$$s(i) = \frac{b(i) - a(i)}{\max\{a(i), b(i)\}} \in [-1, 1].$$

$s \approx 1$: the point is well inside its cluster; $s \approx 0$: it lies on the border between two clusters; $s < 0$: it is probably in the wrong cluster. The **silhouette score** of a clustering is the mean of $s(i)$ over all points; we choose the k with the highest score.

Example: Silhouette by hand

Four points on a line, clustered as $\{1, 2\}$ and $\{8, 9\}$.

Point 1: $a = |1 - 2| = 1$. $b =$ mean distance to $\{8, 9\} = \frac{7+8}{2} = 7.5$. $s = \frac{7.5-1}{7.5} = 0.867$.
Point 2: $a = 1$, $b = \frac{6+7}{2} = 6.5$, $s = \frac{6.5-1}{6.5} = 0.846$.
Point 8: $a = 1$, $b = \frac{7+6}{2} = 6.5$, $s = 0.846$. **Point 9:** $a = 1$, $b = 7.5$, $s = 0.867$.

Mean silhouette $= \frac{0.867+0.846+0.846+0.867}{4} =$ **0.856**: a very good clustering. If instead the clusters were $\{1, 2, 8\}$ and $\{9\}$, the point 8 would have $a = \frac{7+6}{2} = 6.5$ and $b = 1$, so $s = \frac{1-6.5}{6.5} = -0.846 < 0$: a clearly wrong assignment.

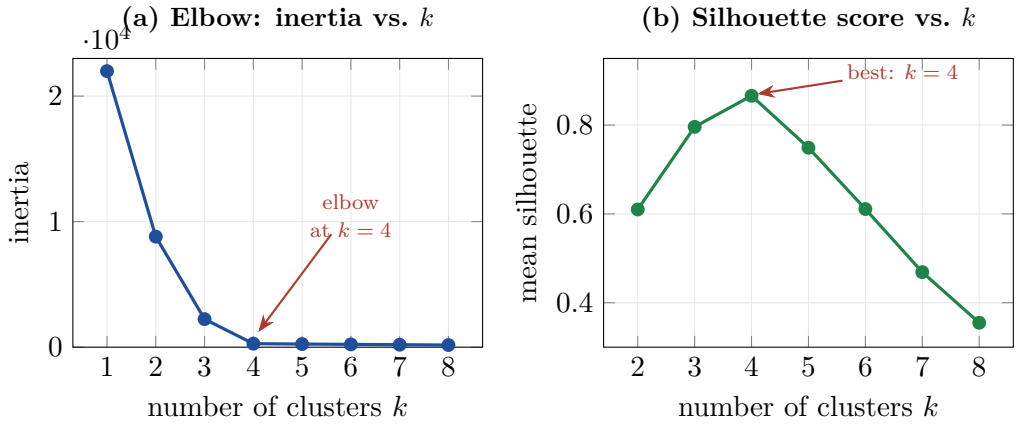


Figure 7.3. Choosing k for 300 points drawn from 4 blobs (real K-Means results).

Reading the Figure

(a) Elbow plot. Going from 1 to 2 to 3 to 4 clusters the inertia collapses (21 990 → 8 805 → 2 236 → 280). After $k = 4$ it decreases only slowly (280 → 250 → 223 …): extra clusters are just splitting blobs that were already compact. The **bend (elbow) at** $k = 4$ gives the answer, matching the four blobs that generated the data.

(b) Silhouette plot. The score peaks at $k = 4$ (0.866) and falls on both sides: with $k = 3$ two blobs are forced together and with $k = 5$ a blob is cut in two. The highest silhouette agrees with the elbow.

Common Mistake

The elbow is not always sharp, and for real data different methods may disagree. Use the elbow and silhouette as guidance together with *domain knowledge* (for example, a company may only want 3 customer segments because it has 3 marketing teams).

Try It Yourself

```python
from sklearn.datasets import make_blobs
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score

X, y = make_blobs(n_samples=300, centers=4, cluster_std=0.7,
                  random_state=7)

for k in range(2, 9):
    km = KMeans(n_clusters=k, init="k-means++", n_init=10,
                random_state=0).fit(X)
    print(k, round(km.inertia_, 1),
          round(silhouette_score(X, km.labels_), 3))
# k=4 -> inertia 279.9, silhouette 0.866 (best)
```

Challenge: cluster the **iris** measurements with $k = 3$ and compare the clusters with the true species using `adjusted_rand_score`.

K-Means

Advantages

- Simple, fast and easy to implement
- Scales to very large datasets (cost $O(nkd)$ per iteration)
- Always converges
- Gives centroids that summarise each group

Disadvantages

- Must choose k in advance
- Result depends on initialisation (local minimum)
- Assumes round, similar-sized clusters
- Sensitive to outliers and feature scaling

Applications

- Customer segmentation
- Image compression and segmentation
- Document and news grouping
- Vector quantisation, feature engineering

7.3 Limits of K-Means

K-Means is fast and popular, but it quietly assumes that clusters are **round blobs of similar size and density**. When this is not true, it fails. The main limits are:

1. k **must be chosen** before running the algorithm.
2. **Local minima:** different starts can give different answers (reduce with K-Means++ and `n_init`).

3. **Cluster shape:** it uses distance to a centroid, so it can only produce convex, roughly spherical clusters. Moons, rings and elongated clusters are cut incorrectly.
4. **Cluster size and density:** it tends to split a big cluster and merge small ones.
5. **Outliers:** one extreme point drags a centroid away (the mean is not robust).
6. **Feature scale:** features with a large range dominate the distance, so features must be standardised.
7. **Hard assignments:** every point belongs to exactly one cluster, with no notion of uncertainty.

Example: The outlier problem

One-dimensional data $\{1, 2, 3, 100\}$ with a single cluster centre. The centroid is the mean

$$\mu = \frac{1 + 2 + 3 + 100}{4} = 26.5,$$

which is nowhere near the three points that actually form the group $(1, 2, 3)$ and is not even close to the outlier. A single bad point has pulled the centre far away. (The *median*, 2.5, would have been unaffected: this is why K-Medians and K-Medoids exist.)

(a) K-Means with $k = 2$
ARI $= 0.22$ **(wrong)**

(b) DBSCAN ($\varepsilon = 0.3$)
ARI $= 0.97$ **(correct)**

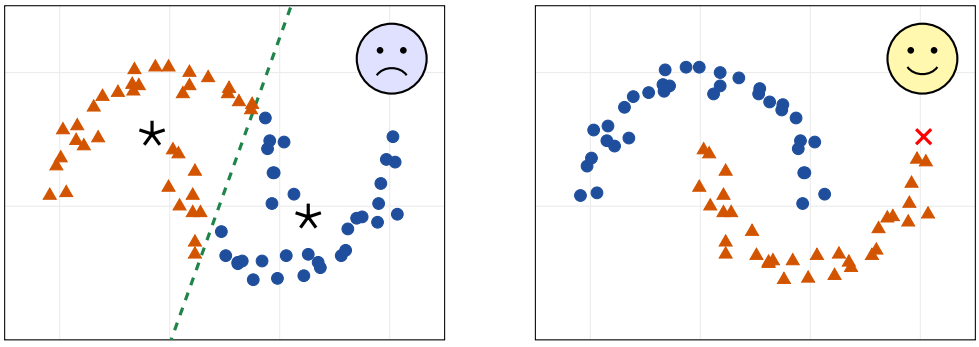


Figure 7.4. Two interleaving half-moons (a 70-point sample): K-Means cuts them with a straight line, while DBSCAN follows their shape.

Reading the Figure

The data are two half-moon shapes. **Panel (a):** K-Means looks for two round clusters. Its two centroids (stars) lie between the moons, and the cluster boundary is the *straight green dashed line* halfway between them. This line slices *through both moons*, so each cluster contains a piece of each moon: the **adjusted Rand index** (ARI, 1 = perfect agreement with the true groups, 0 = random) is only 0.22. **Panel (b):** DBSCAN (Section 7.7) follows the dense curved bands and recovers both moons exactly (ARI = 0.97; the red cross is a point it declares noise).

Dataset (300 points)	K-Means	DBSCAN	Gaussian mixture
Round blobs	**1.00**	0.93 (14 noise)	**1.00**
Unequal sizes and spreads	0.99	0.91 (9 noise)	**1.00**
Stretched (elongated) blobs	0.52	0.99 (2 noise)	**1.00**
Two moons	0.23	**1.00**	0.46
Two concentric circles	−0.00	**1.00**	−0.00

Adjusted Rand index; real results; DBSCAN parameters were chosen per dataset ($\varepsilon = 0.2$ for moons and circles, 0.5–0.7 for blobs, 5 points minimum). **Read the table:** on round blobs all methods succeed, but K-Means collapses on the elongated blobs (0.52), the moons (0.23) and the circles (0.00, no better than random). Gaussian mixtures cope with elongated clusters, and DBSCAN with arbitrary shapes. No single algorithm is best for every shape.

7.4 Using Clustering for Image Segmentation

Definition: Image segmentation

Image segmentation is the task of dividing an image into regions of pixels that belong together (for example sky, grass and a house). **Colour segmentation** with K-Means treats every pixel as a point (R, G, B) in three-dimensional colour space, clusters the pixels into k groups, and replaces every pixel by the *centroid colour* of its cluster.

Procedure.

1. Reshape the image of size $H \times W \times 3$ into a list of $H \cdot W$ points with 3 features.
2. Run K-Means with k clusters (here k = the number of colours wanted).
3. Replace each pixel by the centroid of its cluster and reshape back to $H \times W \times 3$.

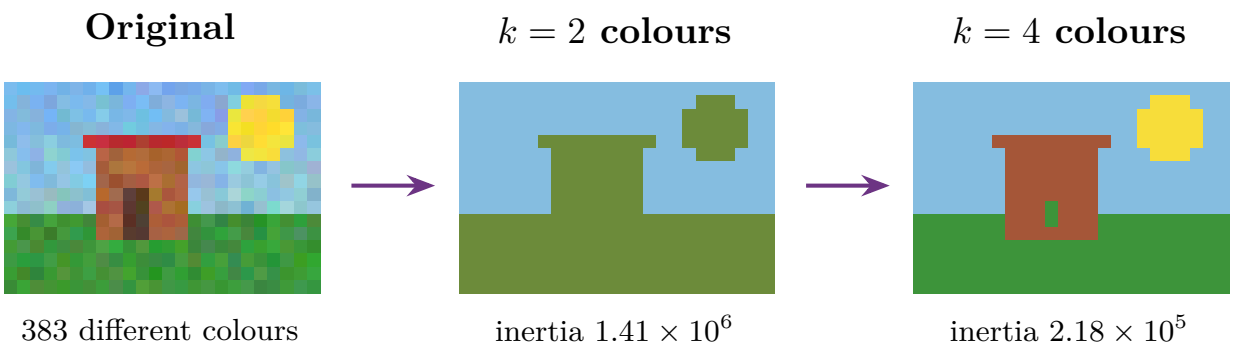


Figure 7.5. Colour segmentation of a small synthetic landscape (24×16 pixels) with K-Means.

Reading the Figure

The picture on the left is a tiny cartoon landscape (sky, sun, grass, a house with a roof and a door), with a little random noise added to every pixel so that there are 383 different colours among the 384 pixels. In the middle, K-Means with $k = 2$ finds only two colour groups: "sky-like" (light blue) and "ground-like" (dark green): the sun, house and roof are lost. On the right, with $k = 4$ the centroid colours are sky blue $(133, 189, 224)$, grass green $(61, 148, 58)$, sun yellow $(247, 221, 58)$ and house brown $(165, 86, 56)$, so the **main objects reappear** and the noise vanishes. The inertia falls from 1.41×10^6 to 2.18×10^5 as k grows from 2 to 4 (and to 1.26×10^5 for $k = 6$).

Segmentation as compression (vector quantisation). After clustering, each pixel needs only an **index** into a small **palette** of k colours.

Example: How much smaller is the image?

An image of N pixels stored with 24 bits per pixel needs $24N$ bits. After K-Means with k colours it needs $N\lceil \log_2 k \rceil$ bits for the indices plus $24k$ bits for the palette.

Image	Original	After K-Means	Reduction
Our $24 \times 16 = 384$ pixels, $k = 4$	9216 bits	$384 \cdot 2 + 96 = 864$ bits	10.7×
100×100 pixels, $k = 8$	240 000 bits	$10\,000 \cdot 3 + 192 = 30\,192$ bits	7.9×
1 megapixel photo, $k = 16$	24 000 000 bits	4 000 384 bits	6.0×

Common Mistake

Colour segmentation uses *only colour*, not position or meaning. Two different objects with similar colours (a green car and green grass) will fall in the same cluster, and shadows can split one object into several clusters. For real object recognition we use richer methods (such as neural networks, Unit V).

Try It Yourself

```
import numpy as np
from sklearn.cluster import KMeans

# image: a NumPy array of shape (height, width, 3), values 0-255
X = image.reshape(-1, 3)                          # one row per
    pixel
km = KMeans(n_clusters=4, n_init=10, random_state=0).fit(X)
segmented = km.cluster_centers_[km.labels_]     # centroid colour
```

```
segmented = segmented.reshape(image.shape).astype(np.uint8)
print(km.cluster_centers_.round())              # the 4-colour palette
```

Challenge: load a photograph with `matplotlib.image.imread` and try $k = 2, 4, 8, 16$; look at how the picture and its palette change.

Image segmentation with K-Means

Advantages

- Very simple and fast
- Reduces the number of colours (compression)
- Removes noise and gives clean regions
- No training labels needed

Disadvantages

- Uses only colour, not position or shape
- Must choose k
- Same-coloured objects merge; shadows split objects
- Not a true object-recognition method

Applications

- Image compression and posterisation
- Pre-segmentation for medical and satellite images
- Background removal in simple scenes
- Colour palette extraction

7.5 Using Clustering for Preprocessing

Clustering can be used *before* supervised learning to create better input features.

Definition: Clustering as preprocessing

After fitting K-Means with k clusters, every instance $\boldsymbol{x}$ can be represented by its k **distances to the centroids**, $(\|\boldsymbol{x} - \boldsymbol{\mu}_1\|, \ldots, \|\boldsymbol{x} - \boldsymbol{\mu}_k\|)$ (or by a one-hot cluster index). This new representation replaces, or is added to, the original features, and a supervised model is trained on it.

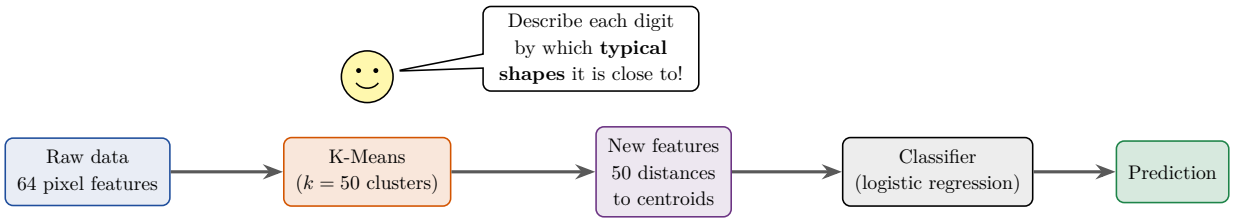


Figure 7.6. K-Means as a preprocessing step in a pipeline.

Reading the Figure

Read the pipeline from left to right. The raw data (here, handwritten digits as 64 pixel values) are first given to K-Means, which finds 50 *typical shapes* (the centroids). Each digit is then described by 50 numbers: how far it is from each typical shape. A digit that looks like a "typical 7" will be very close to the centroid of a 7-cluster and far from the others.

A classifier then learns from these distance features. The cartoon says it best: instead of describing a digit pixel by pixel, we describe it by which typical shapes it resembles.

Real results on the 8×8 handwritten digits (1347 training and 450 test images):

Model	Test accuracy
Logistic regression on the 64 raw pixels	0.973
K-Means ($k = 50$) → logistic regression on 50 distances	0.976

The clustered features reach the same accuracy while using fewer inputs (50 instead of 64) and the improvement is small (0.973 → 0.976), because logistic regression is already strong on this easy dataset. Preprocessing with clustering helps most when the original data are high-dimensional, noisy, or not linearly separable. The value of k is a hyperparameter chosen by cross-validation on the final accuracy.

Clustering for preprocessing

Advantages

- Reduces dimensionality (many raw features → k distances)
- Adds non-linear, "typical shape" features that help simple models
- Fully automatic, no labels needed to build the features
- Can reveal useful structure

Disadvantages

- Adds a hyperparameter k to tune
- Information about fine details can be lost
- Gain is small if the data are already easy to classify
- Needs scaled features

Applications

- Digit and image recognition pipelines
- Customer-behaviour features for prediction
- Text and sensor-data feature engineering
- Speeding up classifiers on large data

7.6 Using Clustering for Semi-Supervised Learning

In many projects we have **lots of unlabelled data but only a few labels**, because labelling is slow and expensive (Chapter 1). Clustering gives a clever way to use the unlabelled data.

Definition: Label propagation by clustering

(1) Cluster all the data (labelled and unlabelled) into k clusters with K-Means. (2) For each cluster pick the **representative instance** (the one closest to the centroid) and obtain its label from a human. (3) **Propagate** that label to all other instances of the cluster. (4) Train an ordinary supervised classifier on the (partly pseudo-)labelled data.

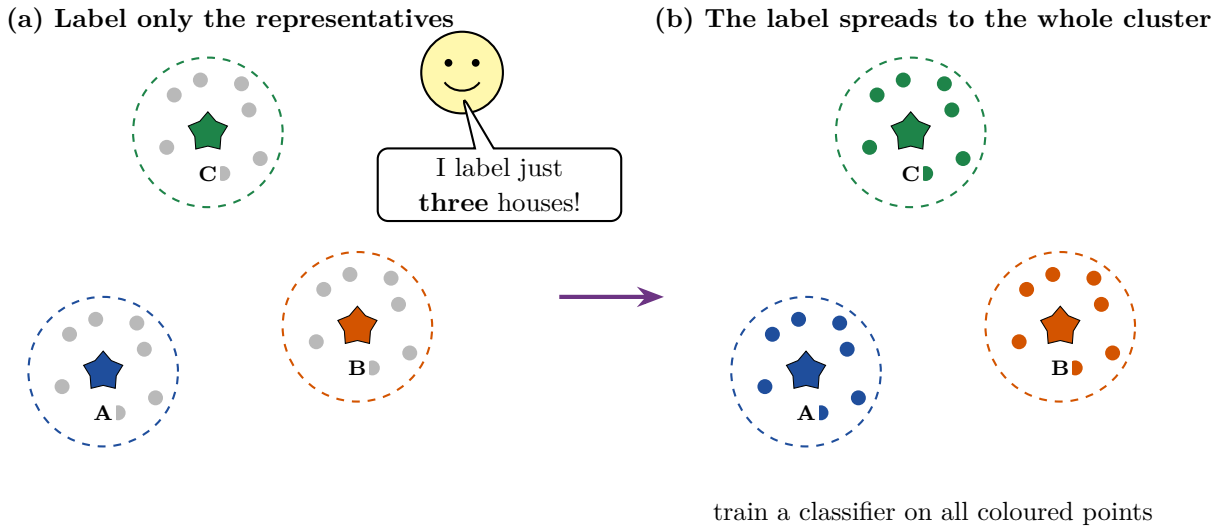


Figure 7.7. Label propagation: a few labels become many pseudo-labels.

Reading the Figure

(a) The data have been clustered into three groups (dashed circles). Grey dots are unlabelled; the **stars** are the cluster representatives (closest to each centre), the only instances a human has to label (as classes A, B and C). **(b)** Every point inherits the label of its cluster's representative (shown by colour), so three human labels have turned into labels for all the points. A classifier is then trained on this large, mostly correct, training set. The assumption is that points in the same cluster usually share the same class.

Real results on the digits data (1347 training images, 450 test images). We can afford to label only 50 images:

Strategy (50 labels only)	Training labels used	Test accuracy
Label 50 *random* images	50	0.813
Cluster ($k = 50$), label the 50 *representative* images	50	0.896
Propagate each representative's label to its *whole* cluster	1347 (93.5% correct)	0.909
Propagate only to the 20% of each cluster closest to the centroid	288 (98.3% correct)	**0.924**

Read the table: the same budget of 50 human labels gives accuracy 0.813 if the images are chosen at random, but 0.896 if we label the *cluster representatives*, because they cover every kind of digit. Propagating labels adds more: 0.909 for the full cluster, and 0.924 when we propagate only to the points *close to the centroid*, where the label is most reliable (98.3% correct instead of 93.5%). For comparison, a classifier trained on all 1347 true labels reaches 0.973.

Clustering for semi-supervised learning

Advantages

- Needs very few human labels
- Chooses *which* instances to label (cheap and effective)
- Uses the structure of the unlabelled data
- Easy to combine with any classifier

Disadvantages

- Assumes points in a cluster share a class
- Wrong propagated labels add noise
- Depends on a good clustering and on k
- Less accurate than having all true labels

Applications

- Labelling large image collections (photo tagging)
- Medical imaging where experts' time is costly
- Text and speech corpora
- Active learning workflows

7.7 DBSCAN

7.7.1 The Idea: Crowds at a Party

Imagine a big party seen from above. Wherever people stand in a **dense crowd**, you see a group. People standing alone at the edges are not part of any group. Two crowds separated by an empty space are two different groups, even if each crowd has a strange shape (a ring around the dance floor, a long queue). **DBSCAN** (Density-Based Spatial Clustering of Applications with Noise) clusters exactly like that: it finds *dense regions* and treats isolated points as **noise**.

7.7.2 Definitions

DBSCAN has two parameters: the **radius** ε ("arm's reach") and the **minimum number of points** MinPts ("how many friends make a crowd").

Definition: ε-neighbourhood

The ε-neighbourhood of a point p is the set of all points within distance ε of p (including p itself): $N_\varepsilon(p) = \{q : \ d(p, q) \leq \varepsilon\}$.

Definition: Core, border and noise points

- A **core point** has at least MinPts points in its ε-neighbourhood: $|N_\varepsilon(p)| \geq \text{MinPts}$. It is in the *inside* of a dense region.
- A **border point** is *not* a core point but lies within ε of a core point. It is on the *edge* of a cluster.
- A **noise point** (outlier) is neither core nor border.

Definition: Density-reachable and density-connected

A point q is **directly density-reachable** from p if p is a core point and $q \in N_\varepsilon(p)$. q is **density-reachable** from p if there is a chain of points $p = p_1, p_2, \ldots, p_m = q$ in which each p_{i+1} is directly density-reachable from p_i. Two points are **density-connected** if both are density-reachable from a common core point.

Definition: DBSCAN cluster and algorithm

A **cluster** is a maximal set of density-connected points. The algorithm:

1. Pick an unvisited point p and find its ε-neighbourhood.
2. If p is **not** a core point, mark it provisionally as noise (it may later turn out to be a border point of some cluster).
3. If p **is** a core point, start a **new cluster**: add all points in its neighbourhood; then for every core point added, add *its* neighbours too, and so on until the cluster cannot grow.
4. Repeat until every point has been visited.

7.7.3 Worked Example: DBSCAN by Hand

Data: ten points $A(1,1)$, $B(1,2)$, $C(2,1)$, $D(2,2)$, $E(3,2)$, $F(8,8)$, $G(8,9)$, $H(9,8)$, $I(9,9)$, $J(5,5)$ with $\varepsilon = 1.2$ and MinPts $= 3$ (counting the point itself).

Example: Classifying each point

Neighbours within distance 1.2 (note that $d(A,D) = \sqrt{2} = 1.414 > 1.2$):

Point	ε-neighbourhood	Size	Type
A	$\{A, B, C\}$	3	**core**
B	$\{B, A, D\}$	3	**core**
C	$\{C, A, D\}$	3	**core**
D	$\{D, B, C, E\}$	4	**core**
E	$\{E, D\}$	2	**border** (within ε of the core point D)
F	$\{F, G, H\}$	3	**core**
G	$\{G, F, I\}$	3	**core**
H	$\{H, F, I\}$	3	**core**
I	$\{I, G, H\}$	3	**core**
J	$\{J\}$	1	**noise**

Building the clusters. Start at A (core): add B and C. B is core: add D. D is core: add E (border; E is not core, so the cluster does not grow from E). The first cluster is $\{A, B, C, D, E\}$. Next, F (core): add G and H, then I. The second cluster is $\{F, G, H, I\}$. Finally J belongs to no cluster: **noise**.

Result: 2 clusters and 1 noise point. We never had to say "two clusters"; DBSCAN found them from the density.

Effect of ε. If we enlarge ε to 1.5, then $d(A, D) = 1.414$ and $d(C, E) = 1.414$ are now within reach, so E has two neighbours (D and C) besides itself, i.e. $|N_\varepsilon(E)| = 3$, and becomes a core point; the clusters stay the same but the first cluster is now more tightly connected. If we enlarge ε to 8 every point is a neighbour of many others and all ten points merge into a single cluster.

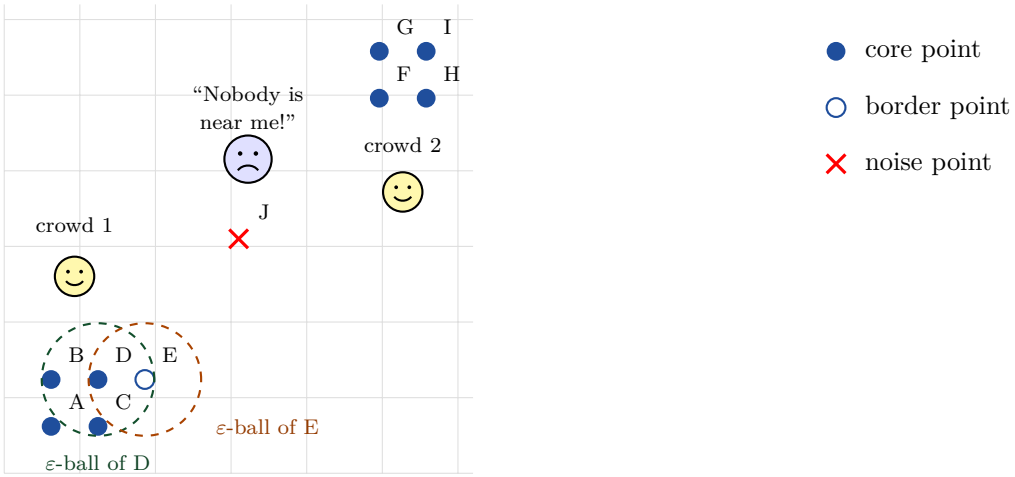


Figure 7.8. DBSCAN on the ten-point example ($\varepsilon = 1.2$, MinPts $= 3$).

Reading the Figure

Filled blue dots are core points: each has at least 3 points (itself included) within the dashed radius. The **white-filled dot E** is a border point: the orange dashed circle around it contains only two points (E and D), so it is not core, but it lies inside the green circle of the core point D, so it joins D's cluster. The **red cross J** has nobody nearby: noise (the sad face). There are two crowds (the smiley faces): the lower-left group with five points and the upper-right group with four. Clusters were *not* given in advance.

7.7.4 Choosing ε and MinPts

- **MinPts:** a common rule is MinPts $\geq d + 1$ and often $2d$ (d = number of features); larger values make DBSCAN more robust to noise. For 2-D data MinPts $= 4$ or 5 is typical.
- **ε:** use the **k-distance graph**. For every point compute the distance to its MinPts-th nearest neighbour, sort these distances from small to large and plot them; choose ε at the *knee* (sharp bend) of the curve.

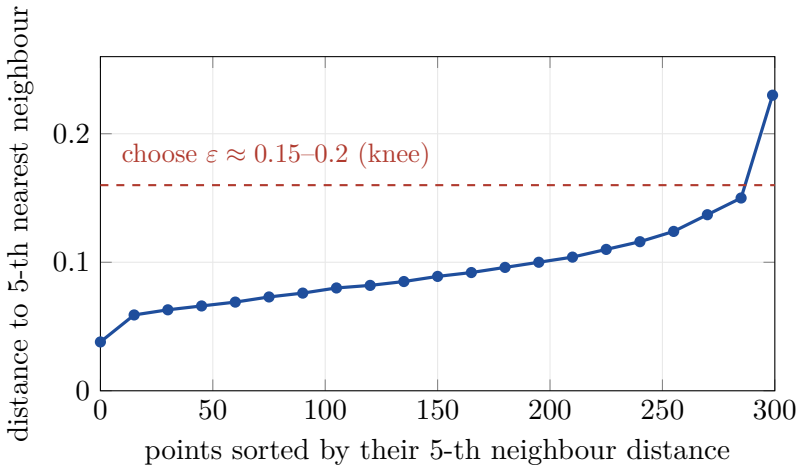


Figure 7.9. k-distance graph for 300 points of the two-moons data (real values, MinPts = 5).

Reading the Figure

Each point on the curve is a data point, sorted by how far its 5-th nearest neighbour is. Most points have a small value (< 0.1): they sit inside dense regions. Only at the far right does the curve **bend upward sharply**: those are the few isolated points whose neighbours are far away (noise candidates). The ε value at the knee separates "dense" from "sparse", so $\varepsilon \approx 0.15$–0.2 is a good choice here. (We used $\varepsilon = 0.2$ and obtained a perfect clustering.)

Sensitivity to ε (real results on two moons with noise, MinPts = 5):

ε	0.05	0.1	**0.2**	0.3	0.5 and 1.0
Clusters found	4	14	**2**	1	1
Noise points	278	38	**1**	0	0
ARI	0.00	0.20	**0.99**	0.00	0.00
Diagnosis	almost all noise	fragmented	**correct**	moons merged	all merged

A radius that is too small labels almost everything as noise, and one that is too large merges everything into one cluster, so ε must be chosen with care.

Try It Yourself

```
from sklearn.datasets import make_moons
from sklearn.cluster import DBSCAN
from sklearn.metrics import adjusted_rand_score

X, y = make_moons(n_samples=300, noise=0.08, random_state=0)
db = DBSCAN(eps=0.2, min_samples=5).fit(X)

labels = db.labels_                          # -1 means noise
```

```
print(set(labels), (labels == -1).sum()) # {0, 1}  1 noise point
print(adjusted_rand_score(y, labels))    # 0.99
print(len(db.core_sample_indices_))      # number of core points
```

Challenge: change `eps` to 0.05, 0.1 and 0.5 and observe the number of clusters and noise points.

DBSCAN

Advantages

- Finds clusters of *any shape*
- No need to specify the number of clusters
- Identifies **noise/outliers** automatically
- Robust to outliers; only two parameters

Disadvantages

- Hard to choose ε (sensitive!)
- Struggles when clusters have *different densities*
- Poor in very high dimensions (distances lose meaning)
- No cluster centres, no soft membership

Applications

- Geographic and location clustering
- Anomaly and fraud detection
- Astronomy, sensor and trajectory data
- Image and spatial data analysis

7.8 Gaussian Mixture Models

7.8.1 The Idea: Overlapping Hills

K-Means says "this point belongs to cluster 2, full stop". But what about a point right between two groups? A more honest answer is "60% cluster 1, 40% cluster 2". A **Gaussian mixture model (GMM)** gives exactly this kind of **soft** answer. It assumes that the data were produced by several overlapping bell-shaped (Gaussian) hills, and tries to find the hills.

Definition: Gaussian mixture model

A **Gaussian mixture model** assumes that the data come from a mixture of K Gaussian distributions:

$$p(\boldsymbol{x}) = \sum_{k=1}^{K} \pi_k \mathcal{N}(\boldsymbol{x} \mid \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k), \qquad \pi_k \geq 0, \quad \sum_k \pi_k = 1.$$

Each component k has a **mixing weight** π_k (the probability of picking this cluster), a **mean** $\boldsymbol{\mu}_k$ (its centre) and a **covariance matrix** $\boldsymbol{\Sigma}_k$ (its shape, size and orientation). Clustering means finding the parameters and then computing, for each point, the **responsibility**

$$\gamma_{ik} = P(\text{cluster } k \mid \boldsymbol{x}_i) = \frac{\pi_k \mathcal{N}(\boldsymbol{x}_i \mid \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)}{\sum_{j=1}^{K} \pi_j \mathcal{N}(\boldsymbol{x}_i \mid \boldsymbol{\mu}_j, \boldsymbol{\Sigma}_j)},$$

the probability that point i was generated by component k. This is **soft clustering**.

7.8.2 The EM Algorithm

The parameters are learned by the **Expectation–Maximisation (EM)** algorithm, which alternates two steps, very much like K-Means.

Definition: EM algorithm for a Gaussian mixture

Initialise $\pi_k, \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k$ (e.g. with K-Means). Then repeat:

1. **E-step (Expectation):** compute the responsibilities γ_{ik} of every component for every point with the formula above.
2. **M-step (Maximisation):** re-estimate the parameters using the responsibilities as *soft weights*. With $N_k = \sum_i \gamma_{ik}$:

$$\boldsymbol{\mu}_k = \frac{1}{N_k} \sum_i \gamma_{ik} \boldsymbol{x}_i, \qquad \boldsymbol{\Sigma}_k = \frac{1}{N_k} \sum_i \gamma_{ik} (\boldsymbol{x}_i - \boldsymbol{\mu}_k)(\boldsymbol{x}_i - \boldsymbol{\mu}_k)^\top, \qquad \pi_k = \frac{N_k}{n}.$$

until the log-likelihood $\sum_i \ln p(\boldsymbol{x}_i)$ stops improving.

Relation to K-Means. The E-step is a *soft* version of K-Means' assignment step (a point is shared among clusters instead of being given to one), and the M-step is a *weighted* version of the centroid update. K-Means is the special case of a GMM with equal spherical covariances and *hard* assignments.

7.8.3 Worked Example: One EM Iteration by Hand

Example: One-dimensional EM

Five points $x = 1,\ 2,\ 5,\ 8,\ 9$ and $K = 2$ components. Start with $\mu_1 = 2$, $\mu_2 = 8$, $\sigma_1^2 = \sigma_2^2 = 1$ and $\pi_1 = \pi_2 = 0.5$.

E-step (responsibilities). Use $\mathcal{N}(x \mid \mu, 1) = \varphi(x - \mu)$ with $\varphi(z) = \frac{1}{\sqrt{2\pi}} e^{-z^2/2}$.

x	$\varphi(x-2)$	$\varphi(x-8)$	γ_1	γ_2
1	0.2420	≈ 0	≈ 1	≈ 0
2	0.3989	≈ 0	≈ 1	≈ 0
5	0.0044	0.0044	**0.5**	**0.5**
8	≈ 0	0.3989	≈ 0	≈ 1
9	≈ 0	0.2420	≈ 0	≈ 1

The middle point $x = 5$ is equally far from both centres, so it is shared 50/50. This *soft* sharing is what K-Means cannot do.

M-step (new parameters). $N_1 = 1 + 1 + 0.5 = 2.5$ and $N_2 = 0.5 + 1 + 1 = 2.5$.

$$\mu_1 = \frac{1(1)+1(2)+0.5(5)}{2.5} = \frac{5.5}{2.5} = \mathbf{2.2},$$
$$\mu_2 = \frac{0.5(5)+1(8)+1(9)}{2.5} = \frac{19.5}{2.5} = \mathbf{7.8},$$
$$\sigma_1^2 = \frac{1(1-2.2)^2+1(2-2.2)^2+0.5(5-2.2)^2}{2.5} = \frac{1.44+0.04+3.92}{2.5} = \mathbf{2.16},$$
$$\sigma_2^2 = \frac{0.5(5-7.8)^2+1(8-7.8)^2+1(9-7.8)^2}{2.5} = \mathbf{2.16},$$
$$\pi_1 = \pi_2 = \frac{2.5}{5} = \mathbf{0.5}.$$

Both centres moved towards the middle point (because it contributed half a point to each), and the variances grew from 1 to 2.16.

Using the new model. Classify a new point $x = 4$:

$$\mathcal{N}(4 \mid 2.2, 2.16) = \frac{1}{\sqrt{2\pi \cdot 2.16}}\, e^{-1.8^2/(2\cdot 2.16)} = 0.2714\, e^{-0.75} = 0.1282,$$
$$\mathcal{N}(4 \mid 7.8, 2.16) = 0.2714\, e^{-3.8^2/(2\cdot 2.16)} = 0.2714\, e^{-3.34} = 0.0096.$$

$$\gamma_1 = \frac{0.5(0.1282)}{0.5(0.1282) + 0.5(0.0096)} = \mathbf{0.930}, \qquad \gamma_2 = 0.070.$$

The point belongs to component 1 with probability 93% and to component 2 with probability 7%.

7.8.4 Covariance Types and Choosing the Number of Components

The covariance matrix $\boldsymbol{\Sigma}_k$ controls the shape of each hill. The library offers four choices:

Type	Shape of the clusters	Parameters per component
`spherical`	Circles, each with its own radius	1 variance
`diag`	Axis-aligned ellipses	d variances
`tied`	Ellipses with the *same* shape and orientation for all components	one shared full matrix
`full`	Free ellipses of any shape and orientation	a full $d \times d$ matrix

Increasing the number of components K always increases the likelihood, so as with K-Means we cannot just maximise it. We penalise complexity with the **Bayesian information criterion**:

$$\mathrm{BIC} = p \ln n - 2 \ln \hat{L},$$

where p is the number of free parameters, n the number of points and $\hat{L}$ the maximised likelihood. **Lower BIC is better**: the first term punishes too many parameters, the second rewards a good fit.

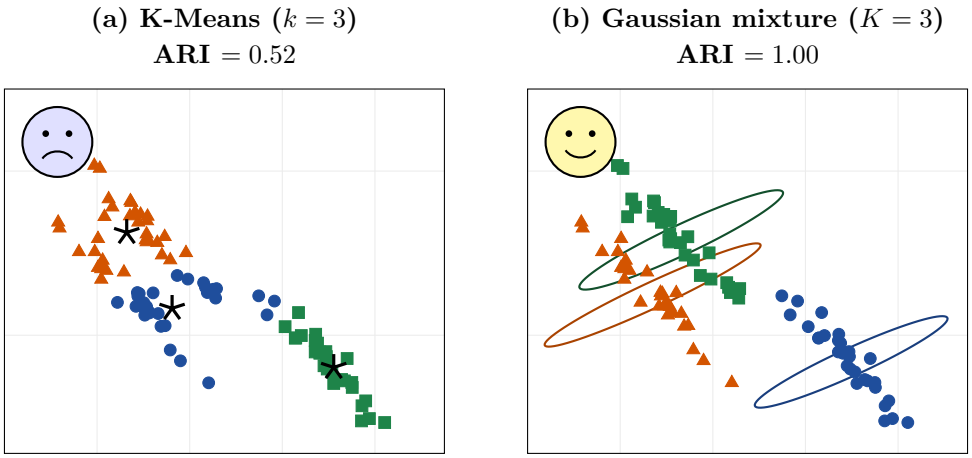


Figure 7.10. Three elongated clusters (90 of the 300 points shown). K-Means cuts them into round pieces; the Gaussian mixture fits tilted ellipses.

Reading the Figure

The three clusters are *stretched and tilted*. **Panel (a):** K-Means (stars are its centroids) can only draw boundaries halfway between its centres, which is right for round clusters but wrong here: two of the elongated clusters are cut crosswise (ARI = 0.52). **Panel (b):** the Gaussian mixture learns a **covariance** for each component. The coloured **ellipses** are the contours (two standard deviations) of the fitted hills: each is long and thin and tilted in the same direction as its cluster, so the three groups are recovered perfectly (ARI = 1.00).

Covariance type	spherical	diag	**tied**	**full**
ARI (true clusters known)	0.50	0.50	1.00	1.00
BIC (lower is better)	2089.3	2085.9	**1608.8**	1639.2

The spherical and diagonal models cannot represent the tilted ellipses and perform like K-Means. The tied model has the lowest BIC because these three clusters have the *same shape* (the data were generated that way), so it needs fewer parameters than the full model for the same fit.

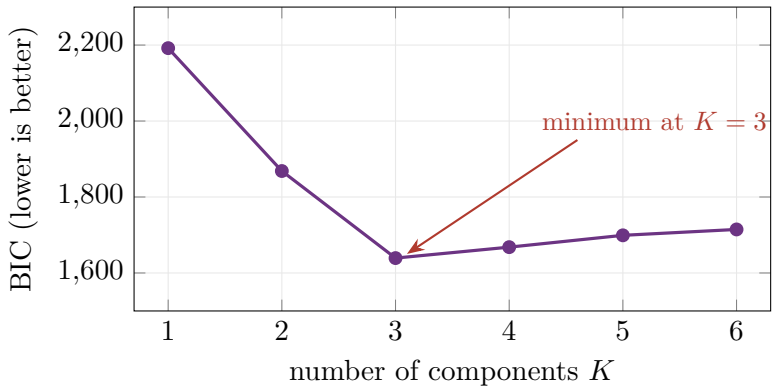


Figure 7.11. BIC for Gaussian mixtures (full covariance) with $K = 1, \ldots, 6$ on the elongated-clusters data.

Reading the Figure

The BIC drops sharply from $K = 1$ (2192) to $K = 3$ (1639) because the model fits the data much better. After $K = 3$ it *rises* again (1668, 1699, 1715): extra components improve the fit only slightly, and the penalty for the extra parameters outweighs the gain. The lowest point, $K = 3$, is the model BIC selects, and it matches the true number of clusters.

7.8.5 Other Uses and Cautions

- **Anomaly detection:** points with very low density $p(\boldsymbol{x})$ are outliers.
- **Generating data:** a fitted GMM can *sample* new, realistic points.
- **Local optima and singularities:** EM can converge to a poor solution (use several initialisations), and a component can collapse onto a single point ($\boldsymbol{\Sigma}_k \to 0$, likelihood $\to \infty$); libraries add a small regularisation term to prevent this.
- **Gaussian assumption:** GMMs work well for blob-like (even elongated) clusters but not for crescents or rings (see the moons result, ARI 0.46).

Try It Yourself

```
from sklearn.datasets import make_blobs
from sklearn.mixture import GaussianMixture
import numpy as np

X, y = make_blobs(n_samples=300, centers=3, random_state=170)
X = X @ np.array([[0.6, -0.64], [-0.4, 0.85]])    # stretch the
    blobs

for k in range(1, 7):
    g = GaussianMixture(n_components=k, covariance_type="full",
                        n_init=5, random_state=0).fit(X)
    print(k, round(g.bic(X), 1))          # lowest at k = 3
        (1639.2)

g = GaussianMixture(3, covariance_type="full", n_init=5,
                    random_state=0).fit(X)
print(g.weights_.round(3))                # mixing weights pi_k
print(g.predict_proba(X[:2]).round(3))    # soft responsibilities
```

Challenge: compare `covariance_type` values `"spherical"`, `"diag"`, `"tied"`, `"full"` using BIC and the adjusted Rand index.

Advantages

- **Soft clustering**: gives membership probabilities
- Handles elongated, tilted and different-sized clusters
- Probabilistic model: BIC for choosing K, anomaly detection, sampling
- Generalises K-Means

Disadvantages

- Must choose K; assumes Gaussian-shaped clusters
- Can get stuck in local optima; sensitive to initialisation
- More parameters and slower than K-Means
- Fails on crescents and rings

Applications

- Customer and market segmentation with overlap
- Speaker and speech recognition
- Anomaly detection and density estimation
- Image and background modelling

7.9 Choosing a Clustering Algorithm

	K-Means	**DBSCAN**	**Gaussian mixture**
Type	Centroid-based	Density-based	Model-based (probabilistic)
Needs number of clusters?	Yes (k)	No	Yes (K)
Other parameters	Initialisation	ε, MinPts	Covariance type
Cluster shape	Round, similar size	**Any shape**	Elliptical
Membership	Hard	Hard (plus noise)	**Soft (probabilities)**
Outliers / noise	Sensitive	**Detects noise**	Can model low density
Speed on big data	**Fast**	Fast with an index	Slower
Main weakness	Non-convex clusters	Varying densities, ε	Non-Gaussian shapes

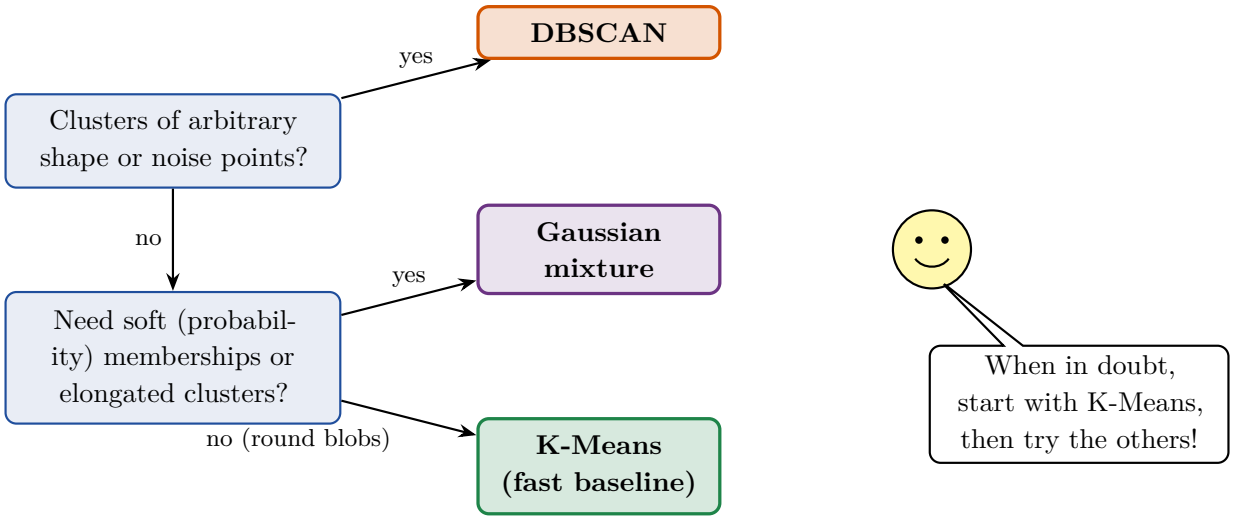


Figure 7.12. A simple guide for choosing a clustering algorithm.

Reading the Figure

Start at the top question. If the clusters may have **arbitrary shapes** (rings, crescents) or you need to find outliers, use **DBSCAN**. If not, ask whether you need **soft memberships** or the clusters are elongated ellipses: then use a **Gaussian mixture**. Otherwise (round, similar-sized blobs) **K-Means** is fast and sufficient. In practice it is common to try K-Means first as a baseline, then compare with the others using the silhouette score or domain knowledge.

Chapter Summary

- **Clustering** groups unlabelled data so that similar items fall together.
- **K-Means:** minimise inertia $J = \sum_j \sum_{x \in C_j} \|x - \mu_j\|^2$; alternate assignment (nearest centroid) and update (mean). Converges to a local minimum; use **K-Means++** and several runs. Choose k by the **elbow** and the **silhouette** $s = \frac{b-a}{\max(a,b)}$.
- **Limits of K-Means:** needs k; local minima; round same-sized clusters only; sensitive to outliers and scale; hard assignments (moons ARI 0.23, circles 0.00).
- **Image segmentation:** cluster pixel colours; replace each pixel by its centroid colour; also compresses the image (e.g. $k = 4$: 10.7× smaller).
- **Preprocessing:** represent instances by distances to k centroids; **semi-supervised:** label representatives and propagate (50 labels: 0.813 → 0.924).
- **DBSCAN:** core, border and noise points with parameters ε and MinPts; finds arbitrary-shaped clusters and noise; choose ε from the k-distance knee.
- **Gaussian mixture:** $p(x) = \sum \pi_k \mathcal{N}(x \mid \mu_k, \Sigma_k)$; EM alternates E-step (responsibilities) and M-step (weighted μ, Σ, π); soft clustering; choose K by BIC.

Exam Corner

A. Very Short Answer Questions (2 marks each)

1. Define clustering. How is it different from classification? **[2 marks]**
2. Define inertia (within-cluster sum of squares). **[2 marks]**
3. Write the two steps of the K-Means algorithm. **[2 marks]**
4. Why does K-Means need several random initialisations? What is K-Means++? **[2 marks]**
5. What is the elbow method? **[2 marks]**
6. Define the silhouette coefficient. What does a negative value mean? **[2 marks]**
7. Name any three limitations of K-Means. **[2 marks]**

8. What is colour segmentation using K-Means? **[2 marks]**

9. Define core point, border point and noise point in DBSCAN. **[2 marks]**

10. What do ε and MinPts mean in DBSCAN? **[2 marks]**

11. What is a Gaussian mixture model? What is a responsibility? **[2 marks]**

12. Differentiate between hard and soft clustering. **[2 marks]**

Model Answer: Q3, Q6 and Q9

Q3. (i) *Assignment step:* assign every point to its nearest centroid. (ii) *Update step:* move each centroid to the mean of the points assigned to it. The two steps are repeated until the assignments stop changing.

Q6. For a point i, $a(i)$ is its mean distance to the other points of its own cluster and $b(i)$ its smallest mean distance to another cluster; $s(i) = \frac{b(i)-a(i)}{\max\{a(i),b(i)\}} \in [-1, 1]$. A negative value ($b < a$) means that the point is closer on average to another cluster than to its own, so it is probably misassigned.

Q9. A *core point* has at least MinPts points (including itself) within distance ε. A *border point* is not core but lies within ε of a core point. A *noise point* is neither core nor border.

B. Short Answer Questions (5 marks each)

1. Explain the K-Means algorithm with a suitable example. **[5 marks]**

2. Explain how the number of clusters k is chosen (elbow and silhouette methods). **[5 marks]**

3. Explain K-Means++ initialisation and why it is better than random initialisation. **[5 marks]**

4. Explain the limitations of K-Means with examples. **[5 marks]**

5. Explain how K-Means is used for image segmentation. **[5 marks]**

6. Explain how clustering can be used for preprocessing. **[5 marks]**

7. Explain how clustering is used for semi-supervised learning. **[5 marks]**

8. Explain the DBSCAN algorithm. How are ε and MinPts chosen? **[5 marks]**

9. Explain Gaussian mixture models and the EM algorithm. **[5 marks]**

10. Compare K-Means, DBSCAN and Gaussian mixtures. **[5 marks]**

Model Answer: B10 (comparison for 5 marks)

Basis	K-Means	DBSCAN	Gaussian mixture
Idea	Nearest centroid	Dense regions	Mixture of Gaussians (EM)
Number of clusters	Given (k)	Found automatically	Given (K)
Shape	Round	Arbitrary	Elliptical
Membership	Hard	Hard; noise labelled	Soft (probability)
Outliers	Sensitive	Detected as noise	Low-density points
Parameters	k	ε, MinPts	K, covariance type

C. Long Answer Questions (10 marks each)

1. Explain the K-Means algorithm in detail. Perform two iterations on a small two-dimensional dataset and compute the inertia. **[10 marks]**

2. Explain how to choose k in K-Means and the limitations of K-Means. Why does it fail on non-spherical clusters? **[10 marks]**

3. Explain the use of clustering for (a) image segmentation, (b) preprocessing and (c) semi-supervised learning, with examples. **[10 marks]**

4. Explain the DBSCAN algorithm with the definitions of core, border and noise points and density-reachability. Classify the points of a given small dataset. **[10 marks]**

5. Explain Gaussian mixture models: the model, responsibilities, the EM algorithm and the choice of the number of components. Work out one E-step and one M-step. **[10 marks]**

6. Compare K-Means, DBSCAN and Gaussian mixture models in terms of working, cluster shape, parameters, advantages and disadvantages. **[10 marks]**

Model Answer: C1 (numerical part)

Data (one dimension): $\{2, 4, 6, 12, 14, 16\}$, $k = 2$, initial centroids $\mu_1 = 3$, $\mu_2 = 13$.
Iteration 1. Assignment: $2, 4, 6 \to \mu_1$ (distances $1, 1, 3$ against $11, 9, 7$); $12, 14, 16 \to \mu_2$.
Update: $\mu_1 = \frac{2+4+6}{3} = 4$, $\mu_2 = \frac{12+14+16}{3} = 14$.
Iteration 2. Assignments are unchanged, so the algorithm has converged.
Inertia: cluster 1: $(2-4)^2 + (4-4)^2 + (6-4)^2 = 8$; cluster 2: $(12-14)^2 + 0 + (16-14)^2 = 8$; $J = \mathbf{16}$.

D. Numerical Problems

1. Find the new centroid of the cluster $\{(1,2),(3,4),(5,0)\}$. *(Answer:* $(3,2)$*.)*

2. Data $\{2,4,6,12,14,16\}$ with initial centroids 3 and 13: perform K-Means and give the final centroids and inertia. *(Answer:* 4 *and* 14*;* $J = 16$*.)*

3. A point has $a = 2$ and $b = 6$. Find its silhouette. Find it also for $a = 5$, $b = 3$. *(Answer:* $\frac{6-2}{6} = 0.667$*;* $\frac{3-5}{5} = -0.4$*.)*

4. K-Means++: points $0, 2, 10$, first centroid 0. Find the probability of choosing each point as the second centroid. *(Answer:* $D^2 = 0, 4, 100$*;* $P = 0,\ 0.038,\ 0.962$*.)*

5. One-dimensional points $\{1, 2, 3, 100\}$: find the mean and the median and comment. *(Answer: mean* 26.5*, median* 2.5*; the mean is dragged by the outlier.)*

6. DBSCAN with $\varepsilon = 1$, MinPts $= 3$ on the one-dimensional points 1, 1.5, 2, 2.4, 5, 9. Classify each point. *(Answer: neighbourhood sizes* $3, 4, 4, 3, 1, 1$*; the first four are core and form one cluster;* 5 *and* 9 *are noise.)*

7. A GMM has $\pi = (0.4, 0.6)$. For a point x the component densities are 0.2 and 0.1. Find the responsibilities. *(Answer:* $\gamma_1 = \frac{0.08}{0.08+0.06} = 0.571$*,* $\gamma_2 = 0.429$*.)*

8. M-step: points $x = 1, 3, 5$ with responsibilities $\gamma_{i1} = (1, 0.5, 0)$. Find N_1, μ_1, π_1 and σ_1^2. *(Answer:* $N_1 = 1.5$*;* $\mu_1 = \frac{1+1.5}{1.5} = 1.667$*;* $\pi_1 = 0.5$*;* $\sigma_1^2 = \frac{(1-1.667)^2+0.5(3-1.667)^2}{1.5} = 0.889$*.)*

9. BIC values for $K = 1, 2, 3, 4$ are $2000, 1700, 1650, 1690$. Which K is chosen? *(Answer:* $K = 3$*, the lowest BIC.)*

10. An image of 100×100 pixels (24 bits per pixel) is quantised to $k = 8$ colours. Find the compressed size and reduction factor. *(Answer:* $10\,000 \times 3 + 8 \times 24 = 30\,192$ *bits;* $240\,000/30\,192 = 7.95$*.)*

E. Multiple-Choice Questions

1. K-Means minimises:
   (a) the number of clusters (b) the within-cluster sum of squares (c) the silhouette (d) the entropy

2. In the update step of K-Means each centroid becomes the:
   (a) median of its points (b) mean of its points (c) farthest point (d) first point

3. K-Means++ is a method for:
   (a) choosing k (b) initialising centroids (c) scaling features (d) removing noise

4. The best k by the silhouette method is the one with the:
   (a) lowest score (b) highest score (c) score nearest 0 (d) largest inertia

5. K-Means fails on two interleaving half-moons because:
   (a) it is too slow (b) it assumes round, convex clusters (c) it needs labels (d) it ignores distance

6. In colour segmentation with K-Means each pixel is a point in:
   (a) 1-D (b) 2-D position space (c) 3-D RGB space (d) 4-D space

7. In DBSCAN a point with fewer than MinPts neighbours but within ε of a core point is a:
   (a) core point (b) border point (c) noise point (d) centroid

8. DBSCAN does *not* require:
   (a) ε (b) MinPts (c) the number of clusters (d) a distance measure

9. A Gaussian mixture model gives each point:
   (a) one label (b) a probability for each cluster (c) no label (d) only a distance

10. The EM algorithm alternates:
    (a) assignment and update (b) E-step and M-step (c) split and merge (d) train and test

11. Which criterion is used to choose the number of Gaussian components?
    (a) Elbow only (b) BIC (c) Accuracy (d) Gini

12. Which algorithm detects noise points automatically?
    (a) K-Means (b) DBSCAN (c) both (d) none

Answers: 1-b, 2-b, 3-b, 4-b, 5-b, 6-c, 7-b, 8-c, 9-b, 10-b, 11-b, 12-b.

F. Practice Problems

1. Run K-Means by hand on $\{1, 2, 3, 10, 11, 12\}$ with $k = 2$ and initial centroids 1 and 2. *(Answer: $\mu = (2,\ 11)$ after two iterations; $J = 4$.)*

2. Compute the silhouette of each point for the clustering $\{1, 2, 3\}$, $\{10, 11, 12\}$.

3. Apply DBSCAN with $\varepsilon = 1.5$, MinPts $= 3$ to the points in Figure 7.8 and explain why E becomes a core point.

4. **Coding:** cluster the *iris* data with K-Means, DBSCAN and a Gaussian mixture, and compare their adjusted Rand indices with the true species.

5. **Coding:** segment a photograph with $k = 2, 4, 8, 16$ colours and plot the palette of each.

6. **Coding:** implement K-Means from scratch with `numpy` (assignment and update steps) and verify it on the six-point example.

Quick Revision

Topic	One-line recall
Clustering	Unsupervised grouping of similar items; no labels.
K-Means	Assign to nearest centroid; centroid = mean; minimise inertia J.
K-Means++	Pick far-apart starting centroids with probability $\propto D^2$.
Choose k	Elbow of inertia curve; maximum mean silhouette.
Silhouette	$s = \frac{b-a}{\max(a,b)} \in [-1, 1]$.
Limits	Needs k; local minima; round clusters; outliers; scaling; hard labels.
Image segmentation	Cluster RGB pixels; replace by centroid; palette compression.
Preprocessing	Use distances to k centroids as features.
Semi-supervised	Label representatives, propagate to clusters.
DBSCAN	Core ($\geq$MinPts in ε), border, noise; arbitrary shapes; ε from k-distance knee.
GMM	$p(x) = \sum \pi_k \mathcal{N}(x \mid \mu_k, \Sigma_k)$; EM (E: responsibilities, M: weighted updates); BIC.
Choosing algorithm	Round blobs: K-Means; odd shapes/noise: DBSCAN; soft/elliptical: GMM.

Chapter 8

Dimensionality Reduction and Principal Component Analysis

Learning Objectives

After studying this chapter you will be able to:

- explain the **curse of dimensionality** and its effects on distances, data requirements and learning algorithms;
- distinguish **projection** from **manifold learning** and name the main methods in each family;
- explain **Principal Component Analysis (PCA)** and carry it out **by hand**: covariance matrix, eigenvalues, eigenvectors, projection and reconstruction;
- use **explained variance** to choose the number of components;
- explain why **feature scaling** is essential for PCA;
- use PCA for **visualisation**, **compression** and **preprocessing**;
- describe **Kernel PCA**, **Incremental PCA** and **Randomised PCA**;
- answer 2-, 5- and 10-mark university questions on this part of Unit IV.

> *Everything should be made as simple as possible, but no simpler.*
>
> — often attributed to Albert Einstein

8.1 Why Reduce the Number of Features?

Real datasets can have a huge number of features: a 64×64 photograph has 4096 pixel values, a gene-expression record has tens of thousands of measurements, and a document may have thousands of word counts. Yet most of these numbers are redundant: neighbouring pixels are almost equal, many genes move together, and most words are rare.

Definition: Dimensionality reduction

Dimensionality reduction is the process of transforming data from a high-dimensional space $\mathbb{R}^d$ into a lower-dimensional space $\mathbb{R}^m$ ($m < d$) while *keeping as much of the useful information as possible*. It is an *unsupervised* technique: it uses only the features $\boldsymbol{x}$, not the labels.

Why do we do it?

- **Visualisation:** humans can look at 2-D or 3-D plots, not at 50-D data.
- **Speed and memory:** fewer features make training and prediction faster.
- **Fighting the curse of dimensionality:** many learning algorithms perform worse as features are added (next section).
- **Removing noise and redundancy:** the discarded directions are often mostly noise.
- **Compression:** store 30 numbers instead of 64.

8.2 The Curse of Dimensionality

Definition: Curse of dimensionality

The **curse of dimensionality** (a term coined by Richard Bellman) refers to the collection of problems that appear when the number of features d becomes large: the volume of the space grows so fast that *any fixed amount of data becomes extremely sparse*; distances between points become almost equal and therefore lose meaning; and the amount of data needed to learn reliably grows exponentially with d.

8.2.1 Cartoon: A Lonely Point in a Big Space

Put the same ten points in a line, in a square and in a cube. In one dimension they crowd together and every point has many close neighbours. In three dimensions the same ten points are lost in a huge space.

Reading the Figure

In all three drawings there are exactly ten blue points, but the **space they live in grows**: a line, a square, a cube. The faces show how it feels to be a point. In 1-D a ball of radius 0.1 around a point covers 20% of the whole line, so a point is surrounded by friends (happy face). In 2-D the same radius covers only $\pi(0.1)^2 = 3.1\%$ of the square, so neighbours are rarer (neutral face). In 3-D it covers $\frac{4}{3}\pi(0.1)^3 = 0.42\%$ of the cube: most points are *alone* (sad face). With 10 features the fraction is 2.6×10^{-10}. To keep the same density of data

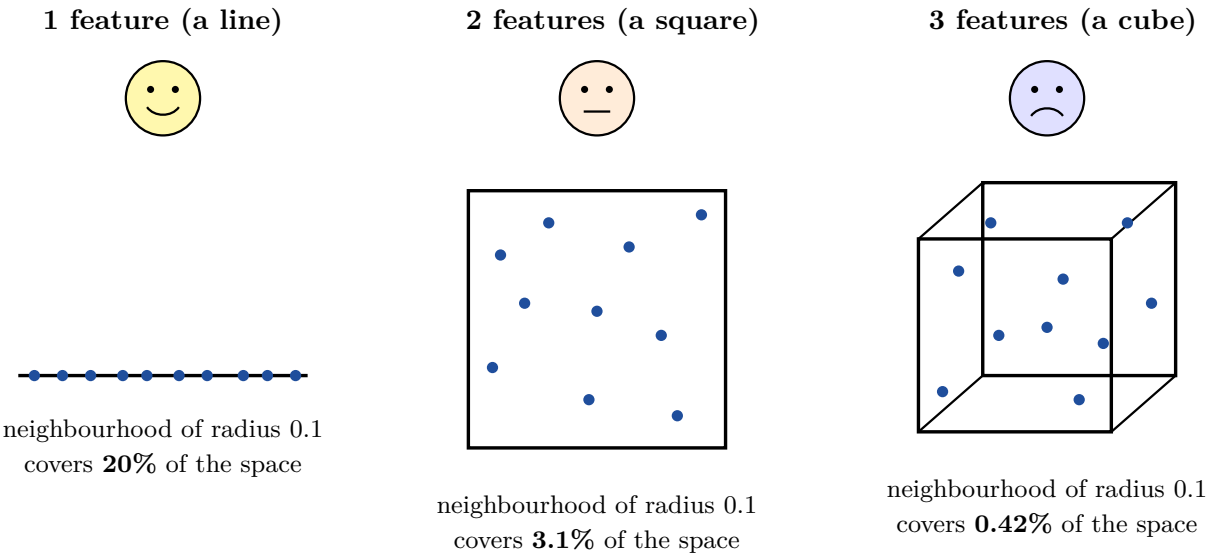


Figure 8.1. The same ten points become more and more isolated as the number of dimensions grows.

we would need an exponentially larger sample.

8.2.2 Mathematical Facts

1. Volume of a neighbourhood shrinks. The volume of a ball of radius r in d dimensions is $V_d(r) = \dfrac{\pi^{d/2}}{\Gamma(d/2+1)}\, r^d$. For $r = 0.1$ inside a unit cube:

Dimension d	1	2	3	5	10
Fraction of the unit cube covered	0.2	0.031	0.0042	5.3×10^{-5}	2.6×10^{-10}

2. Almost every point is near the boundary. Take a random point in the unit hypercube. The probability that it lies within 0.001 of at least one border (of its $2d$ faces) is $1 - (1 - 0.002)^d$:

d	1	2	10	100	1000	10 000
P(near a border)	0.2%	0.4%	2.0%	18%	86%	$\approx 100\%$

In high dimensions practically *all* data lie in a thin shell near the surface: there is hardly any "inside".

3. Distances concentrate. Pick random points uniformly in the unit hypercube. We measured the distance from one query point to 500 others and computed the **relative contrast** $\dfrac{d_{\max} - d_{\min}}{d_{\min}}$ (how much farther is the farthest point than the nearest?).

Dimension d	2	3	10	100	1000
Mean distance between two random points	0.55	0.66	1.23	4.03	12.9
Nearest point	0.03	0.06	0.50	3.46	12.0
Farthest point	1.27	1.00	1.50	4.88	13.6
Relative contrast	**40.7**	16.9	2.0	0.41	**0.13**

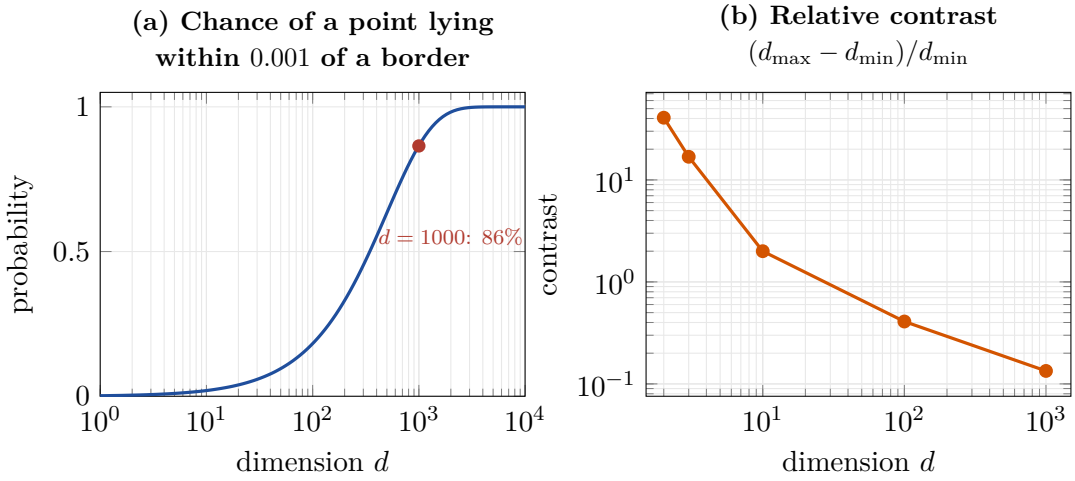


Figure 8.2. Two consequences of the curse of dimensionality (panel b is from a real simulation).

Reading the Figure

Panel (a) shows $1 - 0.998^d$ against the number of dimensions (note the logarithmic horizontal axis). It starts near 0, but by $d = 1000$ (red dot) a random point is almost certainly (86%) next to a border, and by $d = 10\,000$ it is certain. **Panel (b)** shows what happens to distances: in 2-D the farthest point is 40 times farther than the nearest one, so "nearest neighbour" is a meaningful idea; in 1000-D the farthest point is only 13% farther than the nearest, so *every point is about equally far* from every other. Algorithms that rely on distance (k-NN, K-Means, DBSCAN, kernel methods) lose their power.

4. Data requirements explode. To have just *one* training example in each cell of a grid that splits every feature into 10 intervals we need 10^d examples:

Features d	1	2	3	6	10
Cells (examples needed)	10	100	1000	10^6	10^{10}

8.2.3 Real Effect on a Learning Algorithm

We took the 4 informative features of the *iris* data and added k extra features that are pure random noise. Then we measured the test accuracy of 5-nearest-neighbours (average of 30 random

splits, features standardised).

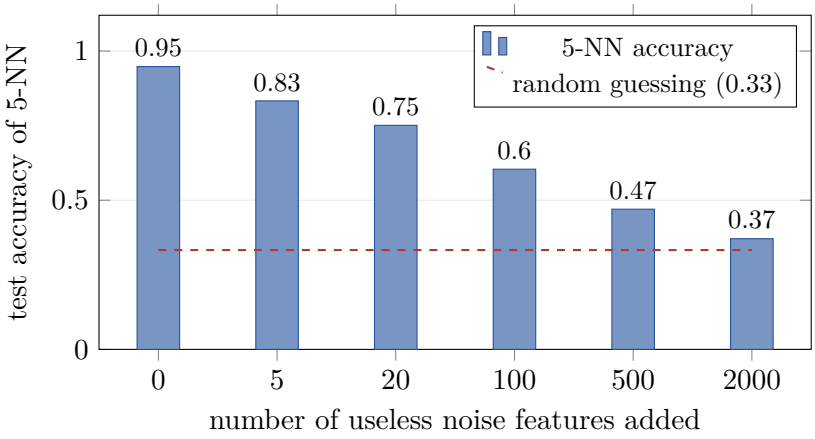


Figure 8.3. k-NN accuracy on iris as useless features are added (real results).

Reading the Figure

Each bar is the accuracy of the same classifier on the same flowers; the only difference is the number of extra random columns. With no noise features the accuracy is 94.8%. Adding just 5 junk features drops it to 83.3%, 100 junk features to 60.4%, and with 2000 junk features it is 37.1%, almost the level of random guessing among three classes (33%, red dashed line). The noise features dominate the distances, so the true neighbours are no longer the nearest. This is why reducing the dimensions (or selecting good features) before learning can help a lot.

Key Idea

Remedies for the curse: collect more data (rarely enough), *select* only useful features, or *reduce* the dimensions with methods such as PCA. These work because real data rarely fill the whole high-dimensional space: they lie close to a much lower-dimensional *surface*.

8.3 Main Approaches for Dimensionality Reduction

There are two main families of methods, based on two different ideas about the shape of the data.

Definition: Projection

Projection reduces dimensions by projecting every point *orthogonally onto a lower-dimensional linear subspace* (a line, a plane, ...) chosen to keep as much information as possible. A projection is a *linear* map $\boldsymbol{z} = \mathbf{W}^\top \boldsymbol{x}$. *Analogy:* a shadow cast on a wall.

Definition: Manifold and manifold learning

A m-dimensional **manifold** is a shape that locally looks like m-dimensional flat space, even though it may be bent or rolled inside a higher-dimensional space (a sheet of paper rolled into a scroll is a 2-D manifold in 3-D). The **manifold hypothesis** says that real high-dimensional data lie close to a low-dimensional manifold. **Manifold learning** methods try to *unroll* that manifold and describe the data by their position *on* it, using *non-linear* maps.

8.3.1 Cartoon: Shadows versus Unrolling

The Swiss roll is the classic example: a flat strip (2-D) rolled up in 3-D. To keep the picture simple we draw its one-dimensional version: a **spiral line** (1-D) rolled up in the plane (2-D). The colour goes smoothly from blue (start of the line) to orange (end).

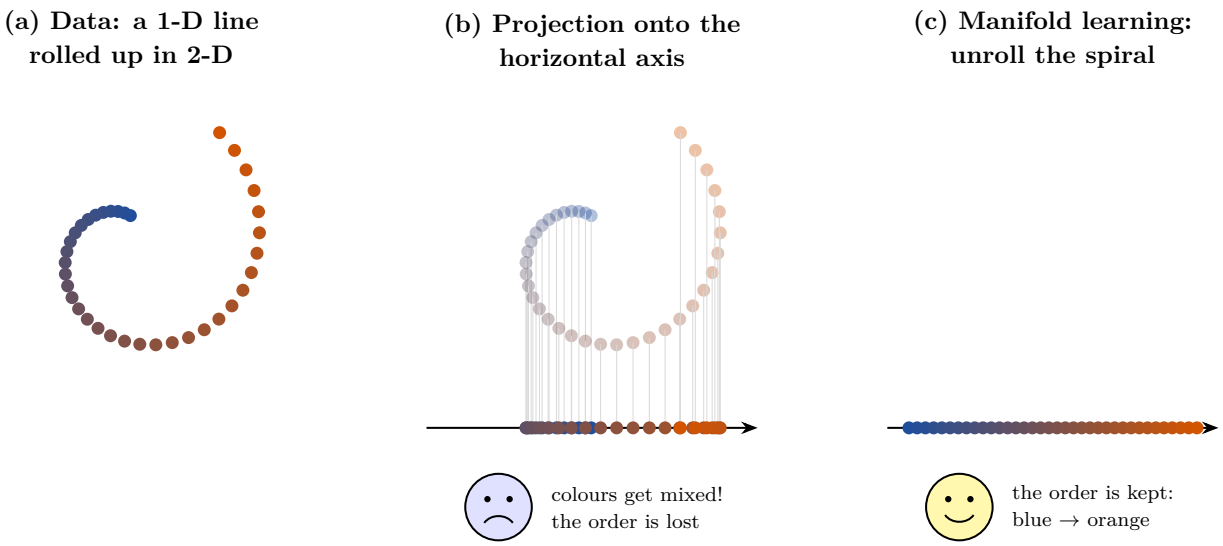


Figure 8.4. Projection versus manifold learning on a rolled-up line.

Reading the Figure

(a) The 36 points really lie on a *line* (1-D) that has been rolled into a spiral, so the ideal one-dimensional description is "how far along the line". **(b)** A linear projection simply drops the vertical coordinate (like a shadow). Points from different turns of the spiral land on top of each other and the colours get mixed: neighbours in the projection are *not* neighbours on the line. **(c)** A manifold-learning method recovers the position along the line, so the colours appear in order from blue to orange. Real data (images, speech, motion) often behave like this: a few hidden factors, bent into a high-dimensional space.

Real results. We generated 1000 points on a 3-D Swiss roll with true position t along the roll, reduced them to *one* dimension, and measured the absolute rank correlation (Spearman) between the recovered coordinate and t (1 = perfect order).

Method (1-D output)	|Spearman correlation| with true position
PCA (linear projection)	0.20
Isomap (manifold learning, 10 neighbours)	**1.00**
Locally Linear Embedding (12 neighbours)	**1.00**

PCA cannot unroll the roll (correlation 0.20), while Isomap and LLE recover the position perfectly.

8.3.2 Main Methods

Method	Family	Main idea
PCA	Projection	Project onto the directions of maximum variance (Section 8.4)
Random projection	Projection	Multiply by a random matrix; distances are approximately preserved (Johnson–Lindenstrauss lemma); very fast
LDA (linear discriminant analysis)	Projection (*supervised*)	Project onto the directions that best *separate the classes*
MDS (multidimensional scaling)	Distance-preserving	Find low-dimensional points whose distances match the original distances
Isomap	Manifold learning	Measure distance *along the manifold* (shortest paths through a neighbourhood graph), then apply MDS
LLE (locally linear embedding)	Manifold learning	Express every point as a linear combination of its neighbours and find low-dimensional points with the same combinations
t-SNE	Manifold learning (visualisation)	Keep points that are neighbours close together in 2-D/3-D; used to draw clusters

Projection methods (PCA, random projection, LDA)

Advantages

- Simple, fast, and mathematically well understood
- Give an explicit map that can be applied to *new* data
- Easy to invert (approximate reconstruction)
- Work for very large datasets

Disadvantages

- Only *linear*: cannot unroll curved data
- Layers of a rolled structure are squashed together
- PCA ignores class labels
- Components can be hard to interpret

Applications

- Compression and noise removal
- Preprocessing before classifiers
- Visualising high-dimensional data in 2-D
- Face recognition (eigenfaces)

Manifold-learning methods (Isomap, LLE, t-SNE)

Advantages

- Capture non-linear structure
- Can unroll curved data
- t-SNE gives very informative cluster pictures
- Need no labels

Disadvantages

- Slow on large datasets
- No simple map for *new* points (e.g. t-SNE)
- Sensitive to parameters (neighbours, perplexity) and noise
- Results may be hard to interpret; distances in the plot can mislead

Applications

- Visualising images, text and genes (t-SNE)
- Motion, speech and image analysis
- Discovering hidden factors of variation
- Data exploration

8.4 Principal Component Analysis (PCA)

8.4.1 The Idea: Find the Best Camera Angle

Imagine photographing a long pencil. If you photograph it from the *side* you see its full length and can tell a lot about it. If you photograph it *end-on* you see only a dot and lose almost all the information. PCA finds, for a cloud of data points, the **best viewing directions**: the ones along which the points are most spread out. Keeping those directions and dropping the rest loses the least information.

Definition: Principal Component Analysis

Principal Component Analysis (PCA) is an *unsupervised linear* dimensionality-reduction technique that finds a new set of **orthogonal (perpendicular) axes**, called **principal components**, ordered by the amount of **variance** of the data along them. The first principal component is the direction of maximum variance; each next one is the direction of maximum variance that is perpendicular to all the previous ones. Projecting the data on the first m components gives the best m-dimensional linear approximation of the data.

Definition: Explained variance

If $\lambda_1 \geq \lambda_2 \geq \cdots \geq \lambda_d$ are the variances along the principal components (the eigenvalues of the covariance matrix), the **explained variance ratio** of component j is $\lambda_j / \sum_{i=1}^{d} \lambda_i$: the fraction of the total variance that the component captures. The **cumulative explained variance** of the first m components is $\sum_{j \leq m} \lambda_j / \sum_i \lambda_i$.

8.4.2 The Mathematics

Let the data matrix $\mathbf{X}$ have n rows (instances) and d columns (features).

Definition: The PCA algorithm

1. **Standardise / centre:** subtract the mean of every feature (and, if features have different units, divide by the standard deviation). Call the centred matrix $\tilde{\mathbf{X}}$.
2. **Covariance matrix:** $\mathbf{C} = \dfrac{1}{n-1}\tilde{\mathbf{X}}^\top\tilde{\mathbf{X}}$ (a $d \times d$ symmetric matrix).
3. **Eigen-decomposition:** find the eigenvalues λ_j and unit eigenvectors $\boldsymbol{w}_j$ of $\mathbf{C}$, $\mathbf{C}\boldsymbol{w}_j = \lambda_j \boldsymbol{w}_j$.
4. **Sort** the eigenvectors by decreasing eigenvalue; the eigenvectors are the principal components.
5. **Project:** keep the first m eigenvectors in a matrix $\mathbf{W}_m$ ($d \times m$) and compute $\mathbf{Z} = \tilde{\mathbf{X}}\mathbf{W}_m$ (an $n \times m$ matrix).
6. **Reconstruct** (approximately): $\hat{\mathbf{X}} = \mathbf{Z}\mathbf{W}_m^\top + \text{mean}$.

Why eigenvectors? The variance of the data projected on a unit vector $\boldsymbol{w}$ is $\boldsymbol{w}^\top\mathbf{C}\boldsymbol{w}$. Maximising it under the constraint $\|\boldsymbol{w}\| = 1$ with a Lagrange multiplier gives $\mathbf{C}\boldsymbol{w} = \lambda\boldsymbol{w}$: the best direction is an eigenvector of $\mathbf{C}$, and the variance along it equals the eigenvalue λ. Equivalently, PCA minimises the *mean squared reconstruction error*: the error left after keeping m components equals the sum of the discarded eigenvalues $\lambda_{m+1} + \cdots + \lambda_d$.

Computation by SVD. In practice PCA is computed with the *singular value decomposition* $\tilde{\mathbf{X}} = \mathbf{U}\mathbf{S}\mathbf{V}^\top$: the columns of $\mathbf{V}$ are the principal components and $\lambda_j = s_j^2/(n-1)$.

8.4.3 Worked Example: PCA by Hand, Step by Step

Example: Two features, five points

Data (a small study of two related measurements):

$$A(1,1), \quad B(2,3), \quad C(3,2), \quad D(4,5), \quad E(5,4).$$

Step 1: centre the data. The means are $\bar{x} = \frac{1+2+3+4+5}{5} = 3$ and $\bar{y} = \frac{1+3+2+5+4}{5} = 3$.

Point	A	B	C	D	E
$(x-\bar{x},\ y-\bar{y})$	$(-2,-2)$	$(-1,0)$	$(0,-1)$	$(1,2)$	$(2,1)$

Step 2: covariance matrix ($n-1 = 4$):

$$s_{xx} = \tfrac{4+1+0+1+4}{4} = 2.5, \quad s_{yy} = \tfrac{4+0+1+4+1}{4} = 2.5, \quad s_{xy} = \tfrac{4+0+0+2+2}{4} = 2.0,$$

$$\mathbf{C} = \begin{bmatrix} 2.5 & 2 \\ 2 & 2.5 \end{bmatrix}.$$

Step 3: eigenvalues. Solve $\det(\mathbf{C} - \lambda\mathbf{I}) = (2.5 - \lambda)^2 - 2^2 = 0$, so $2.5 - \lambda = \pm 2$:

$$\lambda_1 = 4.5, \qquad \lambda_2 = 0.5.$$

Total variance $= 4.5 + 0.5 = 5 = s_{xx} + s_{yy}$ ✓.

Step 4: eigenvectors. For $\lambda_1 = 4.5$: $(\mathbf{C} - 4.5\mathbf{I})\boldsymbol{w} = \begin{bmatrix} -2 & 2 \\ 2 & -2 \end{bmatrix} \boldsymbol{w} = \mathbf{0} \Rightarrow w_x = w_y$; normalised, $\boldsymbol{w}_1 = (\frac{1}{\sqrt{2}}, \frac{1}{\sqrt{2}}) = (0.707, 0.707)$. For $\lambda_2 = 0.5$: $\begin{bmatrix} 2 & 2 \\ 2 & 2 \end{bmatrix} \boldsymbol{w} = \mathbf{0} \Rightarrow w_x = -w_y$; $\boldsymbol{w}_2 = (\frac{1}{\sqrt{2}}, -\frac{1}{\sqrt{2}}) = (0.707, -0.707)$. They are perpendicular: $\boldsymbol{w}_1 \cdot \boldsymbol{w}_2 = 0.5 - 0.5 = 0$.

Step 5: explained variance.

$$\text{PC1: } \frac{4.5}{5} = \mathbf{90}\%, \qquad \text{PC2: } \frac{0.5}{5} = 10\%.$$

Step 6: project all points onto the components, $z_1 = \frac{(x-\bar{x})+(y-\bar{y})}{\sqrt{2}}$, $z_2 = \frac{(x-\bar{x})-(y-\bar{y})}{\sqrt{2}}$:

Point	A	B	C	D	E	Variance
z_1 (PC1)	-2.828	-0.707	-0.707	2.121	2.121	$\frac{8+0.5+0.5+4.5+4.5}{4} = 4.5$
z_2 (PC2)	0	-0.707	0.707	-0.707	0.707	$\frac{0+0.5+0.5+0.5+0.5}{4} = 0.5$

The variances of the new coordinates are exactly the eigenvalues 4.5 and 0.5.

Step 7: reduce to one dimension and reconstruct. Keep only z_1. Reconstruction: $\hat{\boldsymbol{x}} = (3, 3) + z_1(0.707, 0.707)$:

Point	A	B	C	D	E
Original	(1, 1)	(2, 3)	(3, 2)	(4, 5)	(5, 4)
Reconstructed	(1, 1)	(2.5, 2.5)	(2.5, 2.5)	(4.5, 4.5)	(4.5, 4.5)
Squared error	0	0.5	0.5	0.5	0.5

Total squared error $= 2.0 = (n-1)\lambda_2 = 4(0.5)$ ✓: the information lost is exactly the discarded variance. We have compressed 10 numbers into 5 while keeping 90% of the variance.

Check with the SVD. The singular values of the centred matrix are $s_1 = \sqrt{18} = 4.243$ and $s_2 = \sqrt{2} = 1.414$; $s_j^2/(n-1) = 18/4 = 4.5$ and $2/4 = 0.5$, the same eigenvalues.

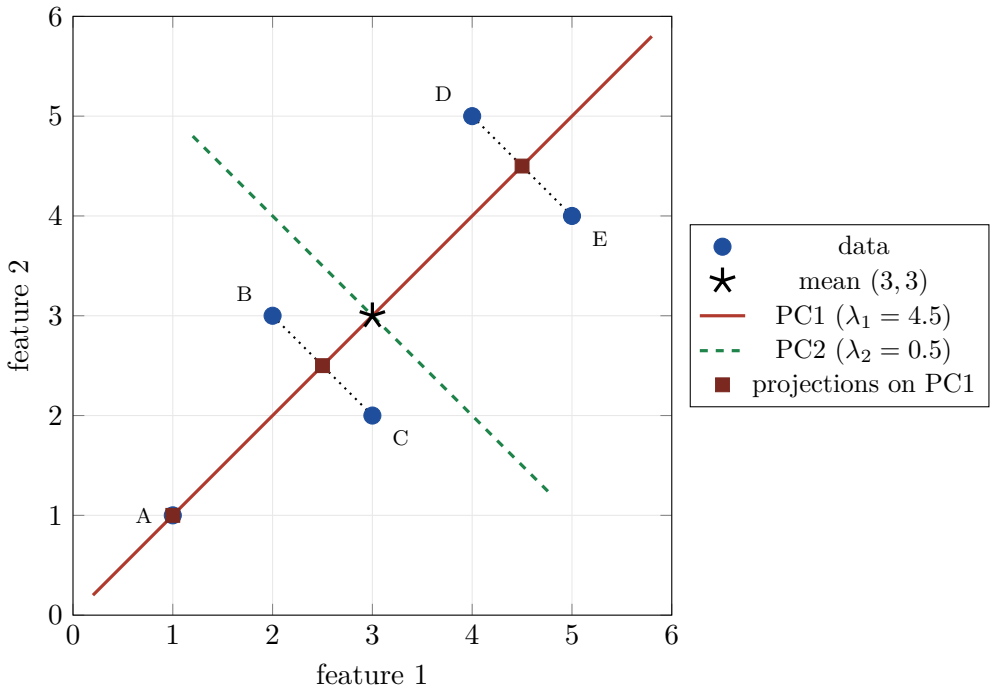


Figure 8.5. PCA of the five-point example: the principal axes and the projections on the first component.

Reading the Figure

The blue dots are the five data points and the black star is their mean. The **red solid line** is the first principal component: it runs along the direction in which the points are most spread out (the diagonal $y = x$, variance 4.5). The **green dashed line** is the second component, perpendicular to the first (the other diagonal, variance only 0.5). The **dotted segments** drop each point perpendicularly onto the red line; the red squares are the resulting projections. Points A, B, C, D, E are only a short distance from the red line (these distances are the information we lose), whereas their positions *along* the red line (the information we keep) are widely spread. Keeping just the red coordinate describes the data well with one number per point instead of two.

8.4.4 Choosing the Number of Components

How many components should we keep? There are three common methods.

1. **Cumulative explained variance threshold:** keep the smallest m with cumulative variance $\geq 95\%$ (or 90%, 99%).
2. **Scree plot (elbow):** plot the explained variance of each component; keep the components before the curve flattens.
3. **Task-based:** treat m as a hyperparameter and pick the value giving the best cross-validated accuracy of the final model.

For **visualisation** we always use $m = 2$ or 3.

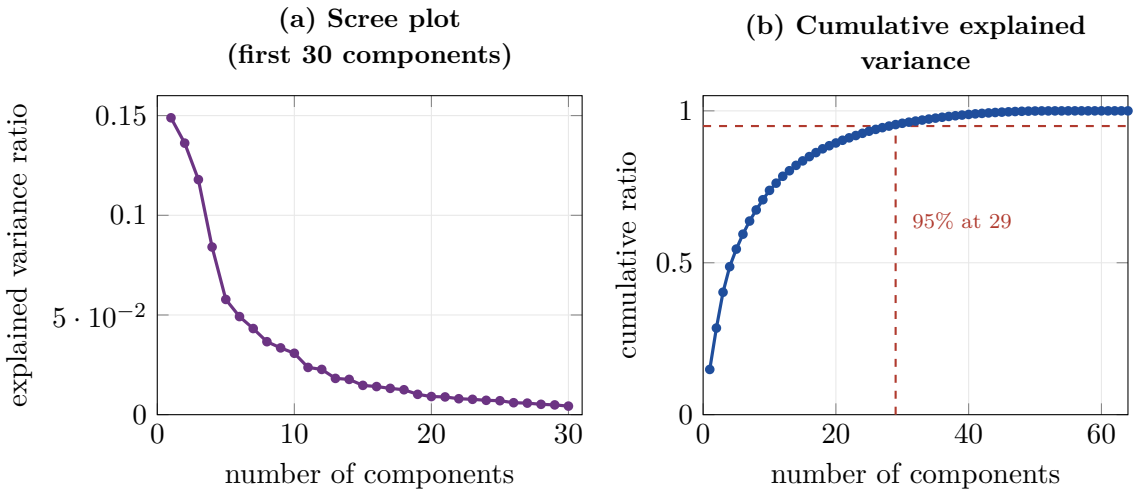


Figure 8.6. PCA of the 64-pixel handwritten digits (1797 images): explained variance (real data).

Reading the Figure

(a) Scree plot. Each point is one principal component. The first component explains 14.9% of the variance, the second 13.6%, the third 11.8%, and then the values fall quickly: components beyond about the 20th each explain well under 1%. **(b) Cumulative plot.** Adding components raises the total variance explained: 54.5% with 5 components, 73.8% with 10, 89.4% with 20, 95.9% with 30 and 100% with all 64. The red lines show the rule "keep 95%": it needs **29** components, less than half of the original 64 pixels.

8.4.5 Real Example 1: Visualising the Iris Flowers in 2-D

The iris data have four measurements per flower. PCA to two dimensions lets us *see* the data.

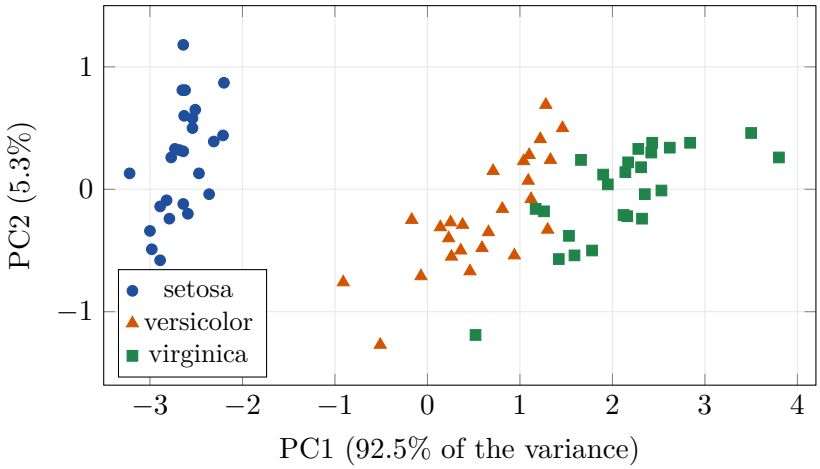


Figure 8.7. Iris flowers (half of each class shown) in the plane of the first two principal components.

Reading the Figure

Every dot is a flower, drawn by its two PCA coordinates instead of four measurements. The **setosa** flowers (blue circles) form a compact group far to the left, completely separated from the others. **Versicolor** (orange triangles) and **virginica** (green squares) sit to the right and overlap a little. Two numbers per flower are enough to see the class structure, and the two axes together keep $92.5 + 5.3 = 97.8\%$ of the variance. The first component is mostly *size*: its loadings are $(0.36, -0.09, 0.86, 0.36)$ for (sepal length, sepal width, petal length, petal width), dominated by petal length. (The PCA was fitted without using the species labels; the colours were added afterwards.)

	PC1	PC2	PC3	PC4
Explained variance, raw features	0.925	0.053	0.017	0.005
Explained variance, standardised features	0.730	0.229	0.037	0.005

8.4.6 Real Example 2: Compressing Handwritten Digits

Each digit image has 64 pixels. We apply PCA to all 1797 images and rebuild one digit (a "3") from only a few components.

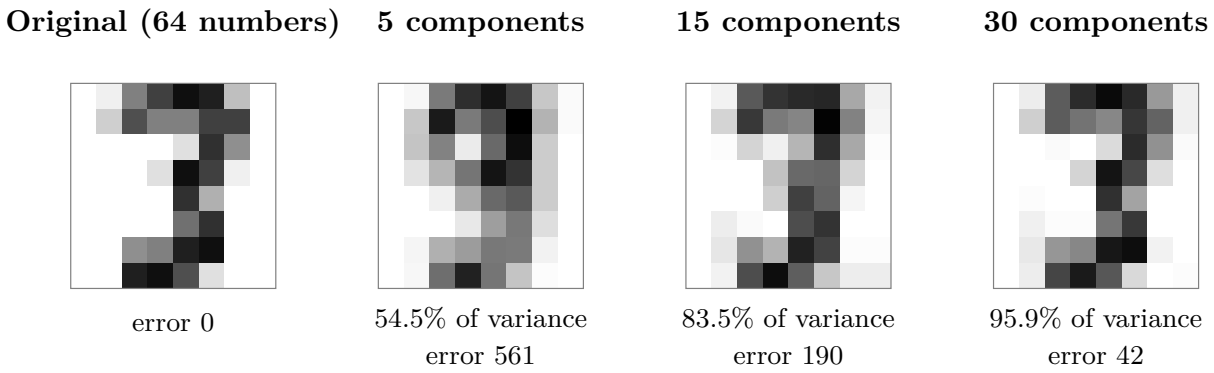


Figure 8.8. One handwritten digit reconstructed from 5, 15 and 30 principal components (real data; error = sum of squared pixel errors).

Reading the Figure

The first picture is the original 8×8 image (dark = more ink). With only **5 components** we get a blurry blob: the rough position of the ink is right but details are missing (squared error 561). With **15 components** the shape of the "3" emerges (error 190), and with **30 components** the digit is almost identical to the original (error 42, against 1149 for describing it by the average digit alone). Each extra component adds detail; the first few capture the coarse shape.

Now we use PCA as *preprocessing* for a classifier. We trained logistic regression on the PCA-reduced digits (1347 training and 450 test images):

Components	2	5	10	20
Test accuracy	0.636	0.867	0.922	0.951
Reconstruction MSE (per pixel)	13.31	8.56	4.96	2.07
Components	29	40	64 (all)	
Test accuracy	0.967	**0.976**	0.973	
Reconstruction MSE	0.90	0.24	0	

Read the table: with only **29** of 64 features (a 55% reduction) accuracy is 0.967, close to the full 0.973, and with **40** components it is 0.976, slightly *better* than with all features: the discarded components were mostly noise. Using PCA(0.95), the library keeps the smallest number of components explaining 95% of the variance (here 28 on the training data).

8.4.7 Real Example 3: Why You Must Scale the Features

PCA looks for directions of maximum *variance*, so a feature with large numbers will dominate. The wine dataset has 13 chemical measurements; one of them, proline, has values in the hundreds or thousands while the others are around 0–10.

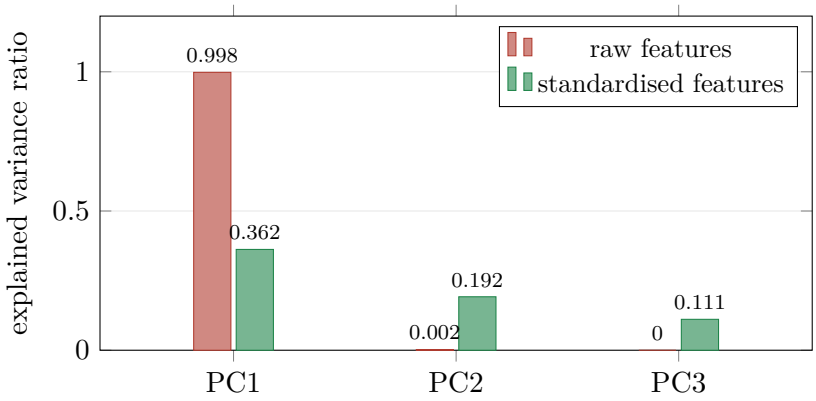


Figure 8.9. PCA on the wine data with and without standardising the features (real results).

Reading the Figure

Red bars (raw data): the first component alone claims 99.8% of the variance and the others are almost 0. This is an *artefact of units*: the loading of proline on PC1 is 1.00 and all other features have loadings near 0, so PC1 is simply "the proline column". **Green bars (standardised data):** after scaling every feature to mean 0 and standard deviation 1, the variance is shared among several components (36.2%, 19.2%, 11.1%): the 13 features all contribute. Without scaling, the PCA would have thrown away all the other chemistry.

Common Mistake

Always standardise the features before PCA when they have different units or ranges (for example height in metres and weight in grams). Only skip scaling when all features are measured in the same units (e.g. pixel intensities).

8.4.8 Properties, Limits and Variants

Properties.

- The principal components are **orthogonal**, and the new coordinates $z_1, z_2, \ldots$ are **uncorrelated**.
- The total variance is preserved: $\sum \lambda_j = \text{trace}(\mathbf{C})$.
- PCA is **sensitive to outliers** and to the **scale** of the features.

Limits.

- **Linear only:** it cannot unroll curved data (Swiss roll result above).
- **High variance $\neq$ useful:** PCA ignores labels; a direction with small variance may be exactly the one that separates the classes (then LDA is better).
- Components are combinations of all features and may be hard to interpret.

Variants.

Definition: Randomised, Incremental and Kernel PCA

- **Randomised PCA:** uses a stochastic algorithm that quickly approximates the first m components; much faster when $m \ll d$.
- **Incremental PCA:** processes the data in mini-batches, so it works when the dataset does not fit in memory or arrives as a stream (online learning).
- **Kernel PCA:** applies PCA *after* an implicit non-linear mapping, using the *kernel trick* of Chapter 6 (the RBF kernel $e^{-\gamma\|\boldsymbol{a}-\boldsymbol{b}\|^2}$, for example). It can find non-linear structure.

Example: Kernel PCA on two concentric circles

Two concentric circles (300 points) cannot be separated by a line. We reduced them to 2-D with PCA and with RBF Kernel PCA ($\gamma = 10$), then trained logistic regression:

Representation	Logistic-regression accuracy
Raw 2-D coordinates	0.51
Linear PCA (2 components)	0.51
RBF Kernel PCA (2 components)	**1.00**

Linear PCA only rotates the circles, so a straight line still cannot separate them. Kernel PCA "unfolds" them so that the two rings become linearly separable. (The kernel parameter γ must be tuned, just as for SVMs.)

Try It Yourself

```python
from sklearn.datasets import load_digits
from sklearn.decomposition import PCA
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
import numpy as np

X, y = load_digits(return_X_y=True)
Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.25,
                                      random_state=42)

pca = PCA().fit(X)
cum = np.cumsum(pca.explained_variance_ratio_)
print(np.searchsorted(cum, 0.95) + 1)        # 29 components

for k in [2, 10, 29, 40]:
    p = PCA(n_components=k).fit(Xtr)
    clf = LogisticRegression(max_iter=3000).fit(p.transform(Xtr),
        ytr)
    print(k, round(clf.score(p.transform(Xte), yte), 3))
# 2 -> 0.636, 10 -> 0.922, 29 -> 0.967, 40 -> 0.976

# keep enough components for 95% of the variance
print(PCA(n_components=0.95).fit(Xtr).n_components_)   # 28
```

Challenge: apply `PCA(2)` to the iris data (with and without `StandardScaler`) and compare the explained variance ratios with the table above.

Principal Component Analysis

Advantages

- Fast and simple; solved exactly by an eigen-decomposition
- Removes redundancy and noise; produces uncorrelated features
- Fewer features: faster and less overfitting
- Enables 2-D/3-D visualisation and compression

Disadvantages

- Only captures *linear* structure
- Must scale features first; sensitive to outliers
- High variance does not always mean high usefulness (ignores labels)
- Components are hard to interpret

Applications

- Image compression and eigenfaces
- Preprocessing for classifiers and clustering
- Visualising high-dimensional data
- Noise reduction; finance and genetics

Kernel PCA

Advantages

- Captures non-linear structure
- Can unfold circles and curved clusters
- Same easy interface as PCA
- Uses the familiar kernel trick

Disadvantages

- Needs the kernel and its parameters to be tuned
- Memory and time grow with the square of the number of points
- Reconstruction (pre-image) is difficult
- Components are even harder to interpret

Applications

- Pre-processing for classifiers on curved data
- Non-linear visualisation and denoising
- Pattern recognition tasks

8.5 Choosing a Dimensionality-Reduction Method

	PCA	**Kernel PCA / Isomap / LLE**	**t-SNE**
Type	Linear projection	Non-linear	Non-linear (visualisation)
Keeps	Global variance	Non-linear structure	Local neighbourhoods
New data?	Yes (`transform`)	Yes (kernel PCA) / limited	**No**
Speed	**Very fast**	Slower	Slow
Inverse (reconstruction)	Yes	Hard	No
Typical output	2 to hundreds of dimensions	few dimensions	2-D or 3-D picture
Use for	Preprocessing, compression	Curved data	Looking at clusters

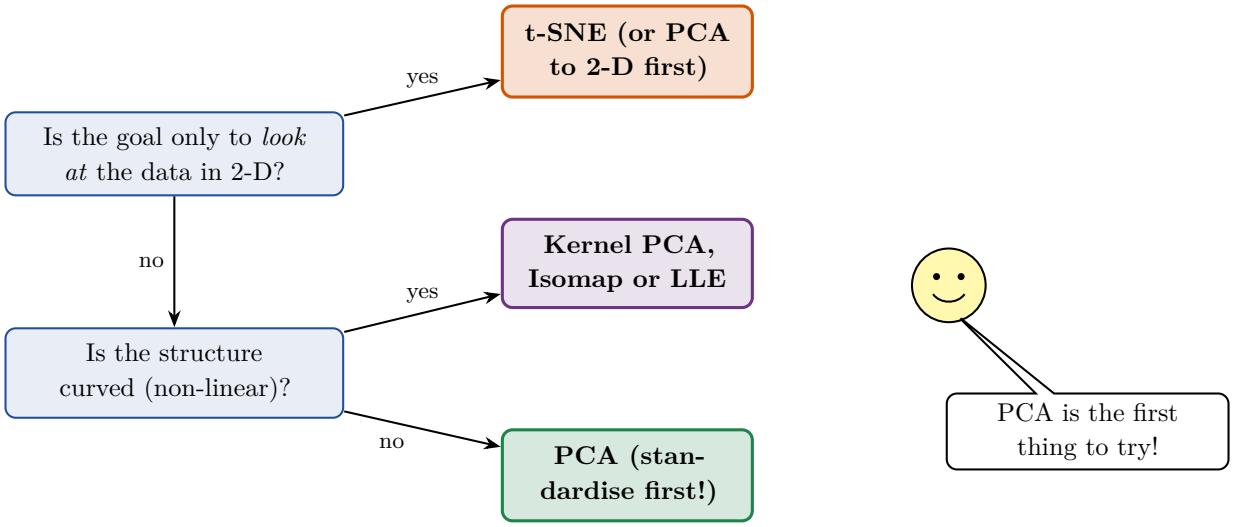


Figure 8.10. A simple guide for choosing a dimensionality-reduction method.

Reading the Figure

Start from the top question. If you only want to *draw* the data, use t-SNE (often after reducing to about 50 dimensions with PCA) or simply PCA with two components. If you need the reduced features for *learning*, ask whether the data lie on a curved surface: if yes, try a non-linear method such as Kernel PCA, Isomap or LLE; if not, use ordinary PCA, which is fast, invertible and applicable to new data. Remember to standardise the features before PCA.

Chapter Summary

- **Dimensionality reduction** maps d features to $m < d$ while keeping the useful information (unsupervised).
- **Curse of dimensionality:** space grows exponentially, so data become sparse (10^d cells), almost all points lie near the boundary ($1 - 0.998^d$), distances concentrate (contrast $40.7 \to 0.13$ from 2-D to 1000-D) and algorithms such as k-NN degrade ($94.8\% \to 37.1\%$ with junk features).
- **Projection** (linear: PCA, random projection, LDA) vs. **manifold learning** (non-linear: Isomap, LLE, t-SNE, kernel PCA); the manifold hypothesis says real data lie near a low-dimensional manifold.
- **PCA:** centre (and scale) the data, covariance matrix $\mathbf{C}$, eigenvalues λ_j (variances) and eigenvectors (components), project $\mathbf{Z} = \tilde{\mathbf{X}}\mathbf{W}_m$. Explained variance ratio $\lambda_j / \sum \lambda_i$; keep components for 95%; reconstruction error = discarded eigenvalues.
- Worked example: $\mathbf{C} = \begin{bmatrix} 2.5 & 2 \\ 2 & 2.5 \end{bmatrix}$, $\lambda = 4.5, 0.5$, PC1 $= (1,1)/\sqrt{2}$ explains 90%.
- **Scale first:** on the wine data raw PCA gives 99.8% on PC1 (an artefact of units).

- **Variants:** Randomised, Incremental and Kernel PCA (accuracy 0.51 → 1.00 on circles).

Exam Corner

A. Very Short Answer Questions (2 marks each)

1. Define dimensionality reduction. Give two reasons for using it. **[2 marks]**
2. What is the curse of dimensionality? **[2 marks]**
3. Why do distances lose meaning in high dimensions? **[2 marks]**
4. Differentiate between projection and manifold learning. **[2 marks]**
5. What is the manifold hypothesis? **[2 marks]**
6. Name two linear and two non-linear dimensionality-reduction methods. **[2 marks]**
7. Define principal component analysis. **[2 marks]**
8. What is the explained variance ratio? **[2 marks]**
9. What is the relation between eigenvalues of the covariance matrix and the variance along principal components? **[2 marks]**
10. Why must features be standardised before PCA? **[2 marks]**
11. How can the number of principal components be chosen? **[2 marks]**
12. What is Kernel PCA? When is it useful? **[2 marks]**

Model Answer: Q2, Q5 and Q10

Q2. The curse of dimensionality is the set of problems that arise when the number of features is large: the space grows exponentially so data become sparse, distances between points become almost equal, most points lie near the boundary, and the amount of data required grows exponentially, so learning algorithms (especially distance-based ones) overfit or lose accuracy.

Q5. The manifold hypothesis states that real-world high-dimensional data (images, speech, ...) lie close to a lower-dimensional manifold embedded in the high-dimensional space, because only a few hidden factors vary.

Q10. PCA maximises variance, so a feature with a large numeric range dominates the first components regardless of its importance (e.g. on the wine data an unscaled PCA gives 99.8% of the variance to one feature). Standardising puts all features on the same scale.

B. Short Answer Questions (5 marks each)

1. Explain the curse of dimensionality with examples. **[5 marks]**
2. Explain how high dimensionality affects k-NN and other distance-based algorithms. **[5 marks]**
3. Explain the two main approaches to dimensionality reduction with a diagram. **[5 marks]**
4. Explain Isomap, LLE and t-SNE briefly. **[5 marks]**
5. Explain the PCA algorithm step by step. **[5 marks]**
6. Derive why principal components are eigenvectors of the covariance matrix. **[5 marks]**
7. Explain how to choose the number of principal components (scree plot and cumulative variance). **[5 marks]**
8. Explain PCA for compression with reconstruction error. **[5 marks]**
9. Explain Randomised PCA, Incremental PCA and Kernel PCA. **[5 marks]**
10. List the advantages and disadvantages of PCA. **[5 marks]**

Model Answer: B5 (outline for 5 marks)

(1) Standardise or centre the data. (2) Compute the covariance matrix $\mathbf{C} = \frac{1}{n-1}\tilde{\mathbf{X}}^\top\tilde{\mathbf{X}}$. (3) Compute its eigenvalues λ_j and unit eigenvectors $\boldsymbol{w}_j$. (4) Sort the eigenvectors by decreasing eigenvalue (these are the principal components) and choose the number m to keep using the explained variance ratio $\lambda_j / \sum \lambda_i$. (5) Project: $\mathbf{Z} = \tilde{\mathbf{X}}\mathbf{W}_m$. (6) Optionally reconstruct $\hat{\mathbf{X}} = \mathbf{Z}\mathbf{W}_m^\top + \text{mean}$; the error equals the sum of the discarded eigenvalues. Add a small 2-D example and a diagram of the principal axes.

C. Long Answer Questions (10 marks each)

1. Explain the curse of dimensionality in detail. Discuss its effects on volume, distances, data requirements and learning algorithms, and suggest remedies. **[10 marks]**
2. Explain projection and manifold learning. Describe PCA, Isomap, LLE and t-SNE, and compare them. **[10 marks]**
3. Explain PCA with the full algorithm and perform it by hand on a small two-dimensional dataset: covariance matrix, eigenvalues, eigenvectors, projection and explained variance. **[10 marks]**
4. Explain how the number of components is chosen in PCA and discuss the importance of feature scaling. Explain how PCA is used for compression, visualisation and preprocessing. **[10 marks]**
5. Describe the variants of PCA (Randomised, Incremental and Kernel PCA) and explain when each is used. **[10 marks]**

6. Discuss the advantages, disadvantages and limitations of PCA and compare it with non-linear dimensionality-reduction methods. **[10 marks]**

Model Answer: C3 (a second numerical solution)

Data $(1,2),(3,4),(5,6)$. Means $(3,4)$; centred points $(-2,-2),(0,0),(2,2)$. Covariance $(n-1=2)$: $s_{xx}=\frac{4+0+4}{2}=4$, $s_{yy}=4$, $s_{xy}=4$; $\mathbf{C}=\begin{bmatrix}4 & 4\\ 4 & 4\end{bmatrix}$.
$\det(\mathbf{C}-\lambda\mathbf{I})=(4-\lambda)^2-16=0\Rightarrow\lambda_1=8,\ \lambda_2=0$. $\boldsymbol{w}_1=\frac{1}{\sqrt{2}}(1,1)$. Explained variance of PC1 $=8/8=100\%$.
Projections $z_1=\frac{(x-3)+(y-4)}{\sqrt{2}}=(-2.828,\ 0,\ 2.828)$. The points lie exactly on a line, so one component describes them *without any loss*.

D. Numerical Problems

1. The eigenvalues of a covariance matrix are 6, 3, 1. Find the explained variance ratios and the number of components needed for at least 85% of the variance. *(Answer:* $0.6, 0.3, 0.1$*; two components give* $90\% \geq 85\%$*.)*

2. Find the eigenvalues and eigenvectors of $\mathbf{C}=\begin{bmatrix}3 & 1\\ 1 & 3\end{bmatrix}$ and the explained variance of PC1. *(Answer:* $(3-\lambda)^2=1\Rightarrow\lambda=4,2$*;* $\boldsymbol{w}_1=(1,1)/\sqrt{2}$*,* $\boldsymbol{w}_2=(1,-1)/\sqrt{2}$*; PC1 explains* $4/6=66.7\%$*.)*

3. Perform PCA on $(1,2),(3,4),(5,6)$ and find the explained variance of PC1. *(Answer:* $\mathbf{C}=\begin{bmatrix}4 & 4\\ 4 & 4\end{bmatrix}$*,* $\lambda=8,0$*;* 100%*.)*

4. The mean is $(1,1)$ and the first component is $\boldsymbol{w}_1=(0.6,0.8)$. Project $(4,3)$ and reconstruct it. *(Answer: centred* $(3,2)$*;* $z=3(0.6)+2(0.8)=3.4$*;* $\hat{\boldsymbol{x}}=(1,1)+3.4(0.6,0.8)=(3.04,3.72)$*.)*

5. A point is uniformly distributed in the hypercube $[0,1]^{50}$. Find the probability that it lies within 0.001 of a border. *(Answer:* $1-0.998^{50}=1-0.905=0.095$*.)*

6. With 5 intervals per feature, how many cells are there for $d=8$ features? *(Answer:* $5^8=390\,625$*.)*

7. What fraction of the unit square is covered by a disc of radius 0.1 (ignoring edge effects)? *(Answer:* $\pi(0.1)^2=0.0314$*, i.e.* 3.1%*.)*

8. A covariance matrix has total variance 10; the first eigenvalue is 8. How much variance is lost if only PC1 is kept? *(Answer:* 2*, i.e.* 20%*.)*

9. Digit images have 64 features; PCA keeps 29 components. What fraction of the numbers is stored and what is the reduction? *(Answer:* $29/64=45\%$*;* $64/29=2.2\times$ *smaller.)*

10. Singular values of a centred matrix with $n = 5$ rows are $s_1 = 4.243$ and $s_2 = 1.414$. Find the eigenvalues and the explained variance of PC1. *(Answer: $\lambda = s^2/(n-1) = 4.5,\ 0.5$; 90%.)*

E. Multiple-Choice Questions

1. In high dimensions the relative contrast between the nearest and farthest neighbour:
   (a) increases (b) decreases (c) stays the same (d) becomes infinite
2. Which method is a *non-linear* dimensionality reduction?
   (a) PCA (b) Random projection (c) Isomap (d) LDA with linear kernel
3. The first principal component is the direction of:
   (a) minimum variance (b) maximum variance (c) maximum mean (d) minimum distance
4. Principal components are:
   (a) parallel (b) orthogonal (c) identical (d) random
5. The variance along a principal component equals:
   (a) the eigenvalue (b) the eigenvector (c) the mean (d) the trace
6. PCA is computed from the eigen-decomposition of the:
   (a) confusion matrix (b) covariance matrix (c) distance matrix (d) weight matrix
7. If eigenvalues are $4, 2, 2, 2$ the explained variance ratio of PC1 is:
   (a) 0.25 (b) 0.4 (c) 0.5 (d) 0.8
8. Before PCA with features in different units we should:
   (a) do nothing (b) standardise the features (c) add noise (d) remove labels
9. Which PCA variant handles data that do not fit in memory?
   (a) Kernel PCA (b) Incremental PCA (c) Sparse PCA (d) none
10. Kernel PCA is used to capture:
    (a) linear structure only (b) non-linear structure (c) labels (d) outliers
11. The reconstruction error of PCA with m components equals:
    (a) the sum of the first m eigenvalues (b) the sum of the discarded eigenvalues (c) zero
    (d) the largest eigenvalue
12. t-SNE is mainly used for:
    (a) compression (b) visualisation (c) regression (d) scaling

Answers: 1-b, 2-c, 3-b, 4-b, 5-a, 6-b, 7-b, 8-b, 9-b, 10-b, 11-b, 12-b.

F. Practice Problems

1. For the data $(2,0), (0,2), (-2,0), (0,-2)$ find the covariance matrix and eigenvalues. What does the result say about PCA on this data? *(Answer: $\mathbf{C} = \begin{bmatrix} 8/3 & 0 \\ 0 & 8/3 \end{bmatrix}$; equal eigenvalues, so no direction is preferred: PCA cannot reduce this data.)*

2. Using the five-point example, compute z_1 for a new point $(6, 6)$ and its reconstruction. *(Answer: centred* $(3, 3)$*;* $z_1 = \frac{6}{\sqrt{2}} = 4.243$*; reconstruction* $(6, 6)$*, exactly on PC1.)*
3. **Coding:** apply PCA to the *wine* data with and without `StandardScaler` and plot the first two components, coloured by wine type.
4. **Coding:** use `PCA(n_components=0.95)` to compress the *digits* data and measure the accuracy of k-NN before and after.
5. **Coding:** use `sklearn.manifold.TSNE` to draw the digits in 2-D, coloured by digit.
6. Draw a labelled diagram of PCA on a two-dimensional elongated cloud with both principal axes.

Quick Revision

Topic	One-line recall
Dimensionality reduction	$d \to m < d$ keeping useful information; unsupervised.
Curse of dimensionality	Sparse data (10^d cells), points near border ($1-0.998^d$), distances concentrate, k-NN degrades.
Projection	Linear maps to a subspace: PCA, random projection, LDA.
Manifold learning	Unroll curved data: Isomap, LLE, t-SNE, kernel PCA.
PCA steps	Centre/scale → covariance → eigen-decomposition → sort → project.
Eigenvalue / eigenvector	Variance along component / direction of the component.
Explained variance	$\lambda_j / \sum \lambda_i$; keep $\approx 95\%$ (digits: 29 of 64).
Reconstruction error	Sum of discarded eigenvalues.
Scaling	Always standardise unless same units (wine: 99.8% artefact).
Variants	Randomised (fast), Incremental (mini-batches), Kernel (non-linear).
Limits	Linear; ignores labels; sensitive to outliers/scale.

Chapter 9

Artificial Neural Networks and MLPs with Keras

Learning Objectives

After studying this chapter you will be able to:

- explain the **artificial neuron** and compare it with a biological neuron;
- train a **perceptron** by hand and explain why a single neuron cannot solve **XOR**;
- describe the architecture of a **multilayer perceptron (MLP)**, count its parameters and compute a **forward pass** by hand;
- compare the common **activation functions** and explain why non-linearity is essential;
- explain **backpropagation** and carry it out by hand on a small network;
- choose the **output layer** and **loss function** for regression, binary and multi-class problems;
- build, train, evaluate and tune MLPs with **Keras** (Sequential and Functional API);
- control **overfitting** with dropout, L2 regularisation and early stopping;
- answer 2-, 5- and 10-mark university questions on neural networks.

> *We can only see a short distance ahead, but we can see plenty there that needs to be done.*
> — Alan Turing, 1950

Unit V is about **neural networks**, the models behind modern image recognition, speech assistants and language translation. A neural network is built from many very simple units, **neurons**. In this chapter we start with a single neuron, grow it into a network, understand how the network *learns* (backpropagation) and then build real networks in Python with **Keras**.

9.1 From Biological to Artificial Neurons

9.1.1 The Biological Inspiration

The human brain contains about 86 billion neurons. Each neuron receives electrical signals from other neurons through its **dendrites**, adds them up in the cell body (**soma**), and, if the total is strong enough, sends a pulse along its long **axon** to other neurons. Learning corresponds to changing the *strength* of the connections.

Definition: Artificial neuron

An **artificial neuron** computes a weighted sum of its inputs plus a bias and passes the result through an **activation function** φ:

$$z = \sum_{i=1}^{n} w_i x_i + b = \boldsymbol{w} \cdot \boldsymbol{x} + b, \qquad a = \varphi(z).$$

The **weights** w_i say how important each input is (they play the role of connection strengths), the **bias** b shifts the threshold at which the neuron fires, and a is the **output** (activation).

Definition: Artificial neural network (ANN)

An **artificial neural network** is a model made of many artificial neurons arranged in **layers** and connected by weights. The weights are learned from data by minimising a loss function with gradient descent.

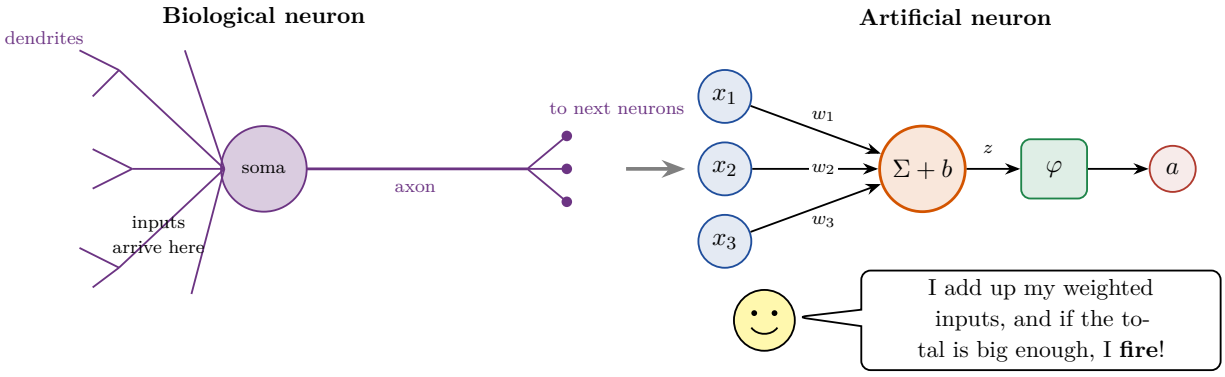


Figure 9.1. A biological neuron and its mathematical model.

Reading the Figure

Left: a biological neuron. Signals enter through the **dendrites** (left), are combined in the **soma** (the body) and the result travels out along the **axon** to the next neurons. **Right:** the artificial copy. The inputs x_1, x_2, x_3 (blue) are multiplied by the weights w_1, w_2, w_3 (the strengths of the connections), added up together with the bias b in the orange circle to give $z = \boldsymbol{w} \cdot \boldsymbol{x} + b$, and passed through the activation function φ (green) to give the output $a = \varphi(z)$. This tiny calculation is repeated millions of times in a large network.

Year	Milestone
1943	McCulloch and Pitts propose the first mathematical model of a neuron
1958	Rosenblatt invents the **perceptron**
1969	Minsky and Papert show that a single perceptron cannot learn XOR; interest in neural networks fades
1986	Rumelhart, Hinton and Williams popularise **backpropagation** for training multilayer networks
2012	A deep convolutional network wins the ImageNet competition; the **deep learning** boom begins

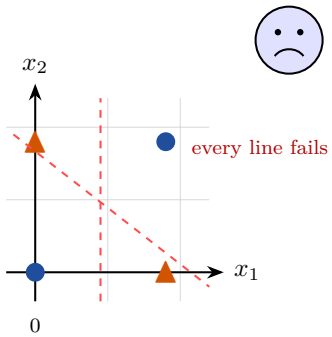
9.1.2 Logic with Neurons

With the step activation $\varphi(z) = 1$ if $z > 0$ and 0 otherwise, one neuron can implement simple logic gates:

Gate	Weights	Bias	Output
AND	$(1,\ 1)$	-1.5	$\text{step}(x_1 + x_2 - 1.5)$
OR	$(1,\ 1)$	-0.5	$\text{step}(x_1 + x_2 - 0.5)$
NOT	(-1)	$+0.5$	$\text{step}(0.5 - x_1)$

For example, AND with inputs $(1,1)$: $z = 1 + 1 - 1.5 = 0.5 > 0 \Rightarrow 1$; with $(1,0)$: $z = -0.5 \Rightarrow 0$. But a single neuron **cannot** compute the **XOR** (exclusive or) function, which is 1 when the inputs *differ*. In the plane, the points $(0,1)$ and $(1,0)$ (output 1) lie on one diagonal and $(0,0)$ and $(1,1)$ (output 0) on the other: no single straight line separates them.

(a) XOR is not linearly separable



(b) A network with a hidden layer solves it

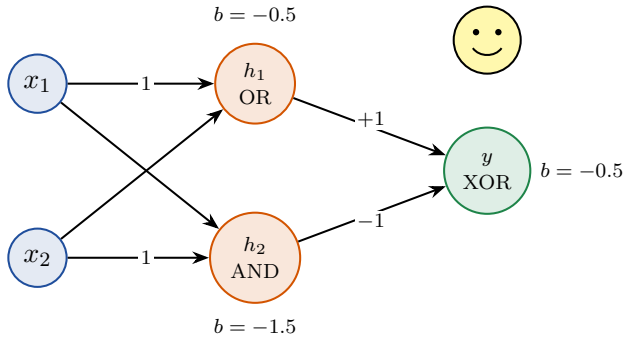


Figure 9.2. (a) No straight line separates the XOR classes. (b) Two hidden neurons and one output neuron compute XOR exactly.

Reading the Figure

(a) The blue circles $(0,0)$ and $(1,1)$ have XOR $= 0$ and the orange triangles $(0,1)$ and $(1,0)$ have XOR $= 1$. Any straight line (the red dashed lines are two attempts) leaves a point of each kind on the wrong side. **(b)** The fix is to add a *hidden layer*. The first hidden neuron is an OR gate and the second is an AND gate. The output neuron computes $\text{step}(h_1 - h_2 - 0.5)$, i.e. "OR but *not* AND", which is exactly XOR:

x_1	x_2	$h_1 = \text{OR}$	$h_2 = \text{AND}$	$z = h_1 - h_2 - 0.5$	y
0	0	0	0	-0.5	**0**
0	1	1	0	$+0.5$	**1**
1	0	1	0	$+0.5$	**1**
1	1	1	1	-0.5	**0**

Key Idea

One neuron draws one straight line; a network of neurons can draw complicated shapes. Hidden layers let the network build new features (here "OR" and "AND") that make the problem linearly separable.

9.2 The Perceptron

Definition: Perceptron

The **perceptron** (Rosenblatt, 1958) is the simplest neural network: a single neuron (a *threshold logic unit*) with a step activation,

$$\hat{y} = \begin{cases} 1 & \text{if } \boldsymbol{w} \cdot \boldsymbol{x} + b > 0 \\ 0 & \text{otherwise,} \end{cases}$$

used as a *linear binary classifier*.

Definition: Perceptron learning rule

Present the training examples one at a time. For an example $(\boldsymbol{x}, y)$ with prediction $\hat{y}$ update

$$w_j \leftarrow w_j + \eta\,(y - \hat{y})\,x_j, \qquad b \leftarrow b + \eta\,(y - \hat{y}),$$

where $\eta > 0$ is the **learning rate**. If the prediction is correct, $y - \hat{y} = 0$ and nothing changes. If the neuron predicted 0 but the true class is 1, the weights are *increased* along $\boldsymbol{x}$; if it predicted 1 but the truth is 0 they are *decreased*. One pass through all examples is an

epoch.

Perceptron convergence theorem. If the two classes are *linearly separable*, the perceptron learning rule is guaranteed to find a separating line in a finite number of updates.

9.2.1 Worked Example: Training a Perceptron to Learn AND

Example: End-to-end perceptron training

Training data: $(0,0) \to 0$, $(0,1) \to 0$, $(1,0) \to 0$, $(1,1) \to 1$. Start with $\boldsymbol{w} = (0,0)$, $b = 0$, learning rate $\eta = 1$.

Epoch 1. The first three examples give $z = 0$, so $\hat{y} = 0$, which is correct. The fourth, $(1,1)$ with $y = 1$, gives $z = 0 \Rightarrow \hat{y} = 0$: wrong, error $y - \hat{y} = +1$. Update: $\boldsymbol{w} = (0,0) + 1 \cdot (1,1) = (1,1)$, $b = 0 + 1 = 1$.

Epoch 2. $(0,0)$: $z = 1 \Rightarrow \hat{y} = 1$ (wrong, error -1): $\boldsymbol{w} = (1,1)$, $b = 0$. $(0,1)$: $z = 1 \Rightarrow \hat{y} = 1$ (wrong): $\boldsymbol{w} = (1,0)$, $b = -1$. $(1,0)$: $z = 0 \Rightarrow 0$ correct. $(1,1)$: $z = 0 \Rightarrow 0$ (wrong, error $+1$): $\boldsymbol{w} = (2,1)$, $b = 0$.

The remaining epochs follow the same pattern; the table shows the state at the end of every epoch:

End of epoch	w_1	w_2	b	Mistakes in the epoch
1	1	1	$+1$	1
2	2	1	0	3
3	2	1	-1	3
4	2	2	-1	2
5	2	1	-2	1
6	2	1	-2	**0**

Epoch 6 has no mistakes, so training stops with $\boldsymbol{w} = (2,1)$ and $b = -2$. **Check:** $(1,1) \to 2 + 1 - 2 = 1 > 0 \Rightarrow 1$; $(1,0) \to 0 \Rightarrow 0$; $(0,1) \to -1 \Rightarrow 0$; $(0,0) \to -2 \Rightarrow 0$. The perceptron has learned AND. Its decision boundary is the line $2x_1 + x_2 - 2 = 0$.

Reading the Figure

Each panel shows the decision boundary (the line $\boldsymbol{w} \cdot \boldsymbol{x} + b = 0$) at the end of an epoch. Points on the side the weight vector points to are classified as 1. In the first panel the line lies far to the lower-left, outside the picture, so *every* point is classified as 1: the three blue points are wrong. In the second and third panels the line moves but still misclassifies something. In the last panel (green) the line $2x_1 + x_2 = 2$ separates the orange triangle

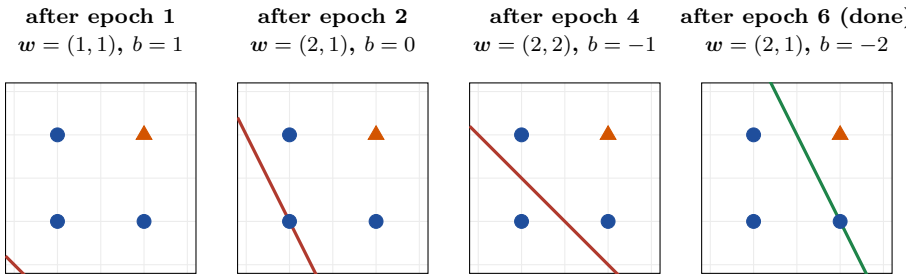


Figure 9.3. The decision boundary of the perceptron while it learns AND (blue circles: class 0, orange triangle: class 1).

$(1, 1)$ from the three blue circles (the blue point $(1, 0)$ lies exactly on the line, where $z = 0$ gives class 0). The line was moved a little each time a mistake was made.

Limits. The perceptron produces only a hard 0/1 answer (no probability), and it works only for **linearly separable** data: on XOR the learning rule never converges. Replacing the step function by a smooth activation and the rule by gradient descent gives logistic regression (Chapter 4) and, with hidden layers, the MLP.

Perceptron

Advantages

- Very simple; easy to understand and implement
- Fast online learning; guaranteed to converge on separable data
- Building block of all neural networks

Disadvantages

- Only linear decision boundaries (cannot learn XOR)
- Gives class labels, not probabilities
- Does not converge if the data are not linearly separable

Applications

- Linearly separable binary classification
- Historical basis of neural networks
- Simple online classifiers

9.3 The Multilayer Perceptron (MLP)

Definition: Multilayer perceptron (MLP)

A **multilayer perceptron** is a feed-forward neural network made of an **input layer**, one or more **hidden layers** and an **output layer**. Every neuron of a layer is connected to every neuron of the next layer (a **fully connected** or **dense** layer), and every hidden neuron uses a non-linear activation function. Information flows only forward, from input to output. A network with many hidden layers is called a **deep neural network** (DNN).

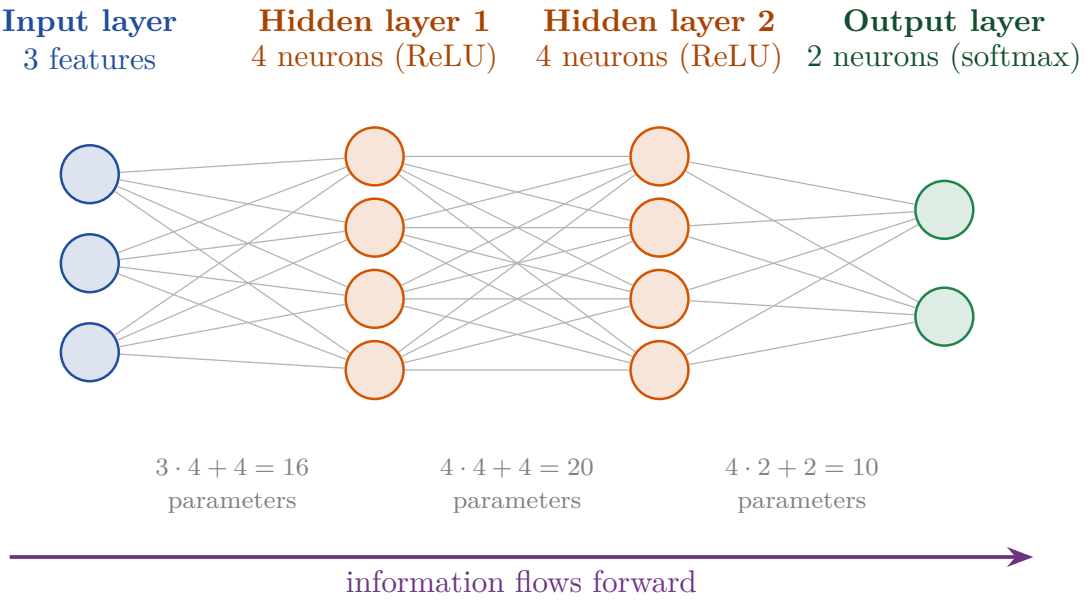


Figure 9.4. A fully connected MLP with two hidden layers (a 3–4–4–2 network).

Reading the Figure

Every circle is a neuron and every line a weight. The **blue** input layer just holds the three feature values. Each **orange** hidden neuron is connected to *all* neurons of the previous layer, computes its weighted sum and applies the activation function. The **green** output layer produces the final answer (here two class probabilities). The numbers below the picture count the **parameters** (weights plus biases) between layers: this small network has $16 + 20 + 10 = 46$ numbers to learn.

Counting parameters. A dense layer with n_{in} inputs and n_{out} neurons has $n_{\text{in}} \cdot n_{\text{out}}$ weights and n_{out} biases:

$$\#\text{parameters} = n_{\text{in}}\, n_{\text{out}} + n_{\text{out}}.$$

For the network used later in this chapter (784 inputs, hidden layers of 300 and 100 neurons, 10 outputs): $(784 \cdot 300 + 300) + (300 \cdot 100 + 100) + (100 \cdot 10 + 10) = 235\,500 + 30\,100 + 1\,010 = \mathbf{266\,610}$ parameters.

9.3.1 Forward Pass: A Worked Example

Example: Computing the output of a 2–2–1 network

Inputs $\boldsymbol{x} = (1,\ 0.5)$. Hidden layer (sigmoid): weights $\mathbf{W}_1 = \begin{bmatrix} 0.4 & -0.6 \\ 0.7 & 0.2 \end{bmatrix}$ (row = neuron), biases $\boldsymbol{b}_1 = (0.1,\ -0.2)$. Output neuron (sigmoid): weights $\boldsymbol{w}_2 = (0.5,\ -0.3)$, bias $b_2 = 0.05$.

Hidden layer:

$$z_1 = 0.4(1) + (-0.6)(0.5) + 0.1 = 0.2, \qquad a_1 = \sigma(0.2) = 0.5498,$$
$$z_2 = 0.7(1) + 0.2(0.5) - 0.2 = 0.6, \qquad a_2 = \sigma(0.6) = 0.6457.$$

Output neuron:

$$z_{\text{out}} = 0.5(0.5498) + (-0.3)(0.6457) + 0.05 = 0.1312, \qquad \hat{y} = \sigma(0.1312) = \mathbf{0.5328}.$$

The network predicts a probability of 0.53 for class 1. This network has $(2{\cdot}2{+}2){+}(2{\cdot}1{+}1) = 9$ parameters.

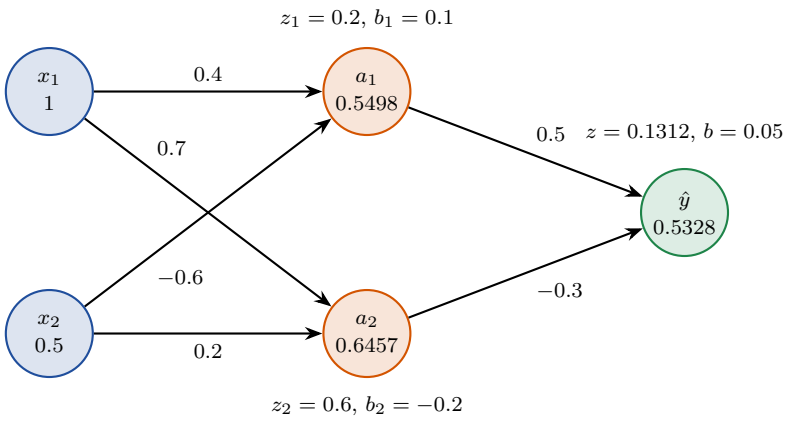


Figure 9.5. The forward pass of the worked example with all intermediate values.

Reading the Figure

Follow the numbers from left to right. The inputs (1 and 0.5) are multiplied by the weights on the arrows and added to the bias to give the pre-activations $z_1 = 0.2$ and $z_2 = 0.6$ (written above and below the hidden neurons). The sigmoid squashes these into the hidden activations 0.5498 and 0.6457 (the values inside the orange circles). These become the inputs of the output neuron, which gives the prediction 0.5328.

9.3.2 Activation Functions

Definition: Activation function

An **activation function** φ is the non-linear function applied to the weighted sum z of a neuron. The most common ones are:

Name	Formula	Range	Derivative
Step	1 if $z > 0$ else 0	$\{0, 1\}$	0 (not trainable)
Sigmoid	$\sigma(z) = \dfrac{1}{1 + e^{-z}}$	$(0, 1)$	$\sigma(z)(1 - \sigma(z))$
Tanh	$\tanh z = \dfrac{e^{z} - e^{-z}}{e^{z} + e^{-z}}$	$(-1, 1)$	$1 - \tanh^2 z$
ReLU	$\max(0, z)$	$[0, \infty)$	1 if $z > 0$, else 0
Leaky ReLU	$\max(\alpha z, z)$, $\alpha \approx 0.01$	$(-\infty, \infty)$	1 if $z > 0$, else α
Softmax	$\dfrac{e^{z_k}}{\sum_j e^{z_j}}$	$(0, 1)$, sums to 1	(used in output layer)

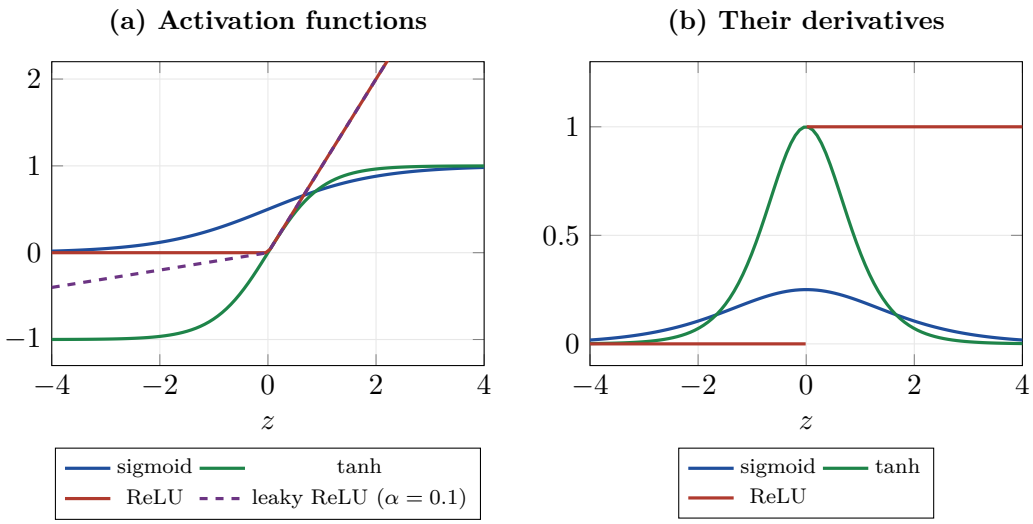


Figure 9.6. Common activation functions and their derivatives.

Reading the Figure

(a) The **sigmoid** (blue) squeezes any input into $(0, 1)$ with an S-shape; **tanh** (green) does the same into $(-1, 1)$ and is centred on zero; **ReLU** (red) is 0 for negative inputs and the identity for positive ones; **leaky ReLU** (purple dashed) lets a small amount through for negative inputs. **(b)** The derivatives matter for training because backpropagation multiplies them. The sigmoid derivative never exceeds 0.25 and almost vanishes for large $|z|$: multiplying many such small numbers makes gradients in early layers tiny (the **vanishing-gradient problem**). The ReLU derivative is exactly 1 for positive inputs, so gradients flow unchanged; this is why ReLU is the default hidden activation.

Where used	Typical choice
Hidden layers	**ReLU** (default); leaky ReLU if many neurons "die"; tanh in some recurrent networks
Output, binary classification	sigmoid (probability of class 1)
Output, multi-class	softmax (probabilities of the classes, sum to 1)
Output, regression	none (linear); ReLU or softplus if the output must be positive

Example: Why non-linearity is essential

Without activation functions, two layers collapse into one. Let $\mathbf{W}_1 = \begin{bmatrix} 1 & 2 \\ 0 & 1 \end{bmatrix}$ and $\mathbf{W}_2 = \begin{bmatrix} 2 & 0 \\ 1 & 1 \end{bmatrix}$. Then $\mathbf{W}_2(\mathbf{W}_1\boldsymbol{x}) = (\mathbf{W}_2\mathbf{W}_1)\boldsymbol{x}$ with

$$\mathbf{W}_2\mathbf{W}_1 = \begin{bmatrix} 2 & 4 \\ 1 & 3 \end{bmatrix},$$

a single linear layer. No matter how many linear layers we stack, the network can only compute a linear function. The non-linear activation between layers is what gives deep networks their power.

Definition: Universal approximation theorem

A feed-forward network with a *single hidden layer* of enough neurons and a non-linear activation can approximate *any continuous function* on a bounded region to any accuracy. In practice, deeper networks with fewer neurons per layer learn complex functions more efficiently than very wide shallow ones.

Multilayer perceptron (neural network)

Advantages

- Learns complex non-linear relationships automatically
- Universal approximator; works for classification and regression
- Learns its own features in hidden layers
- Scales with more data and computing power

Disadvantages

- Needs a lot of data and computation
- Many hyperparameters to tune (layers, neurons, learning rate)
- Black box: hard to interpret
- Can overfit; sensitive to feature scaling and initialisation

Applications

- Image and speech recognition
- Credit scoring and fraud detection
- Demand and price forecasting
- Medical diagnosis, recommender systems

9.4 Training a Neural Network: Loss Functions and Backpropagation

9.4.1 Loss Functions and Output Layers

The network and its loss must match the task:

Task	Output neurons	Output activation	Loss function
Regression (1 value)	1	none	MSE (or MAE / Huber)
Multivariate regression	one per target	none	MSE
Binary classification	1	sigmoid	binary cross-entropy $-[y \ln p + (1-y)\ln(1-p)]$
Multi-label classification	one per label	sigmoid	binary cross-entropy for each label
Multi-class (exclusive)	one per class	softmax	categorical cross-entropy $-\ln p_{\text{true class}}$

In Keras the multi-class loss is called `sparse_categorical_crossentropy` when labels are integers $0, 1, \ldots, K-1$ and `categorical_crossentropy` when they are one-hot vectors.

9.4.2 Gradient Descent and Its Variants

Training minimises the loss $J(\boldsymbol{\theta})$ over all weights and biases $\boldsymbol{\theta}$ with gradient descent (Chapter 4): $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \eta \nabla J$.

Variant	Gradient computed on	Properties
Batch gradient descent	all n training examples	smooth but slow
Stochastic gradient descent (SGD)	one example	fast but noisy
Mini-batch gradient descent	a small batch (e.g. 32)	best compromise; GPU-friendly

Terminology. An **epoch** is one full pass over the training set; the number of weight updates per epoch is $\lceil n/\text{batch size} \rceil$. For 55 000 training images and batches of 32 that is $\lceil 55\,000/32 \rceil = 1719$ updates per epoch.

9.4.3 Backpropagation

Definition: Backpropagation

Backpropagation is the algorithm that computes the gradient of the loss with respect to *every weight and bias* of a network efficiently, using the chain rule of calculus. One training step has four stages:

1. **Forward pass:** compute and store all pre-activations $z^{(l)}$ and activations $a^{(l)}$ layer by layer.
2. **Loss:** compare the output with the target.
3. **Backward pass:** starting from the output layer, propagate the error backwards, computing for each layer $\delta^{(l)} = \partial J/\partial z^{(l)}$:

$$\delta^{(L)} = \frac{\partial J}{\partial a^{(L)}} \odot \varphi'(z^{(L)}), \qquad \delta^{(l)} = (\mathbf{W}^{(l+1)\top}\delta^{(l+1)}) \odot \varphi'(z^{(l)}).$$

4. **Update:** $\partial J/\partial \mathbf{W}^{(l)} = \delta^{(l)}\, a^{(l-1)\top}$, $\partial J/\partial \boldsymbol{b}^{(l)} = \delta^{(l)}$, then $\mathbf{W}^{(l)} \leftarrow \mathbf{W}^{(l)} - \eta\, \partial J/\partial \mathbf{W}^{(l)}$.

9.4.4 Worked Example: One Step of Backpropagation

Example: A 1–1–1 network with sigmoid neurons

Input $x = 1$, target $y = 1$. Parameters: $w_1 = 0.5$, $b_1 = 0$ (hidden neuron), $w_2 = 0.8$, $b_2 = 0$ (output neuron). Both neurons use the sigmoid; the loss is $J = \frac{1}{2}(a_2 - y)^2$.

1. Forward pass.

$$z_1 = w_1 x + b_1 = 0.5, \qquad a_1 = \sigma(0.5) = 0.6225,$$
$$z_2 = w_2 a_1 + b_2 = 0.4980, \qquad a_2 = \sigma(0.498) = 0.6220,$$
$$J = \tfrac{1}{2}(0.6220 - 1)^2 = \mathbf{0.0714}.$$

2. Backward pass (chain rule). Use $\sigma'(z) = a(1-a)$.

$$\frac{\partial J}{\partial a_2} = a_2 - y = -0.3780,$$
$$\delta_2 = \frac{\partial J}{\partial z_2} = -0.3780 \times a_2(1-a_2) = -0.3780 \times 0.2351 = -0.0889,$$
$$\frac{\partial J}{\partial w_2} = \delta_2\, a_1 = -0.0889 \times 0.6225 = -0.0553, \qquad \frac{\partial J}{\partial b_2} = \delta_2 = -0.0889,$$
$$\frac{\partial J}{\partial a_1} = \delta_2\, w_2 = -0.0889 \times 0.8 = -0.0711,$$
$$\delta_1 = \frac{\partial J}{\partial z_1} = -0.0711 \times a_1(1-a_1) = -0.0711 \times 0.2350 = -0.0167,$$
$$\frac{\partial J}{\partial w_1} = \delta_1\, x = -0.0167, \qquad \frac{\partial J}{\partial b_1} = \delta_1 = -0.0167.$$

3. Update with learning rate $\eta = 1$: $w_2 = 0.8 + 0.0553 = 0.8553$, $b_2 = 0.0889$, $w_1 = 0.5 + 0.0167 = 0.5167$, $b_1 = 0.0167$.

4. Check. New forward pass: $a_1 = \sigma(0.5167 + 0.0167) = 0.6303$, $a_2 = \sigma(0.8553 \times 0.6303 + 0.0889) = 0.6520$ (closer to the target 1) and the loss falls from 0.0714 to **0.0605**. TensorFlow's automatic differentiation gives exactly the same gradients (-0.0167, -0.0167, -0.0553, -0.0889).

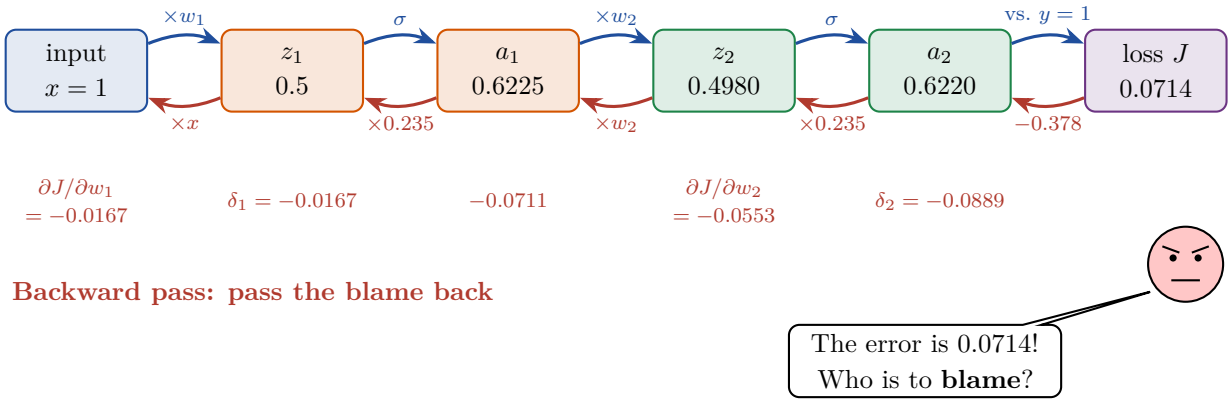


Figure 9.7. Backpropagation on the worked example: values go forward (blue), blame goes backward (red).

Reading the Figure

Blue arrows (top): the forward pass. The input flows through the multiplication by w_1, the sigmoid, the multiplication by w_2 and the sigmoid to produce $a_2 = 0.622$; comparing with the target gives the loss 0.0714. **Red arrows (bottom):** the backward pass. The error (-0.378, meaning "the output is too small") is passed back along the same path, each step multiplied by the local derivative (the sigmoid slopes 0.2351 and 0.2350 and the weights). On the way, every neuron receives its **share of the blame**: $\delta_2 = -0.0889$, then $\delta_1 = -0.0167$ (smaller, because the error has been damped on its way back). The final red numbers under the boxes are the gradients of the weights, which gradient descent subtracts from them.

9.4.5 Why Training Can Fail

- **Vanishing gradients:** with sigmoid or tanh, the derivative factors (each ≤ 0.25 for the sigmoid) shrink the error as it travels backwards (in the example $-0.378 \to -0.0889 \to -0.0167$). In deep networks the first layers hardly learn. Remedies: ReLU activations, careful initialisation, batch normalisation.
- **Exploding gradients:** the opposite; gradients grow and the weights blow up. Remedy: gradient clipping.

- **Bad initialisation:** all-zero weights make all neurons in a layer identical forever (symmetry). Keras uses *Glorot* (Xavier) initialisation by default; *He* initialisation is recommended for ReLU.
- **Bad learning rate:** too small is slow; too large diverges (see the experiment in Section 9.6).

9.4.6 Optimisers

Definition: Common optimisers

- **SGD:** $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \eta \boldsymbol{g}$ with the mini-batch gradient $\boldsymbol{g}$.
- **Momentum:** keeps a velocity that accumulates past gradients, $\boldsymbol{v} \leftarrow \beta \boldsymbol{v} - \eta \boldsymbol{g}$, $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} + \boldsymbol{v}$ (typically $\beta = 0.9$): like a ball rolling downhill it speeds up along consistent directions and dampens oscillations.
- **RMSProp:** divides the step by a running average of recent squared gradients, giving every parameter its own adaptive learning rate.
- **Adam:** combines momentum and RMSProp; the usual default ($\eta = 0.001$).

Example: Momentum by hand

Start with $\theta = 1$, $v = 0$, $\beta = 0.9$, $\eta = 0.1$. First gradient $g = 2$: $v = 0.9(0) - 0.1(2) = -0.2$, $\theta = 0.8$. Second gradient $g = 1.6$: $v = 0.9(-0.2) - 0.1(1.6) = -0.34$, $\theta = 0.8 - 0.34 = 0.46$. The second step (0.34) is larger than a plain SGD step (0.16) because the velocity remembers the first step.

9.5 Implementing MLPs with Keras

9.5.1 Keras and TensorFlow

Definition: TensorFlow and Keras

TensorFlow is an open-source library for numerical computation and large-scale machine learning developed at Google; it runs on CPUs, GPUs and TPUs and provides automatic differentiation. **Keras** is TensorFlow's high-level API for building and training neural networks with a few lines of code. Install with `pip install tensorflow`.

Every Keras project follows the same five steps.

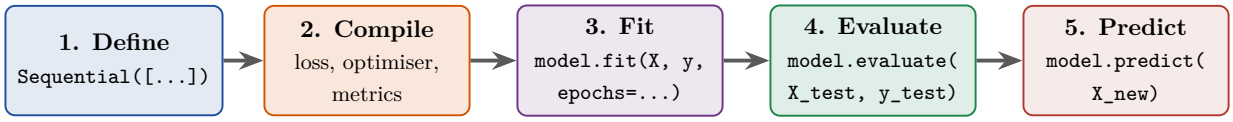


Figure 9.8. The five steps of building a model in Keras.

Reading the Figure

(1) Define the architecture (layers). **(2) Compile:** choose the loss function, the optimiser and the metrics to report. **(3) Fit:** train on the training data for a number of epochs, optionally monitoring a validation set. **(4) Evaluate** on the held-out test set. **(5) Predict** for new data. The data are always scaled first (for images, divide the pixel values by 255).

9.5.2 The Dataset: Fashion-MNIST

Fashion-MNIST contains 70 000 grayscale images of 28×28 pixels (pixel values 0–255) of ten kinds of clothing: 60 000 for training and 10 000 for testing. We keep the last 5000 training images as a **validation set** and scale all pixels to $[0, 1]$.

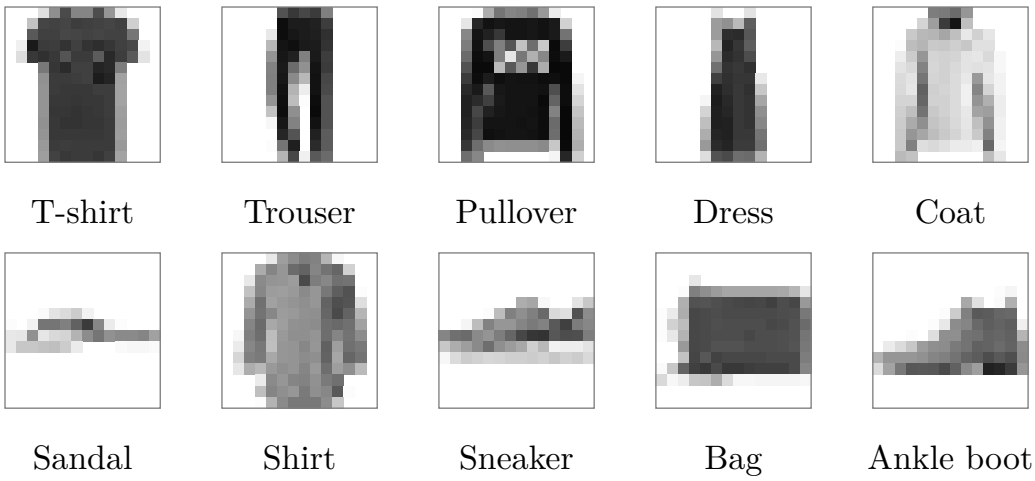


Figure 9.9. One example image from each of the ten Fashion-MNIST classes (shown at 14×14 resolution).

Reading the Figure

Each small square picture is one training example: a grey-scale image in which dark pixels are ink. The ten classes are T-shirt/top, trouser, pullover, dress, coat, sandal, shirt, sneaker, bag and ankle boot. Some classes look very different (trouser, bag), while others are easy to mix up (T-shirt, shirt, pullover and coat all look alike at low resolution). We will see this in the confusion matrix.

9.5.3 Building a Classification MLP with the Sequential API

```
1 import keras
```

```
from keras import layers

(X_train_full, y_train_full), (X_test, y_test) = \
    keras.datasets.fashion_mnist.load_data()
X_train_full = X_train_full.astype("float32") / 255.    # scale to
    [0,1]
X_test = X_test.astype("float32") / 255.
X_train, y_train = X_train_full[:-5000], y_train_full[:-5000]
X_valid, y_valid = X_train_full[-5000:], y_train_full[-5000:]

model = keras.Sequential([
    keras.Input(shape=(28, 28)),
    layers.Flatten(),                           # 28*28 = 784 inputs
    layers.Dense(300, activation="relu"),       # hidden layer 1
    layers.Dense(100, activation="relu"),       # hidden layer 2
    layers.Dense(10, activation="softmax"),     # 10 class probabilities
])
model.summary()
```

Layer	Output shape	Parameters	Calculation
Flatten	(784)	0	turns 28×28 into a list
Dense (300, ReLU)	(300)	235 500	$784 \cdot 300 + 300$
Dense (100, ReLU)	(100)	30 100	$300 \cdot 100 + 100$
Dense (10, softmax)	(10)	1 010	$100 \cdot 10 + 10$
Total		**266 610**	

What each line does.

- `Flatten` converts every 28×28 image into a vector of 784 numbers (it has no parameters).
- `Dense(300, activation="relu")` is a fully connected hidden layer of 300 neurons with the ReLU activation.
- `Dense(10, activation="softmax")` gives one probability per class; they sum to 1.

Compile and train

```
model.compile(loss="sparse_categorical_crossentropy",  # integer
    labels
              optimizer=keras.optimizers.SGD(learning_rate=0.01),
              metrics=["accuracy"])

history = model.fit(X_train, y_train, epochs=30,
                    validation_data=(X_valid, y_valid))
```

`loss="sparse_categorical_crossentropy"` is the right loss because the labels are integers 0–9 and the output is a softmax. `fit` runs 30 epochs of mini-batch SGD (default batch size 32, so

1719 updates per epoch) and records the loss and accuracy on the training and the validation set after every epoch.

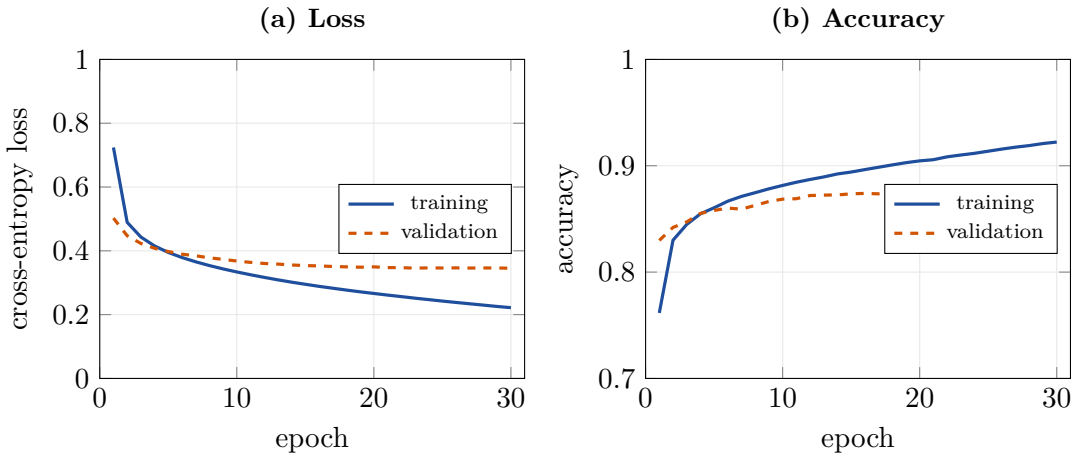


Figure 9.10. Learning curves of the 784–300–100–10 network trained with SGD (learning rate 0.01) for 30 epochs (real run).

Reading the Figure

(a) The blue **training loss** falls steadily from about 0.7 to 0.22; the orange dashed **validation loss** falls too, then flattens near 0.35. **(b)** Training accuracy climbs from 76% (epoch 1) to 92.2% (epoch 30); validation accuracy rises from 83% to 88.1%. At the start the validation accuracy is *higher* than the training accuracy because the training accuracy of an epoch is averaged *while the model is still improving*. Later the curves separate: a growing gap between training (92%) and validation (88%) is the first sign of **overfitting**. Because the validation curves have not turned upwards yet, continuing training would still help a little.

Evaluate and predict

```
model.evaluate(X_test, y_test)          # [loss, accuracy]
proba = model.predict(X_test[:8])       # 8 x 10 probabilities
y_pred = proba.argmax(axis=1)           # most probable class
```

Optimiser (30 epochs)	Train acc.	Validation acc.	Test acc.	Test loss
SGD, learning rate 0.01	0.922	0.881	**0.873**	0.364
Adam, learning rate 0.001	0.955	0.883	**0.877**	0.565

The test accuracy (87.3%) is slightly below the validation accuracy (88.1%), as expected: the test set was never used during training. Adam fits the training data better (95.5%) but its test *loss* is higher (0.565 against 0.364): the network is more over-confident on the images it gets wrong. The first eight test images are all classified correctly, with confidences from 0.667 (an ankle boot) to 1.000 (trousers).

9.5.4 Confusion Matrix: Where Does the Network Go Wrong?

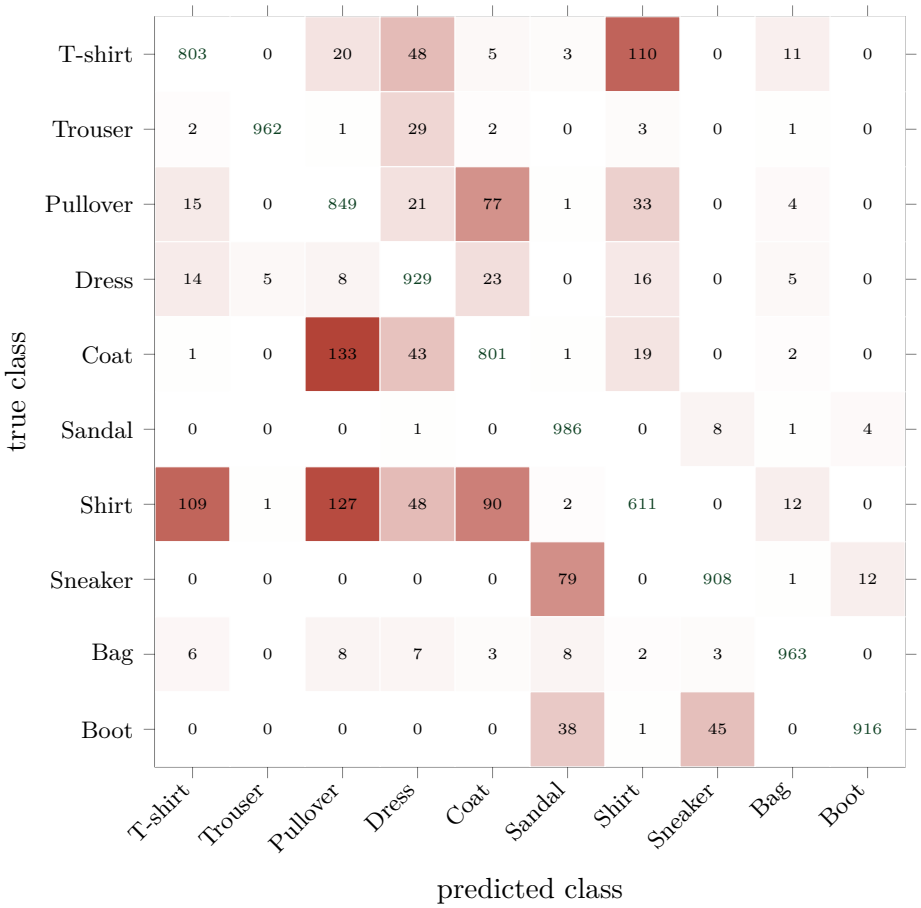


Figure 9.11. Confusion matrix of the Fashion-MNIST network on the 10 000 test images (colour = number of errors; green numbers on the diagonal are correct predictions).

Reading the Figure

Row = the **true** class, column = the class the network **predicted**. The diagonal (green numbers, e.g. 962 trousers correctly recognised out of 1000) counts the correct answers; every other cell counts a particular kind of mistake, shaded darker red when there are more. The biggest confusions are between similar-looking upper-body clothes: coats predicted as pullovers (133), shirts as pullovers (127), T-shirts as shirts (110), shirts as T-shirts (109) and shirts as coats (90); and sneakers predicted as sandals (79). Trousers, bags and sandals are almost never confused.

Class	T-shirt	Trouser	Pullover	Dress	Coat	Sandal	Shirt	Sneaker	Bag	Boot
Accuracy	0.803	0.962	0.849	0.929	0.801	**0.986**	**0.611**	0.908	0.963	0.916

The overall accuracy hides large differences: sandals are recognised 98.6% of the time, but shirts only 61.1%.

9.5.5 Regression MLP

For regression the output layer has a single neuron with *no activation* and the loss is the mean squared error.

```
model = keras.Sequential([
    keras.Input(shape=(10,)),
    layers.Dense(50, activation="relu"),
    layers.Dense(50, activation="relu"),
    layers.Dense(1),                          # no activation: any real value
])
model.compile(loss="mse", optimizer=keras.optimizers.Adam(0.003))
early = keras.callbacks.EarlyStopping(patience=15, restore_best_weights=True)
model.fit(X_train, y_train, epochs=300,
          validation_data=(X_valid, y_valid), callbacks=[early])
```

We tested it on a non-linear benchmark (`make_friedman1`: 10 features, $y = 10\sin(\pi x_1 x_2) + 20(x_3 - 0.5)^2 + 10x_4 + 5x_5 +$ noise with noise standard deviation 1), with features standardised and 3000 examples split into training, validation and test sets.

Model	Test RMSE	Test R^2	Parameters
Predict the mean (no model)	4.94	0.00	1
Linear regression	2.66	0.709	11
MLP (10–50–50–1), Adam, early stopping after 69 epochs	**1.22**	**0.939**	3 151
Noise floor (best any model can do)	≈ 1.0		

The linear model cannot represent the sin and squared terms and leaves an RMSE of 2.66. The MLP learns them and reaches 1.22, close to the irreducible noise level of 1.0 (Chapter 2: reducible and irreducible error).

9.5.6 Functional API: Non-Sequential Networks

The Sequential API only builds a plain stack of layers. The **Functional API** allows branches, several inputs and several outputs, for example the **wide & deep** network, in which the inputs reach the output both through a deep stack (to learn complex patterns) and directly (to keep simple rules).

```
inp    = keras.Input(shape=(8,))
hidden1 = layers.Dense(30, activation="relu")(inp)
hidden2 = layers.Dense(30, activation="relu")(hidden1)
concat  = layers.Concatenate()([inp, hidden2])      # wide + deep path
output  = layers.Dense(1)(concat)
model   = keras.Model(inputs=inp, outputs=output)
```

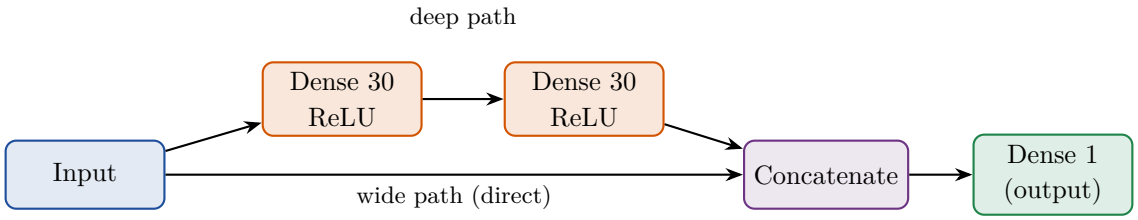


Figure 9.12. A wide & deep network built with the Functional API.

Reading the Figure

The input is used twice: it goes through two hidden layers (the **deep path** on top, which can learn complicated patterns) and also skips directly to the concatenation layer (the **wide path** below, which preserves simple relationships). The concatenation joins both and a final neuron gives the prediction. Such branching structures cannot be written with `Sequential`.

9.5.7 Callbacks, Saving and Loading

- **ModelCheckpoint:** saves the model during training (e.g. only when the validation score improves).
- **EarlyStopping:** stops training when the validation loss has not improved for `patience` epochs and can restore the best weights.
- **TensorBoard:** writes logs for interactive learning-curve visualisation.
- **Saving and loading:** `model.save("my_model.keras")` stores the architecture, the weights and the optimiser state; `keras.models.load_model("my_model.keras")` restores them.

9.6 Fine-Tuning the Hyperparameters

A neural network has many hyperparameters: the number of hidden layers, neurons per layer, learning rate, optimiser, batch size, activation function and regularisation. We illustrate their effect with real experiments on Fashion-MNIST (10 epochs each, same seed).

9.6.1 Number of Neurons

Hidden layers	Parameters	Train acc.	Val. acc.	Test acc.
1 layer of 10 neurons	7 960	0.851	0.845	0.838
1 layer of 100 neurons	79 510	0.870	0.859	0.854
300 + 100 neurons	266 610	0.882	0.869	**0.860**

More neurons give more capacity and better accuracy, with diminishing returns. In practice one builds a network that is a little *too large* and then uses regularisation and early stopping to prevent overfitting (the "stretch pants" approach: buy pants that are too big and let them shrink).

9.6.2 The Learning Rate

The learning rate is the most important hyperparameter. We trained the same network with SGD and four learning rates:

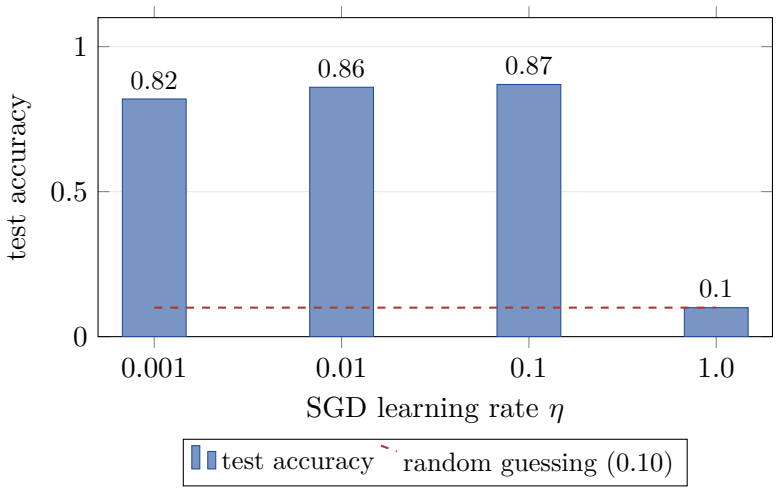


Figure 9.13. Test accuracy after 10 epochs of SGD with different learning rates (real runs).

Reading the Figure

$\eta = 0.001$**:** too small. After 10 epochs the network has hardly learned (81.9%). $\eta = 0.01$**:** reasonable (86.0%). $\eta = 0.1$**:** best (87.0%): faster progress. $\eta = 1.0$**:** too large, training **diverges**: the weights jump over the minimum, the loss stays at about $2.30 = \ln 10$ and the accuracy is 10%, the level of random guessing among ten classes. The same behaviour was shown for linear regression in Chapter 4. A good rule: try values on a logarithmic scale (0.001, 0.003, 0.01, ...) and pick the largest one that trains stably.

9.6.3 Optimiser

Optimiser (10 epochs)	**Train acc.**	**Val. acc.**	**Test acc.**
SGD ($\eta = 0.01$)	0.882	0.869	0.860
Adam ($\eta = 0.001$)	0.919	0.876	**0.871**

Adam converges faster with little tuning, which is why it is the usual first choice.

9.6.4 Fighting Overfitting: Dropout, L2 and Early Stopping

Definition: Regularisation techniques for neural networks

- **L2 regularisation (weight decay):** adds $\lambda \sum w^2$ to the loss, keeping weights small (the same as Ridge regression, Chapter 4).
- **Dropout:** during training, every neuron of a layer is *switched off* with probability p (e.g. 0.3) at each step. The network cannot rely on any single neuron, which acts like training an ensemble of many thinned networks (Chapter 5). At test time all neurons are active.
- **Early stopping:** stop training when the validation loss stops improving (and keep the best weights).
- **Data augmentation:** create extra training examples by small random changes (shifts, rotations, flips of images).

```
keras.Sequential([
    keras.Input(shape=(28, 28)), layers.Flatten(),
    layers.Dense(300, activation="relu",
                 kernel_regularizer=keras.regularizers.l2(1e-3)),   # L2
    layers.Dropout(0.3),                                            # dropout
    layers.Dense(100, activation="relu"), layers.Dropout(0.3),
    layers.Dense(10, activation="softmax")])
```

We trained the same network for 20 epochs with Adam under four regimes:

Regime	Train acc.	Val. acc.	Test acc.	Test loss	Gap (train − val)
No regularisation	0.943	0.880	0.876	0.503	0.063
Dropout 0.3	0.898	0.883	**0.878**	**0.345**	**0.015**
L2, $\lambda = 10^{-3}$	0.884	0.854	0.842	0.498	0.030
Early stopping (8 epochs)	0.911	0.878	0.870	0.367	0.033

Read the table: without regularisation the network reaches 94.3% on the training set but only 88.0% on validation (a gap of 6.3 points, and a high test loss of 0.503): overfitting. **Dropout** shrinks the gap to 1.5 points, slightly improves test accuracy (87.8%) and cuts the test loss from 0.503 to 0.345 (the network is much less over-confident). **L2 with** $\lambda = 10^{-3}$ was *too strong* for this network: the gap narrows (3.0 points) but the whole model is held back (test accuracy falls to 84.2%), a mild case of underfitting; λ is itself a hyperparameter to tune (a smaller value such as 10^{-4} would be the next try). **Early stopping** halted training after only 8 epochs (47 seconds instead of 119) and restored the best weights: test accuracy 87.0% and test loss 0.367. (For early

stopping the training and validation accuracies shown are those of the last epoch, while the test figures use the restored best weights.)

Common Mistake

Always tune hyperparameters on the **validation** set, never on the test set. The test set is used once, at the very end, to report the final performance (Chapter 2).

9.6.5 Practical Guidelines

Hyperparameter	Rule of thumb
Hidden layers	1–2 for simple problems; more layers learn complex patterns (images, speech)
Neurons per layer	enough to fit the data, then regularise (funnel or equal-size layers work well)
Learning rate	most important; try values on a log scale; reduce it if the loss explodes
Optimiser	Adam as a default; SGD with momentum for fine-tuning
Batch size	32 is a good default; larger is faster per epoch but may generalise slightly worse
Activation	ReLU in hidden layers; softmax / sigmoid / none in the output layer
Epochs	set high and use early stopping

Tools such as `keras-tuner` automate the search (random search, Hyperband) over these choices.

9.7 When to Use Neural Networks

	Neural networks (MLP)	Classical models (trees, SVM, linear)
Best for	Images, audio, text, very large datasets	Small to medium tabular data
Data needed	Large	Small to moderate
Feature engineering	Learned automatically	Often required
Training cost	High (GPU helps)	Low
Interpretability	Low	Moderate to high
Tuning effort	High	Lower

Chapter Summary

- An **artificial neuron** computes $a = \varphi(\boldsymbol{w} \cdot \boldsymbol{x} + b)$; neurons are arranged in layers and the weights are learned from data.
- A **perceptron** (step activation) learns by $w_j \leftarrow w_j + \eta(y - \hat{y})x_j$; it solves only linearly separable problems (AND, OR) and fails on **XOR**; a hidden layer solves XOR.
- An **MLP** has input, hidden and output layers; parameters per dense layer $= n_{\text{in}} n_{\text{out}} + n_{\text{out}}$ (784–300–100–10: 266 610).
- **Activations:** sigmoid, tanh, ReLU (default for hidden layers), leaky ReLU, softmax (multi-class output); non-linearity is essential, otherwise layers collapse into one.
- **Loss and output:** MSE with a linear output for regression; sigmoid + binary cross-entropy for binary; softmax + categorical cross-entropy for multi-class.
- **Backpropagation:** forward pass, loss, backward pass with the chain rule ($\delta^{(l)} = (\mathbf{W}^{(l+1)\top}\delta^{(l+1)}) \odot \varphi'$), gradient update. Example: loss 0.0714 → 0.0605.
- **Keras workflow:** define → compile → fit → evaluate → predict; Sequential vs. Functional API; callbacks (early stopping, checkpoints).
- **Results:** Fashion-MNIST MLP 87.3% test accuracy (shirts hardest at 61%); learning rate 1.0 diverges (10%); dropout cuts the train–validation gap from 6.3 to 1.5 points; regression MLP RMSE 1.22 vs. linear 2.66.
- **Regularisation:** L2, dropout, early stopping, data augmentation; tune on the validation set.

Exam Corner

A. Very Short Answer Questions (2 marks each)

1. Define an artificial neuron. **[2 marks]**
2. What is a perceptron? Write its learning rule. **[2 marks]**
3. Why can a single perceptron not solve the XOR problem? **[2 marks]**
4. What is a multilayer perceptron? **[2 marks]**
5. What is an activation function? Name four. **[2 marks]**
6. Why do we need non-linear activation functions? **[2 marks]**
7. Write the formula for the number of parameters of a dense layer. **[2 marks]**
8. Define backpropagation. **[2 marks]**
9. What is the vanishing-gradient problem? **[2 marks]**

10. Which output activation and loss are used for multi-class classification? **[2 marks]**
11. Differentiate between an epoch and a batch. **[2 marks]**
12. What is dropout? **[2 marks]**

Model Answer: Q3, Q6 and Q9

Q3. A perceptron computes $\text{step}(\boldsymbol{w} \cdot \boldsymbol{x} + b)$, whose decision boundary is a straight line. For XOR the points $(0,1),(1,0)$ (class 1) and $(0,0),(1,1)$ (class 0) cannot be separated by any straight line, so no values of $\boldsymbol{w}, b$ classify all four points correctly. A hidden layer is needed.
Q6. Without non-linear activations every layer computes a linear function, and a composition of linear functions is again linear ($\mathbf{W}_2(\mathbf{W}_1\boldsymbol{x}) = (\mathbf{W}_2\mathbf{W}_1)\boldsymbol{x}$). Such a network could not learn non-linear relations such as XOR. The non-linearity lets the network approximate any continuous function.
Q9. In backpropagation the gradient is multiplied by the activation derivative at every layer. For sigmoid/tanh the derivative is $\leq 0.25/\leq 1$ and tiny for large $|z|$, so in deep networks the gradients reaching the early layers become almost zero and those layers hardly learn. ReLU and good initialisation reduce the problem.

B. Short Answer Questions (5 marks each)

1. Explain the artificial neuron and compare it with a biological neuron. **[5 marks]**
2. Explain the perceptron learning algorithm with an example. **[5 marks]**
3. Explain the limitations of the perceptron and how an MLP overcomes them (XOR). **[5 marks]**
4. Explain the architecture of a multilayer perceptron with a diagram. **[5 marks]**
5. Explain sigmoid, tanh and ReLU activation functions with their advantages and disadvantages. **[5 marks]**
6. Explain backpropagation with the chain rule. **[5 marks]**
7. Explain the choice of output layer and loss function for regression, binary and multi-class problems. **[5 marks]**
8. Explain the steps of building a neural network with Keras. **[5 marks]**
9. Explain how to avoid overfitting in neural networks (dropout, L2, early stopping). **[5 marks]**
10. Explain the main hyperparameters of an MLP and how to tune them. **[5 marks]**

Model Answer: B9 (outline for 5 marks)

Overfitting occurs when training accuracy is much higher than validation accuracy. Techniques: (1) *Early stopping*: stop when the validation loss stops improving, keep the best weights. (2) *L2 regularisation*: add $\lambda \sum w^2$ to the loss to keep weights small. (3) *Dropout*: randomly switch off a fraction p of neurons at each training step, which acts

like averaging many thinner networks; all neurons are used at test time. (4) *More data / data augmentation*, (5) *a smaller network*. *Example:* on Fashion-MNIST dropout 0.3 reduced the train–validation gap from 6.3 to 1.5 percentage points and the test loss from 0.503 to 0.345.

C. Long Answer Questions (10 marks each)

1. Explain the perceptron and its learning rule. Train a perceptron by hand to learn the AND function. **[10 marks]**
2. Explain the multilayer perceptron: architecture, forward pass, activation functions and the universal approximation theorem. Show how XOR can be solved. **[10 marks]**
3. Explain the backpropagation algorithm. Work out one forward and one backward pass for a small network and update the weights. **[10 marks]**
4. Explain loss functions, optimisers (SGD, momentum, Adam) and the problems of vanishing and exploding gradients. **[10 marks]**
5. Describe how to implement a classification MLP with Keras for an image dataset: data preparation, model definition, compilation, training, evaluation and learning curves. **[10 marks]**
6. Explain how to fine-tune the hyperparameters of a neural network and how to prevent overfitting, with examples. **[10 marks]**

Model Answer: C1 (numerical part)

AND data, $\boldsymbol{w} = (0,0)$, $b = 0$, $\eta = 1$, $\text{step}(z) = 1$ if $z > 0$. Epoch 1: only $(1,1) \to 1$ is misclassified ($z = 0$): $\boldsymbol{w} = (1,1)$, $b = 1$. Epoch 2: $(0,0)$ wrong $\to b = 0$; $(0,1)$ wrong $\to \boldsymbol{w} = (1,0)$, $b = -1$; $(1,1)$ wrong $\to \boldsymbol{w} = (2,1)$, $b = 0$. Continuing, after epoch 6 there are no errors: $\boldsymbol{w} = (2,1)$, $b = -2$. Check: $(1,1)$: $1 > 0 \to 1$; $(1,0)$: $0 \to 0$; $(0,1)$: $-1 \to 0$; $(0,0)$: $-2 \to 0$.

D. Numerical Problems

1. A neuron has $\boldsymbol{w} = (0.5, 1)$, $b = 0.3$ and input $\boldsymbol{x} = (2, -1)$. Compute z and the output with step, sigmoid and ReLU activations. *(Answer: $z = 1 - 1 + 0.3 = 0.3$; step 1; sigmoid $\sigma(0.3) = 0.574$; ReLU 0.3.)*
2. Perceptron with $\boldsymbol{w} = (0,0)$, $b = 0$, $\eta = 1$ sees $x = (1,1)$ with $y = 1$, $\hat{y} = 0$. Give the update, then the update after $x = (0,1)$, $y = 0$. *(Answer: $\boldsymbol{w} = (1,1)$, $b = 1$; then $z = 0 + 1 + 1 = 2 \Rightarrow \hat{y} = 1$, error -1: $\boldsymbol{w} = (1,0)$, $b = 0$.)*
3. Count the parameters of a 4–5–3 network. *(Answer: $(4 \cdot 5 + 5) + (5 \cdot 3 + 3) = 25 + 18 = 43$.)*
4. Count the parameters of a 784–100–10 network. *(Answer: $78\,500 + 1010 = 79\,510$.)*

5. Compute the softmax of $(1, 2, 3)$. *(Answer:* $e^z = 2.718, 7.389, 20.086$*; sum* 30.193*;* $(0.090, 0.245, 0.665)$*.)*

6. Give $\sigma'(0)$, $\tanh'(0)$ and $\mathrm{ReLU}'(2)$. *(Answer:* 0.25*,* 1*,* 1*.)*

7. Compute the cross-entropy loss of the prediction $(0.1, 0.2, 0.7)$ when the true class is the third. *(Answer:* $-\ln 0.7 = 0.357$*.)*

8. A linear neuron $\hat{y} = wx$ has $w = 0.2$, $x = 2$, target $y = 1$ and loss $\frac{1}{2}(\hat{y} - y)^2$. Compute the gradient and the weight after one step with $\eta = 0.1$. *(Answer:* $\hat{y} = 0.4$*;* $\partial J/\partial w = (\hat{y} - y)x = -1.2$*;* $w = 0.2 + 0.12 = 0.32$*.)*

9. Momentum with $\beta = 0.9$, $\eta = 0.1$, $\theta = 1$, gradients 2 then 1.6. Find θ after two steps. *(Answer:* $v_1 = -0.2$*,* $\theta = 0.8$*;* $v_2 = -0.34$*,* $\theta = 0.46$*.)*

10. How many weight updates per epoch for 55 000 examples and batch size 32? And for 10 epochs? *(Answer:* $\lceil 55\,000/32 \rceil = 1719$*;* $17\,190$*.)*

11. For the XOR network of Figure 9.2, compute the output for input $(1, 0)$. *(Answer:* $h_1 = \mathit{step}(0.5) = 1$*,* $h_2 = \mathit{step}(-0.5) = 0$*,* $z = 1 - 0 - 0.5 = 0.5 \Rightarrow 1$*.)*

E. Multiple-Choice Questions

1. A single perceptron can learn:
   (a) XOR (b) AND (c) any function (d) only regression

2. The default activation for hidden layers in modern networks is:
   (a) step (b) ReLU (c) softmax (d) linear

3. The output activation for 10-class classification is:
   (a) sigmoid (b) ReLU (c) softmax (d) tanh

4. The loss for integer-labelled multi-class classification in Keras is:
   (a) `mse` (b) `binary_crossentropy` (c) `sparse_categorical_crossentropy` (d) `hinge`

5. The number of parameters of a dense layer with 10 inputs and 5 neurons is:
   (a) 50 (b) 55 (c) 15 (d) 60

6. Backpropagation uses:
   (a) the chain rule (b) the Bayes rule (c) Lagrange multipliers (d) clustering

7. The vanishing-gradient problem is most associated with:
   (a) ReLU (b) sigmoid in deep networks (c) dropout (d) softmax

8. One pass of the whole training set through the network is called a(n):
   (a) batch (b) epoch (c) iteration (d) layer

9. Dropout is applied:
   (a) only at test time (b) during training (c) to the labels (d) to the loss only

10. A learning rate that is far too large typically causes:
    (a) slow learning (b) divergence (c) underfitting only (d) no change

11. Which API allows multiple inputs and branching in Keras?
    (a) Sequential (b) Functional (c) Flatten (d) none

12. In a regression MLP the output layer normally has:
    (a) softmax (b) sigmoid (c) no activation (d) ReLU only

Answers: 1-b, 2-b, 3-c, 4-c, 5-b, 6-a, 7-b, 8-b, 9-b, 10-b, 11-b, 12-c.

F. Practice Problems

1. Train a perceptron by hand to learn OR ($\boldsymbol{w} = (0, 0)$, $b = 0$, $\eta = 1$) and give the final weights. *(Hint: after the first mistake on $(0, 1)$ the weights become $(0, 1)$, $b = 1$; continue until no errors. A valid answer is any $\boldsymbol{w}, b$ with $b \leq 0$ and $w_1 + b > 0$, $w_2 + b > 0$.)*

2. Compute the forward pass of a 2–1 network (one sigmoid neuron) with $\boldsymbol{w} = (1, -1)$, $b = 0$ for $\boldsymbol{x} = (2, 1)$. *(Answer: $z = 1$, $a = 0.731$.)*

3. **Coding:** reproduce the Fashion-MNIST MLP with Keras and plot the learning curves.

4. **Coding:** add `Dropout(0.3)` after each hidden layer and compare the train–validation gap with the baseline.

5. **Coding:** train the same network with learning rates 0.001, 0.01, 0.1 and 1.0 and find which diverge.

6. **Coding:** build a wide & deep model with the Functional API for a regression dataset of your choice.

Quick Revision

Topic	One-line recall
Neuron	$a = \varphi(\boldsymbol{w} \cdot \boldsymbol{x} + b)$.
Perceptron	step neuron; $w \leftarrow w + \eta(y - \hat{y})x$; linear separable only; fails XOR.
MLP	input, hidden (ReLU), output layers; fully connected; XOR needs a hidden layer.
Parameters	dense layer: $n_{\text{in}} n_{\text{out}} + n_{\text{out}}$; 784–300–100–10 = 266 610.
Activations	sigmoid $(0, 1)$, tanh $(-1, 1)$, ReLU, softmax; non-linearity is essential.
Outputs and losses	regression: linear + MSE; binary: sigmoid + BCE; multi-class: softmax + CCE.
Backpropagation	forward, loss, backward (δ via chain rule), update.
Optimisers	SGD, momentum ($v \leftarrow \beta v - \eta g$), RMSProp, Adam.
Keras	define, compile, fit, evaluate, predict; Sequential vs. Functional.
Overfitting	dropout, L2, early stopping, augmentation; tune on validation set.
Learning rate	too small slow; too large diverges (1.0: 10% accuracy).

Chapter 10

TensorFlow, and Loading and Preprocessing Data

Learning Objectives

After studying this chapter you will be able to:

- explain the roles of **TensorFlow** and **Keras** and work with **tensors**, **variables** and **broadcasting**;
- compute gradients with **automatic differentiation** (`GradientTape`) and write a **custom training loop**;
- explain how `tf.function` turns Python code into a fast **computation graph**;
- build efficient input pipelines with the `tf.data` API: batching, shuffling, mapping and **prefetching**;
- read data from **CSV** files and **TFRecord** files;
- preprocess **numerical**, **categorical** and **text** features with Keras preprocessing layers, including **one-hot encoding** and **embeddings**;
- build a complete data-to-model pipeline;
- answer 2-, 5- and 10-mark university questions on this part of Unit V.

> *Garbage in, garbage out.*
>
> — an old saying among programmers

In Chapter 9 we used Keras as a black box and handed it ready-made NumPy arrays. Real projects are different: the data live in files, are too big for memory, must be cleaned and encoded, and have to be delivered to the model fast enough to keep the processor busy. This chapter opens the box. First we look at **TensorFlow**, the engine under Keras; then we study how to **load and preprocess data** efficiently.

10.1 TensorFlow Basics

Definition: TensorFlow

TensorFlow is an open-source library for numerical computation and large-scale machine learning. It offers (i) a NumPy-like API for working with *tensors*; (ii) **automatic differentiation**, which computes gradients for any computation; (iii) **graph optimisation** that makes computations fast; and (iv) support for CPUs, GPUs and TPUs. **Keras** is its high-level API for building neural networks.

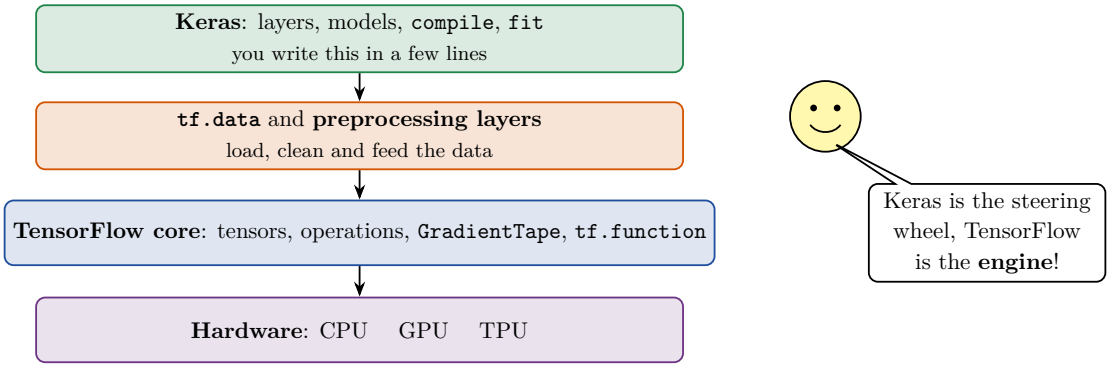


Figure 10.1. The layers of the TensorFlow ecosystem used in this unit.

Reading the Figure

Read the picture from top to bottom like a car. The top box is what you *drive*: Keras models. The next box prepares the *fuel*: the data pipeline. The blue box is the *engine*: TensorFlow's tensor operations and automatic differentiation, which Keras calls for you. At the bottom is the *road*, the hardware that runs the computation. In this chapter we look at the three lower boxes.

10.1.1 Tensors

Definition: Tensor

A **tensor** is a multi-dimensional array of numbers of a single type. It is described by its **rank** (the number of dimensions or axes), its **shape** (the length of each axis) and its **dtype** (the number type, e.g. `float32`, `int32`). Tensors generalise scalars, vectors and matrices.

Rank	Name	Example (shape)	Real-life meaning
0	scalar	5 $()$	one number (a loss value)
1	vector	$[1, 2, 3]$ (3)	the features of one instance
2	matrix	$\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix}$ $(2, 3)$	a table: 2 instances × 3 features
3	3-D tensor	$(2, 2, 3)$	a stack of matrices; e.g. a colour image $(H, W, 3)$
4	4-D tensor	$(32, 28, 28, 1)$	a batch of 32 grey-scale 28×28 images

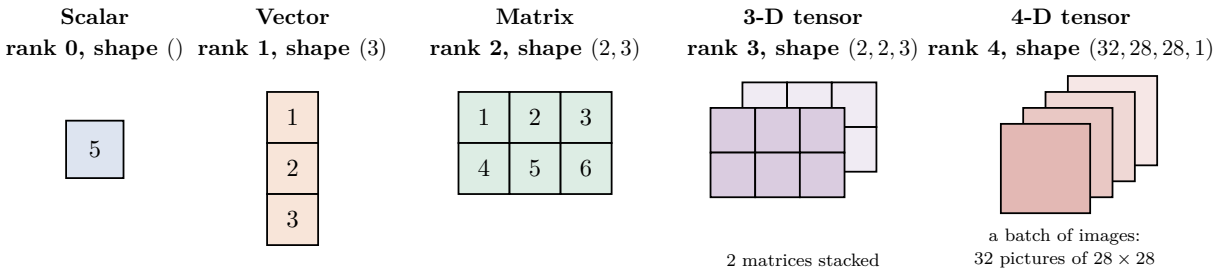


Figure 10.2. Tensors of increasing rank.

Reading the Figure

A **scalar** is a single number. A **vector** is a column of numbers (a row of a dataset: the feature values of one instance). A **matrix** is a table with rows and columns (a whole dataset). A **3-D tensor** is a stack of matrices; for example a colour photograph has height, width and 3 colour channels. A **4-D tensor** adds a "batch" axis, so a mini-batch of 32 grey-scale 28×28 images has shape $(32, 28, 28, 1)$ and $32 \cdot 28 \cdot 28 = 25\,088$ numbers. Neural networks process tensors from layer to layer.

10.1.2 Tensor Operations: Real Outputs

```
import tensorflow as tf

t = tf.constant([[1, 2, 3],
                 [4, 5, 6]])
print(t.shape, t.dtype)          # (2, 3)  int32
print(tf.transpose(t))           # [[1,4],[2,5],[3,6]]
B = tf.constant([[1, 0], [0, 1], [1, 1]])
print(tf.matmul(t, B))           # [[ 4,  5],[10, 11]]
print(t + 10)                    # [[11,12,13],[14,15,16]]
print(tf.reduce_sum(t, axis=0))  # [5 7 9]   (sum of each column)
print(tf.reduce_sum(t, axis=1))  # [ 6 15]   (sum of each row)
print(tf.reshape(t, (3, 2)))     # [[1,2],[3,4],[5,6]]
```

```
print(tf.cast(t, tf.float32).dtype)    # float32
```

Operation	**Result for** $t = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix}$
`tf.transpose(t)`	$\begin{bmatrix} 1 & 4 \\ 2 & 5 \\ 3 & 6 \end{bmatrix}$ (shape $(3, 2)$)
`tf.matmul(t, B)` with $B = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \end{bmatrix}$	$\begin{bmatrix} 1+0+3 & 0+2+3 \\ 4+0+6 & 0+5+6 \end{bmatrix} = \begin{bmatrix} 4 & 5 \\ 10 & 11 \end{bmatrix}$
`t + 10`, `t * 2`	element-wise: $\begin{bmatrix} 11 & 12 & 13 \\ 14 & 15 & 16 \end{bmatrix}$, $\begin{bmatrix} 2 & 4 & 6 \\ 8 & 10 & 12 \end{bmatrix}$
`tf.reduce_sum(t)`	21 (axis 0: $[5, 7, 9]$; axis 1: $[6, 15]$)
`tf.reshape(t,(3,2))`	$\begin{bmatrix} 1 & 2 \\ 3 & 4 \\ 5 & 6 \end{bmatrix}$ (same numbers, new shape)

Broadcasting. When two tensors have different shapes, TensorFlow automatically *stretches* the smaller one along the missing axes (without copying data) so that the shapes match.

$$\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix} + \begin{bmatrix} 10 & 20 & 30 \end{bmatrix} = \begin{bmatrix} 11 & 22 & 33 \\ 14 & 25 & 36 \end{bmatrix}.$$

The row $[10, 20, 30]$ was added to *every* row of the matrix. This is how a bias vector is added to all instances of a batch at once.

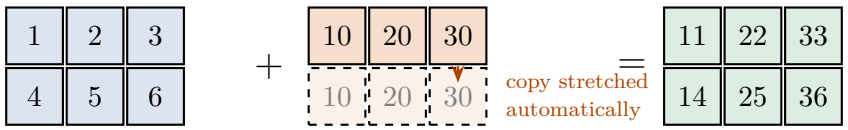


Figure 10.3. Broadcasting: the row vector is added to every row of the matrix.

Reading the Figure

On the left is the 2×3 matrix (blue). In the middle is the row vector $[10, 20, 30]$ (solid orange). Its dashed grey twin below is the *virtual copy* that TensorFlow stretches so that the shapes match; no memory is actually copied. The sum (green) adds 10 to the first column, 20 to the second and 30 to the third in *both* rows.

Variables. A `tf.Variable` is a tensor whose value can be *changed* (`assign`, `assign_add`). The weights and biases of a model are variables, because training updates them. A constant created with `tf.constant` cannot be changed. For example, `v = tf.Variable([1.,2.]); v.assign([3.,4.]); v.assign_add([1.,1.])` leaves $v = [4, 5]$.

10.1.3 Automatic Differentiation: GradientTape

Training a network needs the gradient of the loss with respect to every weight. In Chapter 9 we computed these by hand with backpropagation; TensorFlow computes them *automatically*.

Definition: Automatic differentiation and `GradientTape`

Automatic differentiation (autodiff) computes exact derivatives of a program by applying the chain rule to the elementary operations it performs. In TensorFlow, `tf.GradientTape` *records* every operation on variables executed inside a `with` block (like a tape recorder) and then `tape.gradient(result, variables)` *plays the tape backwards* to obtain the derivatives.

```
x = tf.Variable(2.0)
with tf.GradientTape() as tape:
    y = x**2 + 3*x              # recorded on the tape
print(tape.gradient(y, x))      # dy/dx = 2x + 3 = 7.0
```

Function and point	By hand	TensorFlow output
$y = x^2 + 3x$ at $x = 2$	$2x + 3 = 7$	`7.0`
$y = x^3$ at $x = 2$ (first derivative)	$3x^2 = 12$	`12.0`
$y = x^3$ at $x = 2$ (second derivative, nested tapes)	$6x = 12$	`12.0`
$z = ab + b^2$ at $(a, b) = (2, 3)$	$\partial z/\partial a = b = 3$, $\partial z/\partial b = a + 2b = 8$	`3.0`, `8.0` (and $z = 15$)

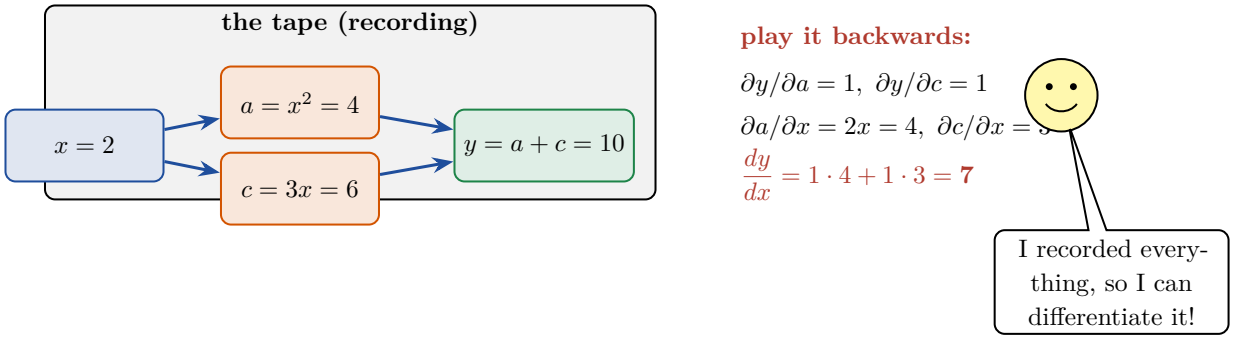


Figure 10.4. How `GradientTape` computes dy/dx for $y = x^2 + 3x$ at $x = 2$.

Reading the Figure

Left: while the `with` block runs, the tape records every operation (blue arrows): from $x = 2$ it computes $a = x^2 = 4$ and $c = 3x = 6$, then $y = a + c = 10$. **Right:** `tape.gradient` walks the tape *backwards* and applies the chain rule: y depends on a and c with slope 1 each; a depends on x with slope $2x = 4$; c depends on x with slope 3. Adding the two paths gives $dy/dx = 1 \cdot 4 + 1 \cdot 3 = 7$, exactly the derivative $2x + 3$ at $x = 2$. This is backpropagation, done automatically for any computation.

10.1.4 A Custom Training Loop

Keras `fit` hides the training loop. With `GradientTape` we can write it ourselves. We retrain the study-hours regression of Chapter 4 (hours $1, \ldots, 5$, marks $35, 45, 50, 65, 70$). We centre the hours (subtract 3) so that gradient descent converges quickly.

```
hours = tf.constant([1, 2, 3, 4, 5.]); marks = tf.constant([35, 45,
    50, 65, 70.])
xc = hours - 3.0
w = tf.Variable(0.0); b = tf.Variable(0.0); eta = 0.05

for step in range(200):
    with tf.GradientTape() as tape:
        pred = w * xc + b
        loss = tf.reduce_mean((pred - marks) ** 2) / 2
    gw, gb = tape.gradient(loss, [w, b])      # automatic gradients
    w.assign_sub(eta * gw)                    # w <- w - eta * gradient
    b.assign_sub(eta * gb)
```

Step	1	10	50	200
slope w	0.900	5.862	8.954	**9.000**
bias b (marks at 3 hours)	2.650	21.267	48.922	**52.998**
loss J	1487.5	572.0	11.2	**2.000**

After 200 steps the loop has found slope $w = 9$ and $b = 53$. The intercept at 0 hours is $b - 3w = 53 - 27 = \mathbf{26}$: exactly the line $\widehat{\text{marks}} = 26 + 9 \times \text{hours}$ that we obtained in Chapter 4 by the least-squares formula (and the minimum loss 2.0 equals $\text{SSE}/(2n) = 20/10$).

Reading the Figure

(a) Both parameters start at 0 and approach their final values: the slope (blue) reaches 9 after about 50 steps, and the bias (orange) rises to 53 a bit more slowly. **(b)** The loss (red) falls from about 1488 at the beginning to its minimum 2.0 (the unavoidable error of the best straight line), which it reaches after roughly 100 steps. The horizontal axes are logarithmic, so the steps near the start are spread out. Keras does exactly this inside `fit`,

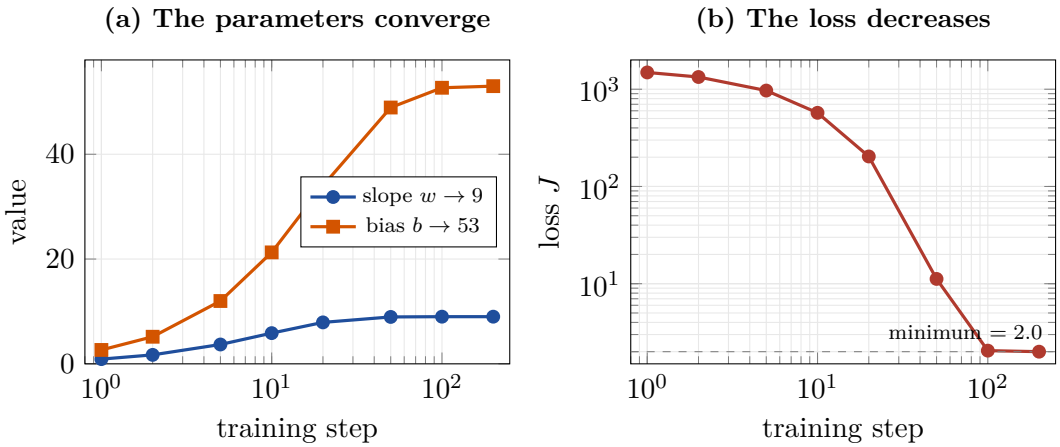


Figure 10.5. Output of the custom `GradientTape` training loop (real run; logarithmic horizontal axis).

with the gradients computed by the same tape.

10.1.5 Speeding Up with `tf.function`

Definition: `tf.function` and computation graph

Decorating a Python function with `@tf.function` (or calling `tf.function(f)`) makes TensorFlow **trace** the function once, converting the operations into a **computation graph**, a data structure that TensorFlow can optimise (merging operations, removing redundant work, parallelising) and run without the Python interpreter. Plain TensorFlow code that runs step by step in Python is called **eager execution**.

We timed a function that performs 20 matrix multiplications followed by `tanh` on a 300×300 matrix, 200 calls each:

Mode	Time per call	Speed-up
Eager (Python, step by step)	22.8 ms	1.0×
`tf.function` (graph)	**15.7 ms**	**1.45×**

The graph version is about 1.45× faster on this machine (the gain depends on the hardware and on how much Python overhead the function has). Keras models are compiled to graphs automatically, which is one reason `model.fit` is fast. **Caution:** Python side effects such as `print` run only while the function is being traced (the first call), not on every call, so use `tf.print` for output inside graph functions.

TensorFlow

Advantages

- Fast numerical computing on CPU, GPU and TPU
- Automatic differentiation: no hand-written gradients
- Graph optimisation and deployment (mobile, web, servers)
- Large ecosystem; Keras on top for quick models

Disadvantages

- Steeper learning curve than Keras alone
- Debugging graph code can be harder
- Large library; heavy to install
- Rapidly changing API (version differences)

Applications

- Image recognition and language processing
- Recommender systems and forecasting
- Research prototypes and production services
- On-device machine learning

10.2 The `tf.data` API

10.2.1 Why Do We Need Input Pipelines?

Real datasets do not fit in memory, live in many files and need preprocessing. If the model waits for data, an expensive GPU sits idle. `tf.data` lets us describe a **pipeline** of simple steps that reads, transforms, shuffles and batches the data *efficiently*.

Definition: `tf.data.Dataset`

A `tf.data.Dataset` represents a (possibly huge or infinite) **sequence of elements**, such as examples or batches. It is created from data in memory or from files and then **transformed** by chaining methods, each returning a new dataset: `map`, `shuffle`, `batch`, `repeat`, `prefetch`, ... Elements are produced *lazily*, only when the model asks for them.

Method	Purpose
`Dataset.from_tensor_slices(t)`	create a dataset whose elements are the *rows* of a tensor (or tuple of tensors (X, y))
`.map(f)`	apply a function to every element (parsing, scaling, augmentation)
`.shuffle(buffer_size)`	randomise the order of the elements
`.batch(n)`	group n consecutive elements into one batch
`.repeat(k)`	repeat the dataset k times (forever if omitted)
`.take(k)`, `.skip(k)`	use the first k elements / skip the first k
`.cache()`	keep the data in memory after the first pass
`.prefetch(n)`	prepare the next batches while the model is busy

10.2.2 Basic Operations: Real Outputs

```
ds = tf.data.Dataset.range(10)                    # 0, 1, 2, ..., 9
list(ds.batch(4))                                 # [0,1,2,3],
    [4,5,6,7], [8,9]
list(ds.batch(4, drop_remainder=True))            # [0,1,2,3],
    [4,5,6,7]
list(ds.map(lambda v: v * v))                     #
    0,1,4,9,16,25,36,49,64,81
list(tf.data.Dataset.range(3).repeat(2))          # 0,1,2,0,1,2
```

`batch(4)` cuts ten elements into batches of $4, 4, 2$ (the last batch is smaller unless `drop_remainder=True`). For features and labels together:

```
X = tf.constant([[1.,2.],[3.,4.],[5.,6.],[7.,8.]]);  y = tf.constant
    ([0,1,0,1])
ds = tf.data.Dataset.from_tensor_slices((X, y))
# elements: ([1.,2.], 0), ([3.,4.], 1), ...   shapes (2,) and ()
list(ds.batch(2))   # ( [[1,2],[3,4]], [0,1] ),  ( [[5,6],[7,8]],
    [0,1] )
```

10.2.3 Shuffling with a Buffer

Training data must be shuffled so that each mini-batch is a random sample. A huge dataset cannot be shuffled completely in memory, so `shuffle(buffer_size)` keeps a **buffer** of the next `buffer_size` elements, repeatedly *draws one at random* from the buffer to output, and refills the freed place with the next element of the source.

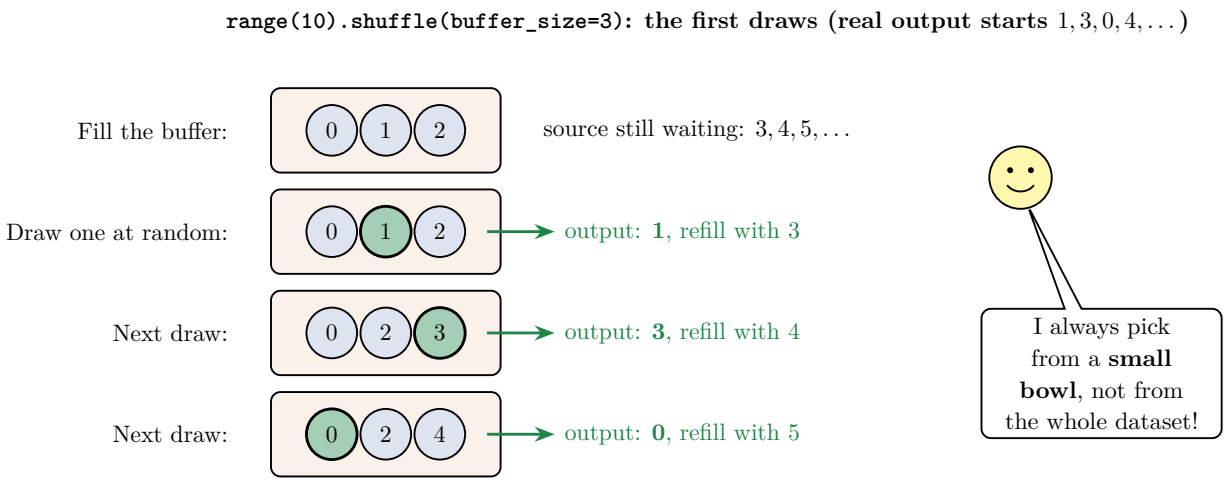


Figure 10.6. How a shuffle buffer of size 3 produces the output $1, 3, 0, \ldots$ (real output below).

Reading the Figure

The bowl holds only three elements. At the start it contains $0, 1, 2$. A random draw gives **1**; the freed place is refilled with the next source element, 3, so the bowl contains $0, 2, 3$; the

next random draw gives **3**, then the bowl is refilled with 4 and the third draw gives **0**, and so on. Continuing the process produces exactly the real output `[1,3,0,4,2,5,6,8,7,9]`. Because an element can only move *a few places* (the bowl is small), the result is only *lightly* shuffled.

Call	Output (seed 42)
`range(10).shuffle(3)`	$1, 3, 0, 4, 2, 5, 6, 8, 7, 9$ (still nearly ordered)
`range(10).shuffle(10)`	$5, 3, 7, 1, 4, 0, 2, 8, 6, 9$ (fully shuffled)

Key Idea

Rule: use a buffer at least as large as the dataset if memory allows (or at least a few thousand elements), and always call `shuffle` *before* `batch`, so that the examples inside a batch are mixed, not the batches themselves.

10.2.4 Performance: Prefetching and Parallel Processing

Imagine a restaurant with one chef and one customer. If the chef cooks a dish, waits while it is eaten, and only then starts the next one, the customer waits a long time. A better kitchen prepares the *next* dish while the customer eats the current one. In an input pipeline the **CPU prepares the next batch while the model (GPU) trains on the current batch**. That is what `prefetch` does.

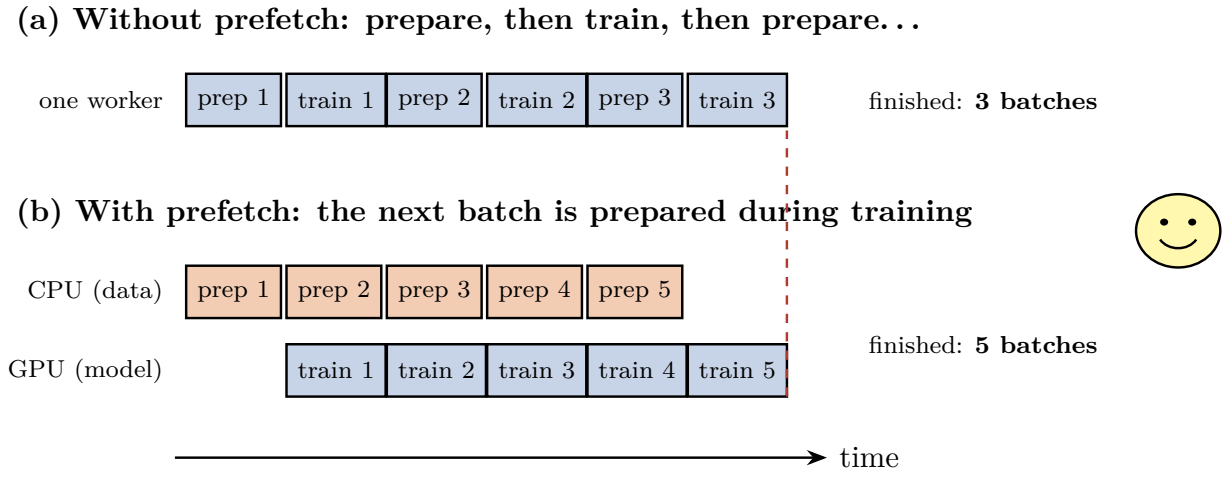


Figure 10.7. Time line of input processing: orange blocks are data preparation, blue blocks are training steps.

Reading the Figure

(a) A single worker alternates between preparing a batch (orange) and training on it (blue), so the model is idle half of the time and only 3 batches are finished in the time shown. **(b)** With prefetch, the data pipeline (top row) prepares batch $k+1$ *at the same time* as the model (bottom row) trains on batch k. After the first preparation the two rows are busy all the time, and in the same time 5 batches are finished. The best case is a speed-up of about $2\times$ when preparation and training take equal time.

We measured this with a pipeline in which preparing one element takes 5 ms and one training step takes 5 ms (100 elements):

Pipeline	Time for 100 steps
No prefetch (prepare, train, prepare, ...)	1.19 s
`.prefetch(1)`	**0.60 s**
Parallel `map` (4 calls) `+ prefetch`	**0.55 s**
Training only (no preparation at all; the lower limit)	0.55 s

Prefetching halves the time (1.19 → 0.60 s), and parallel preparation reaches the lower limit set by the training itself. (Recent TensorFlow versions add prefetching automatically unless disabled; we disabled it for the first row to show the effect.)

```
ds = (tf.data.Dataset.from_tensor_slices((X, y))
        .shuffle(buffer_size=10_000)                          # 1. mix the examples
        .map(preprocess, num_parallel_calls=tf.data.AUTOTUNE)  # 2. prepare in parallel
        .batch(32)                                            # 3. group into batches
        .prefetch(tf.data.AUTOTUNE))                          # 4. overlap with training
```

Good practice	Why
`shuffle` before `batch`	mixes the examples inside each batch
`map` with `num_parallel_calls` set to `AUTOTUNE`	uses several CPU cores for preprocessing
`prefetch(AUTOTUNE)` last	overlaps data preparation with training
`cache()` after expensive steps	avoids repeating work every epoch (if the data fit in memory)

The `tf.data` API

Advantages

- Handles data larger than memory
- Fast: parallel reading, prefetching, caching
- Clean chain of simple operations
- Works directly with Keras `fit`

Disadvantages

- New concepts to learn (lazy datasets, buffers)
- Debugging inside `map` is harder
- Poorly ordered steps (e.g. `batch` before `shuffle`) give subtle bugs
- Little benefit for tiny in-memory datasets

Applications

- Training on large image, text and sensor datasets
- Reading many files (CSV, TFRecord)
- On-the-fly data augmentation
- Production data pipelines

10.3 Reading Data from CSV and TFRecord Files

10.3.1 Reading CSV Files

CSV is the most common file format for tabular data. `tf.data` reads it line by line, so the whole file never has to be loaded into memory.

```
defaults = [0.] * 11                          # 10 features + 1 target, all float

def parse_csv_line(line):
    fields = tf.io.decode_csv(line, record_defaults=defaults)
    return tf.stack(fields[:-1]), tf.stack(fields[-1:])   # (features, target)

def csv_dataset(path, shuffle=False, batch_size=32):
    ds = tf.data.TextLineDataset(path).skip(1)             # skip the header line
    if shuffle: ds = ds.shuffle(1000)
    return (ds.map(parse_csv_line, num_parallel_calls=tf.data.AUTOTUNE)
              .batch(batch_size).prefetch(1))
```

- `TextLineDataset` returns every line of the file as a string.
- `tf.io.decode_csv` splits a line into fields; `record_defaults` gives the type (and the default for missing values) of each column.
- The result has `element_spec` (features of shape (`None`, 10), target of shape (`None`, 1)): `None` is the (variable) batch size.

Real result. We wrote the 1800-row training part of the non-linear regression data of Chapter 9 to a CSV file (first data line: `0.310480,0.720915,0.776527,...`), read it with the pipeline above, and trained the regression MLP with a `Normalization` layer inside the model. Early stopping ended training after 99 epochs and the test RMSE was **1.31** (compared with 1.22 in Chapter 9 with array data and a different random split of the validation set: the same level, and close to the noise floor of 1.0).

10.3.2 TFRecord Files

Definition: TFRecord

TFRecord is TensorFlow's preferred binary file format for large datasets: a file containing a sequence of **records**, each a serialised byte string (usually a *protocol buffer* `tf.train.Example`). The file is read **sequentially**, which is very fast for large datasets on disk or in the cloud, and supports compression and parallel reading of many shards.

A `tf.train.Example` is a dictionary of named *features*, each holding a list of bytes (e.g. an encoded image or a string), floats or 64-bit integers.

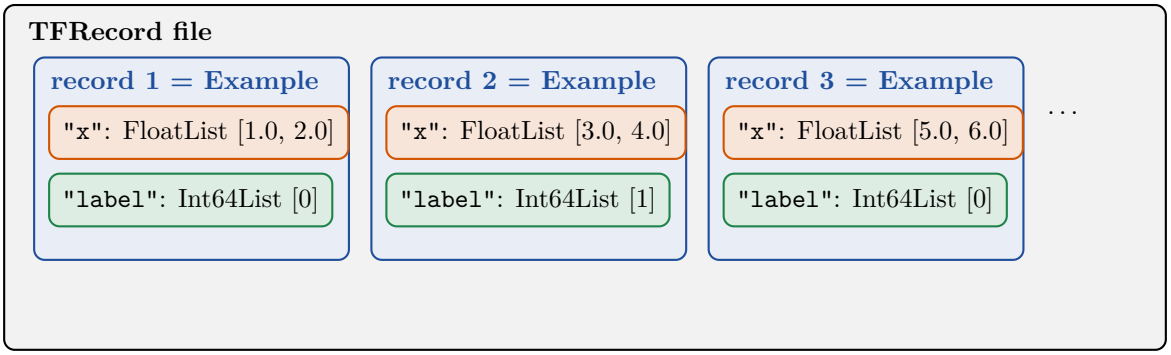


Figure 10.8. The structure of a TFRecord file made of `tf.train.Example` records.

Reading the Figure

The big grey box is the file; it holds a **sequence of records** (blue boxes). Each record is one training example, stored as a dictionary of named features: here `"x"` holds two floating-point numbers (orange) and `"label"` holds one integer (green). Reading the file means reading these records one after another from disk, which is very fast; each record is then *parsed* back into tensors.

```
# writing
with tf.io.TFRecordWriter("toy.tfrecord") as writer:
    for xs, label in [([1., 2.], 0), ([3., 4.], 1), ([5., 6.], 0)]:
        example = tf.train.Example(features=tf.train.Features(feature
            ={
            "x":     tf.train.Feature(float_list=tf.train.FloatList(
                value=xs)),
            "label": tf.train.Feature(int64_list=tf.train.Int64List(
                value=[label])),
        }))
        writer.write(example.SerializeToString())

# reading and parsing
spec = {"x": tf.io.FixedLenFeature([2], tf.float32),
        "label": tf.io.FixedLenFeature([], tf.int64)}
```

```
ds = tf.data.TFRecordDataset("toy.tfrecord").map(
        lambda rec: tf.io.parse_single_example(rec, spec))
for r in ds: print(r["x"].numpy(), int(r["label"]))
# [1. 2.] 0   [3. 4.] 1   [5. 6.] 0
```

The three records took **159 bytes** in total and were read back exactly. For images one usually stores each encoded JPEG as a *bytes feature* together with its label.

Images in folders. For images stored in one folder per class, the function `image_dataset_from_directory` of `keras.utils` creates a labelled `tf.data` dataset directly.

	CSV	**TFRecord**
Format	text, human-readable	binary, compact
Best for	small to medium tabular data	very large datasets, images, audio
Reading speed	slower (text parsing)	fast sequential reads
Inspect with a text editor?	yes	no

TFRecord format

Advantages

- Fast sequential reading of huge datasets
- Compact binary storage; supports compression
- Easy to split into shards and read in parallel
- Integrates directly with `tf.data`

Disadvantages

- Not human-readable
- Extra code to write and parse the records
- Overkill for small datasets
- Schema must be known when parsing

Applications

- Large image and speech datasets
- Distributed and cloud training
- Efficient storage of preprocessed features

10.4 Preprocessing the Input Features

Neural networks need numerical, well-scaled inputs. Text categories must be encoded and numbers must be normalised. There are three places where this can be done:

Where	Remark
Before training (pandas / scikit-learn)	simple, but you must remember to repeat exactly the same steps when using the model later
Inside `tf.data` (`map`)	runs in the pipeline, in parallel with training
As **Keras preprocessing layers inside the model**	the model carries its preprocessing with it, so the same code is used in training and in production (avoids **training–serving skew**)

10.4.1 Numerical Features: Normalization

Definition: `Normalization` layer

`layers.Normalization()` standardises each feature to mean 0 and standard deviation 1, $x' = (x - \mu)/\sigma$. The statistics μ, σ are learned from the training data with `adapt` before training and stored in the layer.

Example: Normalization on three numbers

The training values $10, 20, 30$ give $\mu = 20$ and (population) variance $\frac{100+0+100}{3} = 66.67$, so $\sigma = 8.165$.

$$10 \to \frac{10-20}{8.165} = -1.2247, \qquad 20 \to 0, \qquad 30 \to +1.2247.$$

This is exactly what Keras prints: mean `[20.0]`, variance `[66.67]`, output $-1.2247,\ 0,\ 1.2247$.

```
norm = layers.Normalization()
norm.adapt(X_train)                          # learn the means and
    standard deviations
model = keras.Sequential([keras.Input((10,)), norm,        #
    preprocessing inside
                          layers.Dense(50, activation="relu"), layers.
                              Dense(1)])
```

10.4.2 Categorical Features: One-Hot Encoding

A feature such as `"ocean_proximity"` with values `"<1H OCEAN"`, `"INLAND"`, `"NEAR OCEAN"` must be turned into numbers. Giving them the numbers $0, 1, 2$ would wrongly suggest an order; instead we use **one-hot encoding**.

Definition: One-hot encoding

One-hot encoding represents a categorical value with K possible values by a vector of length K with a 1 at the position of the category and 0 elsewhere. In Keras this takes two layers: `StringLookup` (maps each string to an integer index using a vocabulary learned with `adapt`) and `CategoryEncoding` (converts the index to a one-hot vector).

Example: One-hot encoding, real output

`StringLookup` adapted on the data `["<1H OCEAN","INLAND","NEAR OCEAN","INLAND","<1H OCEAN"]` builds the vocabulary `INLAND`, `<1H OCEAN`, `NEAR OCEAN` (ordered by frequency). Then:

Value	Index	One-hot vector
`INLAND`	0	$[1, 0, 0]$
`NEAR OCEAN`	2	$[0, 0, 1]$
`<1H OCEAN`	1	$[0, 1, 0]$

Unknown categories. If the vocabulary is fixed in advance with one *out-of-vocabulary* slot, a known value gets its index shifted by one (`INLAND`$\rightarrow 2$) and an unseen value such as `ISLAND` falls into slot 0, so the model does not crash on new categories.

10.4.3 Embeddings

One-hot vectors become huge when there are many categories (50 000 words, 1 million users) and treat all categories as equally different. An **embedding** gives every category a short, *learned* vector.

Definition: Embedding

An **embedding layer** is a trainable **lookup table** (a matrix with one row per category and `output_dim` columns). The input is the category's integer index; the output is the corresponding row. The rows are initialised randomly and learned by backpropagation like any other weights, so categories that behave similarly end up with *similar vectors*.

Example: An embedding of 4 categories in 2 dimensions

`layers.Embedding(input_dim=4, output_dim=2)` creates a table of shape $(4, 2)$, initially small random numbers, e.g. row 1 $= (-0.014,\ 0.045)$. Looking up the indices $[1, 2, 1]$ returns a $(3, 2)$ tensor; the first and third rows are identical because both indices are 1.

Parameter count. For 50 categories with 8-dimensional embeddings the table has $50 \times 8 = 400$ trainable numbers, while one-hot encoding would feed 50 numbers (mostly zeros) into the next layer.

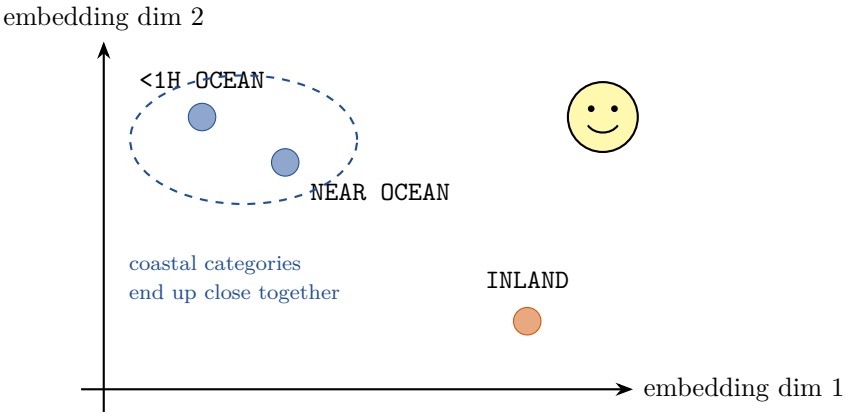


Figure 10.9. Illustration (not a real run) of what a *trained* embedding of the three categories might look like.

Reading the Figure

This is a schematic picture, not the output of an experiment: after training on a house-price task, the vectors of categories that affect the price in the same way (the two coastal categories) tend to lie **close together**, while `INLAND` lies far away. The network discovers this similarity by itself; with one-hot encoding every pair of categories would be equally far apart.

Rule of thumb. Use one-hot encoding for fewer than about 10 categories; use embeddings for more than about 50 (for 10 to 50 either may work, so try both).

10.4.4 Text Features: TextVectorization

Definition: `TextVectorization`

`layers.TextVectorization` converts raw text into sequences of integer word indices. `adapt` builds a **vocabulary** (the most frequent words are given the smallest indices; index 0 is padding and 1 is the "unknown" token `[UNK]`).

Example: TextVectorization, real output

Adapted on `"I love machine learning"`, `"machine learning is fun"`, `"I love fun"`, the vocabulary is

```
['', '[UNK]', 'machine', 'love', 'learning', 'i', 'fun', 'is']
```

(indices 0 to 7). The sentence "I love learning" becomes $[5, 3, 4]$ and "fun fun machine" becomes $[6, 6, 2]$. The integer sequences are then passed to an `Embedding` layer.

Images. Layers such as `Rescaling(1/255)` (scale the pixels), `RandomFlip`, `RandomRotation` and `RandomZoom` (data augmentation; active only during training) can also be placed inside the model.

Common Mistake

Learn preprocessing statistics from the training set only. Call `adapt` on the training data, never on the validation or test data; otherwise information from the test set leaks into the model (Chapter 3, feature scaling).

Keras preprocessing layers

Advantages

- Preprocessing is part of the model: no training–serving skew
- One code path for training and production
- Fast, runs on GPU; includes scaling, encoding, text and augmentation
- Statistics (`adapt`) are saved with the model

Disadvantages

- Must remember to call `adapt` on training data only
- Less flexible than arbitrary Python code
- Large vocabularies increase model size
- Embeddings need enough data to learn well

Applications

- Normalising numerical features
- Encoding categories (one-hot, embeddings)
- Text classification pipelines
- Image augmentation

10.5 A Complete Pipeline from Data to Model

Putting everything together: files are read, parsed, shuffled, batched and prefetched; the model contains the preprocessing layers; and Keras `fit` consumes the dataset directly.

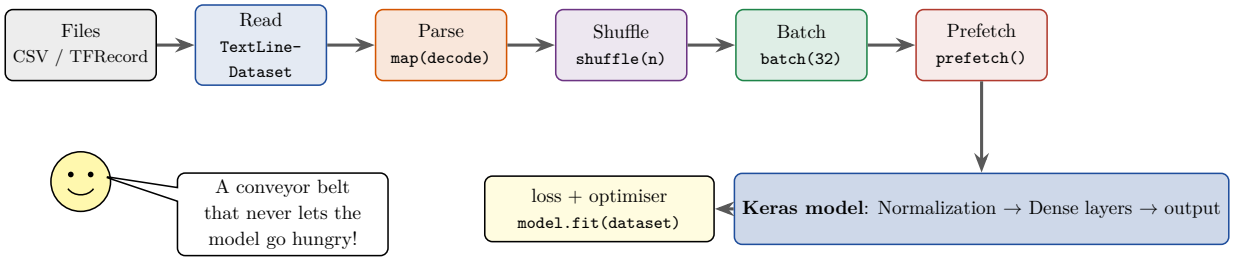


Figure 10.10. A complete input pipeline, from files to the training of a model.

Reading the Figure

Follow the conveyor belt from the left. The raw **files** are **read** line by line or record by record, each element is **parsed** into tensors, the examples are **shuffled** with a buffer, grouped into **batches**, and **prefetched** so the next batch is ready while the model is busy. The model begins with its own preprocessing layers (such as `Normalization`), then the dense layers. The loss and optimiser compare predictions with targets and update the weights; `model.fit(dataset)` runs the whole belt.

Real comparison. We trained the Fashion-MNIST MLP of Chapter 9 for 5 epochs with Adam in two ways: (i) from NumPy arrays; (ii) from a `tf.data` pipeline made of `shuffle(5000)`, `batch(32)` and `prefetch(AUTOTUNE)`.

```
train_ds = (tf.data.Dataset.from_tensor_slices((X_train, y_train))
              .shuffle(5000).batch(32).prefetch(tf.data.AUTOTUNE))
valid_ds = (tf.data.Dataset.from_tensor_slices((X_valid, y_valid))
              .batch(32).prefetch(tf.data.AUTOTUNE))
model.fit(train_ds, epochs=5, validation_data=valid_ds)
```

Input method	Test accuracy	Time per epoch
NumPy arrays	0.870	6.3 s
`tf.data` pipeline	0.875	6.0 s

For a small dataset that fits in memory the two ways give the *same* accuracy (the small difference 0.870 vs 0.875 comes from the different random order of the examples) and similar speed: the advantages of `tf.data` appear for **large datasets, files on disk, and expensive preprocessing**, exactly the situation in practice.

10.6 Putting the Whole Course Together

We have now met the main families of machine-learning methods. The last picture of the book is a map for choosing among them.

Reading the Figure

Start on the left: if you have **no labels** you are in unsupervised territory: clustering to find groups (Chapter 7) or dimensionality reduction to compress and visualise (Chapter 8). If you **do** have labels you are doing supervised learning (regression or classification). For **images, audio, text or very large data**, neural networks built with Keras and fed by `tf.data` (Chapters 9–10) are usually the best choice; for ordinary **tabular data** the classical methods (linear and logistic regression, trees, k-NN, Naive Bayes, SVMs and ensembles such as random forests and boosting; Chapters 3–6) are strong, fast and easier to

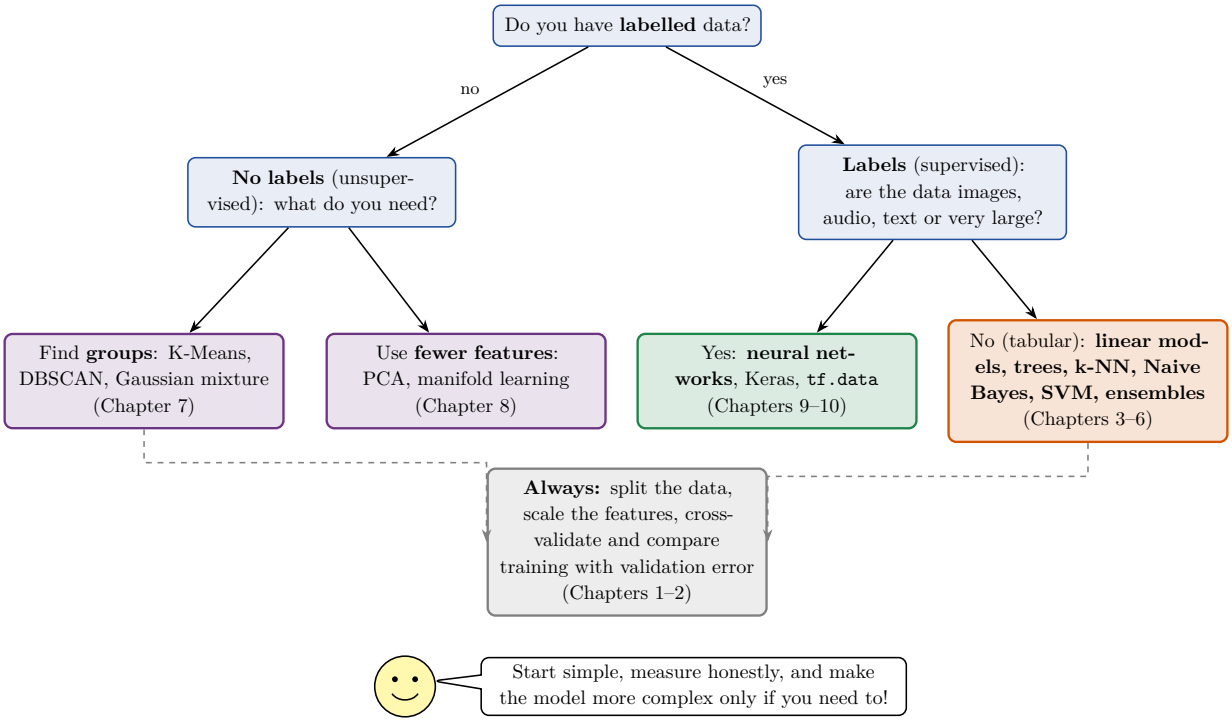


Figure 10.11. A roadmap of the methods studied in this book.

interpret. Whatever the choice, the discipline of Chapters 1 and 2 always applies: split the data, scale the features, cross-validate, and look at the gap between training and validation performance.

Unit	Chapters	Key take-away
I	1–2	Learning means generalising to new data; bias–variance trade-off; test loss, cross-validation, sampling distributions
II	3–4	k-NN, decision trees, Naive Bayes, linear and logistic regression, GLMs, SVM basics
III	5–6	Ensembles reduce variance (bagging) or bias (boosting); kernel SVMs and SVR; Naive Bayes variants
IV	7–8	Clustering (K-Means, DBSCAN, GMM); curse of dimensionality; PCA
V	9–10	Neural networks, backpropagation, Keras; TensorFlow and efficient data pipelines

Chapter Summary

- **TensorFlow** is the numerical engine under Keras: tensors (rank, shape, dtype), operations, broadcasting, variables, automatic differentiation and graph optimisation.
- **Tensors:** scalar (rank 0), vector, matrix, 3-D, 4-D batch of images $(32, 28, 28, 1)$;

`tf.matmul`, `reduce_sum`, `reshape`; broadcasting stretches the smaller tensor.

- **`GradientTape`** records operations and plays them back to get exact gradients (e.g. $dy/dx = 7$ for $x^2 + 3x$ at 2); a custom loop recovers the Chapter 4 regression ($26 + 9x$).
- **`tf.function`** compiles Python into a graph (1.45× faster in our test).
- **`tf.data`:** `from_tensor_slices`, `map`, `shuffle` (before `batch`), `batch`, `prefetch`; prefetching halved the time (1.19 → 0.60 s).
- **CSV** via `TextLineDataset` and `decode_csv`; **TFRecord** for large datasets (sequential binary records of `tf.train.Example`).
- **Preprocessing layers:** `Normalization` (adapt on training data), `StringLookup`+`CategoryEncoding` (one-hot), `Embedding` (learned dense vectors), `TextVectorization`; putting them inside the model avoids training–serving skew.
- A complete pipeline: files → read → parse → shuffle → batch → prefetch → model; for small in-memory data results equal the NumPy route (0.870 vs 0.875).

Exam Corner

A. Very Short Answer Questions (2 marks each)

1. What is TensorFlow? How is it related to Keras? **[2 marks]**
2. Define a tensor. What are rank, shape and dtype? **[2 marks]**
3. What is broadcasting? **[2 marks]**
4. Differentiate between `tf.constant` and `tf.Variable`. **[2 marks]**
5. What is automatic differentiation? **[2 marks]**
6. What does `tf.GradientTape` do? **[2 marks]**
7. What is the purpose of `tf.function`? **[2 marks]**
8. What is a `tf.data.Dataset`? **[2 marks]**
9. Why should `shuffle` be called before `batch`? **[2 marks]**
10. What does `prefetch` do? **[2 marks]**
11. What is a TFRecord file? **[2 marks]**
12. Differentiate between one-hot encoding and embedding. **[2 marks]**

Model Answer: Q3, Q6 and Q9

Q3. Broadcasting is the automatic stretching of a smaller tensor along missing axes so that two tensors of different shapes can be combined element-wise, without copying the data. Example: $\left[\begin{smallmatrix}1 & 2\\ 3 & 4\end{smallmatrix}\right] + [10, 20] = \left[\begin{smallmatrix}11 & 22\\ 13 & 24\end{smallmatrix}\right]$.

Q6. `GradientTape` records all operations performed on variables inside a `with` block; calling `tape.gradient(loss, variables)` replays the recorded operations backwards with the chain rule and returns exact gradients of the loss with respect to the variables. It is used to write custom training loops.

Q9. `shuffle` randomises the order of individual examples, and `batch` groups consecutive examples. If we batch first and then shuffle, only whole batches change their order while each batch always contains the same consecutive examples, so the examples in a batch are not a random sample.

B. Short Answer Questions (5 marks each)

1. Explain tensors with examples of different ranks and the basic tensor operations. **[5 marks]**
2. Explain automatic differentiation with `GradientTape` using an example. **[5 marks]**
3. Explain how a custom training loop is written with TensorFlow. **[5 marks]**
4. Explain `tf.function` and computation graphs. **[5 marks]**
5. Explain the `tf.data` API and its main methods. **[5 marks]**
6. Explain how shuffling with a buffer works. **[5 marks]**
7. Explain how prefetching and parallel processing speed up an input pipeline. **[5 marks]**
8. Explain how CSV data are read with `tf.data`. **[5 marks]**
9. Explain the TFRecord format and the structure of `tf.train.Example`. **[5 marks]**
10. Explain how categorical features are preprocessed (one-hot encoding and embeddings). **[5 marks]**
11. Explain Keras preprocessing layers and their advantages. **[5 marks]**

Model Answer: B7 (outline for 5 marks)

Problem: if the model waits for the data, the GPU is idle. *Prefetch:* `dataset.prefetch(n)` prepares the next n batches in the background while the model trains on the current batch, so data preparation and training overlap. *Parallel map:* `map(f, num_parallel_calls=AUTOTUNE)` preprocesses several elements simultaneously on multiple CPU cores. *Cache:* keep preprocessed data in memory after the first epoch. *Example (real):* with 5 ms of preparation and 5 ms of training per step, 100 steps took 1.19 s without prefetch, 0.60 s with `prefetch(1)` and 0.55 s with parallel map plus prefetch (the training-only limit).

C. Long Answer Questions (10 marks each)

1. Explain TensorFlow: tensors, variables, operations, broadcasting, automatic differentiation and `tf.function`, with code examples. **[10 marks]**
2. Explain how to write a custom training loop with `GradientTape`. Fit a line by gradient descent and show the first update by hand. **[10 marks]**
3. Explain the `tf.data` API: dataset creation, `map`, `shuffle`, `batch`, `prefetch`, with a pipeline for training a model. **[10 marks]**
4. Explain how to load data from CSV files and TFRecord files with TensorFlow, with code. **[10 marks]**
5. Explain the preprocessing of numerical, categorical and text features using Keras preprocessing layers, including embeddings. **[10 marks]**
6. Describe a complete pipeline from raw files to a trained Keras model and discuss its advantages over using in-memory arrays. **[10 marks]**

Model Answer: C2 (first update by hand)

Data $x = (1,2,3,4,5)$ hours centred to $x_c = (-2,-1,0,1,2)$, marks $(35,45,50,65,70)$, model $\hat{y} = wx_c + b$, loss $J = \frac{1}{2n}\sum(\hat{y}-y)^2$, start $w = b = 0$, $\eta = 0.05$.
Errors $\hat{y} - y = -(35,45,50,65,70)$. $\partial J/\partial b = \frac{-265}{5} = -53$; $\partial J/\partial w = \frac{1}{n}\sum(\hat{y}-y)x_c = \frac{-(35)(-2)-(45)(-1)-0-(65)(1)-(70)(2)}{5} = \frac{70+45-65-140}{5} = -18$.
Update: $w = 0 - 0.05(-18) = 0.90$, $b = 0 - 0.05(-53) = 2.65$, which are the values printed after step 1 in the real run ($w = 0.900$, $b = 2.650$).

D. Numerical Problems

1. Give the rank and number of elements of a tensor of shape $(32, 28, 28, 1)$. *(Answer: rank 4; $32 \cdot 28 \cdot 28 \cdot 1 = 25\,088$.)*
2. Compute `tf.matmul(A,B)` for $A = \left[\begin{smallmatrix}1&2\\3&4\end{smallmatrix}\right]$ and $B = \left[\begin{smallmatrix}0&1\\1&0\end{smallmatrix}\right]$. *(Answer: $\left[\begin{smallmatrix}2&1\\4&3\end{smallmatrix}\right]$.)*
3. Compute $\left[\begin{smallmatrix}1&2\\3&4\end{smallmatrix}\right] + [10, 20]$ using broadcasting. *(Answer: $\left[\begin{smallmatrix}11&22\\13&24\end{smallmatrix}\right]$.)*
4. Find dy/dx for $y = 3x^2 + 2x$ at $x = 1$, and the partial derivatives of $z = x^2 y$ at $(x, y) = (2, 3)$. *(Answer: $6x + 2 = 8$; $\partial z/\partial x = 2xy = 12$, $\partial z/\partial y = x^2 = 4$.)*
5. One gradient step: $w = 1$, $x = 2$, $y = 5$, loss $(wx - y)^2$, $\eta = 0.1$. *(Answer: gradient $2(wx - y)x = 2(2 - 5)(2) = -12$; $w = 1 + 1.2 = 2.2$.)*
6. How many batches does a dataset of 1000 examples give with batch size 32, with and without `drop_remainder`? *(Answer: $\lceil 1000/32 \rceil = 32$ batches (31 full plus one of 8); 31 with `drop_remainder=True`.)*
7. Normalise the data $2, 4, 6, 8$ (population standard deviation). *(Answer: $\mu = 5$, $\sigma = \sqrt{5} = 2.236$; -1.342, -0.447, 0.447, 1.342.)*

8. One-hot encode the category with index 2 among 4 categories. How many parameters has an embedding of 50 categories with dimension 8? *(Answer:* $[0, 0, 1, 0]$*;* $50 \times 8 = 400$*.)*

9. A pipeline spends 5 ms preparing a batch and 5 ms training on it, for 100 batches. Estimate the total time without and with perfect overlap. *(Answer:* $100 \times 10 = 1.0$ *s; about* 0.5 *s with overlap. Measured:* 1.19 *s and* 0.60 *s.)*

10. `tf.function` reduces a call from 22.8 ms to 15.7 ms. Find the speed-up and the time saved over 10 000 calls. *(Answer:* $22.8/15.7 = 1.45$*;* $(22.8 - 15.7) \times 10\,000 = 71\,000$ *ms* $= 71$ *s.)*

E. Multiple-Choice Questions

1. A tensor of shape $(2, 3)$ has rank:
   (a) 1 (b) 2 (c) 3 (d) 6

2. Which can be modified after creation?
   (a) `tf.constant` (b) `tf.Variable` (c) a Python tuple (d) a string tensor

3. `tf.GradientTape` is used for:
   (a) plotting (b) automatic differentiation (c) reading files (d) scaling data

4. `@tf.function` converts Python code into a:
   (a) CSV file (b) computation graph (c) database (d) dataset

5. To read the lines of a text file with `tf.data` we use:
   (a) `TFRecordDataset` (b) `TextLineDataset` (c) `range` (d) `Normalization`

6. The correct order of the two methods is:
   (a) `batch` then `shuffle` (b) `shuffle` then `batch` (c) either (d) neither

7. `prefetch` improves performance by:
   (a) overlapping data preparation and training (b) shuffling (c) normalising (d) reducing the batch size

8. A TFRecord stores data as:
   (a) text lines (b) serialised binary records (c) images only (d) CSV

9. To avoid training–serving skew, preprocessing should be:
   (a) done by hand each time (b) part of the model (c) skipped (d) done on the test set only

10. `Normalization.adapt()` should be called on the:
    (a) test set (b) training set (c) validation set (d) all data

11. For a categorical feature with 1000 categories the better encoding is usually:
    (a) one-hot (b) embedding (c) integer coding (d) none

12. In `decode_csv`, `record_defaults` specifies:
    (a) the file name (b) the type and default of each column (c) the batch size (d) the optimiser

Answers: 1-b, 2-b, 3-b, 4-b, 5-b, 6-b, 7-a, 8-b, 9-b, 10-b, 11-b, 12-b.

F. Practice Problems

1. Write TensorFlow code that computes the gradient of $f(x) = x^4$ at $x = 3$ and check it by hand. *(Answer:* $4x^3 = 108$*.)*
2. With a shuffle buffer of size 1, what is the output of `range(5).shuffle(1)`? Why? *(Answer:* $0, 1, 2, 3, 4$*; the buffer always holds one element, so nothing can be reordered.)*
3. **Coding:** write a CSV file with a few columns of your own data, read it with `TextLineDataset` and `decode_csv`, and print the first batch.
4. **Coding:** measure the effect of `prefetch` yourself by adding `time.sleep` in a `map` function and in the training loop.
5. **Coding:** build a model that includes a `Normalization` layer and a `StringLookup` + `CategoryEncoding` branch for a mixed numerical/categorical dataset.
6. Draw the roadmap of Figure 10.11 in your own words and explain when you would choose a neural network instead of a random forest.

Quick Revision

Topic	One-line recall
Tensor	Array with rank, shape, dtype; batch of images $(32, 28, 28, 1)$.
Variable	Mutable tensor (`assign`); model weights.
Broadcasting	Smaller tensor is stretched to match: $[[1, 2], [3, 4]] + [10, 20]$.
GradientTape	Records operations, replays backwards: gradient of $x^2 + 3x$ at 2 is 7.
Custom loop	`with tape:` `loss`; `tape.gradient`; `assign_sub(`η`·grad)`.
tf.function	Traces Python into a graph; $\approx 1.45\times$ faster in our test.
tf.data	`from_tensor_slices`, `map`, `shuffle`, `batch`, `prefetch`.
Shuffle buffer	Random draw from a small bowl; shuffle before batch.
Prefetch	Prepare batch $k + 1$ while training on batch k ($1.19 \to 0.60$ s).
CSV / TFRecord	`TextLineDataset`+`decode_csv` / sequential binary `Example` records.
Preprocessing layers	Normalization, StringLookup+CategoryEncoding, Embedding, TextVectorization; inside the model.
Embedding	Learned lookup table; K categories $\times$ dim parameters.